

ithiema

7 天搞定 Java

深入理解面向对象,自己开发游戏引擎

本套课程配套视频教程已经上线 B 站

<https://www.bilibili.com/video/BV1aA411c7Ab/>





目 录

第 0 章—课程导学	6
1. 课程适合人群.....	6
2. 课程总目标.....	6
3. 内容编排.....	7
4. 学习方法.....	7
第 1 章 计算机系统结构	8
1.1 认识计算机.....	8
1.2 计算机体系结构	10
1.3 认识进制	11
1.3.1 二进制的出现.....	11
1.3.2 二进制计数法.....	12
1.3.3 十六进制计数法	13
1.4 常见数据的二进制表示	14
1.4.1 文本的表示.....	14
1.4.2 图片的表示	15
1.4.3 声音的表示.....	17
1.5 信息处理的原理.....	17
第 2 章 初识 Java	20
2.1 机器语言入门.....	21
2.2 高级编程语言.....	22
2.3 Java 语言概述	24
2.3.1 什么是 Java 语言.....	24
2.3.2 Java 语言的发展历史	25
2.4 认识 JDK、JRE 和 JVM	26
2.5 Java 开发环境搭建	27
2.5.1 JDK 的下载	27
2.5.2 JDK 的安装	31
2.5.3 IDEA 的安装	33
2.5.4 IDEA 的启动	38
2.6 开发第一个 Java 程序	43



2.6.1 使用 IDEA 开发 HelloWorld	43
2.6.2 分析 IDEA 编译代码的流程	50
2.6.3 编写 Java 程序的注意事项	53
2.6.4 JDK 11 的快捷运行	53
第 3 章 Java 编程基础	54
3.1 编程与编程思维	55
3.2 变量	56
3.2.1 变量的声明与赋值	56
3.2.2 应用案例：统计公交车停靠站	57
3.2.2 应用案例：统计公交车累计收入	59
3.3 字面量与常量	62
3.3.1 字面量	62
3.3.1 常量	62
3.4 注释	63
3.5 标识符	64
3.6 关键字	65
3.7 运算符	66
3.7.1 算术运算符	66
3.7.2 赋值运算符	67
3.7.3 比较运算符	68
3.7.4 逻辑运算符	68
3.7.5 逻辑运算符优先级	69
3.8 数据类型	70
3.8.1 基本数据类型	70
3.8.2 复杂数据类型	71
3.8.3 基本数据类型的装箱类	73
3.8.4 数据类型转换	76
3.9 字符串	78
3.9.1 字符与字符串	78
3.9.2 字符串拼接运算	80
第 4 章 条件语句	81
4.1 if 语句	82
4.2 嵌套 if 语句	82
4.3 if-else 语句	84
4.4 switch 语句	85
4.4.1 switch 语句语法	85
4.4.2 应用案例—作文自动生成器	86
4.4.3 switch 比较字符和字符串	88
4.5 三目运算符	89
第 5 章 函数	89



5.1 函数入门	90
5.1.1 什么是函数	90
5.1.2 入口函数	94
5.1.3 剖析 System.out 源码	94
5.2 函数的参数和返回值	96
5.2.1 认识函数参数和返回值	96
5.2.2 应用案例：计算器	98
5.2.3 应用案例：超市找零	100
5.3 格式化输出	101
5.4 函数综合案例——掷骰子	103
5.4.1 掷骰子-基础版	103
5.4.2 掷骰子-升级版	104
5.5 函数特性总结	105
5.6 JDK 文档	105
5.7.1 JDK 文档的查看和阅读	105
5.7.2JDK 文档的生成	109
第 6 章 循环语句	111
6.1 while 循环	112
6.1.1 while 循环入门	112
6.1.2 应用案例：闹钟程序	113
6.1.3 应用案例：计算 100 以内数据和	116
6.2 do-while 循环	116
6.3 for 循环	118
6.3.1 while 循环入门	118
6.3.2 应用案例：水仙花	120
6.3.3 应用案例：九九乘法表	122
6.4 跳转语句	123
6.4.1 break 语句	124
6.4.2 continue 语句	124
6.4.3 应用案例：逢七必过	125
第 7 章 数组	127
7.1 初识数组	127
7.1.1 数组的概念	127
7.1.2 数组的长度	129
7.2 数组在函数中的应用	130
7.2.1 应用案例：计算平均成绩	130
7.2.2 应用案例：查找班级姓“范”的所有同学	131
7.3 数组在增强 for 循环中的应用	132
7.4 二维数组	133
7.4 使用 new 的方式创建数组并赋值	135



7.5 Java 程序内存分析	136
7.5.1 程序加载的内存区域	136
7.5.2 函数调用过程内存分析	137
7.5.3 两个数字交换的内存分析	139
7.5.4 使用数组交换数据的内存分析	142
7.6 数组的动态扩容	143
第8章 面向对象入门	145
8.1 面向对象概述	146
8.1.1 认识面向对象	146
8.1.2 使用面向对象描述事物	146
8.2 类和对象	147
8.2.1 类和对象的关系	147
8.2.2 类的实例化	149
8.3 构造方法	150
8.3.1 定义构造方法	150
8.3.2 构造方法的重载	152
8.4 this 关键字	153
8.5 应用案例：格斗游戏	155
8.5.1 格斗游戏入门	155
8.5.2 打斗过程的内存分析	156
8.5.3 游戏角色的长相设计	157
8.5.4 完善文字版的格斗游戏	159
8.6 toString 方法	163
8.7 静态方法	165
8.8 访问修饰符	166
8.9 包的使用	170
第9章 面向对象进阶	172
9.1 面向对象编程思考	173
9.1.1 面向对象编程的意义	173
9.1.2 应用案例：使用面向对象思想协同开发洗衣机	174
9.2 封装	177
9.2.1 认识封装	177
9.2.2 封装的实现	178
9.2.3 getter 和 setter 方法	181
9.3 继承	183
9.3.1 认识继承	183
9.3.2 继承的实现	185
9.3.3 方法的重写	187
9.4 super 关键字	189
9.5 Object 类	191



9.6 final 关键字	191
9.7 Java 中的抽象	193
9.5.1 规则是更高层次的抽象	193
9.5.2 抽象类和抽象方法的语法	194
9.5.3 应用案例：计算器	194
9.8 案例：使用抽象类模板开发乒乓球游戏	196
9.6.1 乒乓球游戏需求分析	196
9.6.2 游戏框架介绍	197
9.6.3 通过实验理解抽象类中方法的作用	198
9.6.4 快速开发乒乓球游戏	201
9.9 静态常量	208
9.10 接口	210
9.8.1 认识接口	210
9.8.2 接口定义行为标准	212
9.8.3 实现接口	213
9.8.4 检查对象是否实现接口规范	214
9.8.5 接口可以做标记	216
9.8.6 抽象类和接口的区别	217
9.8.7 应用案例：充电宝	218
9.11 通过接口实现多继承	221
9.12 抽象类和接口的选取规则	224
9.10.1 接口和抽象类的设计	224
9.10.2 PPAP 代码的实现	225
9.13 多态	228
第 10 章集合和泛型	230
10.1 集合概述	231
10.2 List 接口	231
10.2.1 ArrayList 集合	232
10.2.2 LinkedList 集合	233
10.3 List 常见 API+循环遍历集合元素	233
10.4 学生信息管理系统的开发	234
10.5 栈和队列	241
10.6 泛型	243
10.7 Map	244
第 11 章 GUI 编程	245
11.1 绘制常见图形	246
11.1.1 绘制对话框	246
11.1.2 绘制划线	247
11.1.3 对抗锯齿	249
11.1.4 绘制好看的线条	250



11.1.5 绘制矩形.....	252
11.1.6 绘制圆形.....	253
11.1.7 绘制文字.....	255
11.1.8 绘制图片.....	257
11.1.9 绘制笑脸.....	260
11.2 使用面向对象思想设计直线.....	262
11.3 绘制动画.....	265
11.3 键盘事件.....	267
第 12 章综合项目-贪吃蛇.....	270
12.1 游戏框架介绍.....	271
12.2 游戏环境搭建.....	274
12.3 贪吃蛇的网格设计.....	275
12.4 贪吃蛇的身子绘制.....	278
12.5 贪吃蛇的上下左右移动.....	281
12.6 绘制苹果.....	286
12.7 吃苹果操作.....	288
12.8 贪吃蛇的速度.....	292
12.5 贪吃蛇的优化.....	294
12.5 案例总结.....	297

第 0 章—课程导学

1. 课程适合人群

本课程从零开始，详细讲解 Java 基础的相关内容，无论你是零基础的小白，还是具有一定开发工作经验的开发人员，学习完本套课程，你都会收获满满的。

本套课程适合的人群包括：

- （1）零基础、对 Java 感兴趣的爱好者。
- （2）深刻体会编程思想，想成为资深开发工程师的同学。

2. 课程总目标

通过本套课程的学习，希望大家可以达成以下目标：

- （1）掌握 Java 常用语法，熟练使用控制语句、方法、数组和集合。
- （2）熟练使用面向对象编程和面向接口编程的编程方法。
- （3）能开发各种常见的小应用，并打包发布供其他人使用。



3. 内容编排

本套课程的内容编排，共分为 4 个模块，具体如图 0-1 所示。

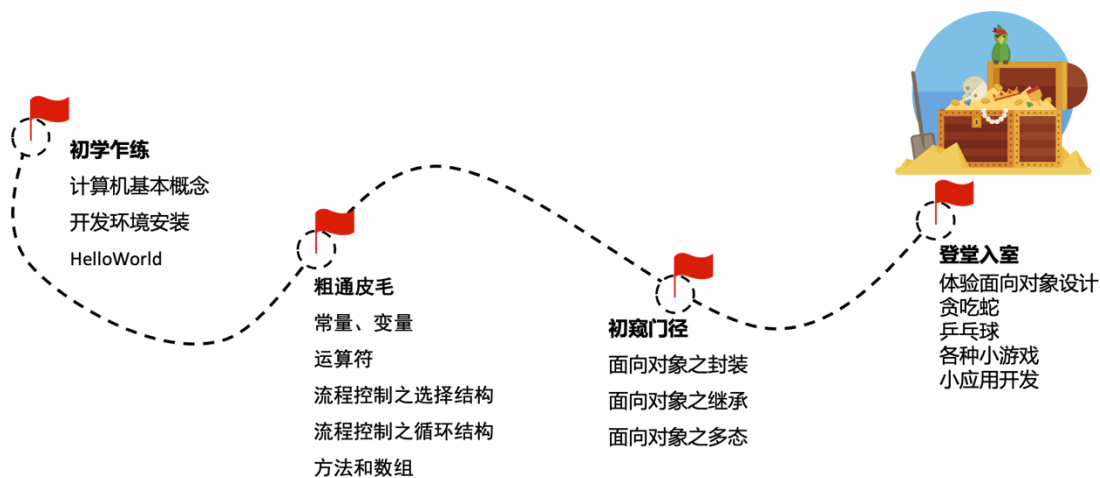


图 0-1 本套课程的内容编排

4. 学习方法

我们根据本套 Java 课程的不同阶段，提供了不同的学习方法，如图 0-2 所示。

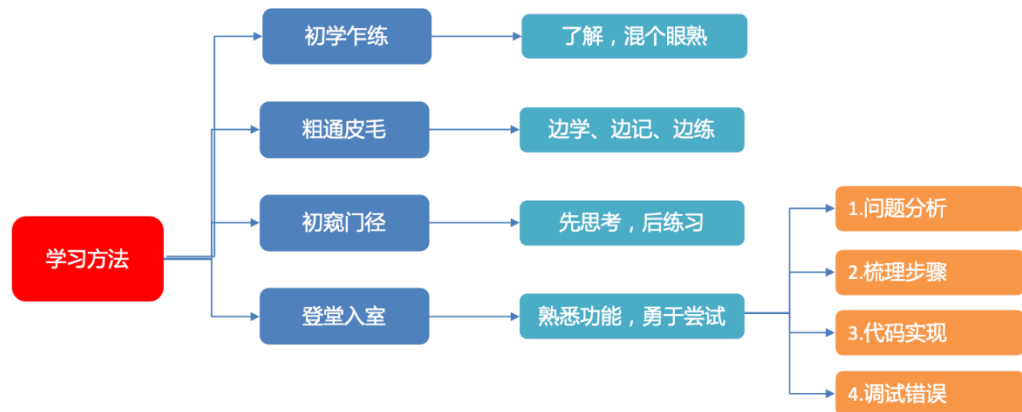


图 0-2 不同阶段的学习方法

关于图 0-2 所示各阶段的学习方法介绍，具体如下：

(1) 初学乍到阶段，主要介绍的是一些计算机的基本概念以及开发环境的搭建，所以该阶段，大家只需要了解即可。

(2) 粗通皮毛阶段，是扎实基本功的阶段，希望大家可以一边学，一边记忆并加以练习，这就好比学习英语的背单词阶段。

(3) 初窥门庭阶段，主要是学习面向对象的编程思想，该阶段需要大家多思考，体会面向对象的编程思想，并结合练习加深理解。

（4）登堂入室阶段，是考验大家解决实际需求的能力，该阶段要求大家能够运用前面所学的知识，对相关业务进行分析，能够梳理出开发思路并最终实现功能，与此同时，在开发项目的过程中，希望大家能够具备调试错误的能力。

学习是一个持之以恒的事情，既然大家选择了 Java 编程，希望能够基于案例、多实践、多感受、多体会，不断提升 Java 技能，大家加油！

第 1 章 计算机系统结构

学习目标

- ◆ 了解计算机，能够说出计算机的组成部分
- ◆ 了解计算机的体系结构，能够说出计算机各部分的功能
- ◆ 熟悉二进制、十六进制，能够进行不同进制的数据转换
- ◆ 了解常见数据的二进制表示方式
- ◆ 了解信息处理的原理，能够说出计算机使用电路输入处理数据

的方式

考虑到零基础同学学习本套课程的难度，我们在正式编写 Java 程序之前，先来介绍一些关于计算机系统结构的基础知识，包括计算机的构成、计算机体系结构、进制、常见数据的二进制表示以及信息处理原理。

1.1 认识计算机

计算机的普及正在改变着我们的世界，从广义上讲，计算机是一种工具，这个工具一点都不神秘，可以把它看做是图 1-1 所示的日常生活中的工具而已。



图 1-1 日常生活中的工具

我们作为工具使用者，如果能熟练掌握工具的使用，就相当于具备了某些技能。这里需要注意的是，我们学习工具的使用时，重点关注的是如何使用工具干活，而不是工具是如何制造出来的。例如，我们学习开挖掘机时，重要的是学习如何控制挖掘机，而不是挖掘机的制造原理。同理，这里告诫各位童鞋，大家在学习 Java 过程中，首先要做的是能够熟练使用 Java 语言编写程序。只有熟练掌握了使用 Java 开发程序的情况下，再去研究 Java 内部原理。

计算机作为一种工具，它可以帮助我们减轻脑力劳动。例如，手机、智能手表、运动手环等都可以看做是计算机，如图 1-2 所示。



图 1-2 计算机设备

那你知道计算机是如何工作的吗？既然计算机可以帮助我们减轻脑力劳动，那么我们先思考一下人类脑力劳动的过程。如果观察人类的思考过程，可以分为 4 个过程，分别是：

- (1) 信息输入 (input)
- (2) 信息存储 (storage)
- (3) 信息处理 (processing)
- (4) 信息输出 (output)

为了大家更好理解上述 4 个过程，我们使用“挨打”的例子来形象描述。假设你走在路上，突然有个陌生人在你后背打了你一拳，此时，人类的思考过程可能使用表 1-1 描述。

表 1-1 “挨打”的思考过程

信息输入	被打疼了
信息存储	记住陌生人的长相
信息处理	根据陌生人的情况，判断妥协还是反击？
信息输出	妥协

如果某个工具同时满足了信息输入、信息存储、信息处理以及信息输出这 4 个条件，那么这个设备基本就可以看做是一个计算机。这里，我们先来看一下计算机信息的输入和输出。

计算机的信息输入，是由输入设备完成的，例如，键盘、鼠标、麦克风、摄像头、运动手环等设备，这些设备都可以搜集数据，被称为输入设备，如图 1-3 所示。



图 1-3 输入设备

同理，数据输出需要用到输出设备，例如，显示器、VR 眼镜、手机屏幕、打印机、雕刻机或者是可以运动控制的机器人，这些都可以理解为输出设备，如图 1-4 所示。



图 1-4 输出设备

至此，我们就介绍了计算机的输入和输出，关于计算机的信息存储和处理，我们将在后面的小节介绍。

1.2 计算机体系结构

在短短不到一百年的时间，我们的社会和生活发生了翻天覆地的变化，计算机结构也发生了巨大的变化。从早期的机械机构，到数码管结构，到现在的微电子大规模集成电路结构，计算机的大小从之前重达几百吨，且需要占据很大面积，发展到现在指甲盖大小甚至更小。

时代在变迁，工艺在发展，但计算机的运算原理始终没有变化，一直都是使用二进制表示输入输出的数据，通过特殊的电路结构完成数据的计算。

计算机芯片产业的竞争是相当激烈的，目前，美国的芯片已经进入 7 纳米阶段，而我们国家的芯片还停留在 14 纳米阶段，世界上各个国家都在追求把芯片电路做的越来越小，这是为什么呢？

电子在电路板上是以光速传播的，如果我们能把电路的长度缩减为原来的二分之一，那计算机的速度就会加快一倍，同时计算机的功耗也会减少原来的二分之一，这也就是为



什么各个国家都在努力发展更小半导体工艺的原因之一。

最后，我们通过一张图总结一下计算机的体系结构，如图 1-5 所示。

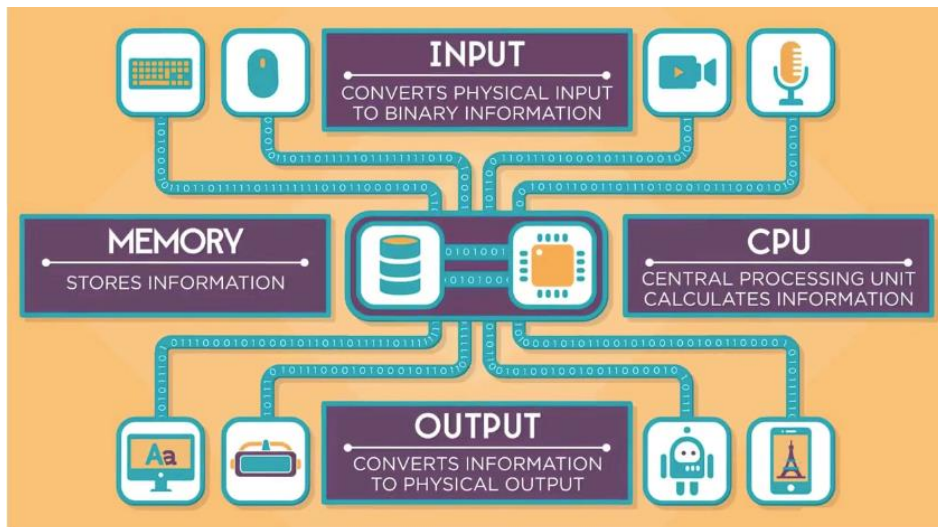


图 1-5 计算机的体系结构

在图 1-5 中，计算机通过键盘、鼠标、摄像机、麦克风等外部设备获取输入(INPUT)，将信息转为二进制数据。内存(MEMORY)负责存储二进制数据，CPU 会从内存中读取数据并进行数据运算，运算后的结果仍然是二进制的。输出设备 (OUTPUT) 负责将运算后的二进制数据转换为人类能理解的物理世界的输出。

1.3 认识进制

1.3.1 二进制的出现

前面我们讲过，计算机内部数据是使用 0 和 1 表示的，0 和 1 在计算机语言中是二进制，所有的数据都可以二进制表示。

之所以使用二进制表示，是因为计算机的内部是由逻辑电路组成的，逻辑电路通常只有两个状态，开关的接通与断开，这两种状态分别使用“1”和“0”表示。

- 如果电路开关断开，则电路不通电，使用 0 表示。
- 如果电路开关打开，则电路通电，使用 1 表示。

我们按照这个思路进行扩展，研究一下电路的数量和表示状态之间的关系，如表 1-2 所示。

表 1-2 电路数量和状态的关系

电路数量	电路状态	状态数量
1 条电路	0 1	2
2 条电路	00 01 10 11	4
3 条电路	000 001 010 011 100 101 110 111	8



4 条电路	0000	0001	0010	0011	0100	0101	0110	0111
	1000	1001	1010	1011	1100	1101	1110	1111

16

不难发现，不同数量的电路，其状态数量是一个排列组合关系，也就是说，如果有 N 条电路，每条电路的状态有 2 种，则 N 条电路共有 2^N 种状态。

1.3.2 二进制计数法

为了大家明白二进制的计数方法，下面，我们将二进制和十进制结合起来进行理解。我们先看一下什么是十进制与二进制。

(1) 十进制：十进制的基本符号是 0~9 这十个数字，即所有的数字都要使用这 10 个基本符号表示。十进制是逢十进一，同一个符号在不同位置上所表示的数值是不同的，如图 1-6 所示。

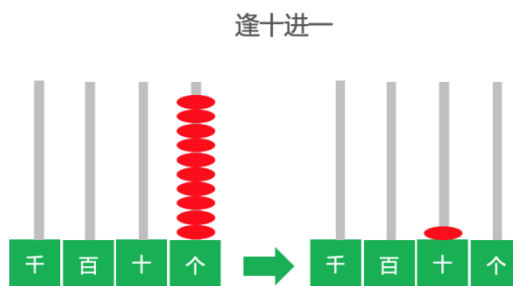


图 1-6 逢十进一

(2) 二进制：二进制的基本符号只有 0 和 1 这两个数字，二进制是逢二进一。同一个符号在不同位置上所表示的数值也是不同的。

以 10 为例，十进制和二进制表示的数字是不同的。

十进制： $10 = 1 \times 10^1 + 0 \times 10^0$ 对应的值就是阿拉伯数字 10

二进制： $10 = 1 \times 2^1 + 0 \times 2^0$ 对应的值就是阿拉伯数字 2

下面再看一个例子，加深对二进制和十进制的理解。

十进制： $1983 = 1 \times 10^3 + 9 \times 10^2 + 8 \times 10^1 + 3 \times 10^0$ 对应的值就是阿拉伯数字 1983

二进制： $1011 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$ 对应的值就是阿拉伯数字 11

了解了十进制和二进制，那么我们就可以让这两种进制建立一些隐射关系。

比如现在有 8 条电路，也就是说，这个电路是由 8 个二进制数字组成的，每个二进制数字（称为位，英文叫 bit）不是 0 就是 1。

8 条电路，共有 8 个 bit，可以表示 256 (2^8) 种不同的组合，如果映射为十进制，表示的数字是 0~255。

同理，32 条电路，共有 32 个 bit，可以表示 4294967296 (2^{32}) 种不同的组合，如果映射为十进制的话，表示的数字是 0~4294967295。

由此可以看出，只有 bit 足够多，我们使用电路可以表示任意的数字。



1.3.3 十六进制计数法

为了方便人类观察和记忆，引入了十六进制。十六进制使用 4 个二进制数据，表示一个十六进制数据。接下来，通过一张表对比看一下二进制和十六进制的表示方式，如表 1-3 所示。

表 1-3 十六进制与二进制的表示方式

十六进制	二进制
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

从表 1-3 可以看出，相比二进制来说，十六进制表示的数据更容易观察记忆，例如，我们计算机的物理地址，采用的就是十六进制表示，如图 1-7 所示。

```
选择管理员: 命令提示符
Microsoft Windows [版本 10.0.19043.1083]
(c) Microsoft Corporation。保留所有权利。

C:\Users\Administrator>getmac /c
错误: 无效参数/选项 - '/c'。
键入 "GETMAC /?" 以了解用法。

C:\Users\Administrator>getmac

物理地址          传输名称
=====
54-E1-AD-06-1B-88 \Device\NPF{...}
06-28-F8-BB-3F-A5 媒体已断开连接
00-28-F8-BB-3F-A9 媒体已断开连接

C:\Users\Administrator>
```

图 1-7 计算机的物理地址



1.4 常见数据的二进制表示

计算机中的信息分为文本、图片和声音，这些信息如何使用计算机的高低电压表示呢？其实，计算机中无论是什么样的数据，都是使用二进制表示的。接下来，我们分别介绍文本、图片以及声音的二进制表示方式。

1.4.1 文本的表示

文本信息的表示，离不开码表。例如，我们要存储一个字符，计算机会将字符以二进制的形式存储在硬盘或者内存中。当我们需要从硬盘或者内存中获取字符的时候，发现二进制又会变成字符，这其实就是码表存在的意义。

码表其实就是一个字符和其对应的二进制相互映射的一张表，例如，表 1-4 就可以看做是一张码表，该码表规定了字符和二进制的映射关系。

表 1-4 码表示例

文本字符	十进制	二进制
a	1	00001
b	2	00010
c	3	00100
d	4	00101
e	5	00110
f	6	00111

世界上有很多种类类似上述形式的码表，用来表示不同形式的信息，通过查询码表可以知道不同二进制代表的含义。

ASCII 是很常见的一个码表，它是美国信息交换标准代码。ASCII 码表使用 7 位二进制数（剩下的 1 位二进制是 0）来表示所有的大写字母、小写字母、数字 0~9，以及美式英语中使用的特殊控制字符。ASCII 码表如图 1-8 所示。

ASCII可显示字符

二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形
0010 0000	32	20	(空格) (sp)	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	\$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o

图 1-8 ASCII 码表

这里再次强调的是，ASCII 码表只能用来字符，不能表示中文汉字。中文汉字也有对应的码表，常见码表如下：

- GB2312 编码：1981 年 5 月 1 日发布的简体中文汉字编码国家标准，收录 7445 个图形字符，其中包括 6763 个汉字。
- BIG5 编码：台湾地区繁体中文标准字符集，共收录 13053 个中文字，1984 年实施。
- GBK 编码：2000 年 3 月 17 日发布，收录 21003 个汉字，包含国家标准 GB13000-1 中的全部中日韩汉字，和 BIG5 编码中的所有汉字。
- Unicode 编码：国际标准字符集，它将世界各种语言的每个字符定义一个唯一的编码，以满足跨语言、跨平台的文本信息转换。

1.4.2 图片的表示

一般情况下，图像分为黑白图像、灰度图像和彩色图像。无论哪种图像，它们都是以像素的形式存储在计算机中的，像素在计算机中是以二进制表示的。那么，不同类型的图形，是如何以二进制形式存储在电脑中的呢？

1. 黑白图像

黑白图像只有黑、白两种颜色，一个像素可以使用二进制的 0 或者 1 表示，其中，0 表示黑色，1 表示白色。图 1-9 展示的是黑白笑脸图像对应的二进制形式。

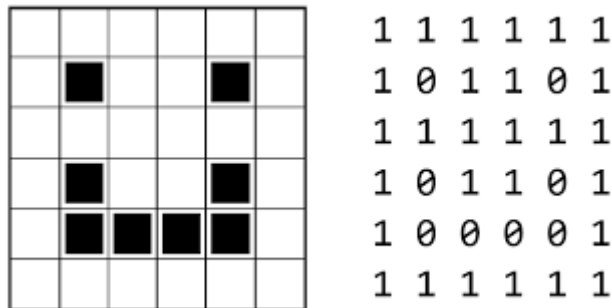


图 1-9 黑白笑脸对应的二进制表示方式

小提示：每个小格子可以理解为一个像素，每个像素都是一个二进制数。

2. 灰度图像

灰度图像，一个像素一般使用 8 个二进制位表示。因为灰度图像不再是黑、白两种颜色，所以不再使用 0 表示黑，1 表示白，而是使用 0 表示黑，使用 255 表示纯白，使用 128 表示黑白之间。图 1-10 展示的灰色数字 8 对应的二进制形式。

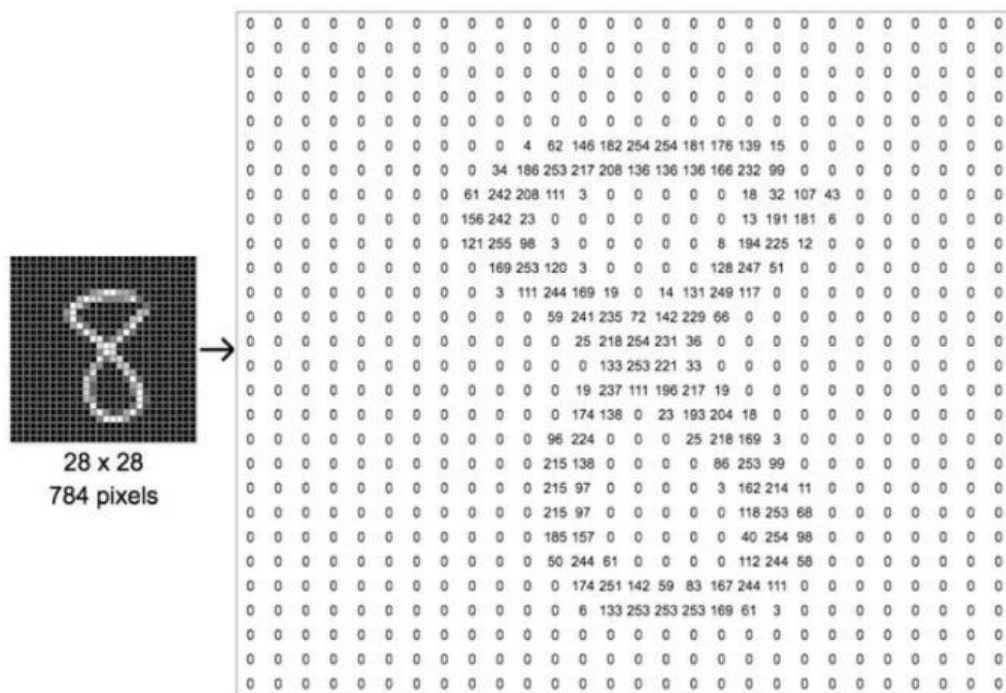


图 1-10 灰色数字 8 对应的二进制形式

3. 彩色图像

彩色图像在计算机的存储方式与前面两种图像是类似的，只不过彩色图像的每个像素使用的是更多位的二进制数表示的，可以表示的颜色也是更加丰富的。例如，图 1-11 美女嘴部图像放大后的像素，都是可以使用二进制数据存储的。

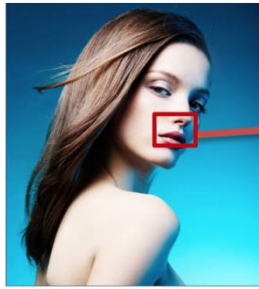


图 1-11 美女嘴部放大图像

1.4.3 声音的表示

声音是物体震动所产生的声波，我们可以将声波的波形转为二进制，从而实现在计算机中存储声音。图 1-12 描述的是声波及声波的二进制表示方式。

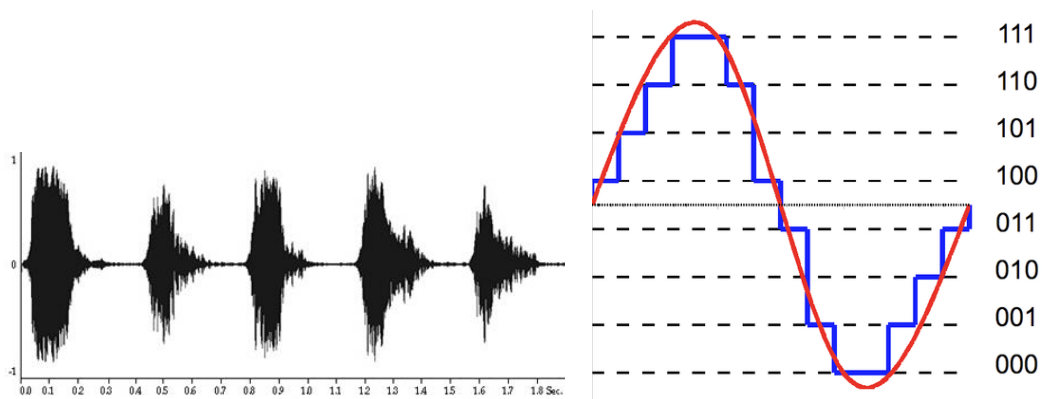


图 1-12 声波及声波的二进制表示方式

1.5 信息处理的原理

了解了计算机的输入、输出以及存储后，接下来，我们来看一下计算机是如何处理信息的。实际上，计算机处理信息的原理比较简单，它是通过电路输入的状态进行处理的。

下面，我们构建一个简单的电路场景。现在有一个由 1 个电池、2 个开关、1 个电灯连接而成的电路，如图 1-13 所示。

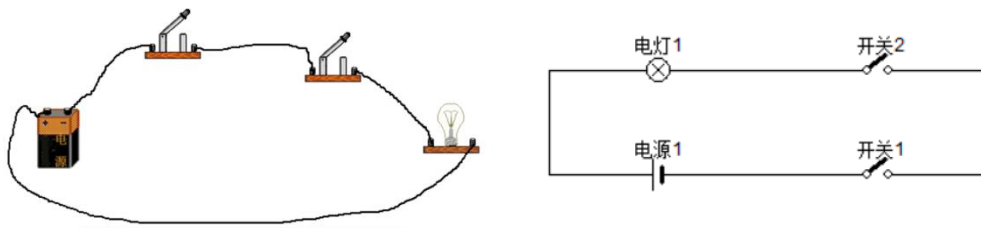


图 1-13 电路图（1）

现在我们对 1-13 的场景做一些约定，具体如下：

(1) 开关：如果开关打开，值为 1，如果开关关闭，值为 0。

(2) 电灯：如果电灯亮，值为 1，如果电灯灭，值为 0。

有了这样的电路之后呢，相当于我们构建出了一个非常简单的计算机的与门，如表 1-5 所示。

表 1-5 电路图 1 构建的计算机与门

输入 1（开关 1）	输入 2（开关 2）	结果（灯的状态）
0	0	0
1	0	0
0	1	0
1	1	1

从表 1-5 可以看出，只有输入 1 和输入 2 的值都是 1 的情况下，结果才为 1，说明只有开关 1 和开关 2 都打开的情况下，电灯才是亮的。

扩展一下，我们可以构造出另外一种电路，如图 1-14 所示。

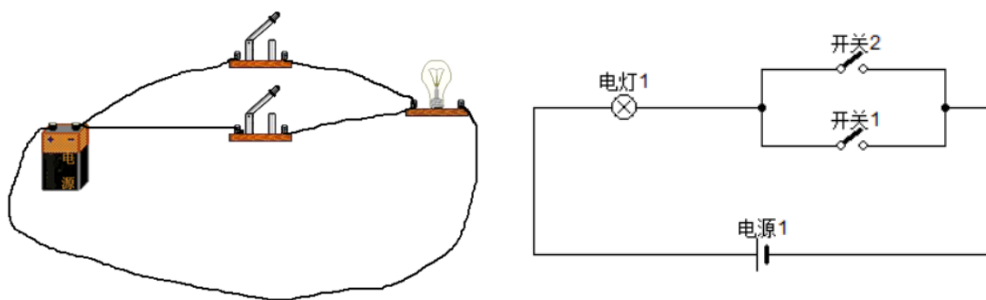


图 1-14 电路图（2）

图 1-14 场景可以构建出一个计算机的或门，如表 1-6 所示。

表 1-6 电路图 2 构建的计算机或门

输入 1（开关 1）	输入 2（开关 2）	结果（灯的状态）
0	0	0
1	0	1
0	1	1
1	1	1

从表 1-6 可以看出，只有输入 1 和输入 2 中有一个值为 1，结果就是 1。说明只要开关 1 或者开关 2，有一个是打开的，电灯就会亮。

之所以举上面两个例子，是想告诉同学们，计算机中实际上是通过二极管、三极管等电子元件组合成图 1-15 中列举的七种基础数字线路。



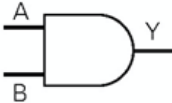
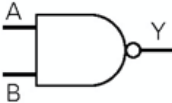
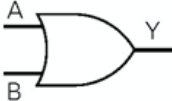
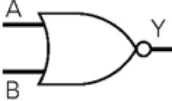
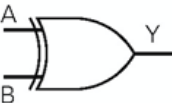
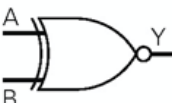
逻辑	真值表	MIL 逻辑符号	逻辑	真值表	MIL 逻辑符号																														
与	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	0	1	0	0	1	1	1		与非	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	1	1	0	1	1	1	0	
A	B	Y																																	
0	0	0																																	
0	1	0																																	
1	0	0																																	
1	1	1																																	
A	B	Y																																	
0	0	1																																	
0	1	1																																	
1	0	1																																	
1	1	0																																	
或	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	1		或非	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	0	
A	B	Y																																	
0	0	0																																	
0	1	1																																	
1	0	1																																	
1	1	1																																	
A	B	Y																																	
0	0	1																																	
0	1	0																																	
1	0	0																																	
1	1	0																																	
异或	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	0		异或非	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	1	
A	B	Y																																	
0	0	0																																	
0	1	1																																	
1	0	1																																	
1	1	0																																	
A	B	Y																																	
0	0	1																																	
0	1	0																																	
1	0	0																																	
1	1	1																																	
非	<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Y	0	1	1	0																												
A	Y																																		
0	1																																		
1	0																																		

图 1-15 七种基础数字线路

在图 1-15 中，共列举了与、或、异或、非、与非、或非、异或非这七种逻辑，每种逻辑中的符号 A、B 可以理解为两个输入，Y 可以理解为输出。

如果将图 1-15 中列举的数字电路再进行一定的组合，就可以组合成一种可以进行加减乘除等复杂运算的数字电路。图 1-16 展示的是全加器，一种使用电路实现两个二进制数相加并求出和的组合线路。

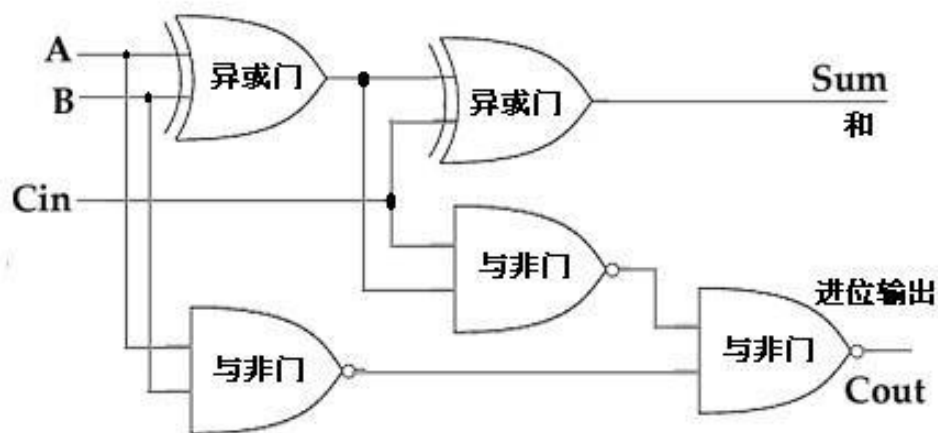


图 1-16 全加器

全加器对应的真值表，如图 1-17 所示。



$C(Carry_{in})$	A	B	$Carry(Carry_{out})$	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

图 1-17 全加器对应的真值表

关于图 1-17 中各列的含义具体如下：

- $C(Carry_{in})$ ：相邻低位的进位数
- A:表示输入的被加数
- B：表示输入的加数
- $C(Carry_{out})$ ：表示向相邻高位进位
- Sum：表示输出的结果和

综上所述，我们可以看出，计算机信息处理的原理就是基于数字电路逻辑实现的。由于关于数字电路的内容不是我们课程的重点内容，所以我们这里不做过多介绍，感兴趣的同学可以自行学习相关课程。

第 2 章 初识 Java

学习目标

- ◆ 了解什么是机器语言，能够说出机器语言编程的过程
- ◆ 了解高级编程语言，能够说出高级编程语言的类型
- ◆ 了解 Java 语言的地位以及发展历史
- ◆ 熟悉 Java 开发环境的搭建，能够独立安装 JDK 以及 IDEA
- ◆ 通过开发第一个 Java 程序，熟悉 IDEA 开发 Java 程序的流程



前面我们了解了计算机的体系结构，接下来要做的事情就是如何让计算机按照我们的要求执行操作。接下来，本章将从机器语言开始，介绍最初计算机编程的模式，然后过渡到高级语言的出现，进而开启 Java 的学习之旅。

2.1 机器语言入门

程序员的主要工作是编程，那什么是编程呢？简单来说，编程就是通过控制指令，告诉机器怎么做。但是，编程并不是直接写代码这么简单，编程是一个自上而下分析解决问题的过程，如图 2-1 所示。



图 2-1 编程的过程

机器语言是最早的编程语言。下面我们看一下，如何通过机器语言控制呼吸灯。表 2-1 列出了控制呼吸灯的部分机器指令和机器操作。

表 2-1 控制呼吸灯的部分机器指令和机器操作

机器指令	机器操作
00000000	停止程序
00000001	完全打开灯泡
00000010	完全关闭灯泡
00000100	把灯泡调暗 20%
00001000	把灯泡调亮 20%
10000001	跳转到位置 1 执行
10000010	跳转到位置 2 执行
.....	

在表 2-1 中，左侧的机器指令是存储在计算机内存中，CPU 可以加载这些机器指令执行相应的操作，并且一次只能加载一条指令。

现在我们要编程控制一个呼吸灯，呼吸灯的业务逻辑按照下列方式呈现。

呼吸灯打开→呼吸灯慢慢变暗→呼吸灯慢慢变亮→呼吸灯慢慢变暗→呼吸灯慢慢变亮

按照编程的过程，我们需要将上述业务逻辑拆分为若干步骤，例如，呼吸灯慢慢变暗可以分解为 3 次调暗 20%，呼吸灯慢慢变亮可以分解为 3 次调亮 20%，这样的话，问题分解后的步骤如下：

呼吸灯打开→调暗 20%→调暗 20%→调暗 20%→调亮 20%→调亮 20%→调亮 20%

分解后的每个步骤对应的机器指令如下：

00000001 00000100 00000100 00000100 00001000 00001000 00001000 10000010

上述最后一个指令 10000010 表示跳转到位置 2 执行，也就是跳到第 2 个机器指令处，

重新执行后续指令，这样整个过程就会循环往复执行下去。

理解了机器语言编程之后，我们再去编写代码，只需要买一个图 2-2 所示的键盘就可以了。但是如果使用机器语言编写代码会非常繁琐，整个编程方法就是写 0 和 1。

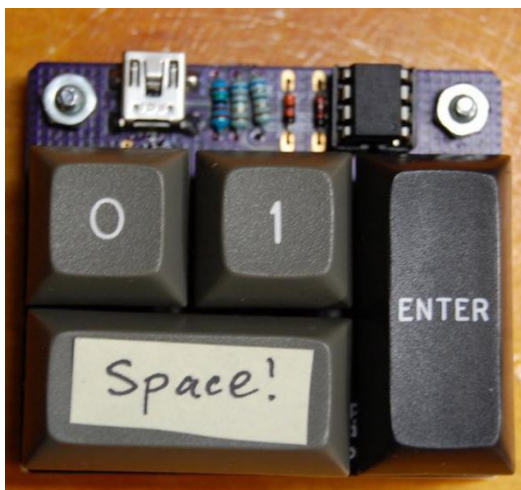


图 2-2 数字只有 0 和 1 的键盘

实际上，上个世纪五六十年代，计算机编程并不是靠写 0 和 1，而是依靠穿孔纸带。编写好的程序被通过打孔的方式记录在纸袋上，有孔的是 1，无孔就是 0。程序需要运行的时候，就通过机器从纸带上读取。图 2-3 就是一种穿孔纸带。



图 2-3 穿孔纸带

通过穿孔纸带这种方式编程，对于人类来说简直就是噩梦，因为如果我们不去执行程序，单纯看纸带是根本无法知晓其含义的。

2.2 高级编程语言

为了解决机器语言存在的问题，我们人类发明了高级编程语言。根据高级编程语言转为机器语言方式的不同，高级编程语言可以分为三类，具体如下：

- 编译型
- 解释型
- 混合型

为了帮助大家理解这三种高级编程语言，下面我们分别以典型的语言进行介绍。



(1) 编译型（以 C/C++ 为例）

使用编译型语言编写的源代码，需要使用某种处理器特定的编译器，将源代码整体翻译成二进制机器指令。以 C/C++ 为例，其源代码执行过程如图 2-4 所示。

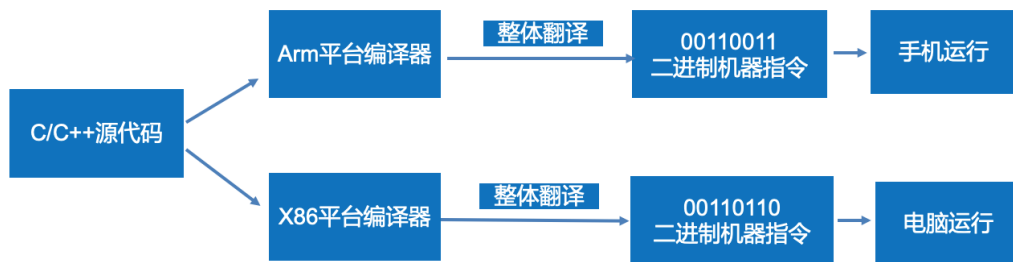


图 2-4 C/C++源代码的执行过程

在图 2-4 中，如果我们编写的 C/C++ 源代码，使用的是 Arm 平台的编译器，那么会被整体翻译成 Arm 平台的机器指令，最终在手机或者支持 Arm 的平板上运行。如果使用 X86 平台的编译器，那么 C/C++ 源代码会被翻译成另外一种形式的二进制机器指令，最终可以在支持 X86 的电脑上运行。

(2) 解释型（以 Python 为例）

解释型语言在程序运行时才会翻译成机器语言，它是按行翻译的，每执行一行都要翻译一次。解释型语言在不同的平台上执行，需要不同平台的解释器。Python 是一种典型的解释型语言，其源代码执行过程如图 2-5 所示。

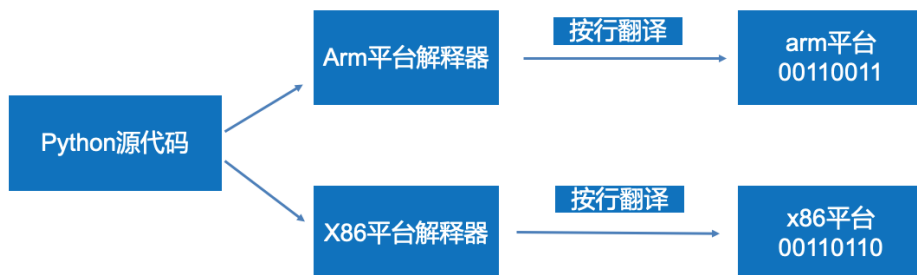


图 2-5 Python 源代码执行过程

(3) 混合型（以 Java 为例）

Java 语言属于混合型，其源代码执行过程如图 2-6 所示。

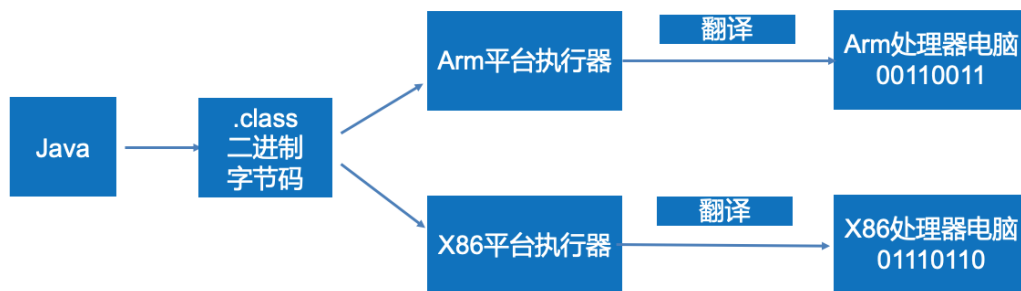


图 2-6 Java 源代码执行过程

在图 2-6 中，Java 源代码首先会被编译成 .class 的二进制字节码（该字节码可以看做加密后的源代码），这个字节码可以“一次编译，到处运行”。

Java 针对不同平台的处理器，开发了不同的执行器，这种执行器统称为 Java 虚拟机。Java 虚拟机主要负责将 .class 二进制字节码文件翻译成机器语言，从而最终交给底层操作系统执行。这样的话，既可以保证程序执行效果，又可以保证 Java 程序的跨平台性。

综上所述，如果对编译型、解释性以及混合型做对比，三者各有优缺点，具体如表 2-2 所示。

表 2-2 三种高级语言的比较

三种高级语言	优点	缺点
编译型语言	整体翻译，运行速度快	可移植性差
解释性语言	开发调试方便	运行效率低 不利于保护程序员的开发成果
混合型语言	半编译、半解释	

2.3 Java 语言概述

2.3.1 什么是 Java 语言

Java 是一门非常优秀的高级编程语言，在各大编程语言排行榜的前三位置上，你几乎都能找到 Java 的身影。接下来，我们以 TIOBE 世界语言排行榜为例，描述 Java 语言在世界编程语言的地位，如图 2-7 所示。

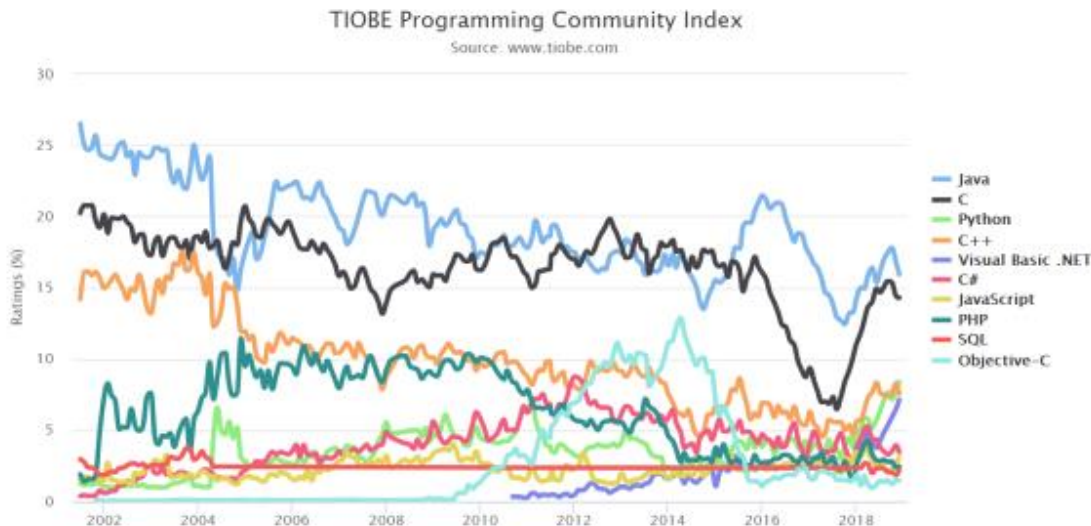


图 2-7 TIOBE 世界语言排行榜

从图 2-7 中可以看出，近 20 年来，Java 的地位一直是不可撼动的，它始终是主流的计算机语言。

除此之外，中国 Java 开发者的数量也是惊人的，在 2021 年 3 月统计的中国编程语言排行榜中，Java 语言的市场份额也是最大的，占据 27.3%，具体如图 2-8 所示。

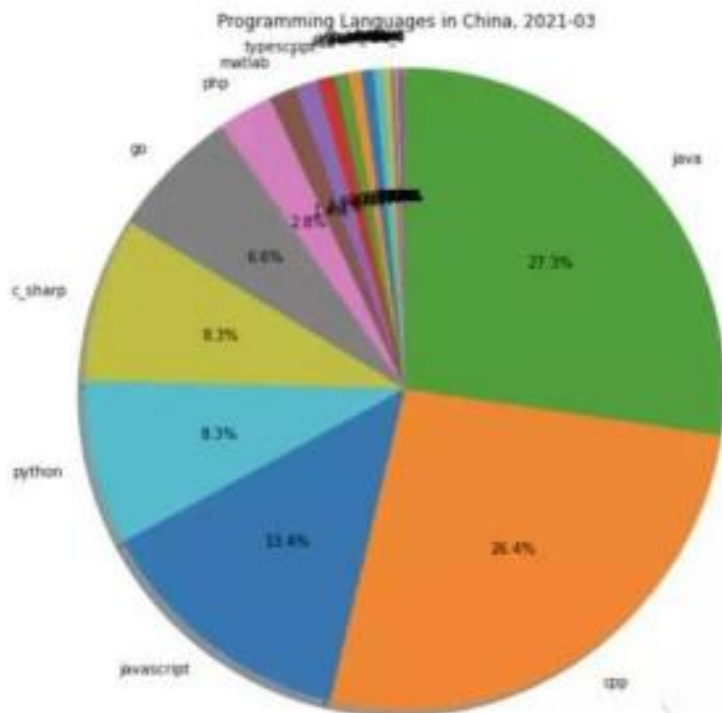


图 2-8 Java 语言的市场份额

Java 语言作为一门优秀的编程语言，它的应用是非常广泛的。下面列举一些使用 Java 语言开发的应用场景，具体如下：

- (1) 桌面应用开发。例如，编程使用的开发工具 IDEA、Clion、PyCharm，国内很多税务管理软件、办公系统软件等，都是使用 Java 语言开发的。因为 Java 语言规范性好、开发漏洞少，使用 Java 编写的应用健壮性会非常好。
- (2) 企业级应用开发。随着业务越来越庞大，企业级的应用开发要求越来越高，例如，春节购票时使用的 12306，可能面临几亿人抢票的场景，服务器每秒要承载的并发访问可能达上千万次。如果要开发这样的企业级应用，Java 是首选语言。
- (3) 移动应用开发。例如，Java 语言可以开发 Android 应用程序以及一些移动端或者嵌入式的应用。
- (4) 科学技术。例如，科学技术软件 matlab 就是使用 Java 语言开发的。
- (5) 大数据开发。例如，Hadoop 框架是使用 Java 语言开发的。
- (6) 游戏开发。例如，MineCraft（我的世界）是一款 Java 开发的游戏。

2.3.2 Java 语言的发展历史

Java 语言是詹姆斯·高斯林发明的，Java 的名字来自于一种咖啡的品种名称，所以 Java 语言的 Logo 是一杯热气腾腾的咖啡。詹姆斯·高斯林等人于 1990 年初开发 Java 语言

的雏形，最初被命名为 Oak。随着 1990 年代互联网的发展，Sun 公司看见 Oak 在互联网上应用的前景，于是改进了 Oak，并于 1995 年 5 月以 Java 的名称正式发布。下面具体讲解一下 Java 语言的发展史。

- 1995 年 5 月 23 日，Java 语言诞生。
- 1997 年，Java 发布了第一个版本，JDK1.1。
- 1998 年 12 月 8 日，JAVA2 企业平台 J2EE 发布。
- 1999 年 6 月，SUN 公司发布 Java 的三个版本：标准版（J2SE）、企业版（J2EE）和微型版（J2ME）。
- 2001 年 9 月 24 日，J2EE 1.3 发布。
- 2002 年 2 月 26 日，J2SE 1.4 发布，自此 Java 的计算能力有了大幅提升。
- 2004 年 9 月 30 日，J2SE 1.5 的发布成为 Java 语言发展史上的又一里程碑。为了表示该版本的重要性，J2SE 1.5 更名为 Java SE 5.0。
- 2005 年 6 月，JavaOne 大会召开，SUN 公司公开 Java SE 6。此时，Java 的各种版本进行了更名，取消了名称中的数字“2”，J2EE 更名为 Java EE，J2SE 更名为 Java SE，J2ME 更名为 Java ME。
- 2009 年 12 月，SUN 公司发布 Java EE 6，同年被 Oracle 公司收购，从此 Java 正式归 Oracle 公司所有。
- 2011 年 7 月 28 日，Oracle 公司发布 Java SE 7。
- 2014 年 3 月 18 日，Oracle 公司发布 Java SE 8（目前主流版本）。
- 2017 年 9 月 21 日，Oracle 公司发布 Java SE 9。
- 2018 年 3 月，Oracle 公司发布 Java SE10。
- 2018 年 9 月，Oracle 公司发布 Java SE11。
- 2019 年 3 月，Oracle 公司发布 Java SE12。
- 2019 年 9 月，Oracle 公司发布 Java SE13。

从 2019 年开始，Java 每间隔半年就发布一个版本，截止 2021 年 4 月，Java 最新的版本是 Java SE16。发布的每个版本都会有很多新特性，这些特性虽然没有像 JDK8 那样做重大改变，但是也会提供一些大家期待已久的特性，例如，Java SE12 更新了一些字符串和集合相关的操作，Java SE13 做了一些关于空指针异常的处理、Lambda 表达式的优化等。如果大家想了解每个 Java SE 版本的新特性，可以查阅网站自行学习。

2.4 认识 JDK、JRE 和 JVM

Java 开发中有 3 个比较重要的概念：JDK，JRE 和 JVM，具体介绍如下：

1. JVM

JVM 的英文全称是 Java Virtual Machine，翻译成中文叫 Java 虚拟机。JVM 是一个虚构出来的计算机，有自己的 CPU 指令集和内存，它规定了程序如何被装载到内存，CPU 如何去理解和执行这些指令。



JVM 的本质是一个规范，它提供了字节码运行时环境，可以加载、验证和执行字节码。JVM 有很多种不同的实现方式，在后续的课程当中，会学习 JVM 规范的相关知识，学习 JVM 的规范之后，大家也可以实现自己的 JVM。

需要注意的是，JVM 和 JAVA 语言没有必然的联系，JVM 可以运行任意编程语言开发的程序，只要该程序能编译出符合 JVM 规范的字节码，例如，使用 Kotlin、Scala、Groovy、C++、Node.js 开发的程序，如果编译出的字节码符合 JVM 的规范，也可以在 JVM 中运行。

JVM 规范官网：<https://docs.oracle.com/javase/specs/index.html>

2. JRE

JRE 的英文全称是 Java Runtime Environment，翻译成中文叫 Java 运行时环境。JRE 是 JVM 规范的真正实现，JVM 在不同的平台下会以不同形式的文件存在，它不能直接运行，而是需要 JRE 环境中运行。JRE 为 Java 代码编译出的字节码提供了必要的运行时环境，可以把后缀名为.class 的字节码文件装载到内存，然后将装载的内容翻译成能在物理机器上执行的指令。

JRE 内部除了有一个 JVM，还提供了一些运行.class 字节码文件所需要的基础函数库，这些函数库都是由 Oracle 公司提供。

JRE 与平台相关，不同平台的 JRE 是不一样的，在不同平台下使用 JRE 就需要下载对应平台的 JRE，Java 代码的跨平台运行就是通过 JRE 实现的。

3. JDK

JDK 的英文全称是 Java Development Kit，翻译成中文叫 Java 开发环境，它是 Java 程序开发的软件工具集，由 JRE 和开发工具集组成。其中，JDK 中的开发工具集提供了 Java 程序员常用的一些工具，例如，Java 编译工具、监控和调试 JVM 的工具、Java 代码堆栈分析的工具等。

如果只需要运行编译好的 Java 程序，而不需要开发 Java 程序，则只需下载 JRE；如果需要开发 Java 程序，则需要下载 JDK。

值得一提的是，由于 JDK 中包含了 JRE，为了方便使用，Oracle 官网中，Java SE 9 及之后的版本都不再提供单独下载 JRE 的地址，即使用 Java SE 9 及之后的版本运行或者开发 Java 程序都需要安装 JDK。

2.5 Java 开发环境搭建

2.5.1 JDK 的下载

Oracle 公司提供了所有版本的 JDK 下载，建议大家通过 Oracle 官网进行 JDK 的下载。具体步骤如下：

(1) 访问地址 <https://www.oracle.com/java/technologies/javase-downloads.html> 进入 Oracle 官网 JDK 的下载页面，如图 2-9 所示。

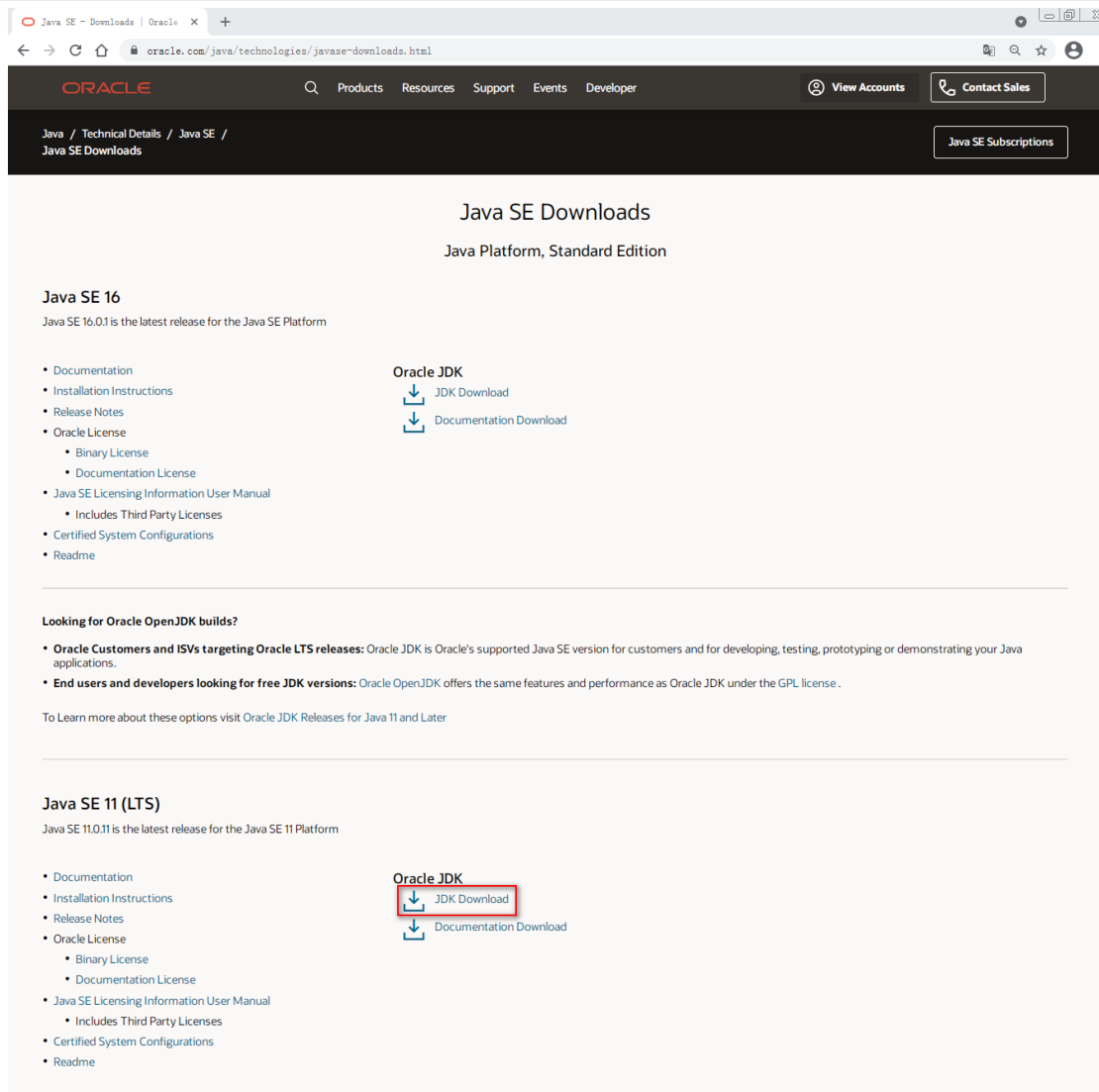


图 2-9 Oracle 官网 JDK 的下载页面

从图 2-9 可以看出，JDK 可以选择下载的版本有很多，例如，Java SE 16 的版本、Java SE 11 (LTS)的版本等，其中 Java SE 16 的版本是官方最新发布的版本。

这里，我们课程中采用的版本是 Java SE 11 (LTS)版本（LTS 是 Long Term Support 的简写）。之所以选择这个版本，是因为该版本是长期演进版，是 Oracle 公司长期支持的版本，与此同时，Java SE 11 (LTS)也是当前国内企业开发选择最多的版本。当然，关于 Java SE 15 和 Java SE 16 版本的新特性，我们也会在后续课程中讲解。

(2) 单击图 2-9 中 Java SE 11 (LTS)右侧的 JDK Download 超链接，进入该版本的下载页面，如图 2-10 所示。



Java SE Development Kit 11 Downloads

Thank you for downloading this release of the Java™ Platform, Standard Edition Development Kit (JDK™). The JDK is a development environment for building applications, and components using the Java programming language.

The JDK includes tools useful for developing and testing programs written in the Java programming language and running on the Java platform.

Important Oracle JDK License Update

The Oracle JDK License has changed for releases starting April 16, 2019.

The new [Oracle Technology Network License Agreement for Oracle Java SE](#) is substantially different from prior Oracle JDK licenses. The new license permits certain uses, such as personal use and development use, at no cost -- but other uses authorized under prior Oracle JDK licenses may no longer be available. Please review the terms carefully before downloading and using this product. An FAQ is available [here](#).

Commercial license and support is available with a low cost [Java SE Subscription](#).

Oracle also provides the latest OpenJDK release under the open source [GPL License](#) at [jdk.java.net](#).

See also:

Java Developer Newsletter: From your Oracle account, select **Subscriptions**, expand **Technology**, and subscribe to **Java**.

Java Developer Day hands-on workshops (free) and other events

Java Magazine

JDK 11.0.11 checksum

Java SE Development Kit 11.0.11

This software is licensed under the [Oracle Technology Network License Agreement for Oracle Java SE](#)

Product / File Description	File Size	Download
Linux ARM 64 Debian Package	145.61 MB	jdk-11.0.11_linux-aarch64_bin.deb
Linux ARM 64 RPM Package	152.16 MB	jdk-11.0.11_linux-aarch64_bin.rpm
Linux ARM 64 Compressed Archive	169.53 MB	jdk-11.0.11_linux-aarch64_bin.tar.gz
Linux x64 Debian Package	149.39 MB	jdk-11.0.11_linux-x64_bin.deb
Linux x64 RPM Package	156.07 MB	jdk-11.0.11_linux-x64_bin.rpm
Linux x64 Compressed Archive	173.46 MB	jdk-11.0.11_linux-x64_bin.tar.gz
macOS Installer	167.15 MB	jdk-11.0.11_osx-x64_bin.dmg
macOS Compressed Archive	167.67 MB	jdk-11.0.11_osx-x64_bin.tar.gz
Solaris SPARC Compressed Archive	184.51 MB	jdk-11.0.11_solaris-sparcv9_bin.tar.gz
Windows x64 Installer	152.05 MB	jdk-11.0.11_windows-x64_bin.exe
Windows x64 Compressed Archive	171.53 MB	jdk-11.0.11_windows-x64_bin.zip

图 2-10 Java SE Development Kit 11 下载页面

从图 2-10 可以看到，列表中有多个不同操作系统的 Java SE Development Kit 11.0.11 下载链接，可以根据自己的需求选择对应的版本进行下载。

小提示：

JDK 11 已经明确不支持 32 位的 Windows 操作系统，如果当前选择安装 JDK 的操作系统是 32 位的，建议先重装一个 64 位的 Windows 操作系统，再进行 JDK 11 的安装。

(3) 单击图 2-10 中右下角的 `jdk-11.0.11_windows-x64_bin.exe` 超链接，选择选择下载 Windows 系统对应的版本进行下载，此时，会弹出下载提示对话框，如图 2-11 所示。

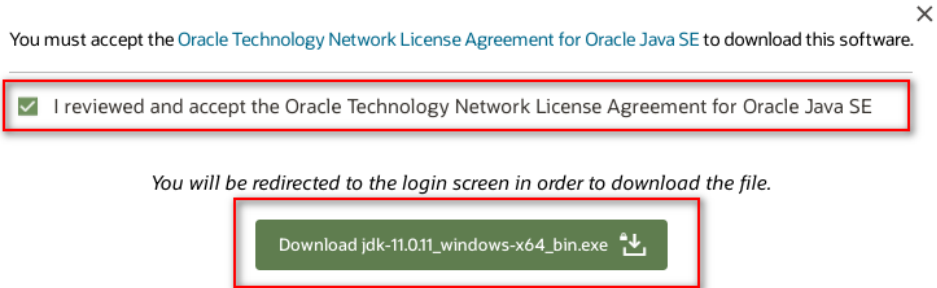


图 2-11 下载提示对话框

(4) 勾选图 2-11 中的单选框，表示接受 Oracle 的网络许可协议。勾选单选框后，对话框下端的下载按钮从禁用变化为可用状态，此时，可以单击该下载按钮继续 JDK 的下载。由于下载 JDK 需要先登陆 Oracle 的网站，此时单击下载按钮会进入 Oracle 的登录页面，如图 2-12 所示。

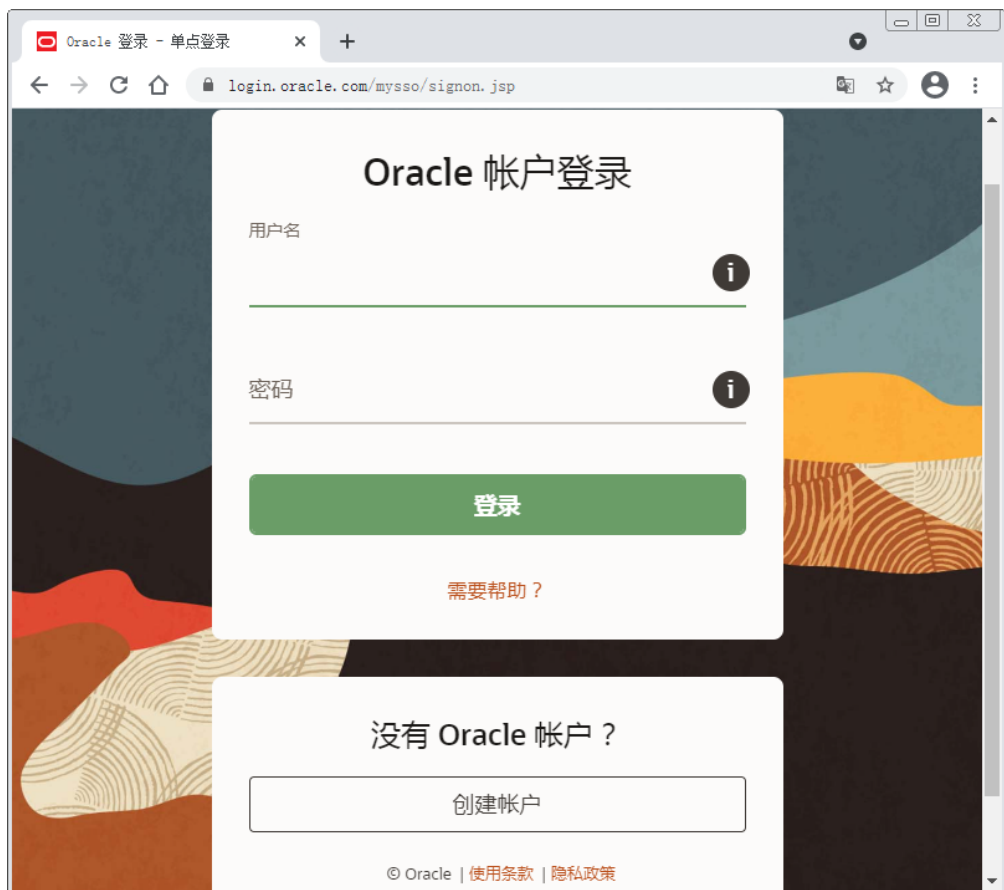


图 2-12 Oracle 的登录页面

如果没有 Oracle 的账号，可以单击图 2-12 下端的创建账户，先注册账户再使用账户进行登录。输入正确的账号和密码登录后，会自动继续 JDK 的下载。

至此，在 Oracle 官网下载 JDK 全部完成。



2.5.2 JDK 的安装

JDK 下载完成后，就可以开始进行 JDK 的安装，JDK 的安装比较简单，根据安装向导一直单击下一步即可，具体步骤如下：

(1) 双击从 Oracle 官网下载的安装文件 “jdk-11.0.11_windows-x64_bin.exe”，进入 JDK 安装界面，如图 2-13 所示。



图 2-13 JDK 11 的安装界面

(2) 单击图 2-13 中的“下一步”按钮进入 JDK 的定制安装界面，如图 2-14 所示。



图 2-14 JDK 的定制安装界面

在图 2-14 所示界面的左侧有 2 个功能模块可供选择，单击这 2 个功能模块任意一个，在界面右侧的功能说明区域会显示该模块所包含的功能说明。在界面右侧还有一个

“更改”按钮，如果想要更改 JDK 的安装目录，可以单击该按钮进行更改；如果不想更改 JDK 的安装目录，可以使用默认的安装路径。

(3) 在这里选择不更改 JDK 的安装目录，单击“下一步”按钮进行 JDK 的安装，安装完毕会进入安装完成界面，如图 2-15 所示。



图 2-15 JDK 安装完成界面

单击图 2-15 中的“关闭”按钮，会关闭当前界面，此时已经完成 JDK 11 的安装。

JDK 安装之后，可以对其安装的结果进行验证。单击系统中的“开始”按钮，在系统的搜索栏中输入命令 cmd，会显示命令提示符的图标，单击命令提示符的图标会打开 Windows 命令处理程序，如图 2-16 所示。



图 2-16 Windows 命令处理程序

此时，在命令行窗口中输入 java 并回车执行，如果显示图 2-17 所示的内容，说明 JDK 安装成功了。



```
管理员: C:\Windows\system32\cmd.exe
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation. 保留所有权利。

C:\Users\tk>java
用法: java [options] <主类> [args...]
      (执行类)
或 java [options] -jar <jar 文件> [args...]
      (执行 jar 文件)
或 java [options] -m <模块> [<主类>] [args...]
      java [options] --module <模块> [<主类>] [args...]
      (执行模块中的主类)
或 java [options] <源文件> [args]
      (执行单个源文件程序)

将主类、源文件、-jar <jar 文件>、-m 或
--module <模块><主类> 后的参数作为参数
传递到主类。

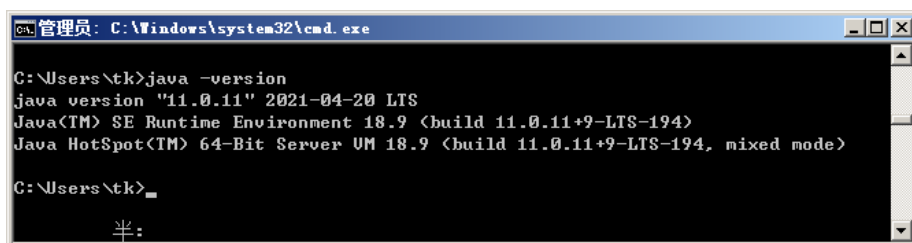
其中，选项包括:

-cp <目录和 zip/jar 文件的类搜索路径>
-classpath <目录和 zip/jar 文件的类搜索路径>
--class-path <目录和 zip/jar 文件的类搜索路径>
使用 ; 分隔的，用于搜索类文件的目录，JAR 档案

半:
```

图 2-17 java 命令的显示内容

如果想要查看当前安装的版本，可以在命令中输入 `java -version` 并回车执行，如图 2-18 所示。



```
管理员: C:\Windows\system32\cmd.exe

C:\Users\tk>java -version
java version "11.0.11" 2021-04-20 LTS
Java(TM) SE Runtime Environment 18.9 <build 11.0.11+9-LTS-194>
Java HotSpot(TM) 64-Bit Server VM 18.9 <build 11.0.11+9-LTS-194, mixed mode>

C:\Users\tk>_

半:
```

图 2-18 查看安装的版本

2.5.3 IDEA 的安装

理论上编写 Java 代码有一个记事本就够了，但是在实际开发中，开发人员会希望这个记事本拥有更多的功能和控件，例如下列内容。

- (1) 能对编写的代码添加一些不同的颜色，以便于进行阅读。
- (2) 能够自动提示一些关键代码；能够分步运行编写的程序，以方便查看程序的每一步执行流程，更便捷的发现程序的逻辑错误。
- (3) 拥有更加漂亮的控件，如按钮、面板、菜单和窗口，并且这些控件整齐排列，方便操作。

上述的这些功能和控件的需求 IDE（全称 Integrated Development Environment，集成开发环境）工具都已经实现好了，我们可以把 IDE 当做强大的文本编辑器、文档管理器、代码调试器、运行管理器或项目构建器。

IDEA 是一款 JAVA 开发的 IDE 工具，它由 JetBrains 公司开发，被认为是 Java 业界公认为最好的 IDE。接下来，分步骤讲解 IDEA 的下载安装，具体如下：

(1) 访问地址 <https://www.jetbrains.com/idea/>，进入 JetBrains 官网中 IDEA 的首页面，如图 2-19 所示。

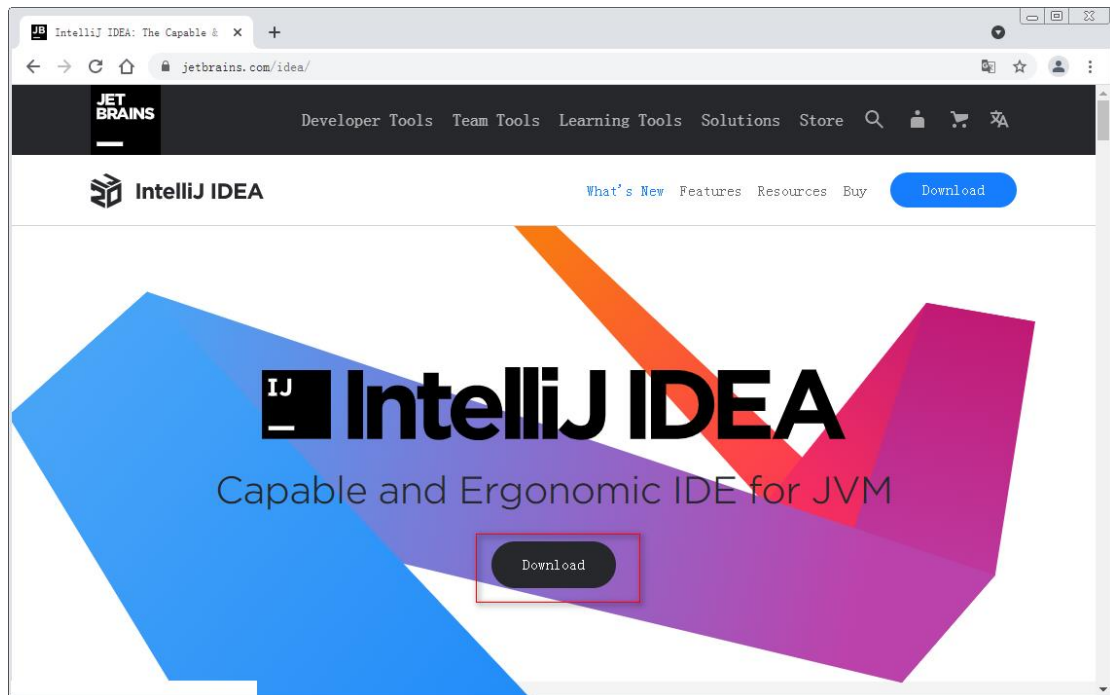


图 2-19 JetBrains 官网中 IDEA 的首页面

从图 2-19 可以看到，官方宣称 IDEA 是适用于 JVM 的功能强大且符合人体工程学的 IDE，大体意思可以理解为 IDEA 是一款可以让使用者感受到更舒适、更高效率与更少的压力的 IDE。

(2) 单击图 2-19 中页面中间的 Download 按钮，进入到 IDEA 的下载页面，如图 2-20 所示。

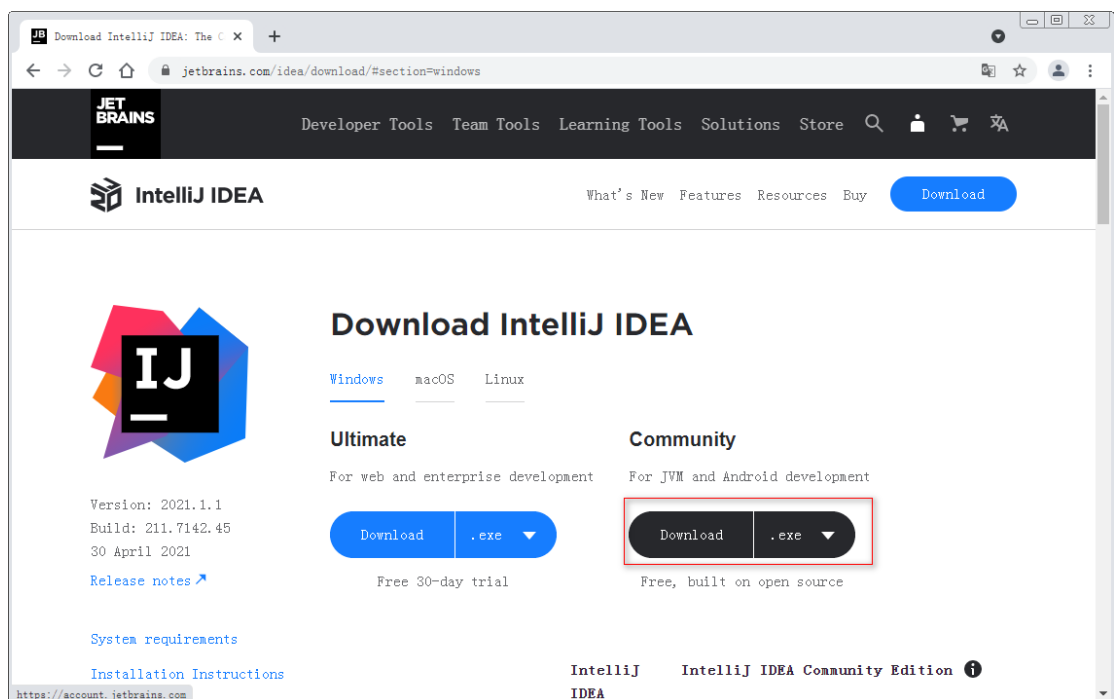


图 2-20 IDEA 的下载页面

从图 2-20 可以看到，官方提供了适用于 Windows 平台、macOS 平台和 Linux 平台的 IDEA 下载，默认显示的是适用于 Windows 平台的 2 款 IDEA 产品 Ultimate 和 Community，其中 Ultimate 指的是旗舰版（提供 30 天免费试用，免费试用后需要收费），Community 指的是社区版（开源免费）。社区版开源满足本系列课程使用的需求，在此选择社区版即可。

（3）单击图 2-20 中 Community 下方的 Download 按钮，即可进行 IDEA 的下载，下载完成后，会在下载目录中生成一个 .exe 后缀名的 IDEA 安装文件。

（4）下载完成后，右键 IDEA 安装文件，选择以管理员方式运行安装文件，接下来会出现图 2-21 所示的 IDEA 安装的欢迎界面。



图 2-21 IDEA 安装欢迎的界面

（5）单击图 2-21 的“Next”按钮，进入选择 IDEA 安装路径的界面，如图 2-22 所示。

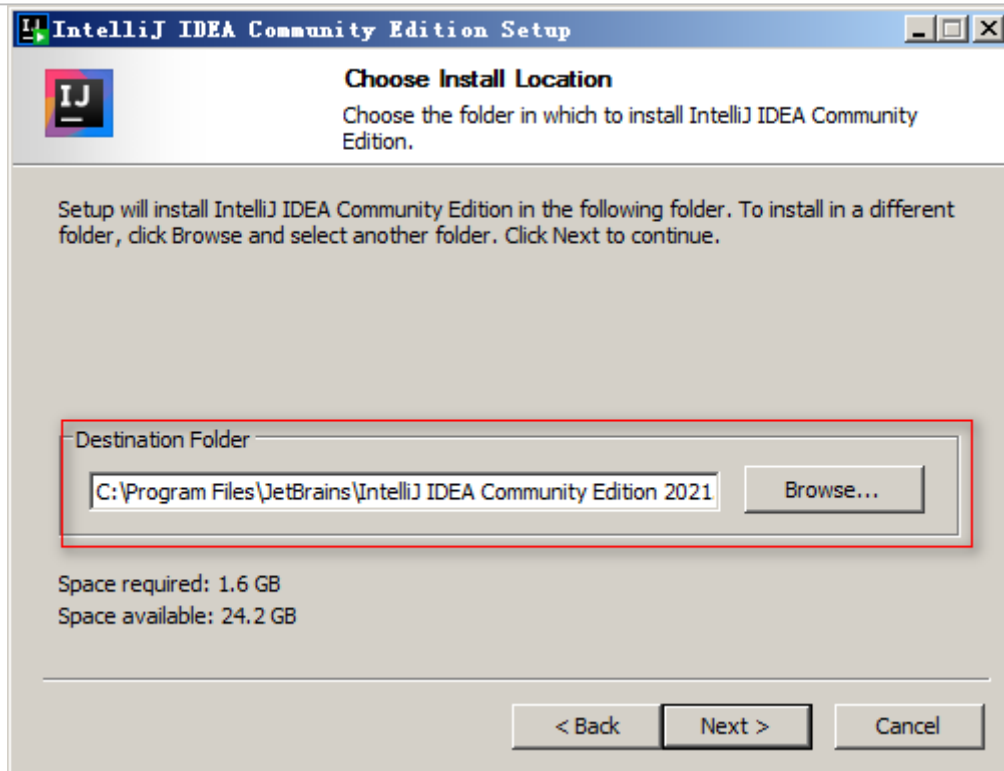


图 2-22 选择 IDEA 安装路径的界面

图 2-22 中，“Browse”按钮之前的文本框对应的是 IDEA 安装的目标文件夹的路径，程序会提供一个默认的安装路径，如果不想在默认提供的路径下安装 IDEA，可以单击“Browse”按钮自行进行选择。

(6) 单击图 2-22 的“Next”按钮，进入 IDEA 安装选项的配置界面，如图 2-23 所示。

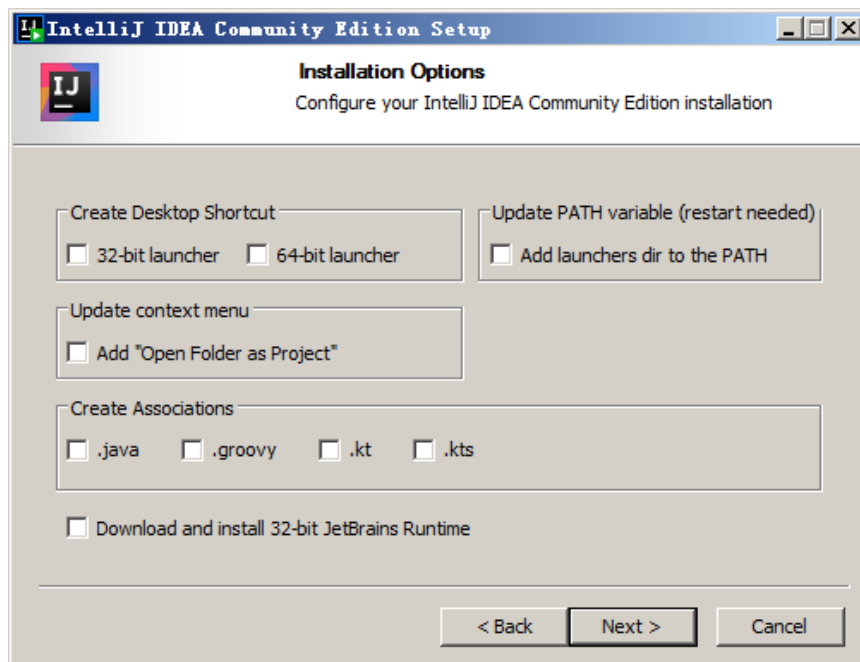


图 2-23 IDEA 安装选项的配置界面

图 2-23 中提供了创建桌面快捷方式、创建关联文件等选项，安装时可以根据自己的需求进行配置。



(7) 单击图 2-23 的“Next”按钮，进入选择开始菜单文件夹的界面，如图 2-24 所示。

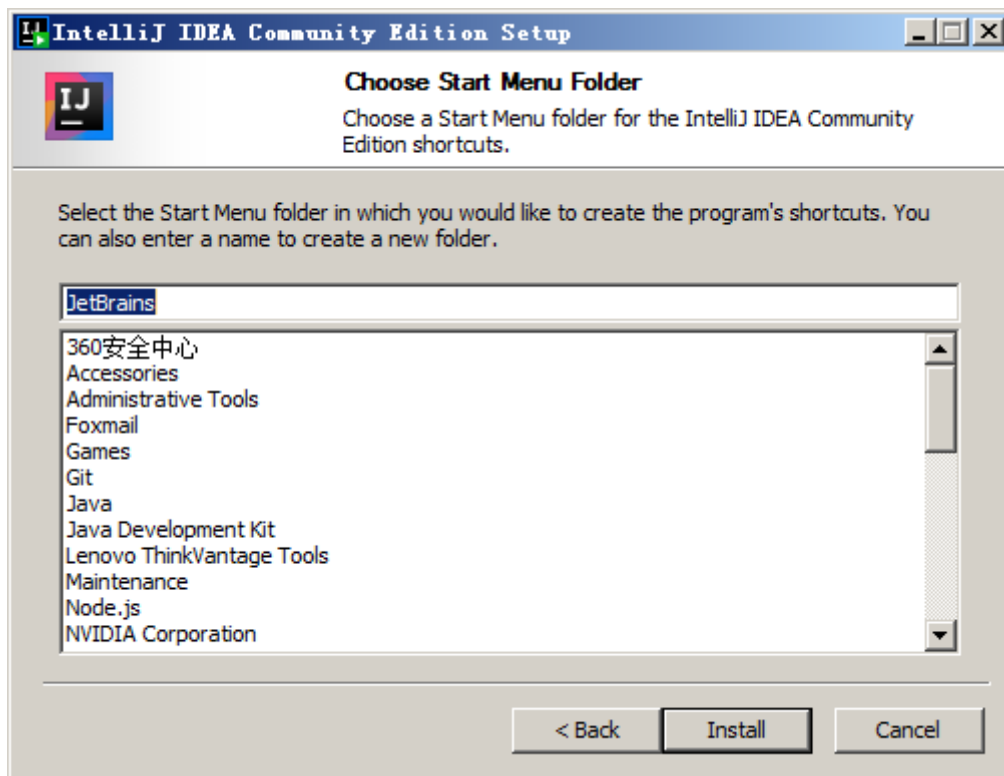


图 2-24 IDEA 安装选项的配置界面

(8) 单击 2-24 中的“Install”按钮，进入 IDEA 安装界面，如图 2-25 所示。

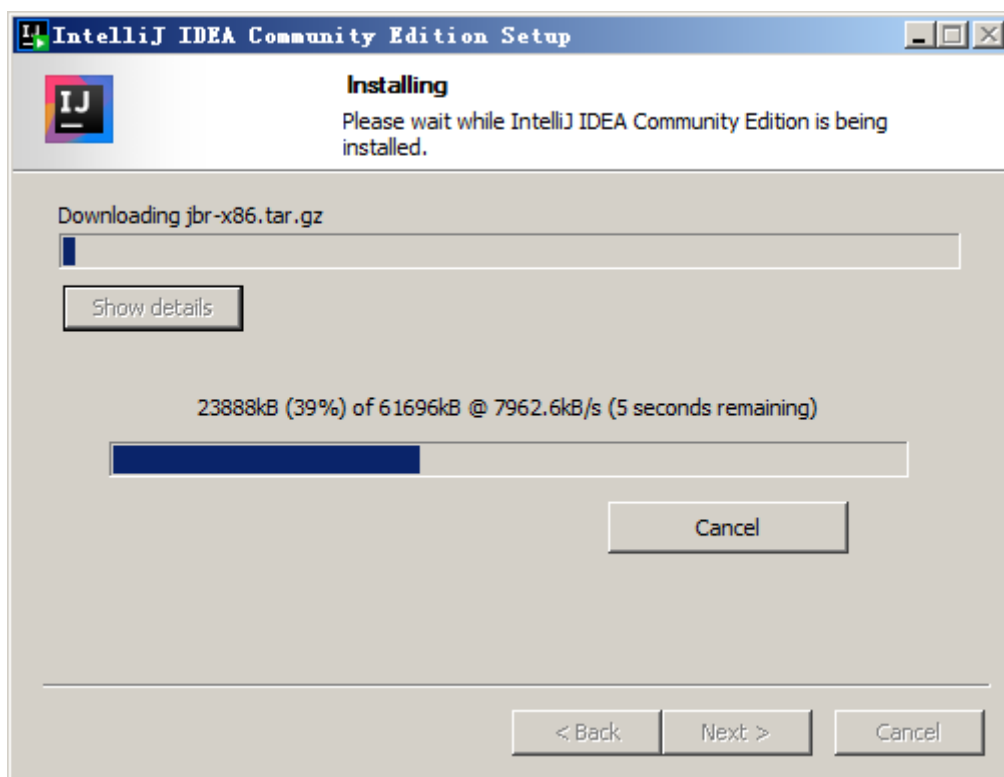


图 2-25 IDEA 安装界面

(9) 从图 2-25 可以看到 IDEA 的安装进度，等 IDEA 安装完成后，会自动进入 IDEA 安装完成的界面，如图 2-26 所示。

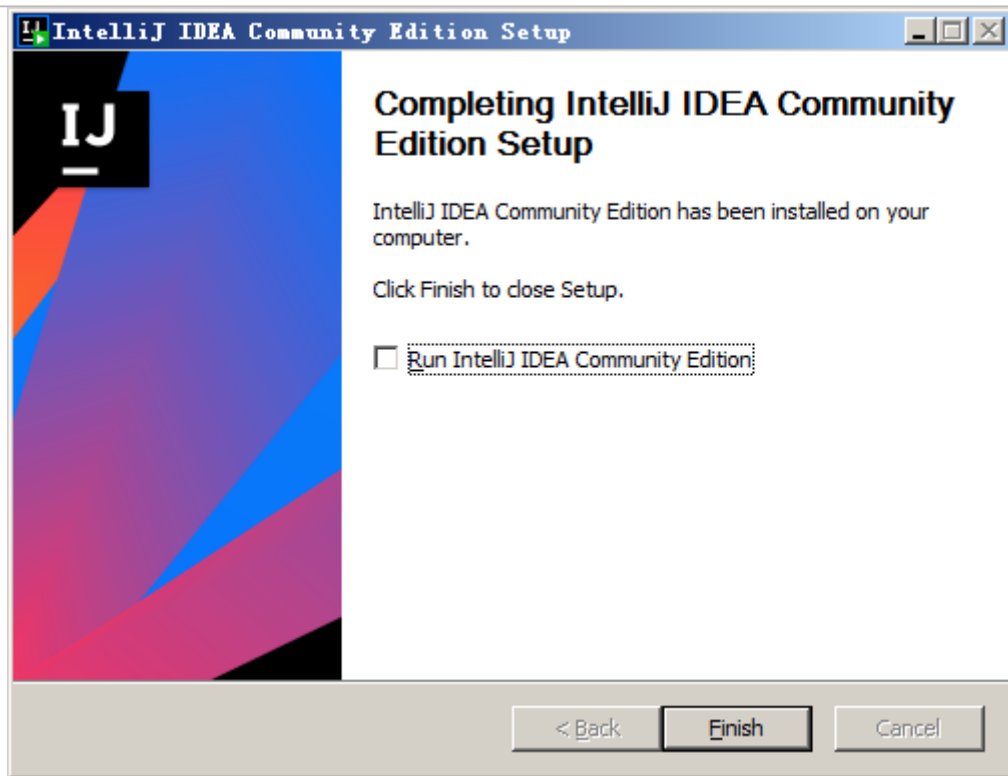


图 2-26 IDEA 安装完成界面

(10) 单击图 2-26 中的“Finish”按钮，完成 IDEA 的安装。
至此，IDEA 的安装已经完成。

2.5.4 IDEA 的启动

完成了 IDEA 的安装后，接下来就可以启动 IDEA,具体步骤如下。

(1) 如果安装时没有设置桌面的快捷方式，可以在电脑的开始菜单中找到 IDEA 启动文件的快捷方式，也可以在 IDEA 安装目录的 bin 文件夹下找到 IDEA 启动文件(idea64.exe)。双击启动文件，会出现使用 IDEA 用户协议的界面，如图 2-27 所示。

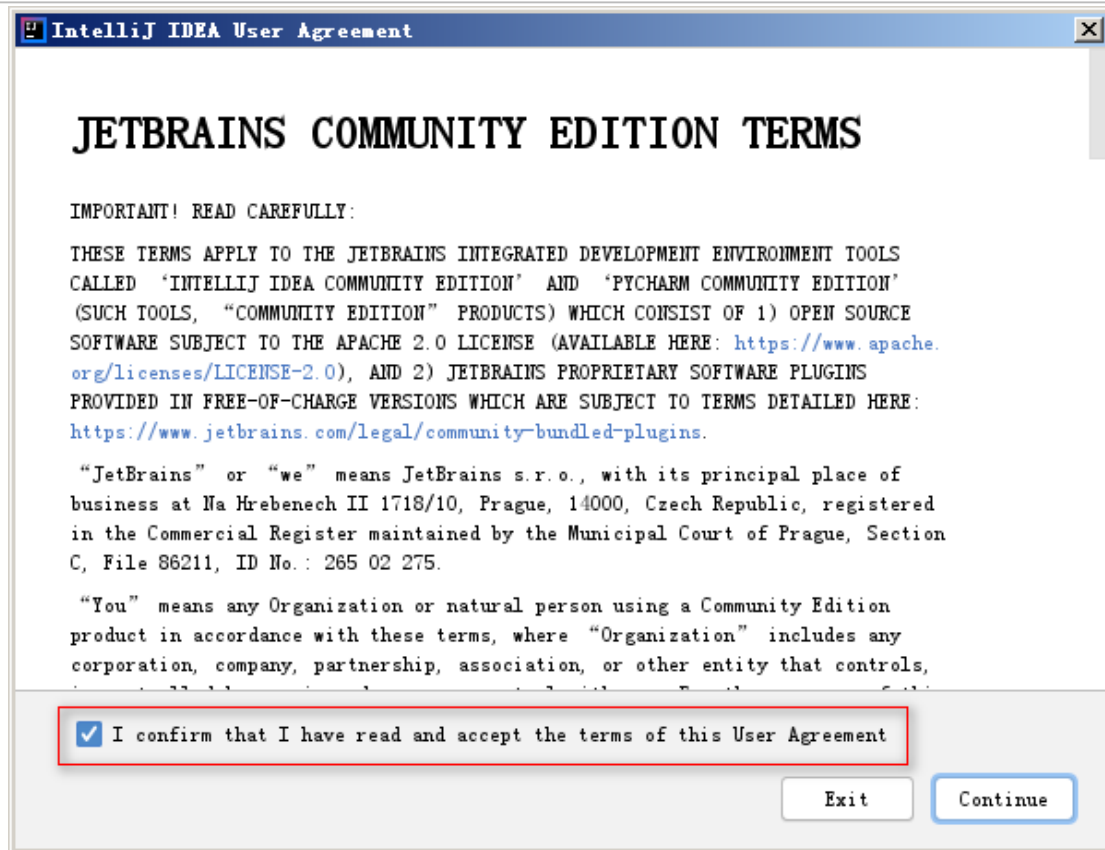


图 2-27 IDEA 用户协议的界面

第一次启动 IDEA 时，需要同意 IDEA 的用户协议，否则将无法继续进行下一步操作。

(2) 单击图 2-27 下端的单选框后，单击“Continue”按钮，进入 IDEA 欢迎界面，如图 2-28 所示。

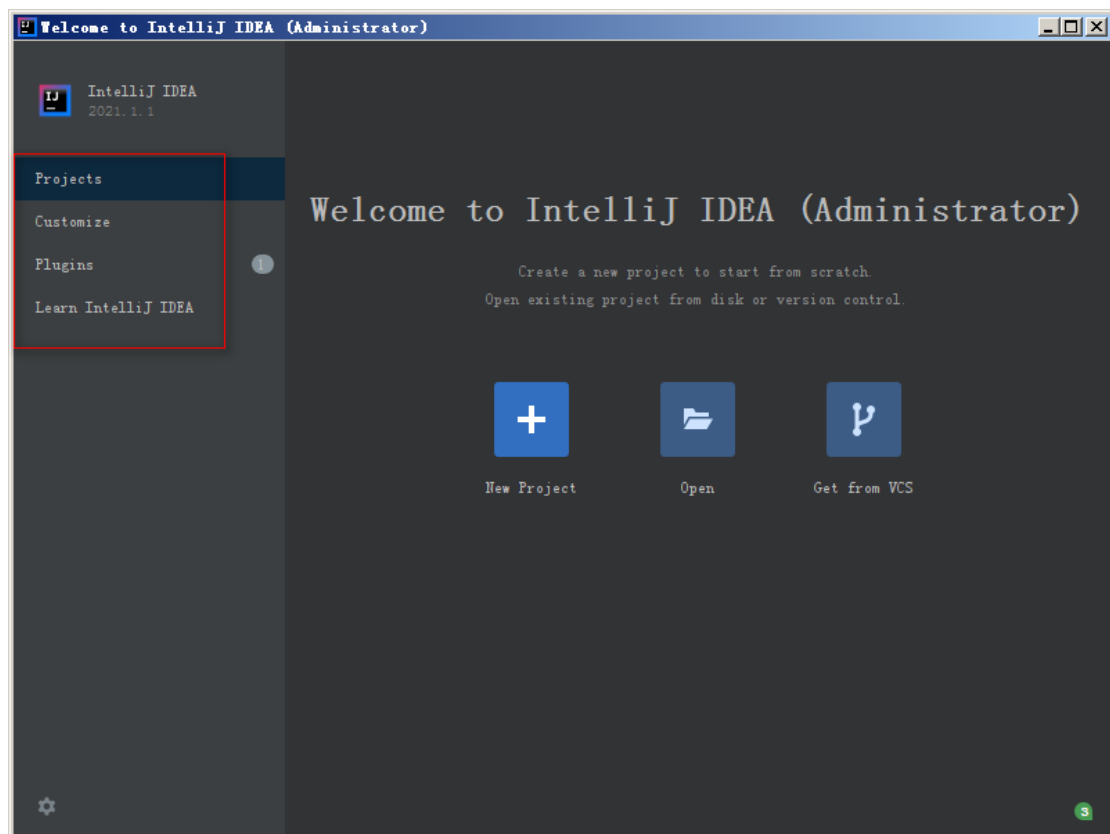


图 2-28 IDEA 欢迎界面

从图 2-28 可以看出，第一次启动 IDEA 后，欢迎界面的左侧有 4 个链接，分别为 Projects、Customize、Plugins 和 Learn IntelliJ IDEA，单击这 4 个链接的任意一个，都会在界面的右侧区域显示对应链接可以选择或设置的功能。界面默认被选中的是 Projects 链接，此时右侧区域显示了 3 个和工程相关的图标，分别为 New Project、Open 和 Get from VCS，单击这 3 个图标依次可以创建一个新的工程、打开一个已经存在的工程和从版本控制库中加载一个工程。

(3) 单击图 2-28 左侧的 Customize 链接后，右侧区域的设置项可以设置软件的风格，如图 2-29 所示。

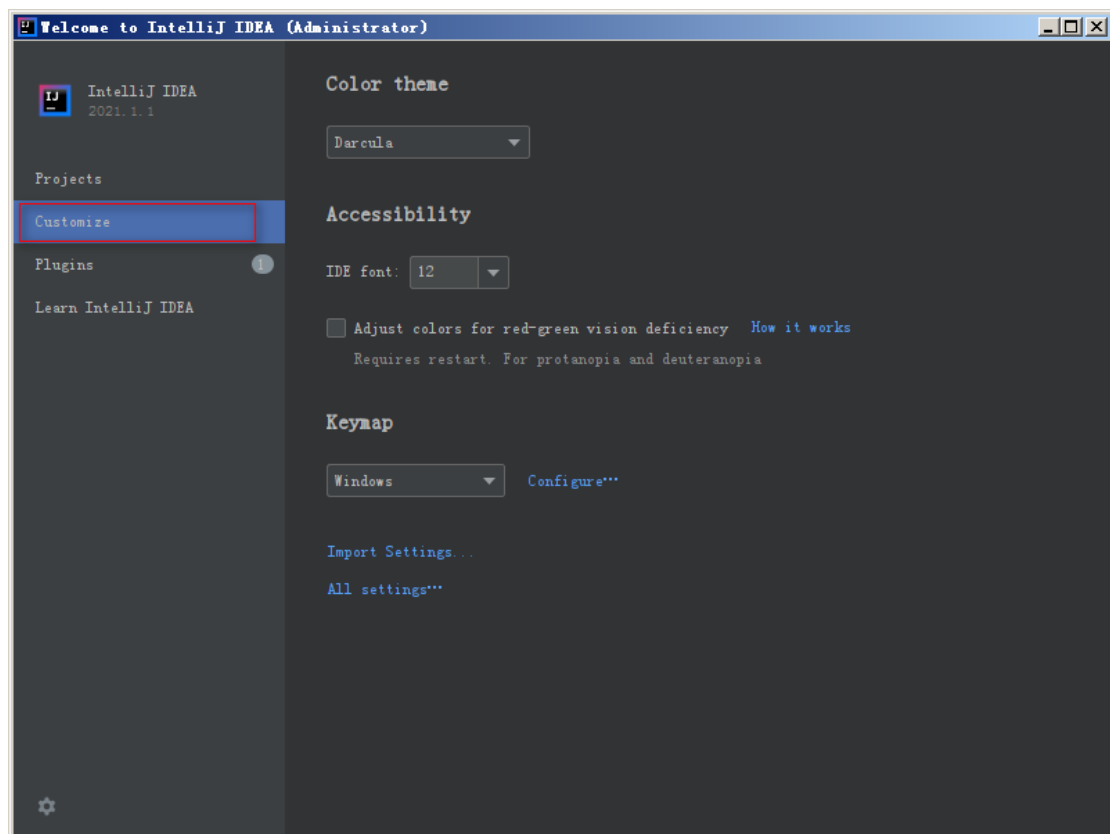


图 2-29 设置软件的风格

图 2-29 中，Color theme 下的下拉框中的值表示软件的颜色主题；font 右侧下拉框中的值表示 IDE 的字体大小；KeyMap 下的下拉框中的值表示映射的快捷键；使用时可以根据自己的需求对修改对应的配置。

(4) 单击图 2-29 左侧的 Plugins 链接后，右侧区域的会展示插件的列表，如图 2-30 所示。

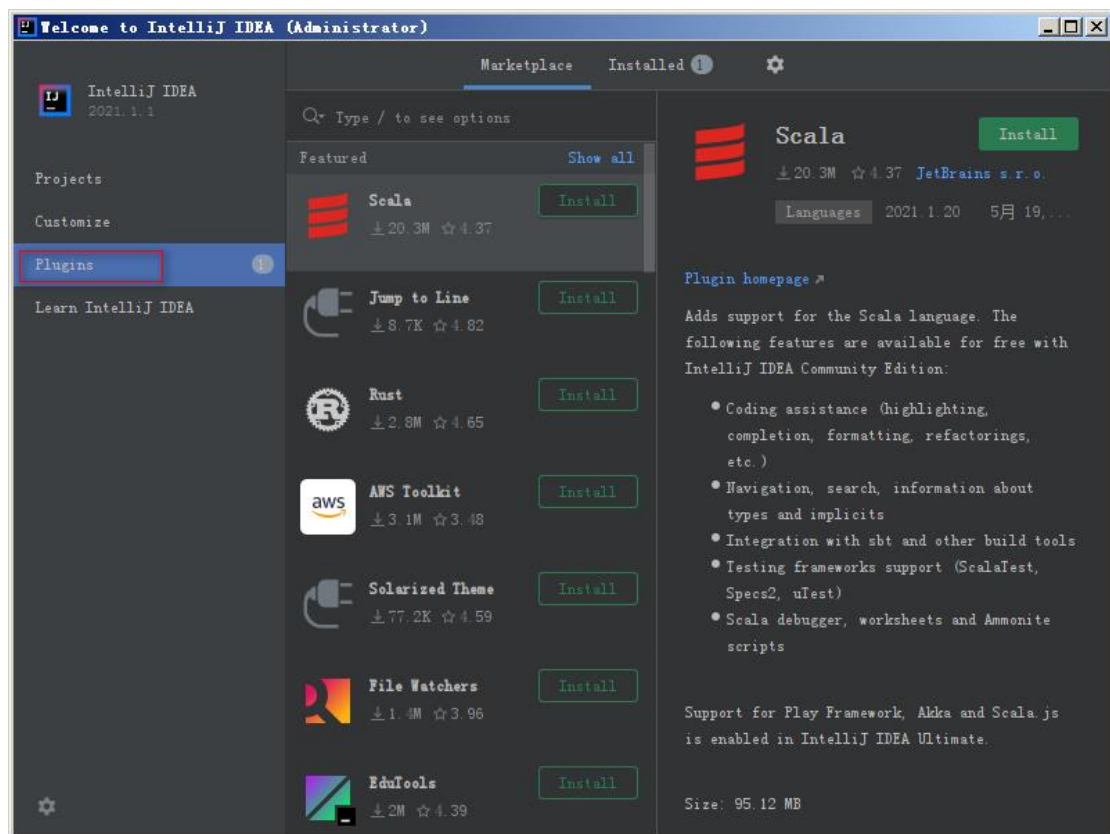


图 2-30 插件列表

从图 2-30 可以看出，界面中展示了很多插件，在后续的开发过程中，可以在此处选择安装所需的插件。

(5) 单击图 2-30 左侧的 Learn IntelliJ IDEA 链接后，右侧区域提供了学习 IDEA 的教程的链接，如图 2-31 所示。

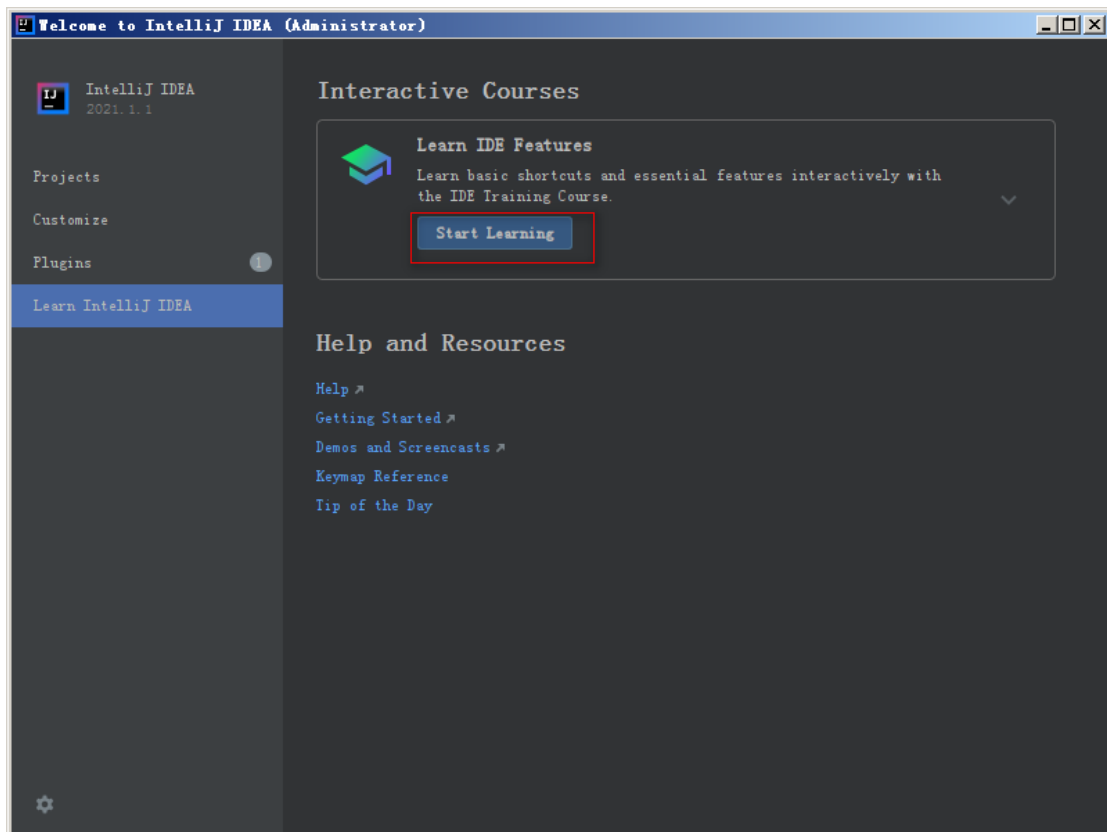


图 2-31 学习 IDEA 的使用教程

单击图 2-31 中的 “Start Learning” 按钮，会跳转到 IDEA 的使用教程中，在教程中可以学习 IDEA 相关功能的使用，例如快捷键等。

2.6 开发第一个 Java 程序

2.6.1 使用 IDEA 开发 HelloWorld

通过前面的学习，相信大家对 IDEA 开发工具已经有了一个基本的认识，本节将学习如何使用 IDEA 完成第一个 Java 程序 HelloWorld 的编写和运行，具体步骤如下。

1. 创建项目

在 IDEA 中，以项目（Project）的形式管理 Java 程序，一个 Java 程序可以用一个项目进行表示，所以编写 Java 程序需要先创建一个项目。在 IDEA 中创建项目的步骤如下。

（1）启动 IDEA 后，会进入 IDEA 欢迎界面，如图 2-32 所示。

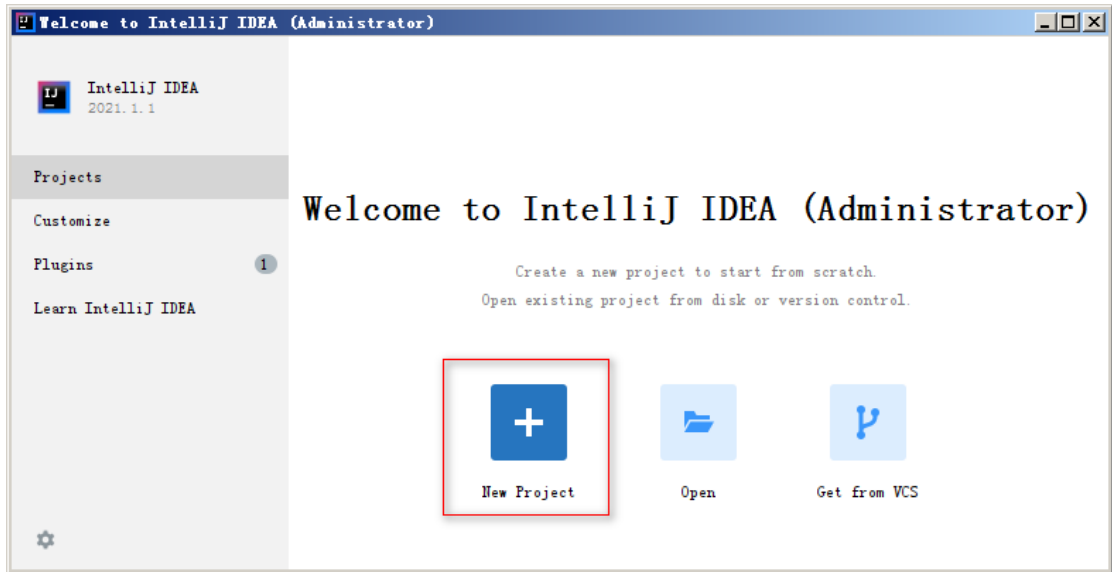


图 2-32 IDEA 欢迎界面

(2) 单击图 2-32 的“New Project”图标，进入 New Project 对话框，如图 2-33 所示。

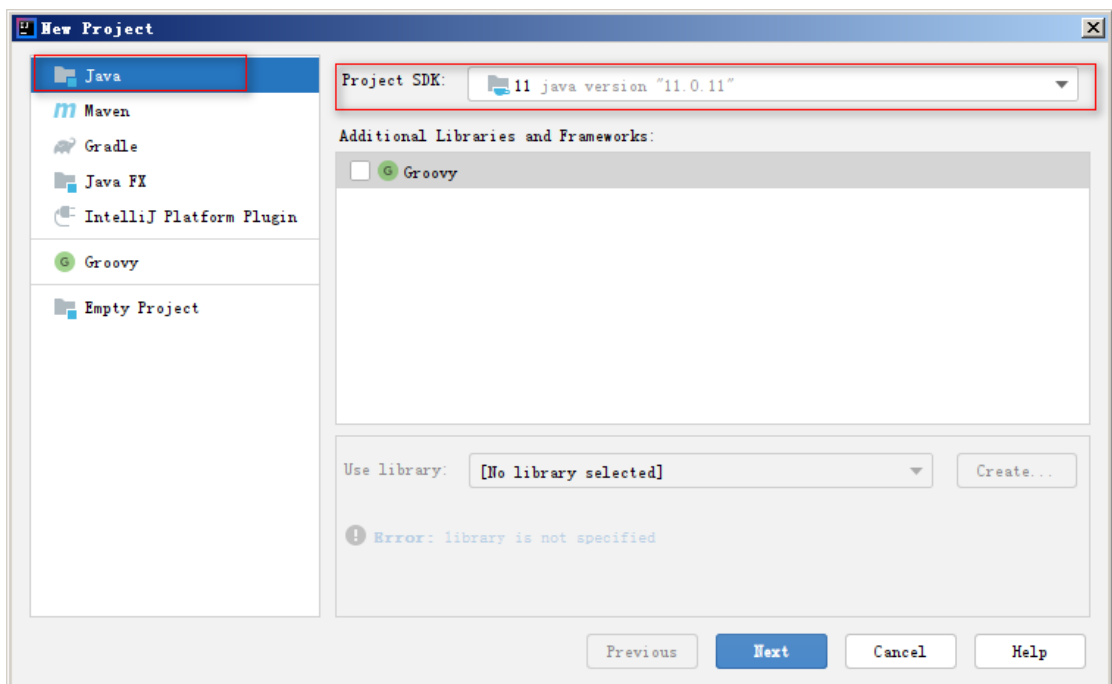


图 2-33 New Project 对话框

从图 2-33 可以看出，New Project 对话框左侧默认选中的就是 Java，在此保存默认即可。对话框右侧 Project SDK 对应的下拉框中是创建本项目使用的 JDK 版本，在此选择本章安装好的 JDK 版本即可。

(3) 单击 2-33 中的“Next”按钮，创建项目使用的模板的界面，如图 2-34 所示。

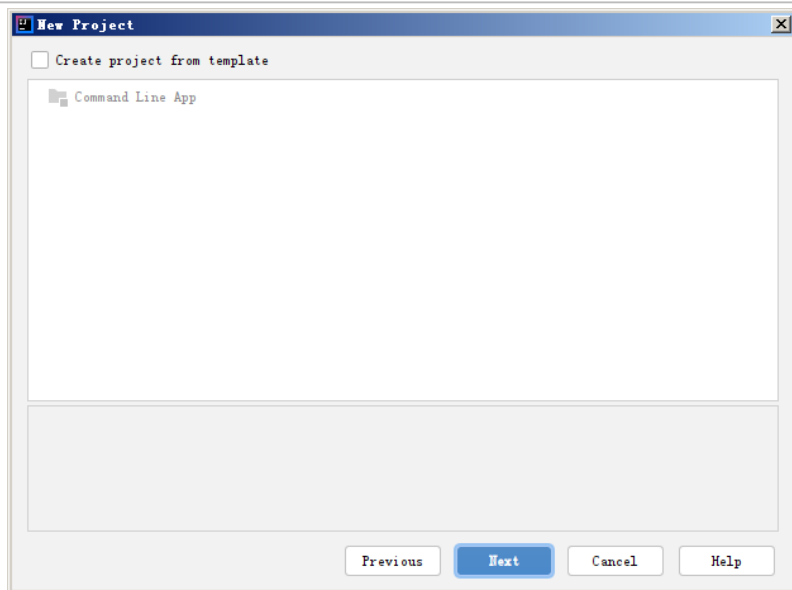


图 2-34 IDEA 欢迎界面

本次创建项目不使用任何模板，所以不用勾选图 13-3 中任何选项。

(4) 单击图 2-34 的“Next”按钮，进入项目名称及项目源代码存放目录的设置界面，如图 2-35 所示。

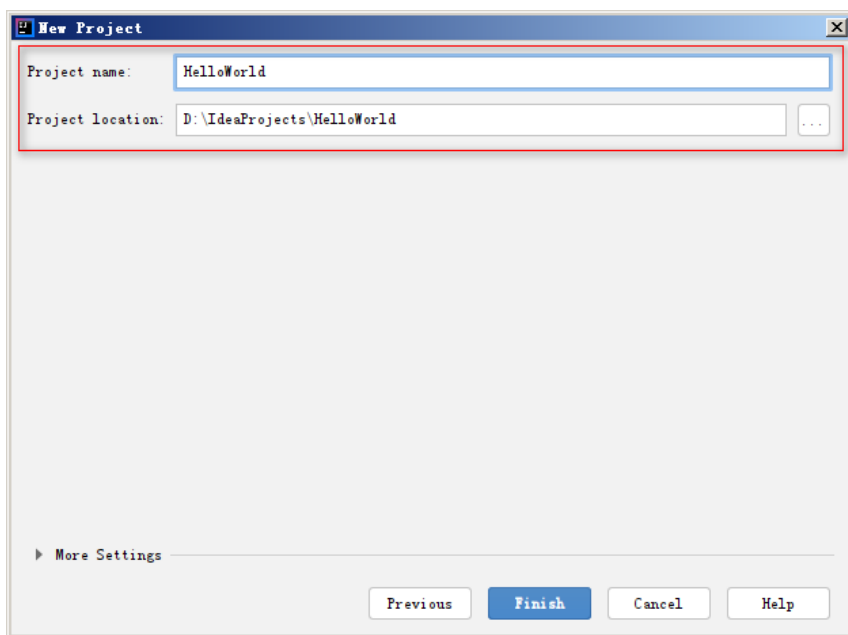


图 2-35 项目名称及项目源代码存放目录的设置界面

图 2-35 中，Project name 输入框用于设置的是项目名称，Project location 输入框用于设置项目源代码存放目录，如果目录不存在，系统会自动创建目录。需要注意的是，源代码存放目录最好不要出现中文或者空格。

(5) 单击图 2-35 的“Finish”按钮，就可以完成项目 HelloWorld 的创建。如果 Project location 填写的目录此时在还不存在，会弹出文件夹不存在的对话框，如图 2-36 所示。

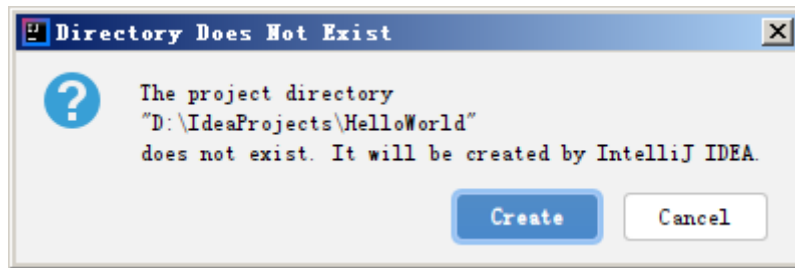


图 2-36 文件夹不存在的对话框

图 2-36 中提示文件夹 D:\IdeaProjects\HelloWorld 还不存在，是否让 IDEA 帮助创建。

(6) 单击图 2-36 中的“Create”按钮，完成对应文件夹的创建。此时 IDEA 会对 HelloWorld 项目进行创建。HelloWorld 项目创建成功后，IDEA 会在 D:\IdeaProjects\HelloWorld 目录下自动创建一些文件夹和文件，此时可以在 IDEA 中看到 HelloWorld 的项目结构，如图 2-37 所示。

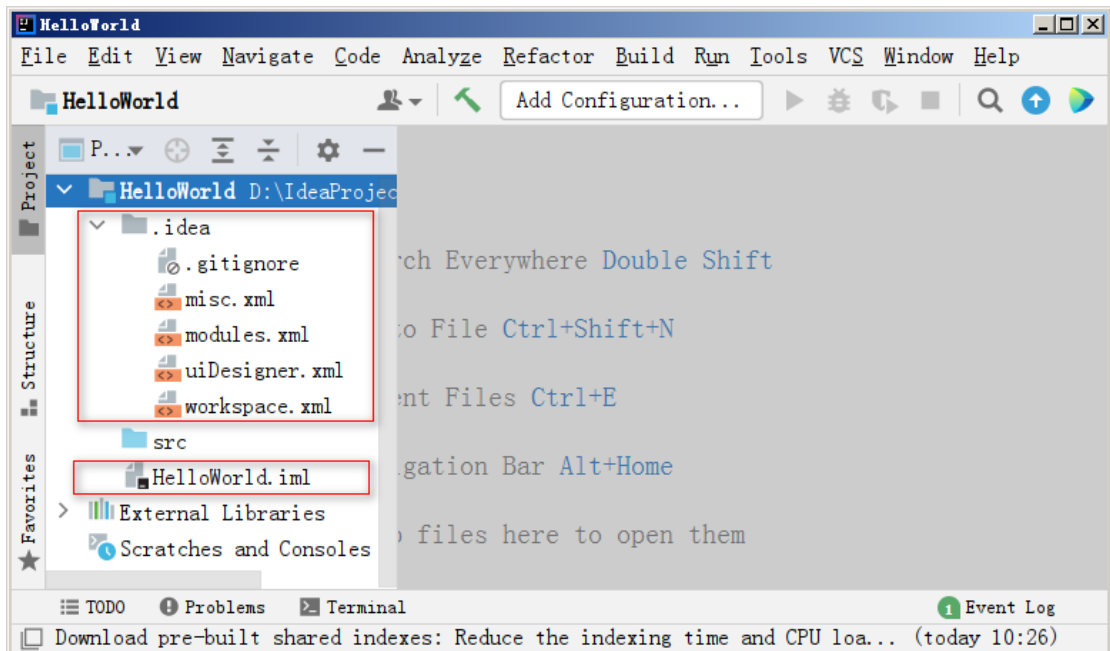


图 2-37 HelloWorld 的项目结构

图 2-37 中，左侧是 HelloWorld 的项目组成的结构，其中.idea 目录下的所有文件，以及 HelloWorld.iml 文件都是 IDEA 开发工具使用的配置文件，不需要开发者操作；src 是 source 单词的缩写，程序的源文件存放在这个目录下；External Libraries 是扩展类库，即 Java 程序编写和运行所依赖的 JDK 中的类。窗口上端是 IDEA 的菜单栏，菜单栏中提供的工具在后续的学习中会慢慢进行讲解。

2. 创建 Java 类

创建好项目之后，可以在项目中创建 Java 类，具体步骤如下。

(1) 右键单击 HelloWorld 项目下的 src 文件夹，依次选择“New”→“Java Class”，会出现一个 New Java Class 界面，如图 2-38 所示。

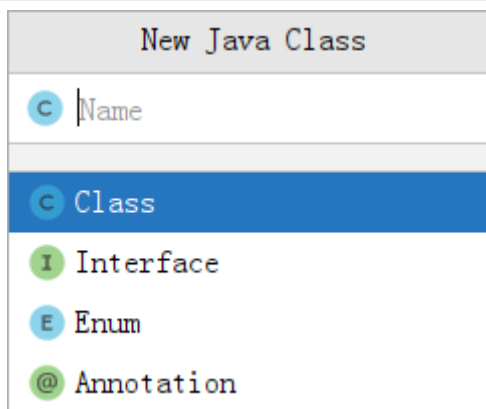


图 2-38 New Java Class 界面

图 2-38 中可以创建多种类型，默认选中的 Class 是我们需要创建的类型，文本框中填写的内容用于设置创建的 Class 的名称。

(2) 在 New Java Class 界面的文本框中，输入 HelloWorld，按下键盘的“Enter”键以完成 Java Class 的创建。创建完成后，src 文件夹下会生成一个名称叫 HelloWorld 的文件，并且自动将该文件在 IDEA 的右侧区域打开，如图 2-39 所示。

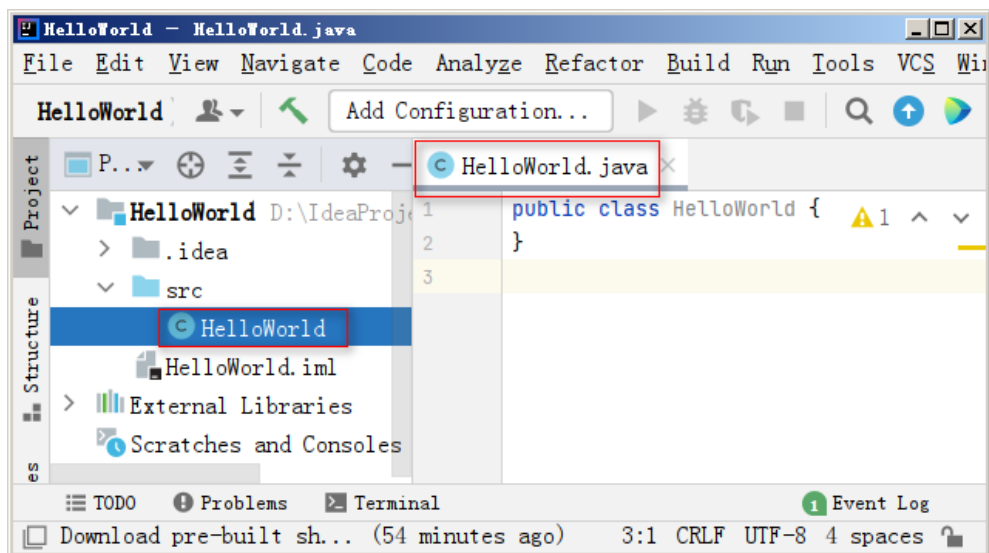


图 2-39 创建的 HelloWorld 文件

从图 2-39 可以看出，创建的 HelloWorld 文件是以 .java 为后缀名，该文件用于编写代码，是 Java 的源文件。编写的 Java 程序中最小单位是类，一个 Java 程序至少拥有一个类，IDEA 自动在 HelloWorld.java 中生成了一些代码，其中 class 代表类，HelloWorld 是类的名称。

3. 编写 HelloWorld 程序

Java 源文件创建好之后，就可以在源文件中编写逻辑代码。在此以在控制台打印 HelloWorld 为例，讲解 HelloWorld 程序的编写，具体步骤如下。

(1) 在创建好的 HelloWorld.java 文件中，输入 main，此时会弹出一个提示内容，如图 2-40 所示。

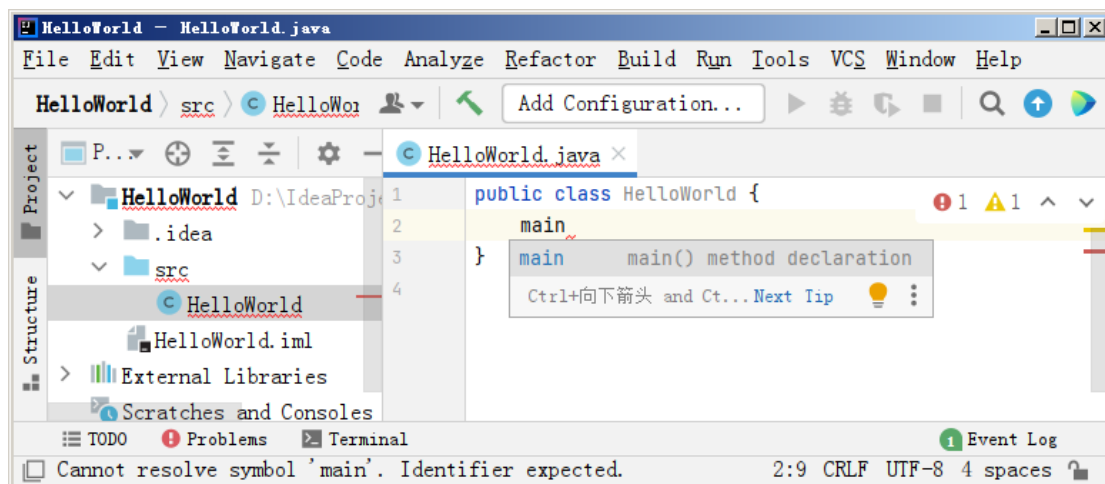


图 2-40 输入 main 后的提示信息

(2) 出现图 2-40 所示的提示信息后，按下键盘的 Enter 键，此时会在文件中自动补全一些代码，具体如文件 1-1 所示。

文件 1-1 HelloWorld.java

```
public class HelloWorld {  
    public static void main(String[] args) {  
  
    }  
}
```

文件 1-1 中自动补全出来的代码是 main() 方法。main() 方法是 Java 程序的入口，它的格式是固定的，格式内代码的具体含义会在后续进行详细讲解。

(3) 在 main() 方法中编写代码，输出 HelloWorld，具体代码如下。

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("HelloWorld");  
    }  
}
```

上述代码中，第 3 行代码是一个输出语句，用于向控制台输出内容。这里告诉大家一个小技巧，我们可以输入“sout”后根据提示内容快速补全“System.out.println()”这行代码。

4. 运行 HelloWorld 程序

Java 程序编写完成后，可以在 IDEA 中运行代码，Java 程序运行方式有 2 种，具体如下：

第 1 种方式：在左侧编辑区绿色三角形按钮上点击，选择“Run ‘HelloWorld.main()’”菜单运行 HelloWorld 程序，如图 2-42 所示。

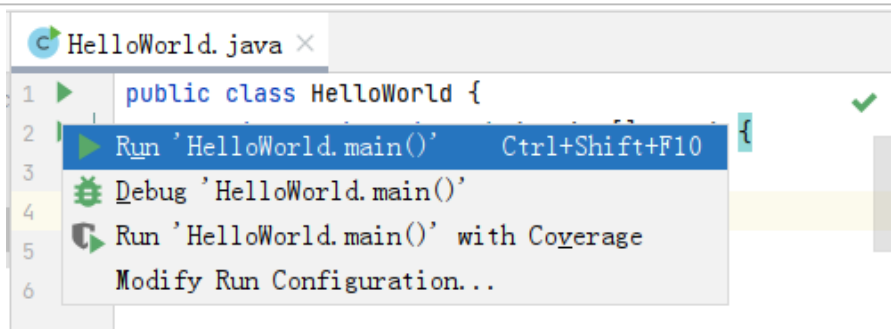


图 2-42 程序运行方式（1）

程序运行后，控制台打印的内容如图 2-43 所示。

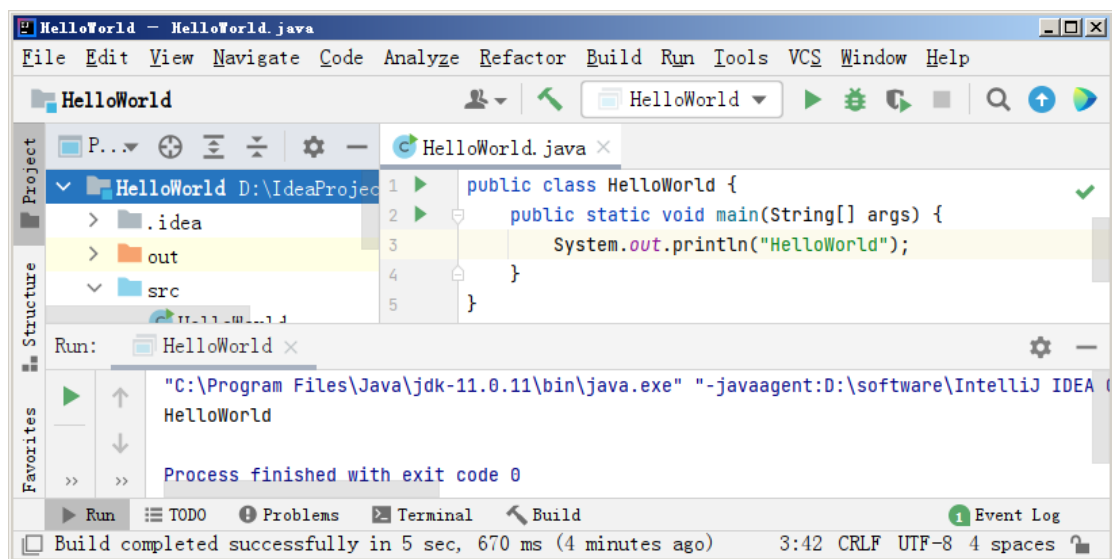


图 2-43 程序运行结果（1）

第 2 种方式：在空白处右键，选择“Run ‘HelloWorld.main()’ ” 菜单运行 HelloWorld 程序，如图 2-44 所示。

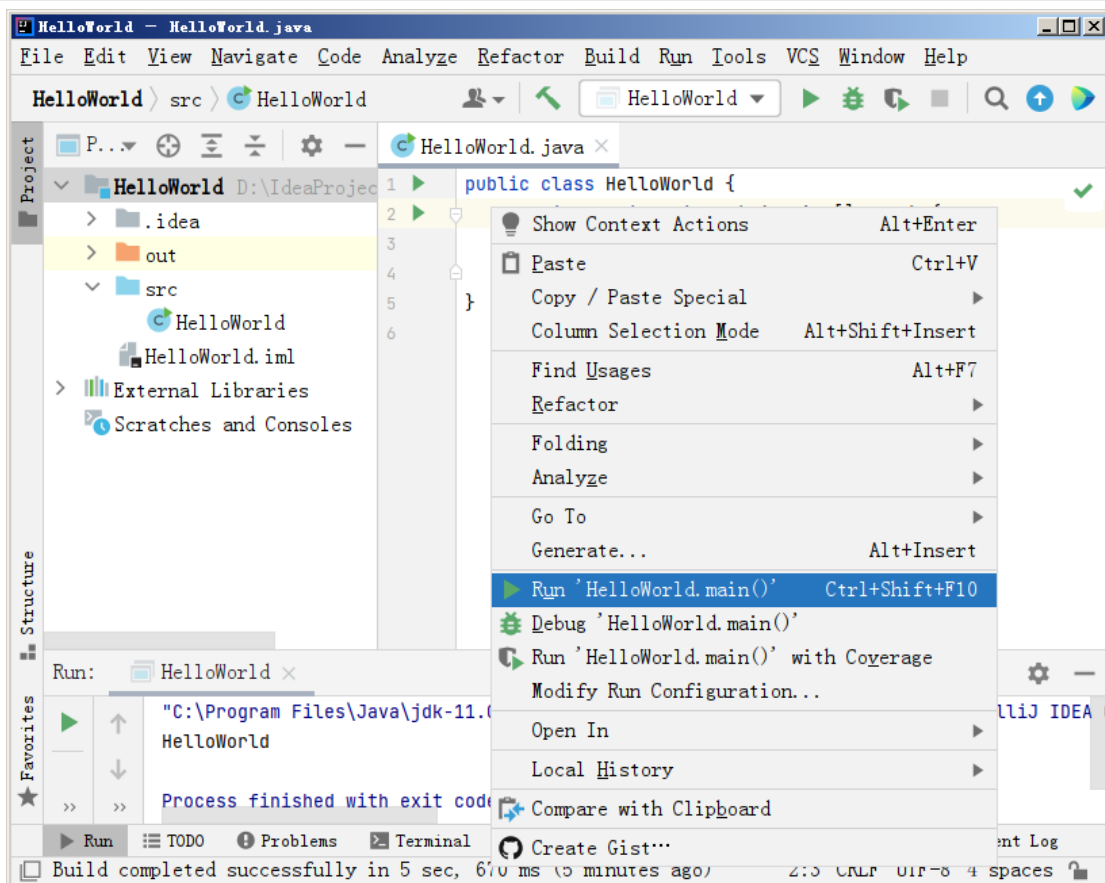


图 2-44 程序运行方式（2）

程序运行后，控制台打印的内容如图 2-45 所示。

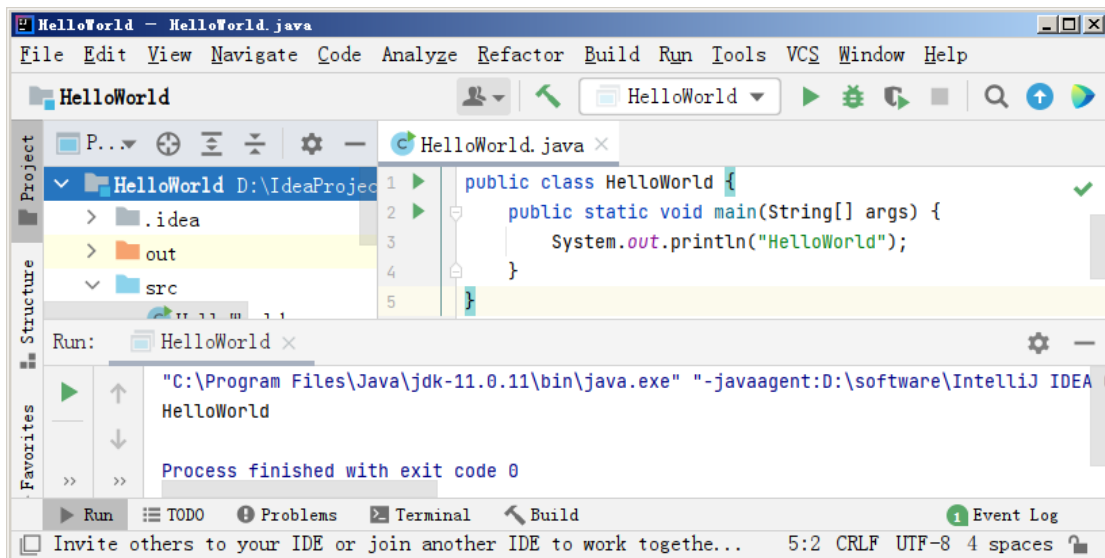


图 2-45 程序运行结果（2）

2.6.2 分析 IDEA 编译代码的流程

我们知道，Java 作为一种高级编程语言，它的执行过程分为“编写→编译→运行”，如图 2-46 所示。

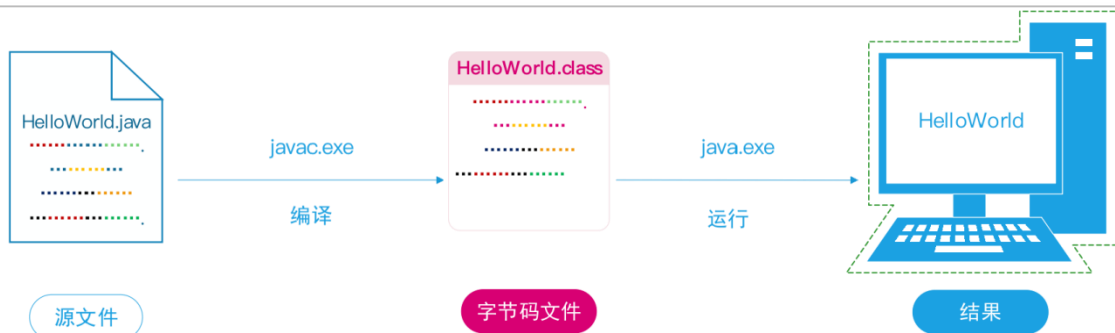


图 2-46 Java 程序执行过程

而我们使用 IDEA 编写的 HelloWorld 程序，它只需要单击  按钮就完成了整个过程，这个时候，可能有的同学会有疑问，IDEA 是如何编译 Java 程序的呢？下面我们回顾一下，HelloWorld 程序在 IDEA 中的运行过程，具体如下：

(1) 当我们在 IDEA 中创建一个名为 HelloWorld 的 Java 文件，并在文件中编写代码，这个过程其实就是编写源文件的过程，如图 2-47 所示。

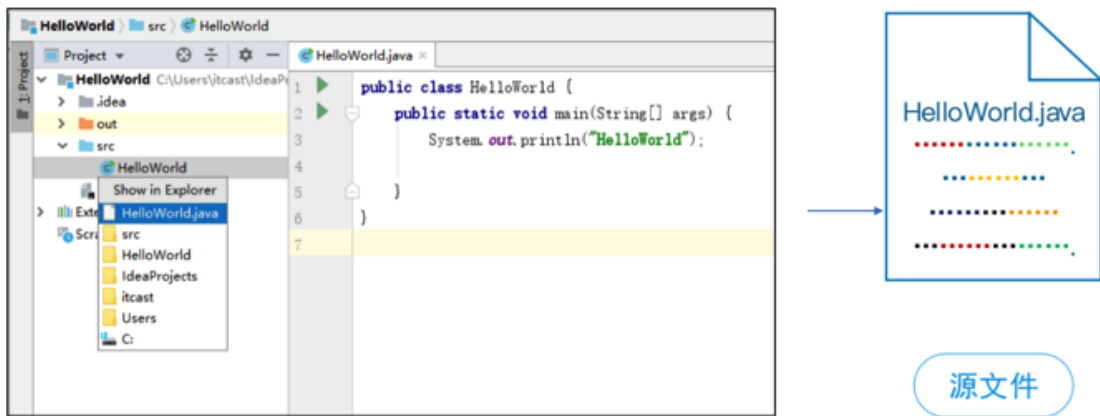


图 2-47 编写 Java 源文件

图 2-47 编写的 Java 源文件位于本地。选中 HelloWorld.java 文件，右击选择“Open”→“Explorer”，即可看到刚刚编写的 Java 源文件，如图 2-48 所示。

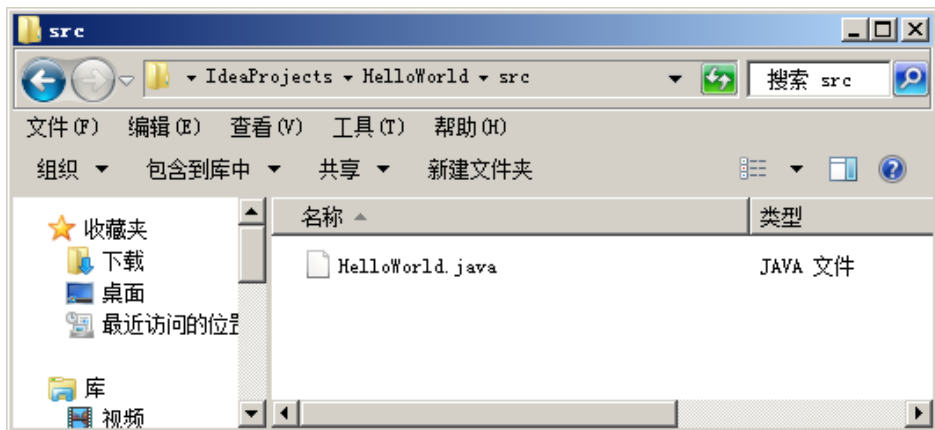


图 2-48 Java 源文件

(2) 单击  按钮运行 Java 程序时，实际上是将手动编译和运行进行了整合，IDEA 会自动调用 javac 命令和 java 命令将源代码编译为字节码文件并运行，如图 2-49 和图 2-50 所



示。

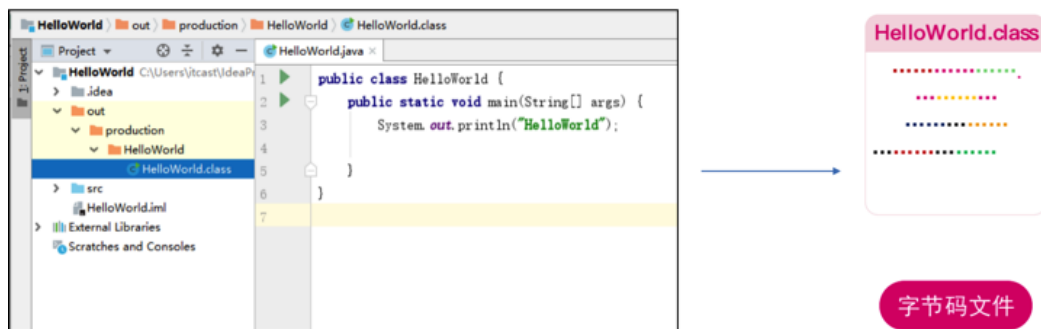


图 2-49 将源文件编译为字节码文件

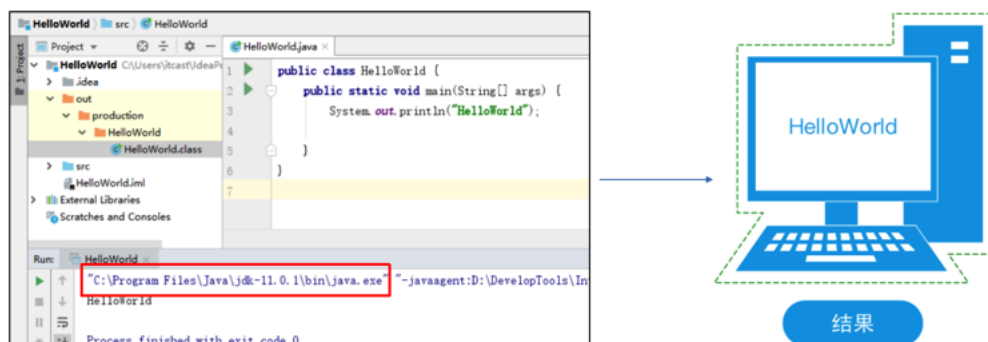


图 2-50 运行 Java 程序

单击  按钮运行 Java 程序后，图 2-48 所示的 Java 文件会被编译出对应的 class 文件，如图 2-51 所示。

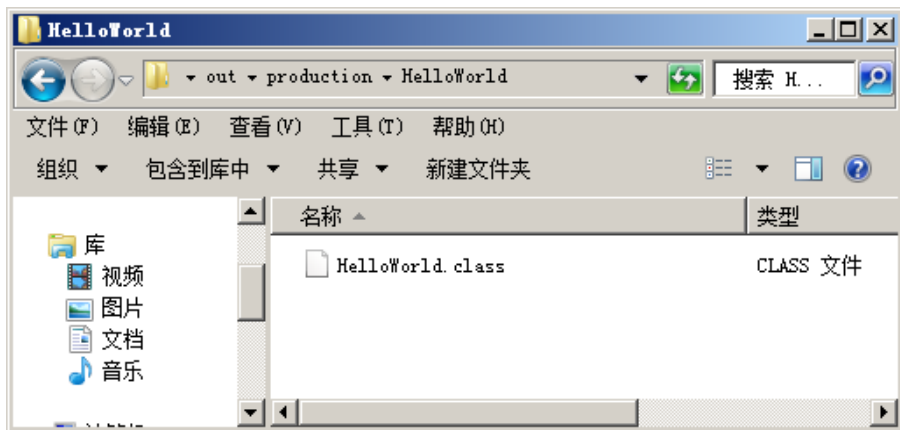


图 2-51 编译出的 class 文件

与此同时，程序会运行字节码文件，并在控制台输出程序结果，如图 2-52 所示。

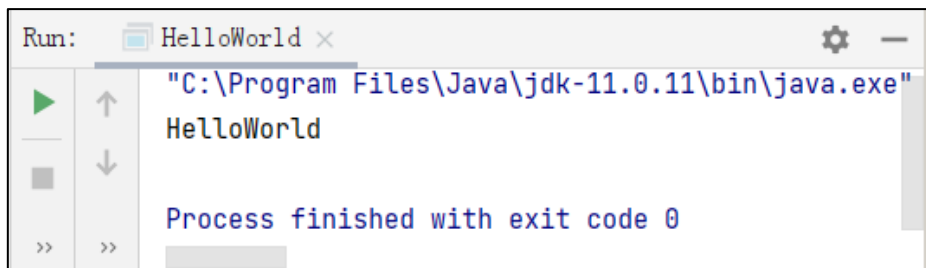




图 2-52 控制台输出程序结果

2.6.3 编写 Java 程序的注意事项

对于初次接触编程的同学，第一次编写 Java 代码时经常会遇到一些问题，下面我们针对常见的问题进行强调，具体如下：

一、中英文输入法导致的问题。

编写 Java 代码时，经常会用到大括号、分号、括号以及双引号等符号，这些符号切记要使用英文输入法，否则程序会报错。图 2-53 所示的就是使用中文双引号导致的程序报错。

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("HelloWorld");  
        System.out.println("HelloWorld"); → 报错  
    }  
}
```

图 2-53 使用中文双引号导致的程序报错

二、代码编写位置错误。

大家在编写代码时，一定要注意代码的编写位置。如果代码放错位置，程序也会报错，示例如图 2-54 所示。

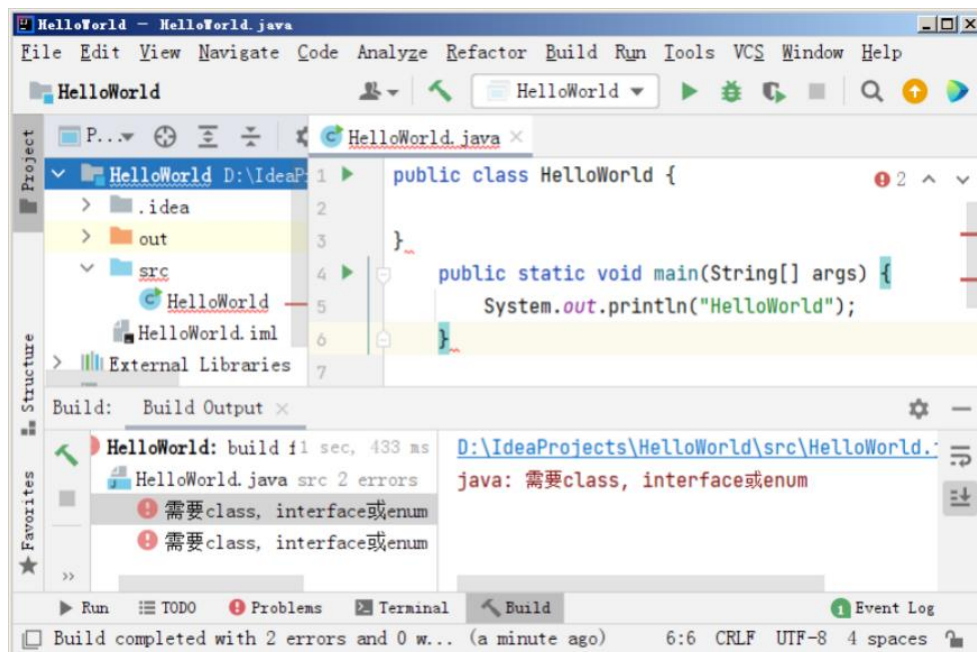


图 2-54 代码放错位置报错

2.6.4 JDK 11 的快捷运行

JDK 11 中提供了一个很不错的功能，这个功能允许我们直接使用 Java 虚拟机运行 Java 源代码。也就是说，我们可以省略 javac 命令编译 Java 源文件，直接使用 Java 命令就直接

运行源代码，而且不会产生.class 文件。

例如，我们在 D 盘的 code 文件夹中，创建一个 HelloWorld.java，具体代码如下：

```
public class HelloWorld{  
    public static void main(String[] args){  
        System.out.println("helloworld");  
    }  
}
```

使用 cmd 命令进入控制台，将当前路径设置为 HelloWorld.java 文件所在的路径，输入 java HelloWorld.java，回车，程序就会运行 HelloWorld.class，并在控制台中打印出 helloworld，如图 2-55 所示。



图 2-55 运行 Java 源文件

需要注意的是，使用 java 命令快捷运行程序是 Java 11 提供的新特性，如果大家使用的 Java 版本低于 11，那么直接使用 java 命令运行 Java 文件会报错，如图 2-56 所示。

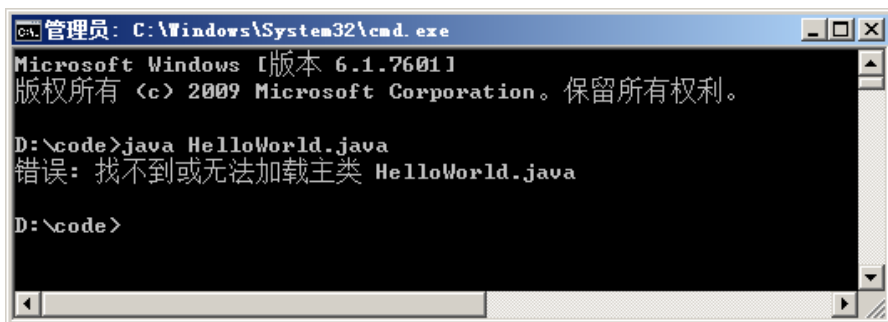


图 2-56 运行 Java 源文件报错

第 3 章 Java 编程基础

学习目标

- ◆ 了解编程与编程思维，能够说出编程和编程思维的含义
- ◆ 掌握变量的声明与赋值，会对不同类型的变量进行声明和赋值



- ◆ 熟悉字面量和常量，能够说出两者的区别
- ◆ 熟悉注释、标识符和关键字的用法
- ◆ 掌握各种运算符的作用，能够使用不同的运算符实现功能
- ◆ 掌握基本数据类型与复杂数据类型，会将基本数据类型转为复杂数据类型以及实现不同数据类型的转换
- ◆ 掌握什么是字符串，会定义字符串变量并实现字符串拼接操作

俗话说，“九层之台，起于累土”。学习 Java 是一样的道理，大家只有夯实基础，才能游刃有余的学习后续内容。接下来，本章将围绕 Java 编程的基础知识进行详细讲解。

3.1 编程与编程思维

先看一个生活中的例子。小时候，我们经常去亲戚朋友家去玩，从家出发之前，妈妈总会对我们说“一会进了门，要先叫叔叔阿姨好”。其实这个时候，妈妈就是在对我们进行“编程”。

编程就是安排一串的指令，告诉计算机应该做什么事，编程的过程和复现人类思考的过程，基本是一样的。

编程思维是我们做事情的思考过程。如果大家想学编程，建议不要着急写代码，而是应该先规划好流程，思考清楚后再动手写代码。

下面我们通过一个游戏，带大家体现一下思维过程，请大家认真听题。

图 3-1 所示的是一辆没有载乘客的公交车出发了，沿着 A~F 站行驶，

公交车到了 A 站，有 1 名乘客上车。

公交车到了 B 站，有 3 名乘客上车，1 名乘客下车

公交车到了 C 站，有 2 名乘客上车，2 名乘客下车

公交车到了 D 站，没有乘客上下车。

公交车到了 E 站，有 2 名乘客上车，1 名乘客下车

公交车到了 F 站，有 1 名乘客上车。

请大家根据公交车的行驶情况，追踪记录公交车上人数的变化过程。

如果我们使用乐高积木表示乘客，用小盒子表示公交车。大家体会一下自己的思考过程。我们在脑子里面追踪记录公交车人数变化的时候，相当于创建了一个小盒子，每当公交车上有人上车，或者下车的时候，我们都会更新脑子中记录的数据，也就是会更新积木的数量。只要持续的对这些数据跟踪，那么最终就会根据小盒子中积木的数量，知道公交车上乘客的数量，这个过程就是我们思维的过程。而编程就是把上述思维过程转为代码的方式。



图 3-1 公交车行驶过程

3.2 变量

3.2.1 变量的声明与赋值

在 Java 编程过程中，使用类似小盒子的编程思想去存储数据，这个小盒子就相当于 Java 中的“变量”。变量包括变量名称和变量的值两部分，其中变量的名称是不会发生变化的，而变量的值是可以改变的，这里变量值的变化就好比小盒子中积木数量的变化。

举一反三，我们可以创建多个盒子，也就是多个变量。同样以前面公交车乘客上下车为例，我们可以创建两个小盒子，分别命名为 `peopleCount` 和 `busStopCount`，其中，`peopleCount` 用于存储的是公交车上的人数，`busStopCount` 用来存储公交车停的站台数量，如图 3-2 所示。

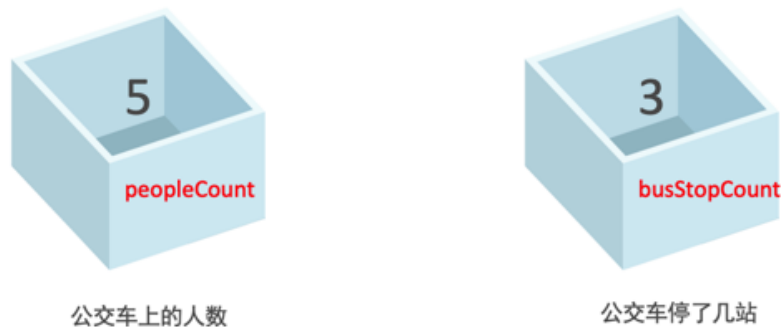


图 3-2 变量

当然，变量不仅可以存储整数值，还可以存储其他类型的数据，比如，小数，字符、字符串文本等，如图 3-3 所示。

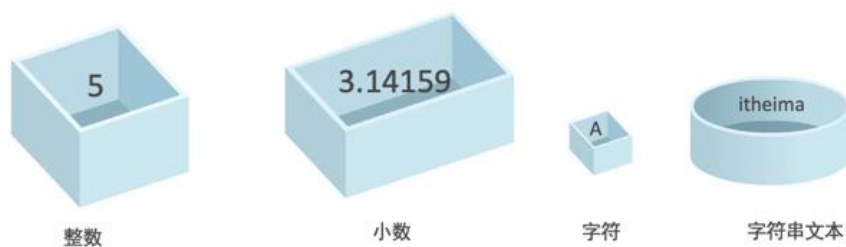




图 3-3 变量存储其他类型的数据

关于不同的变量类型，我们后续会详细讲解，这里大家只需要明白，变量值是可以发生改变的，而变量名不会发生改变。一旦我们创建好了盒子，那么盒子里面只能存放对应类型的数据。也就是说，一旦变量创建好，那么变量可存储的数据类型就指定好了，该变量只能存储对应类型的数据。

例如，公交车上存储乘客数量的变量 `peopleCount`，由于人数只能是整数，所以变量 `peopleCount` 的数据类型只能是整数。Java 中使用 `int` 表示整数类型，可以通过下列方式定义变量 `peopleCount`，这个过程也叫做变量的声明。

```
int peopleCount;
```

上述代码表示创建了一个名称为 `peopleCount` 的变量，该变量的数据类型是整数，它只能存储整数。

我们可以给变量 `peopleCount` 设置一个值，这个过程叫赋值，具体如下：

```
peopleCount=0
```

通常情况下，我们是将变量的声明和赋值同时做到的，具体如下：

```
int peopleCount=0;
```

现在我们以 Java 代码的方式复现一下公交车乘客上下车的过程，具体如图 3-4 所示。



图 3-4 使用 Java 代码复现公交车乘客上下车过程

使用 Java 代码把公交车乘客上下车的情景复现完毕后，接下来，我们输出一下变量 `peopleCount` 的值，代码如下：

```
System.out.println(peopleCount);
```

上述代码最终输出的结果是 5，这个值就是公交车乘客的数量。

3.2.2 应用案例：统计公交车停靠站

如果我们现在还要追踪记录公交车停了几站，同样按照下面的思路编写代码：

- (1) 定义一个新的整型变量 `busStopCount`，并初始化值为 0。
- (2) 每次停靠一次站台，让 `busStopCount` 的值加 1。
- (3) 最后打印 `busStopCount` 的值。

下面就正式进入记录公交车停靠站情景的过程实现。



(1) 创建一个名称为 `BusDemo` 的类，在类的 `main()` 方法中，定义变量 `busStopCount`，示例代码如下：

```
int busStopCount= 0;
```

(2) 公交车到了一站，上来 1 个人，此时，`busStopCount` 的值加 1，代码如下：

```
busStopCount= busStopCount + 1;
```

(3) 公交车又到了一站，上来 3 个人，下去 1 个人，此时，`busStopCount` 的值加 1，代码如下：

```
busStopCount= busStopCount + 1;
```

(4) 公交车又到了一站，上来 2 个人，下去 2 个人，此时，`busStopCount` 的值加 1，代码如下：

```
busStopCount= busStopCount + 1;
```

(5) 公交车又到了一站，没有乘客上车，也没有乘客下车。此时，`busStopCount` 的值加 1，代码如下：

```
busStopCount= busStopCount + 1;
```

(6) 公交车又到了一站，上来 2 个人，下去 1 个人，此时，`busStopCount` 的值加 1，代码如下：

```
busStopCount= busStopCount + 1;
```

(7) 公交车又到了二站，上来 1 个人，此时，此时，`busStopCount` 的值加 1，代码如下：

```
busStopCount= busStopCount + 2;
```

截止目前，我们已经记录好了 `peopleCount` 中存储的值，如果我们希望打印 `peopleCount` 的值，可以使用下面代码实现：

```
System.out.println(busStopCount);
```

上述记录公交车上人数变化和公交车停靠站的完整代码，具体如文件 3-1 所示。

文件 3-1 BusDemo.java

```
public class BusDemo {  
    public static void main(String[] arg){  
        int peopleCount = 0;  
        int busStopCount= 0;  
  
        peopleCount = peopleCount + 1;  
        busStopCount= busStopCount + 1;  
  
        peopleCount = peopleCount + 3;  
        peopleCount = peopleCount - 1;  
        busStopCount= busStopCount + 1;  
  
        peopleCount = peopleCount + 2;  
        peopleCount = peopleCount - 2;  
        busStopCount= busStopCount + 1;  
    }  
}
```

```
        busStopCount= busStopCount + 1;

        peopleCount = peopleCount + 2;
        peopleCount = peopleCount - 1;
        busStopCount= busStopCount + 1;

        peopleCount = peopleCount + 1;
        busStopCount= busStopCount + 2;

        System.out.println(peopleCount);
        System.out.println(busStopCount);
    }
}
```

运行上述程序，控制台的输出结果如图 3-5 所示。

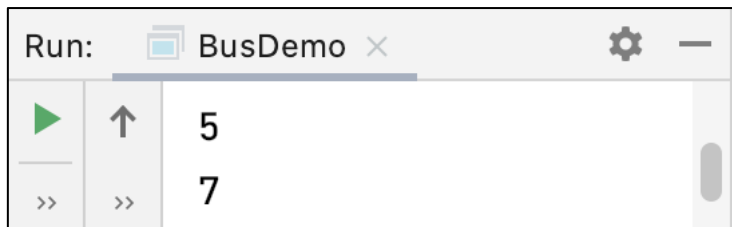


图 3-5 文件 3-1 运行结果

从上述结果可以看出，公交车上共有 5 名乘客，共经过了 7 站。

3.2.2 应用案例：统计公交车累计收入

现实生活中的数据，除了整数，还有小数。

假设出现了一个新需求，公交车公司要求统计一辆公交车的累计收入，现在我们知道的是，不同种类的公交卡，乘车价格是不一样的。

- 普通卡：0.8 元/人
- 学生卡：0.4 元/人
- 老年卡：0.0 元/人

统计公交车累计收入的实现过程具体如下：

(1) 定义一个能存储浮点类型的变量 `totalMoney`，并初始化为 0，示例代码如下：

```
float totalMoney = 0;
```

(2) 公交车到了一站，上来一名成年人，此时 `totalMoney` 值的变化情况如下：

```
totalMoney = totalMoney + 0.8;
```

(3) 公交车又到了一站，上来 3 名成年人，此时，`totalMoney` 值的变化情况如下：

```
totalMoney = totalMoney + 0.8;
totalMoney = totalMoney + 0.8;
```



```
totalMoney = totalMoney + 0.8;
```

(4) 公交车又到了一站，上来 2 名成年人，此时，totalMoney 值的变化情况如下：

```
totalMoney = totalMoney + 0.8;
```

```
totalMoney = totalMoney + 0.8;
```

(5) 公交车又到了一站，此时没有乘客上下车，此时，totalMoney 值没有变化。

(6) 公交车又到了一站，上来 2 名乘客，一名是学生，一名成年人，此时，totalMoney 值的变化情况如下：

```
totalMoney = totalMoney + 0.4;
```

```
totalMoney = totalMoney + 0.8;
```

(7) 公交车又到了一站，上来 1 名老年人，此时，totalMoney 值的变化情况如下：

```
totalMoney = totalMoney + 0.0;
```

接下来，通过一个案例完整演示上述记录公交车上人数变化、公交车停靠站以及公交车收入，具体如文件 3-2 所示。

文件 3-2 BusDemo.java

```
public class BusDemo {  
    public static void main(String[] arg){  
        // 声明一个变量，存储公交车乘客的数量  
        int peopleCount = 0;  
        // 声明一个变量，存储公交车经过了几站  
        int busStopCount= 0;  
        // 声明一个变量，存储公交车的收入  
        float totalMoney = 0.0f;  
        // 公交车过了一站，上来 1 个人  
        peopleCount = peopleCount + 1;  
        busStopCount= busStopCount + 1;  
        totalMoney = totalMoney + 0.8f;  
        // 公交车过了一站，上来 3 个人，下去 1 个人  
        peopleCount = peopleCount + 3;  
        peopleCount = peopleCount - 1;  
        busStopCount= busStopCount + 1;  
        totalMoney = totalMoney + 0.8f;  
        totalMoney = totalMoney + 0.8f;  
        totalMoney = totalMoney + 0.8f;  
        // 公交车过了一站，上来 2 个人，下去 2 个人  
        peopleCount = peopleCount + 2;  
        peopleCount = peopleCount - 2;  
        busStopCount= busStopCount + 1;  
        totalMoney = totalMoney + 0.8f;  
        totalMoney = totalMoney + 0.8f;  
        // 公交车过了一站，没有乘客上下车  
        busStopCount= busStopCount + 1;  
        // 公交车过了一站，上来 2 个人，下去 1 个人
```



```
peopleCount = peopleCount + 2;
peopleCount = peopleCount - 1;
busStopCount= busStopCount + 1;
totalMoney = totalMoney + 0.4f;
totalMoney = totalMoney + 0.8f;
// 公交车过了两站，上来 1 个人
peopleCount = peopleCount + 1;
busStopCount= busStopCount + 2;
totalMoney = totalMoney + 0.0f;
// 打印公交车上乘客的数量以及公交车经过的车站数量
System.out.println("公交车上乘客的数量: ");
System.out.println(peopleCount);
System.out.println("公交车经过的车站数量: ");
System.out.println(busStopCount);
System.out.println("公交车的收入: ");
System.out.println(totalMoney);
}
}
```

运行上述程序，控制台的输出结果如图 3-6 所示。

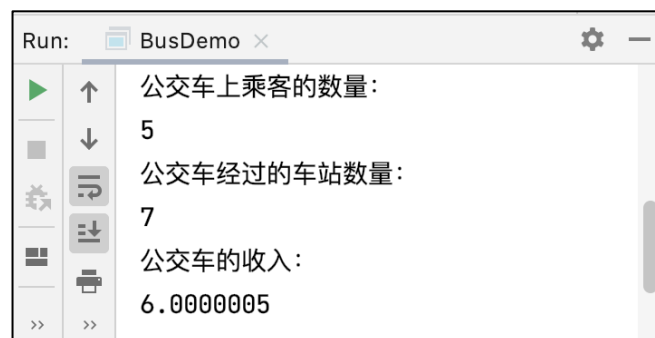


图 3-6 文件 3-2 运行结果（1）

上述控制台的输出结果是 6.0000005，这是怎么回事呢？我们当前定义的 totalMoney 数据类型是 float，该数据类型是一个单精度浮点类型，它只能精确到小数点后面的 7 位。当浮点类型的变量在进行运算时，会导致精度损失，所以多次精度损失后的小数进行累加后，最终产生了了值 0.0000005。

那怎么解决这个问题呢？

通常情况下，float 适用于对精度要求不高的数据，如果我们希望计算的结果精度更高，可以将变量 totalMoney 的类型修改为 double 类型，并将相关计算的数据将 float 修改为 double 类型，示例代码如下：

```
double totalMoney = 0.0;
totalMoney = totalMoney + 0.8;
totalMoney = totalMoney + 0.4;
totalMoney = totalMoney + 0.0;
```

此时，再次运行程序，控制台的输出结果如图 3-7 所示。

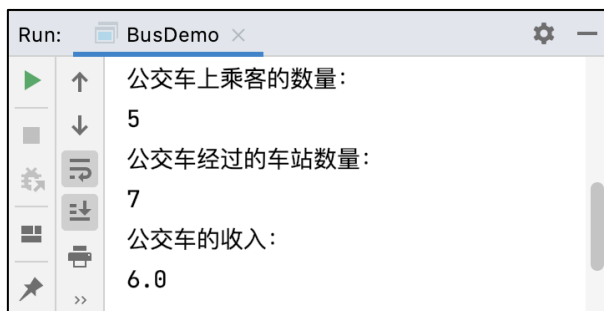


图 3-7 文件 3-2 运行结果（2）

3.3 字面量与常量

3.3.1 字面量

我们知道，计算机的程序是用来做运算的。使用编程语言对数据运算时，如果数据（例如，数字、字符、字符串等）直接写出来就可以人为理解的数据，在 Java 中叫做字面量。例如，下面的数据都属于字面量。

```
0 1 2 3 100 1000 // 整数字面量
'1' 'a' 'b' 'c' '$' // 字符字面量
"hello" "itheima" "hello@itcast.cn" // 字符串字面量
```

需要注意的是，在编写代码的时候，要看清字面量的数据类型，避免程序出错。示例代码如下：

```
int peopleCount=10;
peopleCount = peopleCount + 1; // 正确
peopleCount = peopleCount + '1'; // 错误
```

3.3.1 常量

如果我们希望程序中的某些数值，声明后不能被修改，这种数据类型就是常量。Java 中的常量使用关键字 `final` 修饰。

接下来，看一段示例代码，具体如图 3-8 所示。



```
public class HelloFinalVariabel {  
    public static void main(String[] args) {  
        final double pi=3.1415926;  
        pi=3333;  
    }  
}
```

Cannot assign a value to final variable 'pi'

[Make 'pi' not final](#) [More actions...](#)

`final double pi = 3.1415926`

图 3-8 示例代码

从图 3-8 中可以看出，当使用 final 修饰了 pi 后，如果再次对 pi 的值进行修改时，编译报错，提示不能对 pi 的值进行修改。

需要注意的是，final 关键字不仅可以修饰变量，还可以修饰方法和类，关于使用 final 修饰类和方法的使用，我们将在后面的课程中学习。

3.4 注释

注释是对程序做介绍、解释说明的文字，它能够帮我们理解程序流程、调试错误。

这里我们需要强调的是，注释是给程序员看的，不是给机器看的，即使我们在注释里面写了一首打油诗、写了一篇感悟，都不会影响程序运行。

注释可以分为单行注释、多行注释和文档注释，具体格式如下：

1. 单行注释，格式如下：

```
// 注释文字
```

2. 多行注释，格式如下：

```
/* 注释文字 */
```

3. 文档注释，格式如下：

```
/** 注释文字 */
```

其中，文档注释通常是对一些新开发的框架或者工具所做的说明，这些说明可以生成对应的文档手册供使用者阅读。现阶段我们只对单行注释和多行注释进行介绍。

下面我们在 BusDemo 文件中对之前编写的代码添加单行注释和多行注释，具体如下：

```
/*  
多行注释  
这是一个公交车运行的模拟程序  
公交车每经过一站，都记录人数和停靠站的数量  
*/  
public class BusDemo {  
    public static void main(String[] arg){
```



```
// 声明一个变量，存储公交车乘客的数量
int peopleCount = 0;
// 声明一个变量，存储公交车经过了几站
int busStopCount= 0;
// 公交车过了一站，上来 1 个人
peopleCount = peopleCount + 1;
busStopCount= busStopCount + 1;
// 公交车过了一站，上来 3 个人，下去 1 个人
peopleCount = peopleCount + 3;
peopleCount = peopleCount - 1;
busStopCount= busStopCount + 1;
// 公交车过了一站，上来 2 个人，下去 2 个人
peopleCount = peopleCount + 2;
peopleCount = peopleCount - 2;
busStopCount= busStopCount + 1;
// 公交车过了一站，没有乘客上下车
busStopCount= busStopCount + 1;
// 公交车过了一站，上来 2 个人，下去 1 个人
peopleCount = peopleCount + 2;
peopleCount = peopleCount - 1;
busStopCount= busStopCount + 1;
// 公交车过了两站，上来 1 个人
peopleCount = peopleCount + 1;
busStopCount= busStopCount + 2;
// 打印公交车上乘客的数量以及公交车经过的车站数量
System.out.println("公交车上乘客的数量：");
System.out.println(peopleCount);
System.out.println("公交车经过的车站数量：");
System.out.println(busStopCount);
}
}
```

注意：

文档注释与多行注释相比，从格式上来看，多了一个符号*。从功能上来看，文档注释的内容，后期可以自动生成一个文档，如果在协同开发或者 SDK 开发时，文档注释非常有用。

3.5 标识符

首先我们要明白一个概念——名字只是一个标志。



例如，现在有两个人，父亲叫张翠山，儿子叫张无忌，我们会通过名字区分具体是哪个人，如图 3-9 所示。



图 3-9 使用姓名区分具体人

同理，变量也有对应的名字，我们只需通过名字找到对应的变量，就可以访问其内存空间。Java 中变量的名字，它是一个字符序列，它有一个专业的术语叫标识符。

标识符可以由英文大小写字母、数字、下划线（_）和美元符号（\$）组成，其定义规则如下：

- （1）不能以数字开头。
- （2）不能使用关键字，例如，int、double 等。
- （3）严格区分大小写。

除此之外，标识符还有一些命名规范，例如，驼峰命名，尽量做到见名知意。

这里需要注意的是，定义规则是必须遵守的，而命名规范则是建议性的，不作强制要求。

接下来，看一些变量的命名，具体如下：

```
int zhangsanfengAge = 12;    // 合法标识符
int 8zhangsanfengAge = 12;  // 非法标识符
```

3.6 关键字

其实大家在前面的学习中，已经遇到不少关键字了，比如，class、public、static、void 等。在 Java 中，关键字是被 Java 语言赋予特定含义的单词，通常具有以下特点：

- 组成关键字的单词全部小写。
- 常见的代码编辑器，对关键字有特殊的颜色标记。

Java 常见关键字，如表 3-1 所示。

表 3-1 Java 常见关键字

定义数据类型	class	interface	enum	char
	byte	short	int	long
	float	double	boolean	void
定义数据类型值	true	false	null	
定义流程控制	if	else	switch	case
	default	for	while	do



	break	continue	return	
定义访问权限修饰	public	protected	private	
定义类、函数、变量修饰符	abstract	final	static	synchronized
类与类之间关系	extends	implements		
定义建立实例、引用实例、判断实例	new	this	super	instanceof
异常处理	try	catch	finally	throw
	throws			
用于包	package	import		
其他	native	strictfp	transient	volatile
	assert			

需要注意的是，这些关键字，后续我们都会讲到，大家无须刻意背诵所有的关键字，这里大家只需要有个大致印象即可。

3.7 运算符

3.7.1 算术运算符

算术运算符就是进行算术运算的符号。

Java 中最常见的算术运算符，如表 3-2 所示。

表 3-2 常见算术运算符

类型	运算符
加法运算符	+
减法运算符	-
乘法运算符	*
除法运算符	/
取模（取余）运算符	%
自增运算	++
自减运算	--

这里，以 BusDemo 类中的代码为例，下面代码的效果是等同的。

示例代码一：

```
peopleCount++;
```

等同于

```
peopleCount = peopleCount + 1;
```

示例代码二：

```
totalMoney = totalMoney + 0.8;
```



```
totalMoney = totalMoney + 0.8;
```

```
totalMoney = totalMoney + 0.8;
```

等同于

```
totalMoney = totalMoney + 0.8*3;
```

如果大家想练习算术运算符的使用，可以在 IDEA 中，选择【Tools】→【JShell Console】可进入 JShell Console 窗口，该窗口类似一个灵感记录本，可以方便验证 Java 代码功能的可用性，能够避免新建类，编写 main()方法这个繁琐的过程。

在 JShell Console 中编写代码以及运行效果如图 3-10 所示。

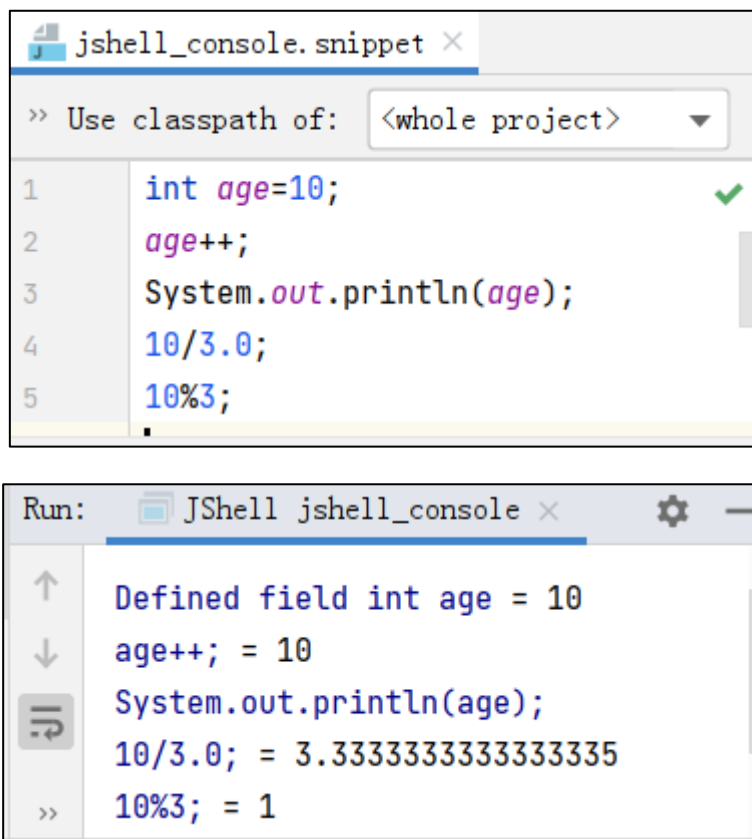


图 3-10 在 JShell Console 中编写代码及运行效果

3.7.2 赋值运算符

赋值运算符分为基本赋值运算符和扩展赋值运算符这两类，其中：

(1) 基本赋值运算符：=

(2) 扩展赋值运算符：

- +=（先加后赋值）
- -=（先减后赋值）
- /=（先除后赋值）
- *=（先乘后赋值）
- %=（先取模后赋值）



由于基本赋值运算符比较简单，我们之前已经使用过很多次了，这里我们对 BusDemo 类中部分代码进行修改，示例代码如下：

```
peopleCount += 3; // 等同于 peopleCount = peopleCount + 3;
peopleCount -= 1; // 等同于 peopleCount = peopleCount - 1;
```

由于扩展赋值运算符的使用基本类似，这里不作过多演示。不过给大家留一个小作业，使用扩展赋值运算符，对之前的公交车案例重新优化。

3.7.3 比较运算符

前面我们使用 if 语句做条件判断时，除了可以直接赋值 true 或 false 之外，还可以通过比较运算实现判断。比较运算符用来比较两个表达式的结果，常用的比较运算符如表 3-3 所示。

表 3-3 常见的比较运算符

运算符	含义	示例
<	小于	3<5, 结果为 true
>	大于	3>5, 结果为 false
>=	大于等于	3>=5, 结果为 false
<=	小于等于	3<=5, 结果为 true
==	等于	3==5, 结果为 false
!=	不等于	3!=5, 结果为 true

除了可以直接比较两个数字之外，我们还可以比较变量的值，例如下面的代码。

```
int x = 10
x>=10    // 结果为 true
x==9     // 结果为 false
```

这里，强调一个初学者非常容易混淆的概念，“=”和“==”，其中，“=”是赋值操作，“==”是逻辑操作，大家一定要区理解。

```
int x = 10;
x==9    // 逻辑操作，结果为 false
x=9;    // 赋值操作，x 的值为 9
```

，比较运算符在实际开发中的应用场景很多，例如，无人驾驶汽车是依靠激光扫描躲避障碍物的，通过判断障碍物的距离，从而决定车辆采取什么样的行动。

3.7.4 逻辑运算符

逻辑运算符用来判断“并且”“或者”“除非”等逻辑关系，它的两端一般连接值为布尔类型的关系表达式，运算结果为布尔值 true 或者 false。

常见的逻辑运算符如表 3-4 所示

表 3-4 常见的逻辑运算符



运算符	含义	示例
&&	逻辑与，并且	3<5, 结果为 true
	逻辑或，或者	3>5, 结果为 false
!	逻辑非，表示否定	3>=5, 结果为 false

为了大家更好理解逻辑运算符的使用，下面我们通过一个“韦小宝找老婆”的故事，加强对三种逻辑关系的理解。

韦小宝最近在相亲找老婆，最初他找老婆的标准既要长得好看，又要身材好。（如图 3-10 所示）。可是，找了几个月后，韦小宝发现找老婆太难了，他就降低了标准，要么长得好看，要么身材好。（如图 3-11 所示）。但是继续找了一段时间后，发现还是太难了，他再次降低了找老婆的标准，只要不是男的，都是可以的，最终找到的老婆如图 3-12 所示。



图 3-10 既好看身材又好



图 3-11 要么好看，要么身材好



图 3-12 最终找到的老婆

上述故事中，韦小宝每次找老婆的标准可以使用逻辑运算进行表示。

- (1) “长得好看，身材又好” 对应的逻辑关系是逻辑与&&
- (2) “长得好看或身材好。”对应的逻辑关系是逻辑或||
- (3) 只要不是男的。对应的逻辑关系是逻辑非!。

3.7.5 逻辑运算符优先级

我们知道，加减乘除优先级是先乘除后加减，例如下面的代码：

```
int a = 2
int b = a + 3 * a    // b的值是 8
```

同样的，逻辑运算也存在优先级，其优先级具体如下：

NOT>AND>OR

下面我们通过一个示例代码，看一下逻辑运算符的优先级。

```
!true||false&&true
```

根据优先级 NOT>AND>OR，上述代码的运算顺序如下：

- (1) 计算!true，其结果为 false。
- (2) 计算 false&&true，其结果为 false
- (3) 计算 (!true) || (false&&true)，也就是 false||false, 其结果为 false。

在实际开发中，建议大家不要在一行代码中出现复杂的逻辑运算。因为复杂的逻辑运算，其运算量会比较大，这样不仅不会提升代码执行速度，而且还会降低代码的可读性。

3.8 数据类型

3.8.1 基本数据类型

在 Java 中，有八种常用的基本数据类型，分别是 byte、short、int、float、double、long、char、boolean。

可能有的同学会觉得疑惑，每种数据类型都是一种盒子，为什么要设计这么多种盒子呢？我们先来看一张图，如图 3-13 所示。



图 3-13 衣柜

图 3-13 所示的是生活中的衣柜，这些衣柜被划分为不同大小的格子，不同的格子具有不同的功能，大格子适合大一些的衣服，小格子适合小一些的衣物。

我们知道计算机使用高低电压表示数据。在内存中，高电压表示 1，低电压表示 0。在硬盘中，使用硬盘有磁表示 1，硬盘无磁表示 0。

在计算机中存储的数据越多，消耗的资源就越多，为了能更合理利用计算机资源，Java

设计了 8 种基本的数据类型，每种数据类型所占用的存储空间是不一样的。关于这八种数据类型的占对应的存储空间具体如表 3-5 所示。

表 3-5 八种基本数据类型及其对应的存储空间

类型	占用空间	取值范围
byte	1 个字节	$-2^7 \sim 2^7-1$
short	2 个字节	$-2^{15} \sim 2^{15}-1$
int	4 个字节	$-2^{31} \sim 2^{31}-1$
long	8 个字节	$-2^{63} \sim 2^{63}-1$
float	4 个字节	$3.4 \times 10^{-38} \sim 3.4 \times 10^{38}$
double	8 个字节	$1.7 \times 10^{-308} \sim 1.7 \times 10^{308}$
char	2 个字节	$0 \sim 2^{16}-1$
boolean	1 个字节	true、false

这里需要告诉大家的是，关于上述基本数据类型所占的空间及取值范围，大家不用刻意去记忆，通常情况下，如果要存储整数，我们会选择 int，如果 int 空间不够，那么会考虑选择使用 long。如果要存储小数，我们可以使用 float 和 double。如果要存储字符，可以选择 char。如果要判断条件真假，可以使用 boolean。

3.8.2 复杂数据类型

学到这里，可能有的同学会有疑问，前面我们已经学会了创建一个 String 类型的盒子，并在盒子里面存储字符串数据，可是为什么学习八种数据类型的时候，没有看到 String 呢？

原因：String 是一个复杂的数据类型。

基本数据类型和复杂数据类型相比，复杂的数据类型，不单可以存储数据，还会拥有一些对数据处理的能力（这里的能力指的是方法）。

这里，我们看一张图来描述一下简单数据类型和复杂数据类型的比较，如图 3-14 所示。

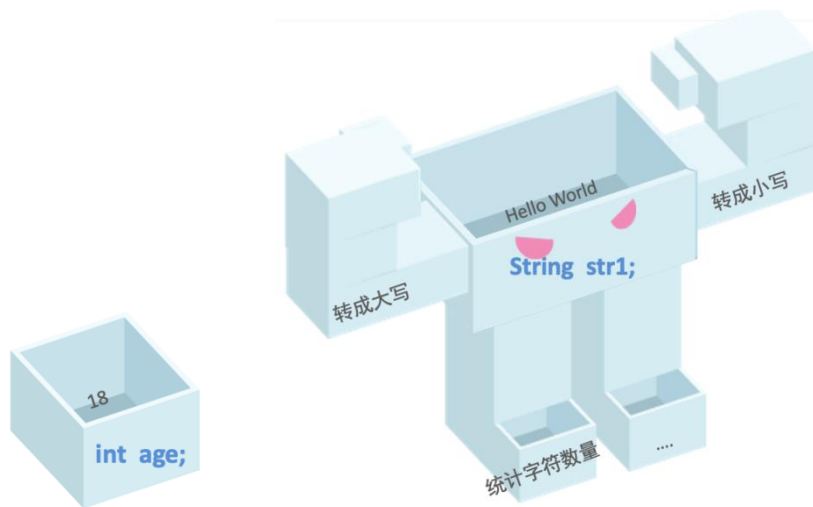


图 3-14 简单数据类型与复杂数据类型的比较



在图 3-14 中，左侧的小盒子表示创建了一个整型变量 `age`，它只能存储整型数字，右侧表示创建了一个 `String` 类型的变量 `str1`，它不仅可以存储“Hello World”，还拥有其他能力（方法），比如转成大写、转成小写、统计字符数量等，这体现的也是简单数据类型与复杂数据类型的区别。

接下来，我们通过一个案例来演示简单数据类型和复杂数据类型的特点，具体如文件 3-3 所示。

文件 3-3 HelloDataType.java

```
/*
研究一下关于数据类型的内容
*/
public class HelloDataType {
    public static void main(String[] args) {
        int age = 18;
        System.out.println(age);
        String helloStr = "Hello World";
        // 将 helloStr 存储的字符串转为小写
        System.out.println(helloStr.toLowerCase());
        // 将 helloStr 存储的字符串转为大写
        System.out.println(helloStr.toUpperCase());
        // 输出字符串 helloStr 的长度
        System.out.println(helloStr.length());
    }
}
```

运行上述程序，控制台的输出结果如图 3-15 所示。

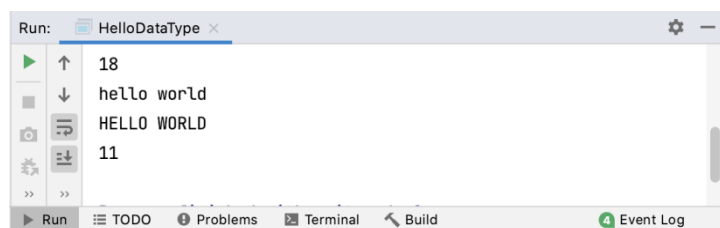


图 3-15 文件 3-3 运行结果

注意：

与简单数据类型相比，复杂数据类型的功能更加强大一些，但是，与此同时，复杂数据类型也会带来一定的资源开销。

图 3-16 和图 3-17 描述了 IDEA 中简单数据类型和复杂数据类型的区别。



图 3-16 简单数据类型

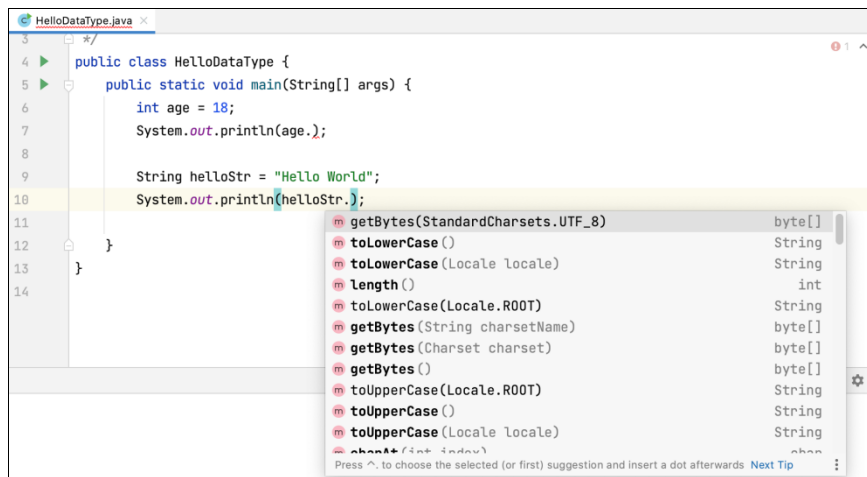


图 3-17 复杂数据类型

通过图 3-16 和图 3-17 的比较，发现复杂数据类型拥有很多 **m** 标注的内容，这些 **m** 就是复杂数据类型拥有的方法（或者能力）。

3.8.3 基本数据类型的装箱类

这里我们引入了一个高端的概念——装箱。

装箱，英文名 **boxing**，它可以让基本的数据类型，拥有一些对数据处理的能力（方法）。

我们知道，基本数据类型只能存储数据，那我们能否让基本数据类型“变身”，拥有像图 3-18 更强大的能力呢？

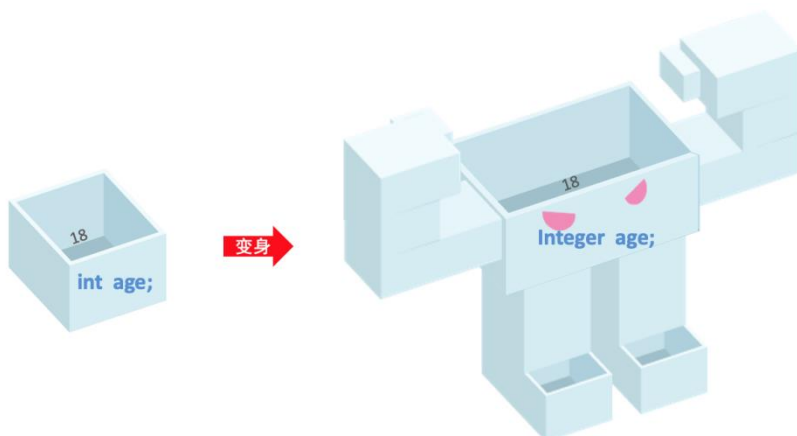


图 3-18 基本数据类型的“变身”

其实，每种基本数据类型都对应一种装箱的“盒子”，如表 3-6 所示。

表 3-6 基本数据类型对应的装箱“盒子”

基本数据类型	装箱
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Char
boolean	Boolean

下面，我们分别定义一个 int 类型和 Integer 类型的变量，代码如下：

```
int age = 18;  
Integer age2 = 18;
```

上述代码中，age2 相当于 age 的升级版。Integer 不仅提供了大量的方法，还提供了很多常量，这些我们都可以在 IDEA 中查看，如图 3-19 所示。

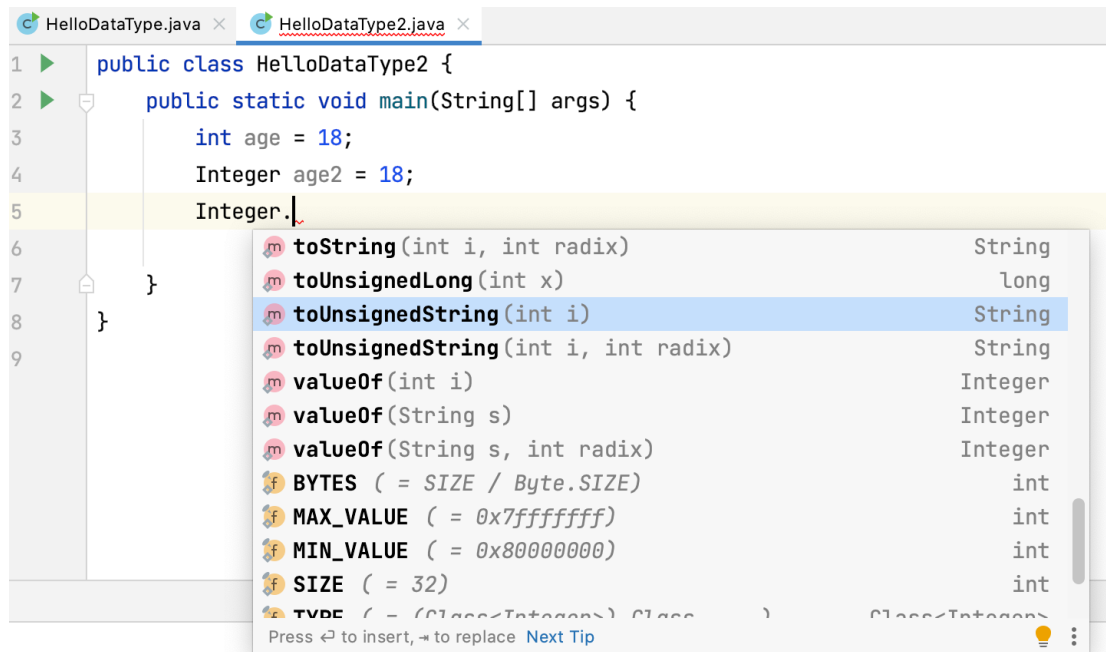


图 3-19 在 IDEA 中查看 Integer

在图 3-19 中，MAX_VALUE 代表了 Integer 的最大值，MIN_VALUE 代表了 Integer 的最小值，SIZE 表示 Integer 占用的位数，BYTE 表示 Integer 占用的字节。

由于每一种数据类型的最大值、最小值、占据 byte 的大小，占据的位数，都是复杂数据类型常用的数据，下面我们将这些装箱类的数据打印出来，具体代码如下：

```
public class HelloDataType2 {  
    public static void main(String[] args) {  
        System.out.println("Integer,最大值,最小值,bytes 的大小,bit 的大小");  
        System.out.println(Integer.MAX_VALUE);  
        System.out.println(Integer.MIN_VALUE);  
        System.out.println(Integer.BYTES);  
        System.out.println(Integer.SIZE);  
  
        System.out.println("Char,最大值,最小值,bytes 的大小,bit 的大小");  
        System.out.println(Character.MAX_VALUE);  
        System.out.println(Character.MIN_VALUE);  
        System.out.println(Character.BYTES);  
        System.out.println(Character.SIZE);  
  
        System.out.println("Short,最大值,最小值,bytes 的大小,bit 的大小");  
        System.out.println(Short.MAX_VALUE);  
        System.out.println(Short.MIN_VALUE);  
        System.out.println(Short.BYTES);  
        System.out.println(Short.SIZE);  
  
        System.out.println("Long,最大值,最小值,bytes 的大小,bit 的大小");
```




```
System.out.println(Long.MAX_VALUE);
System.out.println(Long.MIN_VALUE);
System.out.println(Long.BYTES);
System.out.println(Long.SIZE);

System.out.println("Float, 最大值, 最小值, bytes 的大小, bit 的大小");
System.out.println(Float.MAX_VALUE);
System.out.println(Float.MIN_VALUE);
System.out.println(Float.BYTES);
System.out.println(Float.SIZE);

System.out.println("Double, 最大值, 最小值, bytes 的大小, bit 的大小");
System.out.println(Double.MAX_VALUE);
System.out.println(Double.MIN_VALUE);
System.out.println(Double.BYTES);
System.out.println(Double.SIZE);

System.out.println("Byte, 最大值, 最小值, bytes 的大小, bit 的大小");
System.out.println(Byte.MAX_VALUE);
System.out.println(Byte.MIN_VALUE);
System.out.println(Byte.BYTES);
System.out.println(Byte.SIZE);
}
}
```

3.8.4 数据类型转换

我们先看一个例子，如图 3-20 所示。



图 3-20 丰巢快递柜

图 3-20 展示的是丰巢的快递柜，它有三种规格的柜子，具体如下：

- 小格口：长 45.5CM, 宽 34CM, 高 8CM, 限重 1KG。
- 中格口：长 45.4CM, 宽 34CM, 高 19CM, 限重 3KG。
- 大格口：长 45.5CM, 宽 34CM, 高 29CM, 限重 5KG。



明确了快递柜的规格后，看一下快递存放的原则，具体如下：

- 大格口放大快递
- 小格口放小快递
- 如果把小快递放入大格口没问题，但是空间有点浪费。
- 如果把大快递放入小格口，那就有很大问题。

之所以举上面的案例，是因为在 Java 编程中也会遇到类似的问题。

在 Java 编程过程中，我们可能会遇到不同类型的数据，这些不同类型的数据之间可能会进行运算，而这些数据取值范围不同，存储方式不同，直接进行运算可能会造成数据损失。所以需要将一种类型转换为另外一种类型再进行运算。

数据类型转换的情况，可以分为两种，具体如下：

(1) 自动（隐式）类型转换，例如，四则运算时，小类型会自动提升为大类型，运算结果是大类型。

(2) 强制（显式）类型转换。转换格式为：（新类型）原类型数据

需要注意的是，当且仅当大类型数据可以转换为小类型数据时，才进行转换，否则会造成精度损失或数据错误。

接下来，通过示例代码进行演示，具体如下：

```
public class HelloDataCast {  
    public static void main(String[] args) {  
        int x = 7;  
        int y = 2;  
        double div = x / y;  
        System.out.println(div);  
    }  
}
```

程序运行结果如图 3-21 所示。

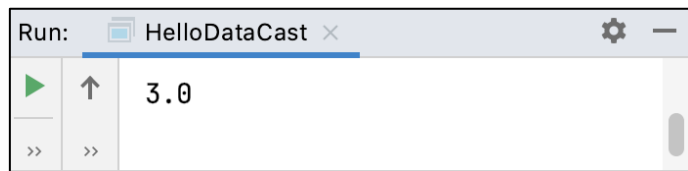


图 3-21 运行结果

从图 3-21 可以看出，程序计算的结果是 3.0，说明在执行 x/y 运算时，运算的结果 3 被自动转为小数类型，也就是说，int 类型被自动转为 double 类型，这个过程就是数据类型的隐式转换。

接下来，我们再看一个显式转换的情况，具体代码如下：

```
public class HelloDataCast {  
    public static void main(String[] args) {  
        int x = 7;  
        int y = 2;  
        double div2 = (double) x / y;  
        System.out.println(div2);  
    }  
}
```

程序的运行结果如图 3-22 所示。

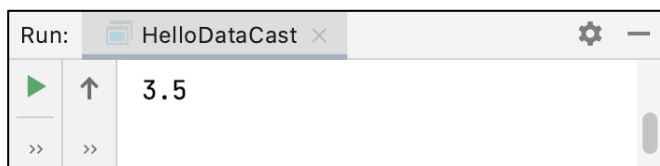


图 3-22 运行结果

从图 3-22 可以看出，程序的运行结果是 3.5，这是因为在执行 x/y 运算时，程序会先将 x 转为 `double` 类型，然后再与 y 进行除法运算，最终程序得到的结果就是 3.5。

需要注意的是，在进行强制类型转换时，一定要考虑数据类型的范围大小，因为取值范围大的数据类型是可以存放取值范围小的数据类型。反之，取值范围小的数据类型，是无法存放取值范围大的数据类型，如果通过强制类型转换，会造成精度丢失或者数据错误。

接下来，看一个示例代码，具体如下：

```
public class HelloDataCast {  
    public static void main(String[] args) {  
        long a = 3243232323233L;  
        int b = (int) a;  
        System.out.println(b);  
    }  
}
```

程序运行结果如图 3-23 所示。

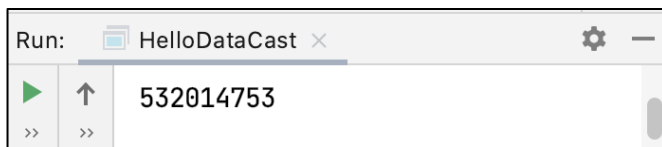


图 3-23 运行结果

从图 3-23 可以看出，输出的数据发生了错误，这是因为要存储的数据太大，而数据类型范围太小导致的。

3.9 字符串

3.9.1 字符与字符串

键盘上，输入法打出来的每一个内容都是字符，如图 3-24 所示。



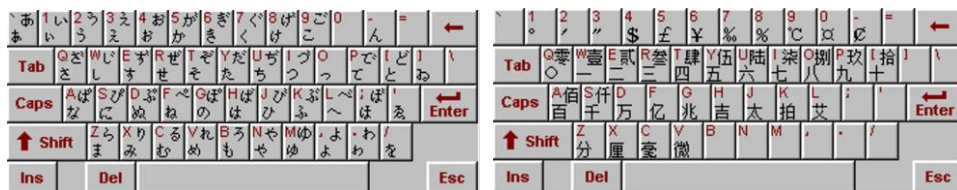


图 3-24 键盘中的字符

在图 3-24 所示的键盘中，像英文字符、标点符号、日文字符、中文字符等都可以看做是一个字符。

如果把一系列的字符串在一起，就变成了“字符串”，示例代码如下：

```
String driverName = "李建国"
String demo = "HELLOWORLD"
String demo2 = "#¥! *てから 5лнийтэмдэ"
```

Java 中的字符串变量有什么用呢？实际开发中，很多信息都是需要使用字符串存储的，比如，姓名、邮箱、英文句子等都可以使用字符串变量存储。

那接下来面临的一个问题，就是字符串如何声明、字符串如何声明？

其实，字符或者字符串变量的声明，和前面学习的整型变量的声明方式是类似的，其中，字符变量使用 char 表示，字符串使用 String 表示。

接下来，我们以代码的方式，演示字符和字符串是如何定义以及使用的。

在 IDEA 中创建一个类，名称为 HelloString，并在 main() 方法中分别定义字符变量和字符串变量，具体如下：

```
/*
 * 当前示例代码演示 char 字符和 String 字符串的初始化
 */
public class HelloString {
    public static void main(String[] args) {
        char a = 'a'; // 声明一个盒子 a, 里面的数据是 char 类型，初始值是字符'a'
        System.out.println(a);
        String str = "你好，世界";
        System.out.println(str);
    }
}
```

运行上述程序，控制台的输出结果如图 3-25 所示。

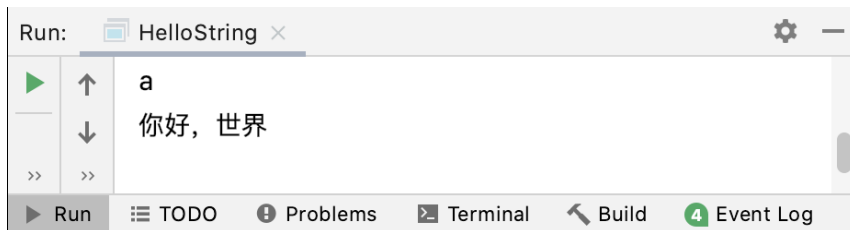


图 3-25 运行结果



3.9.2 字符串拼接运算

大家可能在网上见过类似图 3-26 所示的软件。



图 3-26 自动写作软件

图 3-26 所示的自动写作软件是一个在线写作软件，我们可以把写作文的模式，理解为字符串的拼接。本节课，我们将带领大家开发一个自动写作软件。

进入 21 世纪，比较讲究的就是“套路”。其实写作文也是有套路的，我们看一下大家都是怎么写作文的，如图 3-27 所示。



图 3-27 写作文的“套路”

在图 3-27 中，很多作文的范文都是“... 首先映入眼帘... 金光闪闪的大字...”这样的“套路”，那如何将这些“套路”转换为 Java 代码呢？使用字符串拼接可以实现。

Java 可以使用“+”对字符串进行拼接。接下来，我们通过一段代码来演示，具体如下：

```
public class HelloDiary {  
    public static void main(String[] args) {
```

```
String name = "深圳市儿童公园";  
String diary = "今天天气很好，麻麻带我去" + name  
              + "玩,首先引入眼帘的是, " + "\"" + name  
              + "\"" + name.length() + "个金光闪闪的大字";  
System.out.println(diary);  
}  
}
```

运行上述程序，控制台的输出结果如图 3-28 所示。

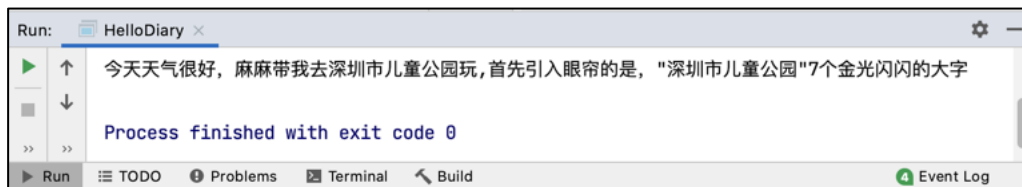


图 3-28 运行结果

需要注意的是，在 Java 中使用双引号表示字符串时，双引号会按照就近原则进行配对。所以如果我们希望字符串本身的内容就是引号，可以使用转义字符\"表示引号。

第 4 章 条件语句

学习目标

- ◆ 掌握 if 语句的语法，会使用 if 语句处理不同的业务场景
- ◆ 掌握嵌套 if 语句的使用，能够使用嵌套 if 语句对不同的业务

场景进行判断处理

- ◆ 掌握 switch 语句的语法，能够使用 switch 语句处理不同的业

务场景

- ◆ 掌握三目运算符的使用，可以准确使用三目运算符判断条件

现实生活中，大家在 12306 网站购票时需要先验证身份，验证通过后可进入购票页面，验证失败则需重新验证。在代码编写工作中，大家可以使用条件语句为程序增设条件，使程序产生分支，进而有选择地执行不同的语句。接下来，本章将针对条件语句的相关知识进行详细讲解。



4.1 if 语句

默认情况下，Java 程序代码都是从上至下按顺序执行的，在执行过程中，没有任何条件判断和流程跳转。但是，现实生活中，很多事物都是需要满足条件的。例如，下面的这个故事。

小红对小明说：“小明，如果这次考试全班第一，我就做你女朋友。”

班级其他同学，听到后都感动了，为了成全小明和小红，他们约定一起交白卷。

小明的眼睛里泛着泪花，他看了一眼体重达二百多公斤的小红，默默的吞下了试卷。

小红微微一笑，在自己的试卷上写下了小明的名字。

故事讲完了，如果我们使用 Java 代码实现，代码是这样的：

```
boolean isFirst = true;
if (isFirst){
    // 这段代码只有小明考试第一名，isFirst 是 true 的时候才执行
    System.out.println("小红变成了小明的女朋友!");
}
```

上述代码中，isFirst 就是一个判断条件，只有 isFirst 的值为 true 时，才会执行大括号 {} 中的代码。

当然，现实生活中还有很多更复杂的条件判断场景，例如，无人驾驶汽车，就是通过一系列的条件判断决定车辆的行驶状态。

如果根据交通灯是红灯还是绿灯，决定车辆是前进还是停止，示例代码如下：

```
boolean isLightGreen = true;
if (isLightGreen){
    System.out.println("gogogo!");
}else {
    System.out.println("stop!");
}
```

如果根据交通灯是红灯、黄灯还是绿灯，判断车辆前进、减速还是停止，示例代码如下：

```
boolean isLightGreen = true;
boolean isLightYellow = false;
if (isLightGreen){
    System.out.println("gogogo!");
}else if (isLightYellow){
    System.out.println("slow!");
}else {
    System.out.println("stop!");
}
```

4.2 嵌套 if 语句

在 if 语句中还有 if 语句，就叫做**嵌套 if 语句**，例如下面的结构：

```
if (条件 1) {
```




```
        if (条件2) {  
            if (条件3) {  
                // do something  
            }  
        }  
    }  
}
```

例如，使用嵌套 if 输出 x、y、z 的值，示例代码如下：

```
public class Test {  
    public static void main(String[] args) {  
        int x = 1;  
        int y = 2;  
        int z = 3;  
        if (x==1){  
            if (y==2){  
                if (z==3){  
                    System.out.println("x=1,y=2,z=3");  
                }  
            }  
        }  
    }  
}
```

那么，你知道嵌套 if 语句有哪些应用场景吗？下面我们看一个日常购物的场景。我们在网站购物后，经常会遇到如图 4-1 所示的评分环节。

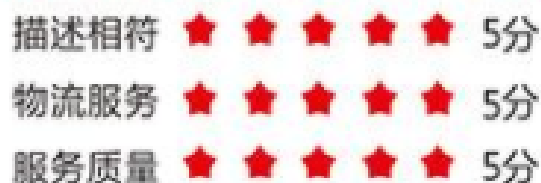


图 4-1 购物评分

要求是这样的：

- (1) 如果评分在 1~5 分之间，输出“谢谢评价”，否则提示“请输入合法的评分 1~5 分”。
- (2) 如果评分低于 2 分，提示“请问您评分低的原因是什么？”

接下来，通过一个案例实现上述需求，具体代码如下：

```
public class Test {  
    public static void main(String[] args) {  
        int rating = 3;  
        if (rating>0 && rating<=5){  
            System.out.println("谢谢评价");  
            if (rating<=2){  
                System.out.println("请问您评分低的原因是什么？");  
            }  
        }  
        if (rating<=0){
```



```
        System.out.println("请输入合法的评分，1~5 分");
    }
    if (rating>5){
        System.out.println("请输入合法的评分，1~5 分");
    }
}
}
```

上述代码中，当变量 rating 的值为 3，程序会输出“谢谢评价”；当变量 rating 的值为 9，程序会输出“请输入合法的评分，1~5 分”；当变量 rating 的值为 2 时，程序会输出“请问您评分低的原因是什么？”

4.3 if-else 语句

条件语句还有一种形式是 if-else，语法格式如下：

```
if (expression) {
    // 代码 1
} else {
    // 代码 2
}
```

上述语法格式中，如果判断条件 expression 为 true，则执行代码 1，否则执行代码 2。下面，我们使用 if-else 语句对 4.2 小节的代码进行改写，修改后的代码具体如下：

```
public class HelloWorld {
    public static void main(String[] args) {
        int rating = 3;
        if (rating > 0 && rating <= 5) {
            System.out.println("谢谢评价");
            if (rating <= 2) {
                System.out.println("请问您评分低的原因是什么? ");
            }
        } else {
            System.out.println("请输入合法的评分，1~5 分");
        }
    }
}
```

上述代码中，如果 rating 的值位于 1~5 之间，那么会执行 if 包含的代码，否则的话，就会执行 else 包含的代码。相比 4.2 小节使用嵌套 if 语句依次判断各个情况，使用 if-else 语句编写的代码更加简洁。

当然，如果有更多的判断条件，可以使用 if-else if-else 组合语句，其语法格式如下：

```
if (expression0) {
    // 代码 1
} else if (expression1) {
```



```
// 代码 2  
...  
} else if (expressionN) {  
    // 代码 n  
}
```

例如，根据你的现金情况，选择购买不同配置的电脑，示例代码如下：

```
int money = ...; //你的现金  
if (money < 1000) {  
    System.out.println("买个上网本");  
} else if (money < 8000) {  
    System.out.println("买个高配台式机");  
} else if (money < 100000) {  
    System.out.println("买个服务器");  
} else {  
    System.out.println("买个数据中心");  
}
```

上述代码中，由于程序是从上至下执行的，所以 if-else if-else 语句的判断顺序是：

- (1) 先判断现金金额是否小于 1000，如果小于 1000，那么可以买个上网本。
- (2) 如果金额不小于 1000，那么会判断金额是否小于 8000，也就是说，判断金额是否在 1000~8000 之间，如果是，则可以买个高配台式机。
- (3) 如果金额不小于 8000，那么会判断金额是否小于 10000，也就是说，判断金额是否在 8000~10000 之间，如果是，则可以买个服务器。
- (4) 如果金额不小于 10000，也就是说，如果金额大于等于 10000，那么可以买个数据中心。

4.4 switch 语句

4.4.1 switch 语句语法

switch 是一个常用的流程控制语句，它通过特定值与其他值的比较，按照比较结果选择执行语句。

switch 语句格式，具体如下：

```
switch (表达式) {  
    case 取值 1:  
        语句块 1;  
        break;  
    ...  
    case 取值 n:  
        语句块 n;  
        break;  
    default:
```



```
    语句块 n+1;  
    break;  
}
```

上述格式中，switch 语句将表达式的值与每个 case 中的取值进行匹配，如果找到了匹配的值，则执行对应 case 后面的语句块；如果没找到任何匹配的值，则执行 default 后的语句。另外，switch 语句中的 break，它的作用是终止 case，并跳出 switch 语句。

比如，我们要把 1~3 对应的星期几的英文单词打印出来，示例代码如下：

```
public class Test {  
    public static void main(String[] args) {  
        int day = 3;  
        switch (day) {  
            case 1:  
                System.out.println("Monday");  
                break;  
            case 2:  
                System.out.println("Tuesday");  
                break;  
            case 3:  
                System.out.println("Wednesday");  
                break;  
            default:  
                System.out.println("i don't know");  
        }  
    }  
}
```

4.4.2 应用案例—作文自动生成器

接下来，我们再回顾一下之前编写的一个作文自动生成器案例，有这么一段话：

妈妈带我去中国银行取钱，

首先映入眼帘的是“中国银行” 4 个金光闪闪的鎏金大字

上面的句子有 1 个需要优化的地方，就是需要将数字 1, 2, 3... 等等转换为中文汉字，转换规则如下：

```
1  → 一  
2  → 二  
3  → 三  
4  → 四  
5  → 五  
6  → 六  
7+ → 很多
```

如果要想实现上述转换，既可以使用 if 语句实现，也可以使用 switch 语句实现。



1. 使用 if 语句实现的代码如下：

```
String name = "人民公园";
int len = name.length();
String count = "";
if (len == 1) {
    count = "一";
} else if (len == 2) {
    count = "二";
} else if (len == 3) {
    count = "三";
} else if (len == 4) {
    count = "四";
} else if (len == 5) {
    count = "五";
} else if (len == 6) {
    count = "六";
} else {
    count = "很多";
}

System.out.println("妈妈带我去" + "\"" + name + "\"" + "玩，首先映入眼帘的" + "\"" + name + "\"" + count + "个鎏金大字");
```

2. 使用 switch 语句实现的代码如下：

```
String name = "人民公园";
int len = name.length();
String count = "";
switch (len) {
    case 1:
        count = "一";
        break;
    case 2:
        count = "二";
        break;
    case 3:
        count = "三";
        break;
    case 4:
        count = "四";
        break;
    case 5:
        count = "五";
        break;
    case 6:
        count = "六";
        break;
    default:
        count = "很多";
}
```



```
        count = "六";  
        break;  
    default:  
        count = "很多";  
}
```

4.4.3 switch 比较字符和字符串

switch 不仅可以比较数值，还可以比较字符和字符串，示例代码如下：

(1) switch 比较字符。

```
char day='一';  
switch (day){  
    case '一':  
        System.out.println("Monday");  
        break;  
    case '二':  
        System.out.println("Tuesday");  
        break;  
    case '三':  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("i don't know");  
}
```

(2) switch 比较字符串。

```
String day="星期一";  
switch (day){  
    case "星期一":  
        System.out.println("Monday");  
        break;  
    case "星期二":  
        System.out.println("Tuesday");  
        break;  
    case "星期三":  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("i don't know");  
}
```

需要注意的是，switch 语句支持字符串这一特性，是从 JDK 8 开始的，如果大家使用的 JDK 版本低于 JDK 8，那么是无法使用 switch 语句判断字符串值的。



4.5 三目运算符

三元运算符又叫“三目运算符”，它是由三部分组成的，格式如下：

(关系表达式)?表达式 1:表达式 2;

上述格式的运算流程如下：

- 如果关系运算符表达式结果为 true, 那么运算结果是表达式 1。
- 如果关系运算符表达式结果为 false, 那么运算结果是表达式 2。

例如，下面的这两段代码是等价的。

```
int x = 0;
if (x > 0) {
    x = 0;
} else {
    x = 1;
}
```

等价于

```
x=x>0?0:1;
```

为了大家更好理解三目运算符，接下来，通过一个计算两个整数最大值的案例演示三目运算符的用途，具体代码如下：

实现代码如下：

```
public class Test {
    public static void main(String[] args) {
        int x = 13;
        int y = 8;
        System.out.println(x > y ? x : y);
    }
}
```

上述代码中，我们首先定义了两个 int 类型的变量，然后使用三目运算符比较 x 和 y 值的大小并返回最大值。与 if-else 语句相比，三元运算符看起来更简洁，我们可以把三元运算符理解为 if-else 语句的简写，大家可以根据自己的编程爱好，自行选择使用哪种方式编写代码。

第 5 章 函数

学习目标

- ◆ 掌握函数的概念，能够说出函数的作用以及构造
- ◆ 了解入口函数，能够说出入口函数的作用



- ◆ 了解查看 System.out 源码的方式，能够说出 println() 和 print() 函数的区别
- ◆ 掌握函数的参数和返回值，能够根据业务场景设计函数的参数和返回值
- ◆ 掌握格式化输出的方式
- ◆ 了解递归
- ◆ 熟悉 JDK 文档的查看以及生成方式

假设现在要开发一个游戏程序，程序在运行过程中，要不断地发射炮弹。发射炮弹的动作需要编写 100 行代码。在学习函数之前，我们可能在每次实现发射炮弹的地方都需要重复地编写这 100 行代码，这样的程序代码是相当臃肿的，而且程序可读性也非常差。

如果使用函数可以很好的解决上述问题，我们可以将发射炮弹的代码提取出来，放在一个 {} 中，并为这段代码起个名字，提取出来的代码可以看作是一个函数。我们可以把函数理解为完成特定功能的代码段，每一个函数都是一个独立的小功能。接下来，本章将针对函数的相关内容进行讲解。

5.1 函数入门

5.1.1 什么是函数

大家都听说过“函数”这个词，对于数学学得好的同学，可能脑子中对函数的印象如图 5-1 所示。对于数学不好的同学，可能对函数的印象如图 5-2 所示。

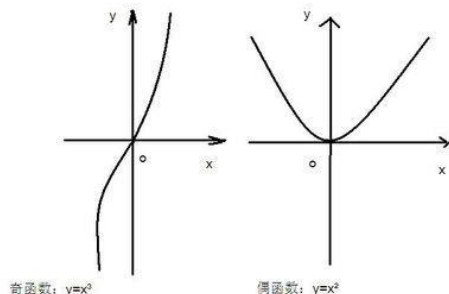


图 5-1 函数画面（1）



图 5-2 函数画面（2）

计算机编程中的函数，和我们数学中的函数不一样。无论你数学学的怎么样，大家只需要认真听我们的课，都可以很好的掌握 Java 编程中的函数。

为了方便大家理解函数的概念，我们先看看乐高积木，如图 5-3 所示。



图 5-3 乐高积木

我们可以通过图 5-3 所示的乐高积木拼出很多好玩的玩具，例如，房子，大象、熊猫等。乐高积木是由很多小片段组成的，这些下片段组合在一起，可以做成一个很棒的大玩具。

其实，计算机程序也是由很多小片段组成的，函数把这些小片段组合在一起，可以提供某种特殊的功能，方便后续重复使用。可以说，函数的作用就是更好的组织和组合代码片段。

我们结合乐高积木来理解一下函数，大家可以把乐高积木想象成程序中的各种函数，如图 5-4 所示。

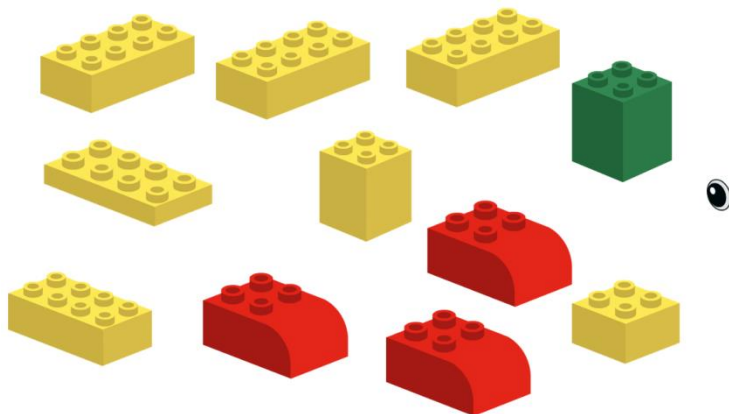
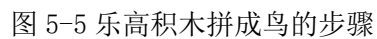


图 5-4 乐高积木小片段

如果我们把这些积木按一定顺序搭建在一起，就可以拼出一个鸟，拼积木步骤如图 5-5 所示。



接下来，我们通过一个示例代码来说明方法的作用，假设现在我们要打印一个图案“ITHEIMA”，我们可以这么做：

这个时候，我们可以把重复的代码抽成一个方法。在IDEA中将代码抽取成方法的方式比较简单，选中要抽取的代码，右击【Refactor】→【Extract Method】，具体如图5-6所示。

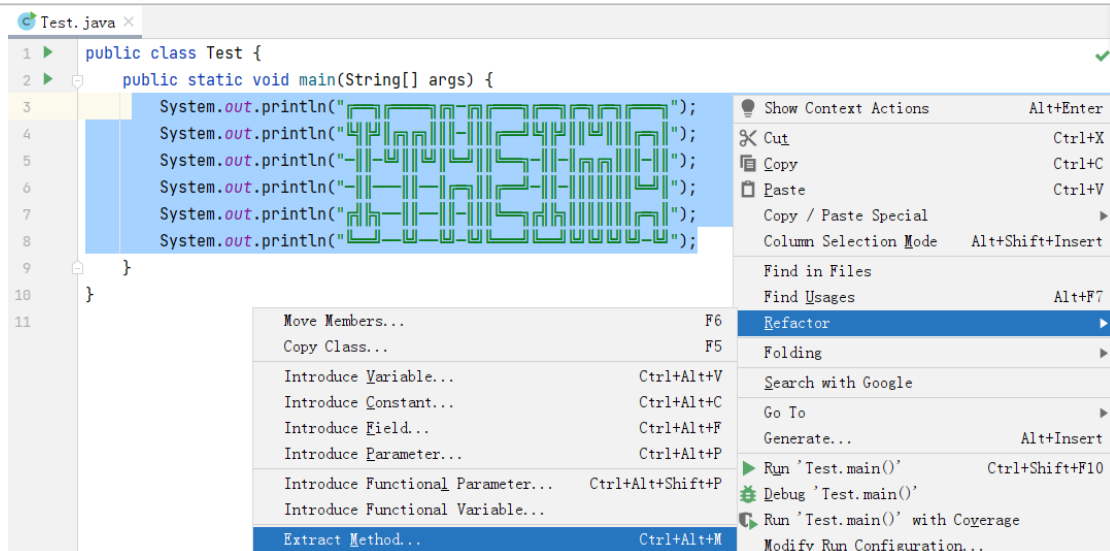


图 5-6 将代码抽取为方法

这样我们就抽取出了一个方法，这里我们将方法命名为 heimalogo。如果我们想调用这个方法，只需要编写代码 heimalogo() 就可以了，具体如图 5-7 所示。

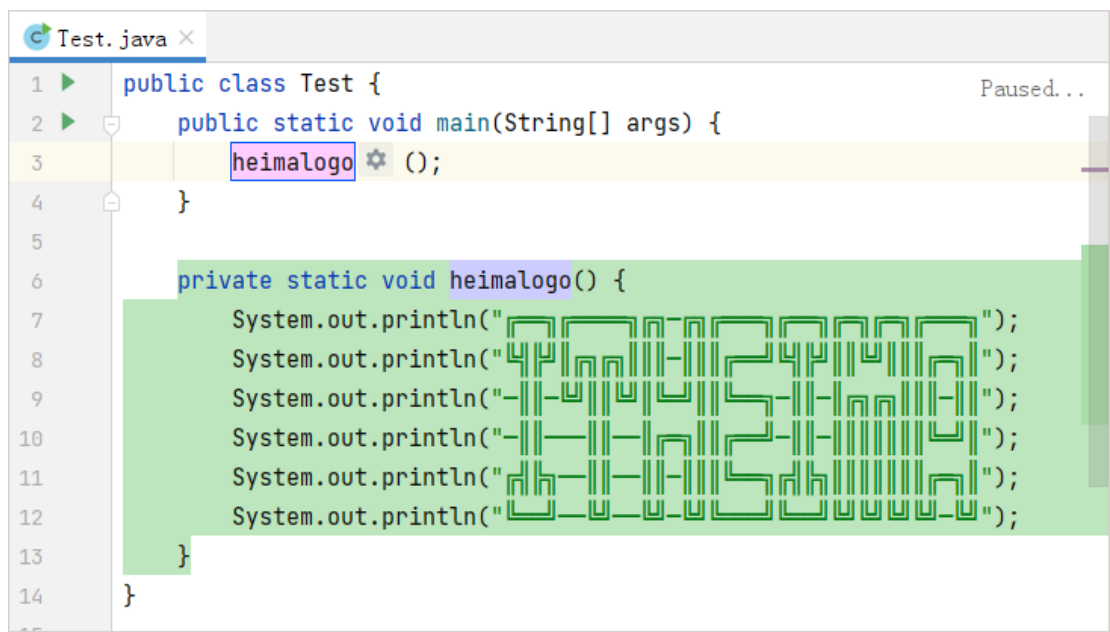


图 5-7 调用方法

如果后续还想连续输出多次 logo，可以调用多次 heimalogo() 方法，例如，在 main() 方法中调用 2 次 heimalog()，如图 5-8 所示

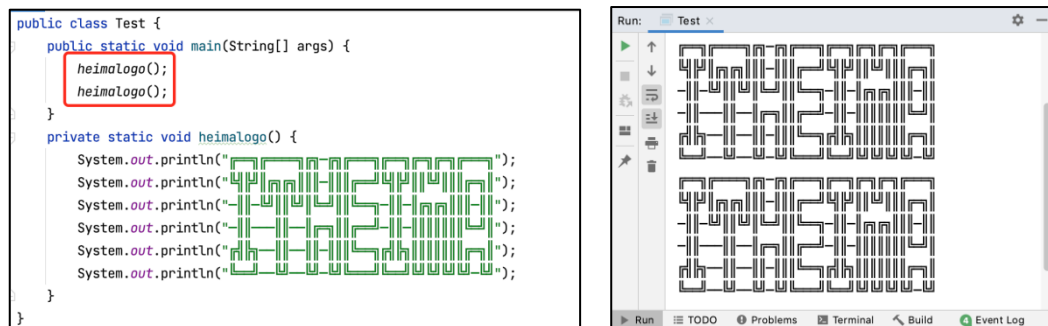


图 5-8 调用 2 次 heimalogo() 方法

将重复代码抽取为方法还有另外一个好处，就是代码维护很方便。假设方法中的代码需要修改，那么我们只需在方法中修改一次即可，具体如图 5-9 所示。

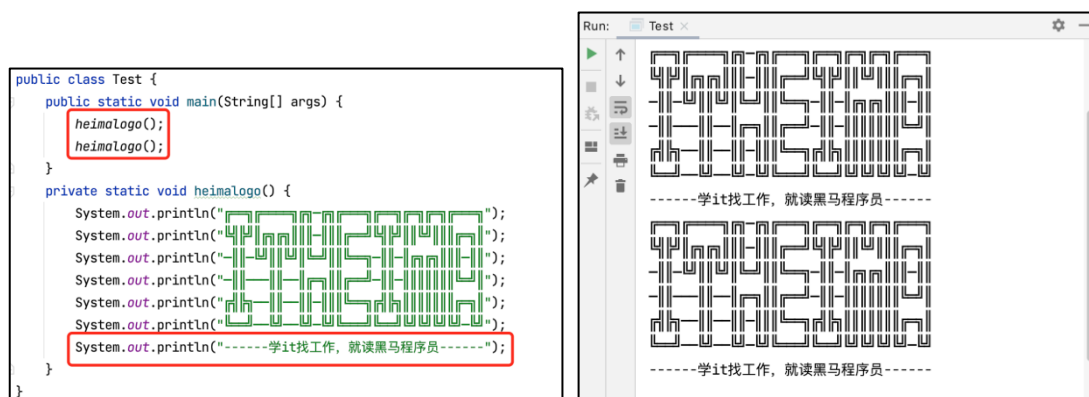


图 5-9 修改方法中的代码

5.1.2 入口函数

Java 中有一个非常重要的函数，即入口函数 `main()`，它是程序运行默认执行的函数，也就是程序执行的入口，示例代码如下：

```
public static void main(String[] args) {  
    // 入口函数中的代码  
}
```

需要注意的是，入口函数 `main()` 的函数名和写法是固定的，不可修改。

5.1.3 剖析 System.out 源码

前面章节的学习中，我们多次使用了 `System.out.println()` 函数输出内容。其实，代码的输出不仅可以使用 `System.out.println()` 函数，还可以使用 `System.out.print()` 函数。

虽然上述两个函数都是用来输出内容的，但是它们还是有一些区别，具体如下：

- (1) `System.out.print()` 函数的作用是把括号里面的内容输出到计算机屏幕。
- (2) `System.out.println()` 函数的作用是作用是把括号里面的内容输出到计算机屏幕，同时换行。

理解了这两个函数的区别后，我们阅读一下它们的源码，剖析一下其内部的实现原理。

在 IDEA 中，按住 `Ctrl` 键的同时，使用鼠标单击 `println` 查看 `println()` 函数源码，如图 5-10 所示。

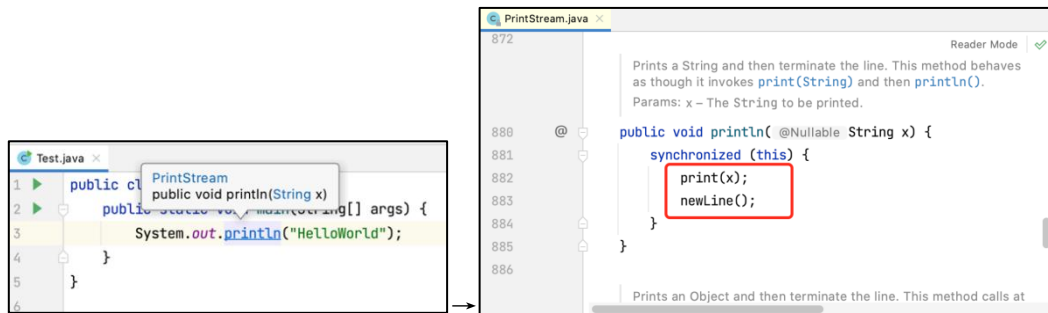


图 5-10 查看 println() 函数源码

从图 5-10 中可以看出,在 println() 函数中,先调用了 print() 函数,再调用了 newLine() 函数,也就是说,println() 函数的功能其实是 print() 函数和 newLine() 函数组合实现的。继续查看 newLine() 函数的源码,如图 5-11 所示。



图 5-11 newLine() 源码

从图 5-11 中,我们继续查看 textOut.newLine() 的源码,具体如图 5-12 所示。

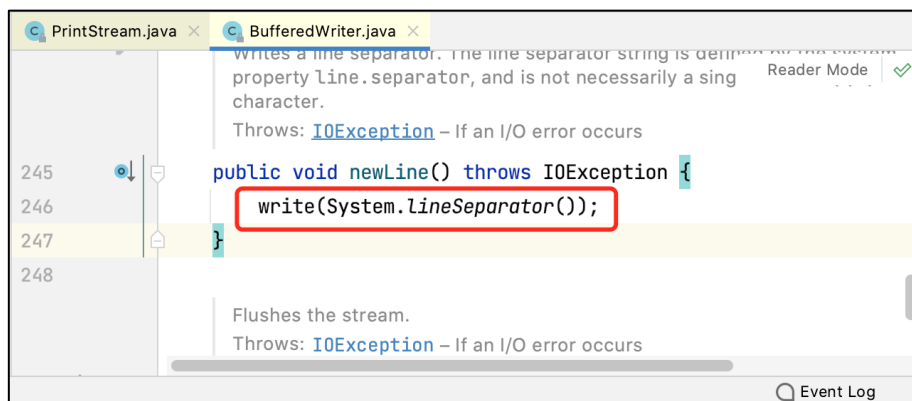


图 5-12 newLine() 源码

在图 5-12 中,继续查看 LineSeparator() 源码,具体如图 5-13 所示。

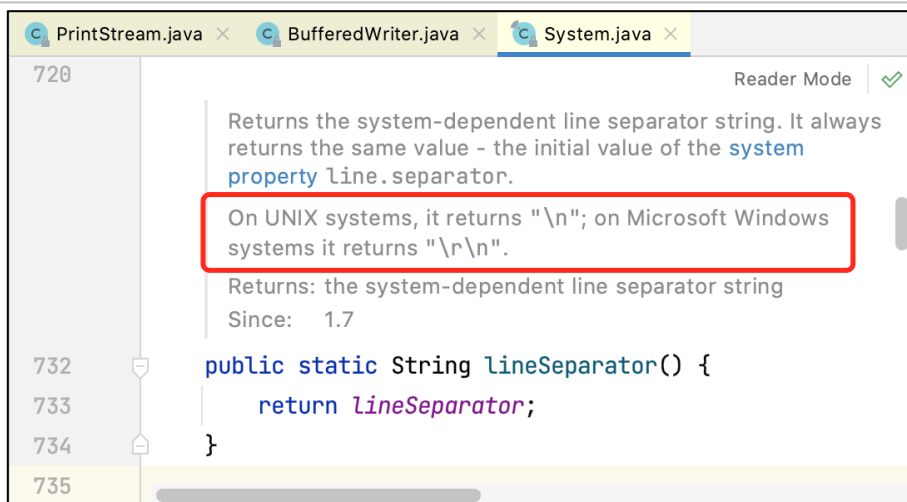


图 5-13 LineSeparator() 源码

在图 5-13 中，LineSeparator() 方法的注释说明了在 UNIX 系统，返回值是“\n”，表示返回一个换行符。在 Windows 系统会返回“\r\n”，表示返回一个回车和换行符。

通过上述源码的阅读，我们可以发现，下面两段代码的效果是等同的。

```
System.out.println("helloworld");
```

等同于：

```
System.out.print("helloworld");
```

```
System.out.print("\r\n");
```

5.2 函数的参数和返回值

5.2.1 认识函数参数和返回值

函数的本质，可以理解为一个或一组功能的组合。例如，我们利用 System.out.println 这样的小函数，组合成了打印 logo 的大函数。这节课我们继续探索函数的本质。

Java 中的函数还有一个重要作用就是处理数据。我们知道，计算机的功能就是处理数据，当我们把二进制数据交给计算机，计算机运算完毕后，会输出新的二进制数据。

如果把人类的消化系统与计算机进行类比的话，可以把人类摄入的食物比作是向计算机输入的数据，这些食物经过唾液腺、胃、肝脏等一系列器官处理后，最终会有产物，这个产物就相当于计算机的输出。图 5-14 描述了人类与计算机的输入输出过程。

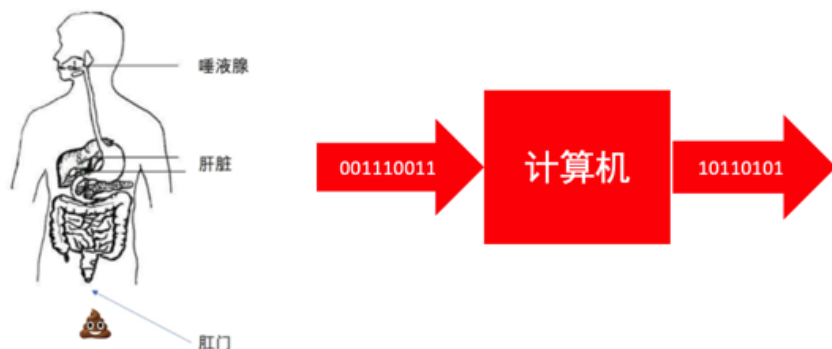


图 5-14 人类和计算机的输入输出过程

如果把人的消化系统类比一个函数，会出现 4 种情况：

- 有时候吃完饭就拉粑粑
- 有时候吃完饭不拉粑粑
- 有时候不吃饭就拉粑粑
- 有时候不吃饭不拉粑粑

我们重新梳理一下上述思路，具体如下：

- 饭→消化系统函数→粑粑
- 饭→消化系统函数→nothing
- nothing→消化系统函数→粑粑
- nothing→消化系统函数→nothing

计算机是用于处理数据的，我们编写的函数一般都是用来处理或者解析数据。有时候，我们会将数据以参数的形式交给函数，函数处理完毕后，会产生输出。如果将函数类比为人的消化系统，处理数据的过程如图 5-15 所示。



图 5-15 函数处理数据的过程

实际上，函数的参数与返回值会有四种场景：

- 有输入，有输出
- 没输入，有输出
- 有输入，没输出
- 没输入，没输出

如果将上述四种场景与消化系统对应的话，如图 5-16 所示

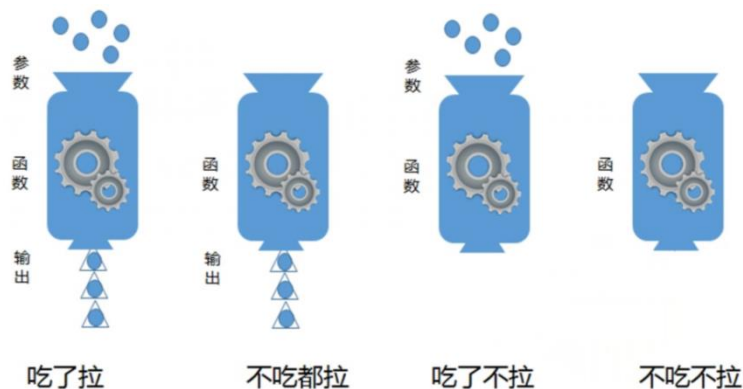


图 5-16 函数与消化系统的场景对应

理解了函数的输入和输出规则之后，下面我们看一下 Java 函数的编写规则。

```
函数修饰符 返回值类型 函数名（函数参数类型 函数名）{  
    函数体语句;  
}
```

下面我们将之前写的代码与上面的函数编写规则对应起来加深理解，如图 5-17 所示。

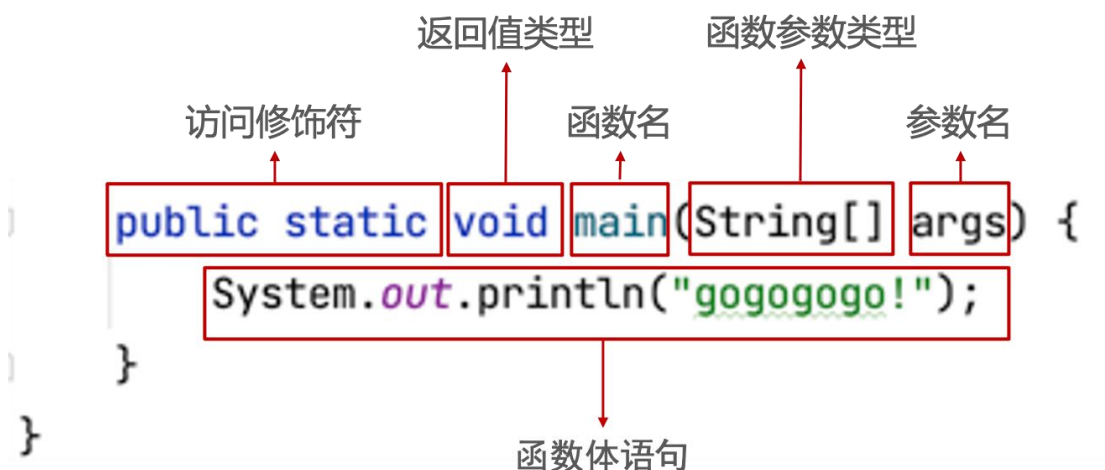


图 5-17 代码与函数编写规则的对应

在图 5-17 中，函数的访问修饰符是 public static, 返回值类型是 void（空类型），表示没有返回值，函数名是 main，表示是程序的入口函数，函数参数类型是 String[]，表示的是字符串数组，参数名是 args。函数体语句是一个输出语句。

5.2.2 应用案例：计算器

理解了函数的参数和返回值的作用后，接下来，我们开发一个计算器，用于实现两个整数的加减乘除四则运算，具体步骤如下：

(1) 创建一个表示计算器的类 Calc，在该类中定义四个函数，分别实现加减乘除算法，具体代码如下：

```
/**  
 * 这是一个相加的函数  
 */
```



```
* @param x 第 1 个加数
* @param y 第 2 个被加数
* @return x+y 的结果
*/
public static int add(int x, int y) {
    return x + y;
}

/**
 * 这是一个相减的函数
 *
 * @param x 减数
 * @param y 被减数
 * @return x-y 的结果
 */
public static int sub(int x, int y) {
    return x - y;
}

/**
 * 这是一个相乘的函数
 *
 * @param x 乘数
 * @param y 被乘数
 * @return x*y 的结果
 */
public static int multi(int x, int y) {
    return x * y;
}

/**
 * 这是一个相除的函数
 *
 * @param x 这是除数
 * @param y 这是被除数
 * @return x/y 的结果
 */
public static int division(int x, int y) {
    return x / y;
}
```

(2) 在 Calc 类的 main() 函数中进行测试，具体代码如下：

```
public static void main(String[] args) {
    int result_01 = add(3, 8);
```



```
System.out.println("x+y=" + result_01);  
int result_02 = sub(10, 5);  
System.out.println("x-y=" + result_02);  
int result_03 = multi(3, 5);  
System.out.println("x*y=" + result_03);  
int result_04 = division(10, 5);  
System.out.println("x/y=" + result_04);  
}
```

(3) 运行程序，结果如图 5-18 所示。

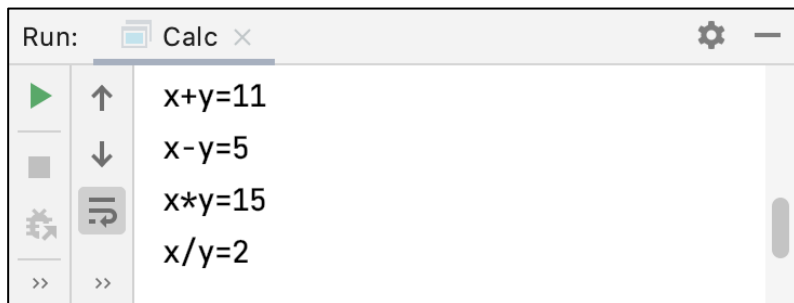


图 5-18 Cala 的运行结果

5.2.3 应用案例：超市找零

函数对外提供某种业务功能，用来解决某种问题。设计函数时，最好可以做到下面几点：

- 函数的名称最好能顾名思义，通过函数名就能基本看出函数的功能。
- 根据业务提供的数据，设计函数的参数。
- 根据业务的输出要求，设计函数的返回值。

下面我们通过一个超市收银的案例来演示如何抽取现实生活中的业务场景并将业务场景设计为函数。

一、业务场景：

图 5-19 描述了一种超市收银的业务场景，某顾客购买了香蕉，价格是 3.99 元，顾客支付了 5 元，超市收银员会操作柜台，自动计算出需要给顾客找零多少钱。



图 5-19 超市收银业务场景

二、设计函数：

(1) 由于上述业务是找零的场景，所以我们可以设计函数名为 makeChange。



(2) 顾客消费了 3.99 元，但是给了收银员 5 元，按照这个逻辑，我们为函数设计两个参数，一个参数表示消费金额 `double itemCost`，一个参数表示给收银员的金额 `double money`。

(3) 这个函数用来计算找零，最终函数的返回值应该是 `double change`。

三、编写函数：

按时设计的函数，编写函数 `makeChange`，示例代码如下：

```
public static double makeChange(double itemCost,double money){  
    return money-itemCost;  
}
```

在 Test 中调用 `makeChange` 函数进行测试，示例代码如下：

```
public class HelloStore {  
    public static void main(String[] args) {  
        double itmeCost = 2.0;  
        double money = 5.0;  
        double change = makeChange(itmeCost, money);  
        System.out.println("找零" + change);  
    }  
    public static double makeChange(double itemCost, double money) {  
        return money - itemCost;  
    }  
}
```

程序的运行结果如图 5-20 所示。

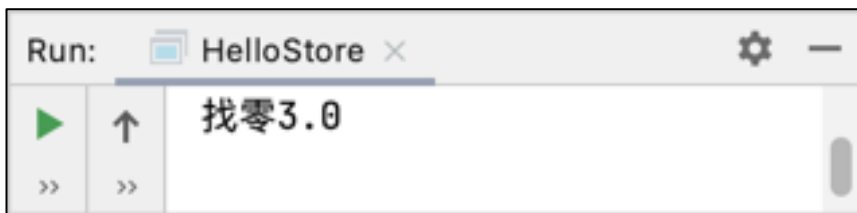


图 5-20 HelloStore 的运行结果

多学一招：实参与形参

Java 中的参数分为实参和形参，其中，形参就是形式参数，用于定义函数时的参数，用来接收调用者传递的参数。实参就是实际的参数，用来调用函数时传递给函数的参数，实参在调用方法之前要预先赋值的。例如，上面的案例中，`makeChange` 函数中的 `itemCost` 和 `money` 都是形参，而 `main` 函数中的 `itmeCost` 和 `money` 都是实参。

5.3 格式化输出

Java 中提供了一个格式化输出的函数 `printf()`，它实现了 C 语言风格的输出，用于输出带各种数据类型占位符的参数，其参数个数是不定的。接下来，通过一张表来罗列不同数据类型的标记占位符，如表 5-1 所示。

表 5-1 不同数据类型的标记占位符

占位符	数据类型	示例代码
-----	------	------



%d	int, short, byte, long	System.out.printf("Display an Integer %d", 1500);
%c	char	System.out.printf("Display a Character %c", 'c');
%f	double, float	System.out.printf("Display a Floating-point Number %f", 123.45);
%s	String	System.out.printf("Display a String %s", "String");

为了大家更好理解格式化输出的效果，接下来，通过一个案例来演示 printf() 函数的用法，如文件 5-1 所示。

文件 5-1 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        System.out.printf("My Name is %s,I was born in %d","Mike",1998);  
        System.out.println("");  
        System.out.printf("The sum of %d and %d is %d",15,40,55);  
        System.out.println("");  
        System.out.printf("Display a Number %f",15.233333);  
        System.out.println("");  
        System.out.printf("Display a Number %.2f",15.23);  
    }  
}
```

文件 5-1 的运行结果如图 5-21 所示。

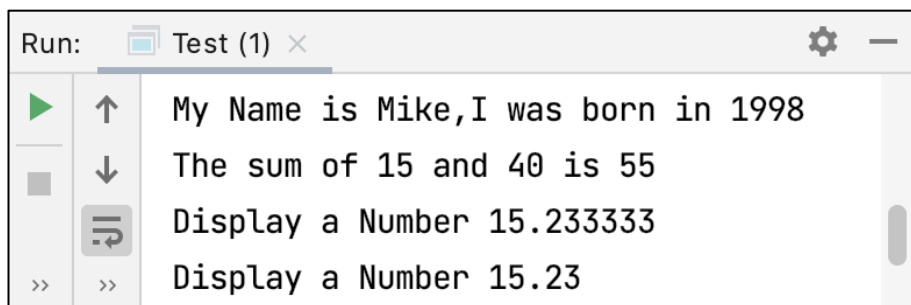


图 5-21 文件 5-1 的运行结果

根据图 5-21 所示的运行结果可知，使用 printf() 函数格式化输出内容时，printf() 函数第 1 个参数中的占位符，依次被后续的数值替代。需要注意的是，使用 printf() 函数格式化输出内容时，要保证占位符和数据类型的一致性。



5.4 函数综合案例——掷骰子

5.4.1 掷骰子-基础版

本节我们通过一个掷骰子的小游戏来继续学习函数的设计。我们的目标是编写一个掷骰子的小游戏，用户每次调用掷骰子的函数就会产生一个 $1\sim 6$ 的结果。

通过对业务分析，我们可以设计一个这样的函数：

(1) 函数名：rollDice

(2) 函数参数：无

(3) 函数的返回值： $1\sim 6$ 之间的一个随机整数。

这里会有一个问题，如何产生 $1\sim 6$ 这样的整数？

JDK 中内置了大量的函数供我们使用，例如，`System.out.println()` 就是在 `System` 类提供的用于打印输出的函数。同样的，在 `Math` 类里面，JDK 也提供了很多方便的函数，这里我们介绍一个随机数的函数 `Math.random()`，它的返回值是一个 $0\sim 1$ 之间的随机生成的小数。

接下来，我们先定义设计好的函数 `rollDice`，具体代码如下：

```
1 public static int rollDice() {  
2     double random = Math.random(); // 范围[0,1)  
3     double result = random * 6 + 1; // 范围[1,7)  
4     return (int) result;  
5 }
```

上述代码中，第 2 行代码使用 `Math.random()` 生成了一个 $0\sim 1$ 的随机小数 `random`。由于我们要产生的随机数范围是 $1\sim 6$ 这样的整数，所以在第 3 行代码中通过 `random*6+1` 这样的操作，将随机数的范围扩大到 $[1, 7)$ ，最后通过 “`(int) result`” 的操作，获取随机小数的整数部分并返回。

在测试类 `HelloDice` 中调用 `rollDice()` 函数，具体代码如下：

```
public class HelloDice {  
    public static void main(String[] args) {  
        int result = rollDice();  
        System.out.println("扔骰子，结果: " + result + "点");  
    }  
    public static int rollDice() {  
        double random = Math.random(); // 范围[0,1)  
        double result = random * 6 + 1; // 范围[1,7)  
        return (int) result;  
    }  
}
```

多次运行程序，发现程序会随机产生 $1\sim 6$ 的数字，运行结果示例如图 5-22 所示。



图 5-22 HelloDice 的运行结果（1）

5.4.2 掷骰子-升级版

假设，现在我们要将上述场景的业务进行升级，因为骰子不仅有 6 个面的，其实还有 5 个面或者 7 个面的，我们能否设计一个通用的函数，任何一个面的骰子，我们都可以摇出来一个随机数。

下面我们对上述设计的 rollDice 函数进行升级，此时设计的函数如下：

- (1) 函数名：rollDice
- (2) 函数参数：int 类型的骰子类型
- (3) 函数的返回值为 1~6 之间的随机整数。

升级后的函数 rollDice，具体代码如下：

```
public static int rollDice(int tpye) {  
    double random = Math.random();  
    double result = random * tpye + 1;  
    return (int) result;  
}
```

在测试类 HelloDice 中再次调用 rollDice() 函数，具体代码如下：

```
public class HelloDice {  
    public static void main(String[] args) {  
        int result = rollDice(7);  
        System.out.println("扔骰子，结果：" + result + "点");  
    }  
    public static int rollDice(int tpye){  
        double random = Math.random();  
        double result = random * tpye + 1;  
        return (int) result;  
    }  
}
```

上述代码在调用 rollDice() 函数时传入的参数是 7，我们多次运行程序，程序是可以随机产生 7 这个数字的，如图 5-23 所示。



图 5-23 HelloDice 的运行结果（2）

通过上述掷骰子的案例，希望大家在设计函数时能够对业务类型进行分析，根据业务的输入设计函数的参数，根据业务输出要求设计函数的返回值。

5.5 函数特性总结

这个小节我们对函数的特性做一些总结，具体如下：

- (1) 函数可以理解成一个黑盒子，它包含很多行代码。但是，我们调用函数的时候，实际上是可以不用关心里面的代码是如何写的。例如，我们前面开发了一个掷骰子函数，后续我们实现掷骰子功能的时候，只需要调用掷骰子函数即可，无需关系该函数内部的代码。
- (2) 函数可以接收参数，参数作为函数的输入，可以在函数内部进行数据运算。
- (3) 函数运算后的结果，可以通过 `return` 返回。
- (4) 调用函数的结果，就是函数返回值，我们可以使用这个返回值实现其他业务逻辑。

5.6 JDK 文档

5.7.1 JDK 文档的查看和阅读

前面我们介绍了 `Math` 类的 `random()` 函数，它用于返回 $0 \sim 1$ 之间的随机数。可能有的同学会有疑问，我们如何自行得知这些类或者函数的作用以及用法呢？

我们先举一个现实生活中的例子，假设我们购买了一台洗衣机，洗衣机厂商通常都会附带一份洗衣机的使用手册或者说明书，如图 5-24 所示。



Hisense 海信洗衣机

XQB60-C3206

简明用户手册

操作步骤

1. 放下排水管。
2. 接好进水管，打开水龙头。
3. 接通电源，按下电源开关。
4. 通过“洗衣程序”键选择洗衣程序或使用默认的洗衣程序。
5. 若需启动洗衣过程，按“过程选择”键，选择洗衣组合，或使用洗衣机默认过程组合。
6. 通过“水位选择”键，选择洗衣水位。
7. 若需更好漂洗效果，可通过“功能选择”键选择“加漂洗”。
8. 若洗涤较少较干净的衣物，可通过“功能选择”键选择“经济洗”。
9. 若需清洗洗衣机桶，可通过“洗衣程序”键选择“桶清洁”。
10. 若需时间预约，可通过“预约时间”键选择预约时间。
11. 取出需要洗涤的衣物口袋中的所有硬币、纸币等异物，然后将衣物均匀地投入洗衣桶内。
12. 投放洗涤剂，也可根据需要投放柔软剂、漂白剂。
13. 盖上洗衣机盖板。
14. 按下“启动/暂停”键，洗衣机开始工作。

洗衣程序

程序	洗涤时间 (约分钟)	漂洗次数	脱水时间 (约分钟)	适用范围
标准	15	2	3	洗涤一般耐、需要经常更换的衣物，如衬衫、罩衫等涤纶、锦纶和混纺类织物。
大件	20	2	4	洗涤一般到非常耐的床单、桌布及内衣裤毛巾、衬衫等棉麻类织物。
轻柔	8	2	2	适合洗涤不太脏的、质地较柔软的衣物，如棉质及合成纤维织物。
快速	6	1	3	穿着时间短的外衣等棉质、涤纶及混纺类织物。
羊毛	6	2	2	有纯羊毛标志，并可机洗的羊毛衣物。
桶清洁	15	1	3	只用于洗衣桶的清洁，默认水位为47L，默认漂洗脱

- 当选择大件程序时，洗涤过程为双重强力洗涤。第一阶段以较低水位洗涤（约为设定水位70%左右），以高转速洗涤和强力漂洗充分瓦解顽固污渍，经过3分钟洗涤后，洗衣机自动补水，并到达设定水位，开始第二阶段洗涤。
- 本表洗涤时间仅为程序在39L水位段时的洗涤时间和脱水时间。
- 当选择“标准”洗衣程序时“过程选择”中“浸泡”选项不可选，“功能选择”仅可选“加水”。
- 标准程序为默认的洗涤程序。

洗涤用水量

水位 [洗涤用水量(L)]	洗涤容量(kg)
7/8 [43/47]	4.0-6.0(羊毛1.0-2.0)
5/6 [35/39]	2.5-4.0(羊毛0-1.0)
3/4 [27/31]	1.5-2.5
1/2 [16/23]	0-1.5

- 受衣物材质及水位传感器精度影响，水位(用水量)可能会稍有出入。
- 洗涤少量衣物时，水位不宜设置过高。

异常现象代码说明

数码指示	报警原因	报警解除
E1	表示两次脱水不平衡状态无法自动处理。 或运行时上盖未盖好。	请人工均匀放置衣物，盖好上盖。 请盖好上盖。
E3	排水5分钟后，仍不到水位。	打开上盖。如果排水管出水口位置过高或未放倒，请放低排水管；如果排水管弯折扭结或被堵塞，请将排水管放顺畅及解决排水管堵塞问题。然后关上上盖，解除报警。
E4	注水或补进水30分钟仍不到设定水位。	打开上盖。如果停水，来水后再用；如果水龙头未打开，请打开水龙头；如果进水管的滤网被堵塞，请按说明书中“进水管的清理”方法清理滤网。然后关上上盖，解除报警。
E5	水位传感不良	请与售后部门联系

下列现象不是故障：
脱水时停止脱水，显示b1或b2：脱水不平衡自动处理状态，这是由于脱水时衣物在洗衣桶内偏向一侧导致不平衡，安全开关动作的缘故。重新进水后，波轮搅拌，消除衣物的侧偏后，便可重新脱水。

注意：当检查了以上各点后仍有异常，请拔下电源插头，尽快与经销单位联系。若异常报警1小时内无处理，洗衣机将自动关机。

规格及技术参数表

型 号	XQB60-C3206		
额定洗涤、脱水容量	6.0kg (标准干布质量)		
额定输入功率	400W		
额定电源	220V~50Hz		
使用水压	0.02MPa~0.8MPa		
质 量	约32 kg		
外形尺寸	宽557mm x 深545mm x 高900mm		
噪声	洗涤≤62dB 脱水≤72dB		
能源效率等级 执行标准： GB 12021.4-2004 电动洗衣机能源效率等级及能源效率等级	2级	耗电量 (千瓦时/工作周期)	0.101
		用水量 (升/工作周期)	138
		洗净比	0.80

图 5-24 洗衣机的使用说明书示例

图 5-24 所示的洗衣机使用说明书中，包含了洗衣机操作步骤、规格及技术参数、异常处理以及洗涤用水量等信息。如果我们看了洗衣机的说明书，那么洗衣机的用法会非常清楚。

同样，Java 也提供了一份“使用说明书”——JDK 文档，链接如下：

<https://docs.oracle.com/en/java/javase/11/docs/api/index-files/index-1.html>

在 JDK 文档中，我们可以找到绝大多数的 Java 使用说明，而且这个 JDK 文档是绝对权威的 Java 开发资料，它包含了关于类、函数、方法等具体说明，所以也称为 API 文档。

下面，我们带大家一起看一下 API 文档中关于 Math 的介绍。

打开 API 文档，通过搜索关键字 Math 找到 Math 类的相关介绍，如图 5-25 所示。

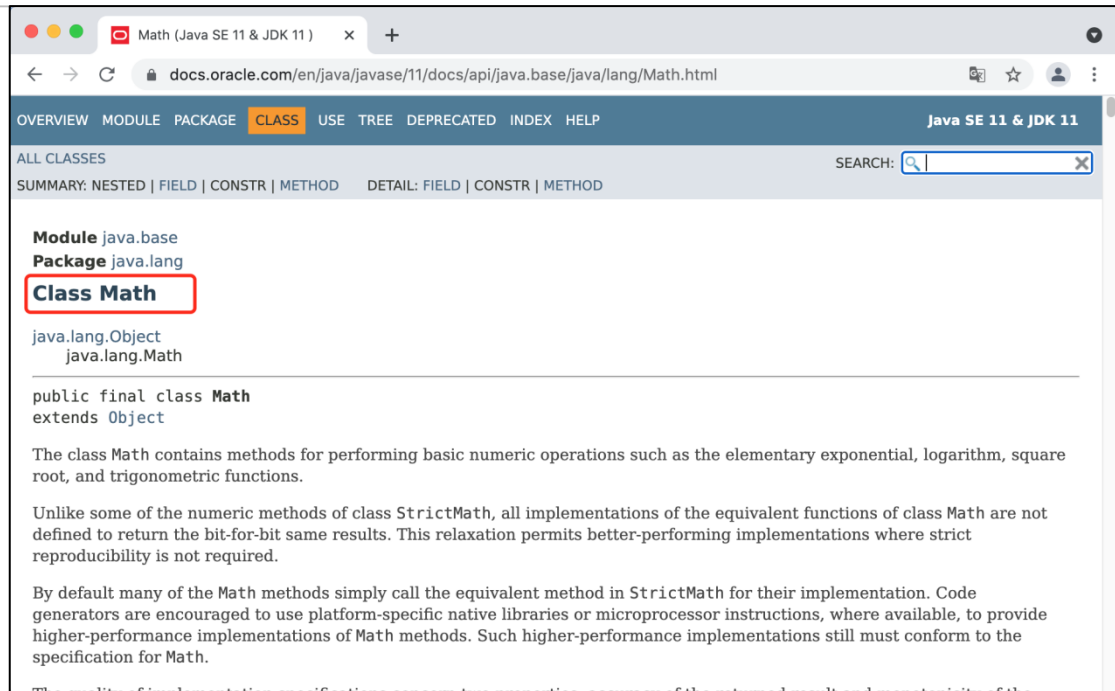


图 5-25 Math 类

通过阅读 Math 类的相关 API，我们发现 Math 类不仅提供了字段，而且还提供了很多函数，如图 5-26 所示。

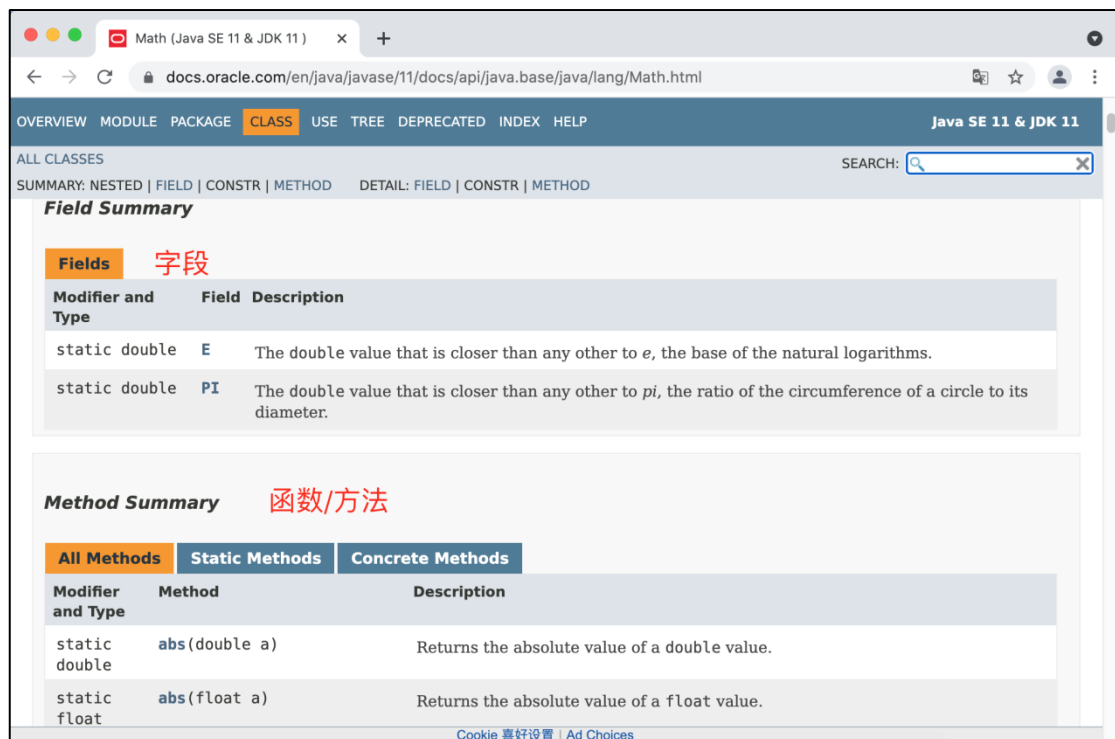


图 5-26 Math 类提供的字段和方法

我们在 Math 类中找到之前讲解的 random 函数，如图 5-27 所示。



static double	pow (double a, double b)	Returns the value of the first argument raised to the power of the second argument.
static double	random ()	Returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0.
static double	rint (double a)	Returns the double value that is closest in value to the argument and is equal to a mathematical integer.
static long	round (double a)	Returns the closest long to the argument, with ties rounding to positive infinity.

图 5-27 random 函数

从图 5-27 中 random() 函数的介绍可知，random() 函数会返回一个大于等于 0 并且小于 1 的双精度的小数。单击 random() 查看 random() 函数的详细介绍，如图 5-28 所示。

Math (Java SE 11 & JDK 11)

docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Math.html#random()

OVERVIEW MODULE PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

Java SE 11 & JDK 11

ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD

DETAIL: FIELD | CONSTR | METHOD

SEARCH: Search

random

public static double random()

Returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0. Returned values are chosen pseudorandomly with (approximately) uniform distribution from that range.

When this method is first called, it creates a single new pseudorandom-number generator, exactly as if by the expression

```
new java.util.Random()
```

This new pseudorandom-number generator is used thereafter for all calls to this method and is used nowhere else.

This method is properly synchronized to allow correct use by more than one thread. However, if many threads need to generate pseudorandom numbers at a great rate, it may reduce contention for each thread to have its own pseudorandom-number generator.

API Note:

As the largest double value less than 1.0 is `Math.nextDown(1.0)`, a value `x` in the closed range `[x1, x2]` where `x1 <= x2` may be defined by the statements

```
double f = Math.random()/Math.nextDown(1.0);
double x = x1*(1.0 - f) + x2*f;
```

Returns:

a pseudorandom double greater than or equal to 0.0 and less than 1.0.

See Also:

`nextDown(double)`, `Random.nextDouble()`

Cookie 喜好设置 | Ad Choices

图 5-28 random() 函数的详细介绍

大家可以自行阅读 random() 函数的相关介绍，熟悉 random() 函数的用法。除了 random() 函数之外，Math 类还提供了其他很多的 API 文档，例如，用于比较最大数的 max() 函数，其相关介绍如图 5-29 所示。

max

public static int max(int a, int b)

Returns the greater of two int values. That is, the result is the argument closer to the value of `Integer.MAX_VALUE`. If the arguments have the same value, the result is that same value.

Parameters:

a - an argument.

b - another argument.

Returns:

the larger of a and b.

图 5-29 max() 函数



在图 5-29 中，`max()` 函数需要传入两个参数 `a` 和 `b`，并返回最大的一个数，`max()` 函数的调用示例如下所示：

```
Math.max(13,5);
```

好了，至此我们介绍了 Java API 文档的阅读方式。

5.7.2JDK 文档的生成

Java 提供了 API 文档供开发者使用，那我们可以自己构建一个 API 文档吗？答案是肯定的。软件开发是一个协同工作，我们可以构建自己的 API 文档供其他程序员使用。下面我们以前面的 `HelloDice` 文件为例，介绍如何构建 API 文档，具体步骤如下：

(1) 使用文档注释对代码进行说明，具体代码如下：

```
/**
 * 这是掷骰子的类
 */
public class HelloDice {
    public static void main(String[] args) {
        int result = rollDice(7);
        System.out.println("扔骰子，结果：" + result + "点");
    }
    /**
     * 掷骰子的方法
     * @param type 筛子是几个面
     * @return 掷骰子后，返回 1~n 的随机整数值
     */
    public static int rollDice(int type) {
        double random = Math.random();
        double result = random * type + 1;
        return (int) result;
    }
}
```

(2) 单击【Tools】→【Generate JavaDoc】进入 Generate JavaDoc 窗口，如图 5-30 所示。

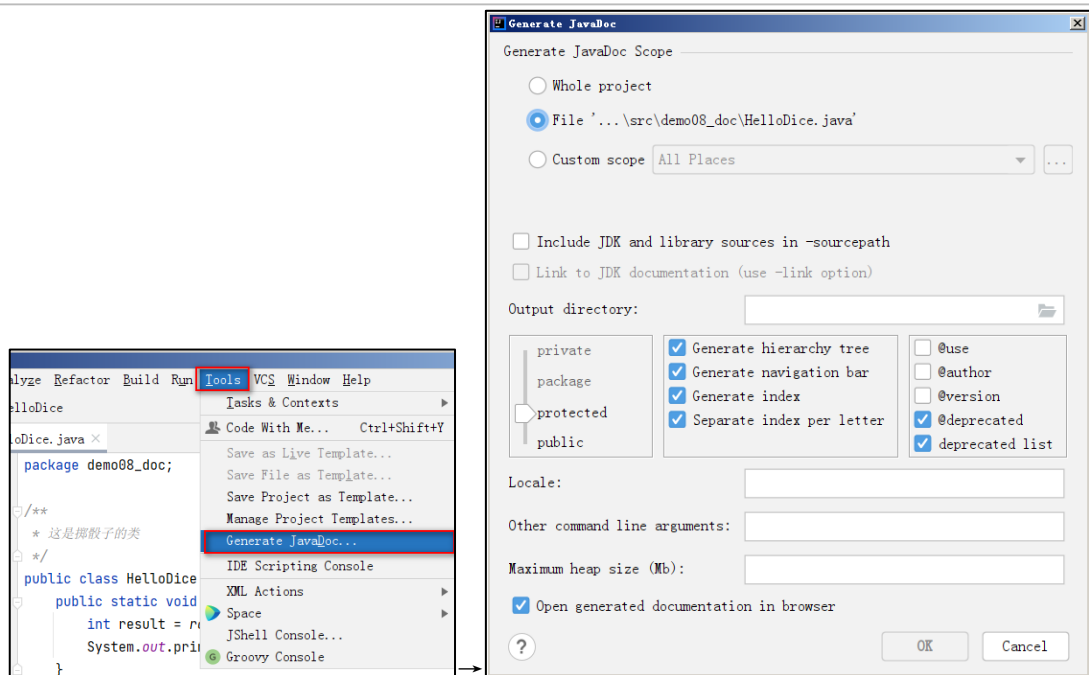


图 5-30 Generate Javadoc 窗口

在图 5-30 中，设置 Output Directory，也就是设置 Java 文档的输出位置，这里我们选择存放在 C:\czjava149\day03\code\doc，这里要保证文件是存在的。因为默认情况下，javadoc 只支持英文，不支持中文，所以，为了使我们设置的 Java 文档支持中文，我们可以在构建 javadoc 文档时，添加中文支持的配置信息，具体如下：

```
-encoding UTF-8 -charset UTF-8 -windowtitle "扔骰子文档"
```

添加配置的位置如图 5-31 所示。

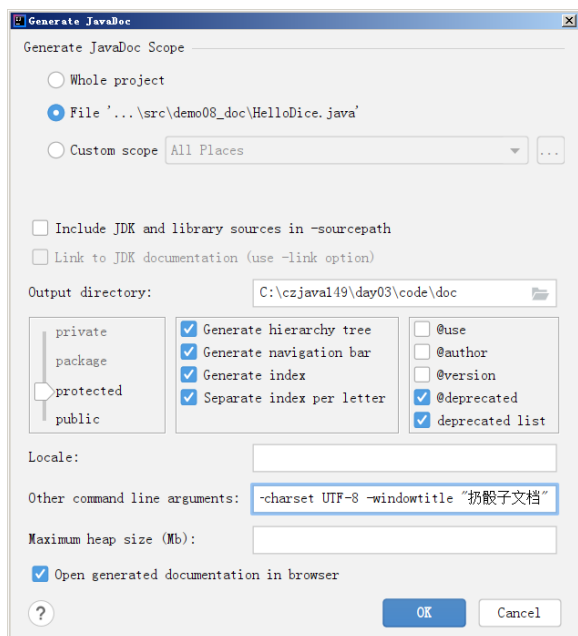


图 5-31 添加中文支持配置

此时，再次单击图 5-31 中的 OK 按钮，文档就可以顺利构建成功。构建好的文档如图 5-32 所示。

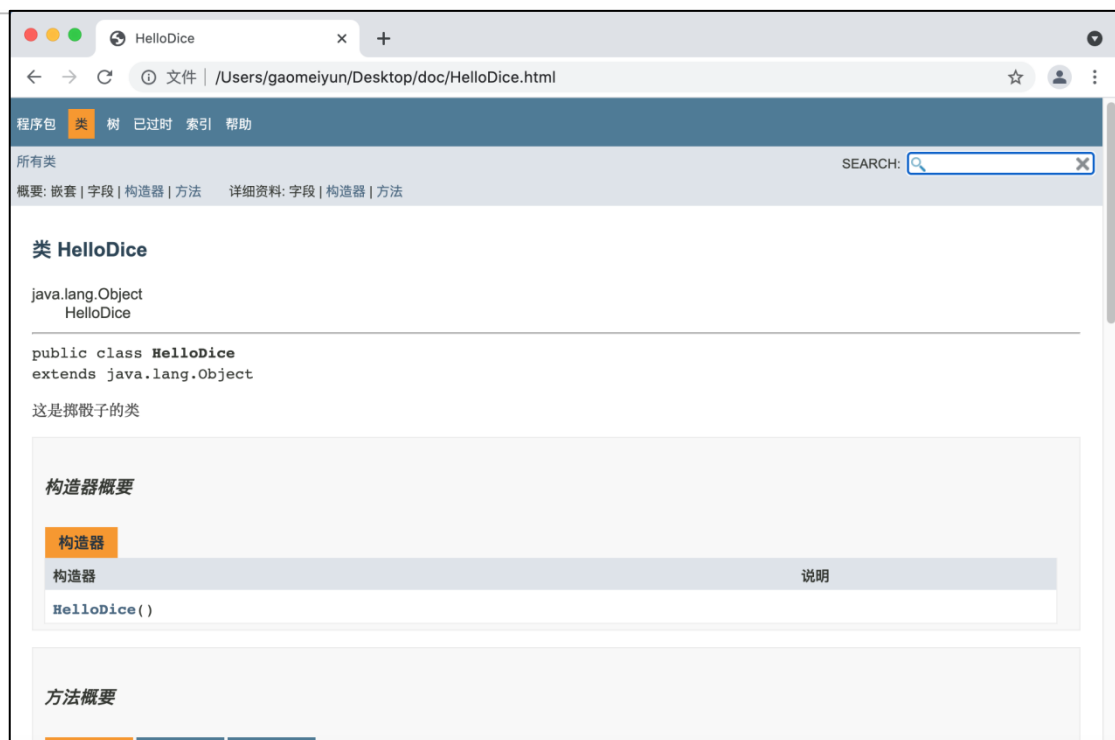


图 5-32 构建好的 Java 文档

好了，构建 Java 文档的介绍就到这里，希望大家在协同开发过程中，要养成一个编写注释的好习惯，这个注释不仅可以自己查看，还可以提供给其他程序员查看。

第 6 章 循环语句

学习目标

- ◆ 掌握 while 循环的语法，能够灵活使用 while 语句开发程序
- ◆ 掌握 do-while 循环的语法，能够灵活使用 do-while 开发程序
- ◆ 掌握 for 循环的语法，能够灵活使用 for 循环开发程序
- ◆ 熟悉 break 和 continue 语句的含义

日常生活中，我们会重复做一些事情。例如，每天我们会拿着牙刷轻柔的在牙齿上重复刷动。在编程过程中，我们也需要重复执行一些代码，这些代码的执行可以使用循环语句实现。在 Java 编程中，循环有三种常见的写法，分别是 while 循环、do-while 循环以及 for 循环，接下来，本章将针对这些循环语句进行讲解。



6.1 while 循环

6.1.1 while 循环入门

while 循环语句的基本格式具体如下：

```
while(条件语句) {  
    循环体;  
}
```

上述格式中，只要条件语句的值为 true，那么循环体语句就会一直执行。如果条件语句的值为 false，那么循环就会结束。

while 循环的执行流程如图 6-1 所示。

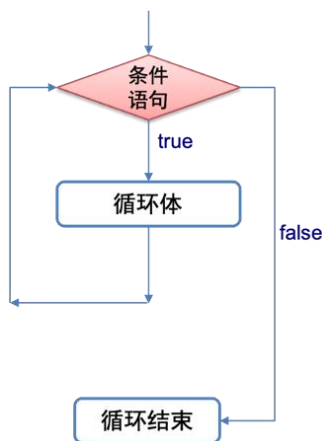


图 6-1 while 循环执行流程

下面通过示例代码，演示 while 循环语句的使用，如文件 6-1 所示。

文件 6-1 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        while (true){  
            System.out.println("itheima");  
        }  
    }  
}
```

上述代码执行后，控制台会不停的打印 itheima, 相当于我们写了无数个 System.out.println("itheima") 语句。如果我们希望停止程序的运行，可以单击控制台左侧的 ■ 按钮终止程序运行，具体如图 6-2 所示。

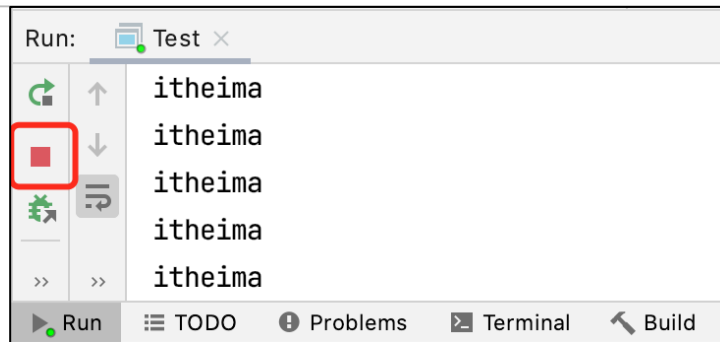


图 6-2 终止程序运行

6.1.2 应用案例：闹钟程序

我们先回顾一下现实生活中的闹铃，当闹铃定时响起时，如果我们不关闭闹铃，此时闹铃会一直“叮铃铃”响，如果使用 while 循环表示的话，将是下面的场景：

```
while (只要没人关闭闹铃) {  
    闹铃会不停的“叮铃铃”  
}
```

在实际开发过程中，很多时候要基于别人开发好的代码上进行二次开发，完成一个闹钟程序。假设现在有一个 Jar 包 itheima-clockalarm.jar，该 jar 包是别人将开发好的代码打包后的形式，我们打开 Jar 包，发现里面有一个编写好的类 Alarm，该类包含一些定义好的方法，具体如图 6-3 所示。



图 6-3 Alarm 类

在图 6-3 中，Alarm 类定义了三个方法，其中，beep() 方法用于让报警器响一声，checkSetting() 方法用于返回报警器的开关状态，init() 方法用于初始化报警器。

了解 itheima-clockalarm.jar 包定义的内容后，接下来，我们基于 Jar 包定义的内容，分步骤开发闹钟程序，具体如下：

1. 导入 itheima-clockalarm.jar 包

(1) 单击“File”→“Project Structure”进入 Project Structure 界面，在该界面选中 Libraries 选项，如图 6-4 所示。

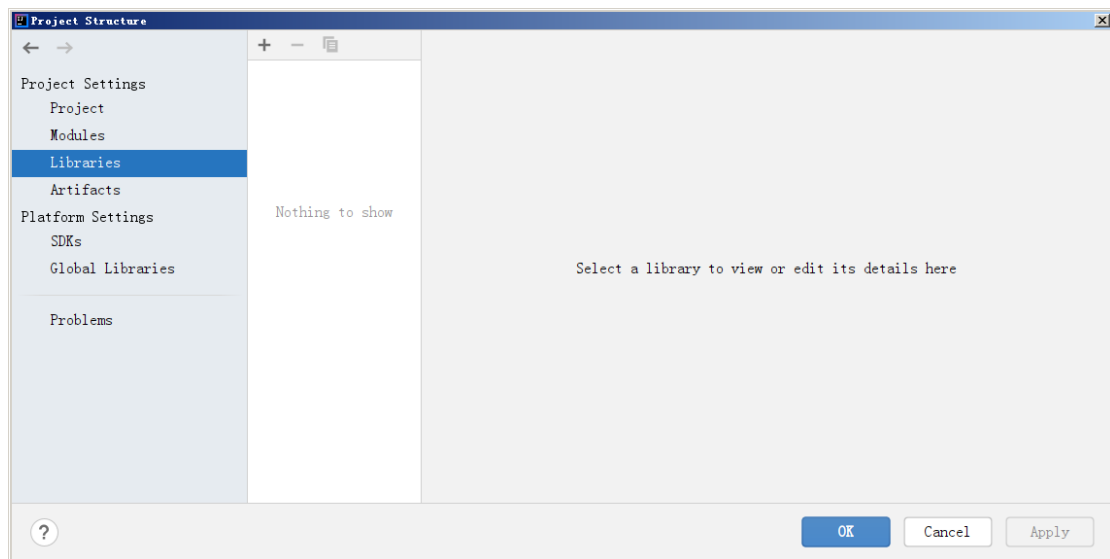


图 6-4 Project Structure 界面

(2) 单击图 6-4 中的  按钮，选择添加 Jar 包，添加后的 Project Structure 界面如图 6-5 所示。

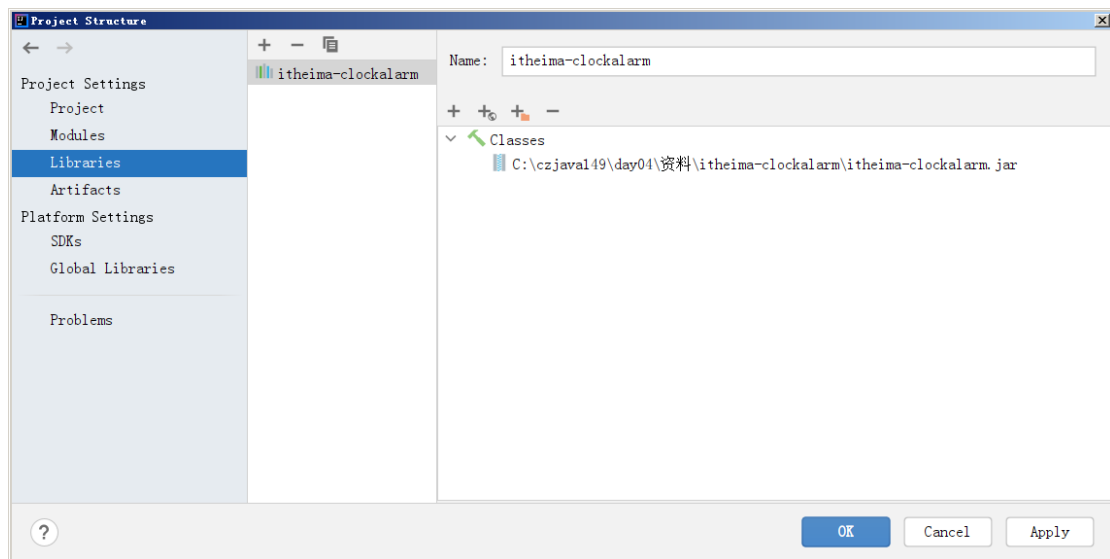


图 6-5 添加 jar 包

(3) 单击图 6-5 的“OK”按钮完成 Jar 包的添加，此时，我们可以在项目下方看到刚刚添加的 Jar 包信息，如图 6-6 所示。

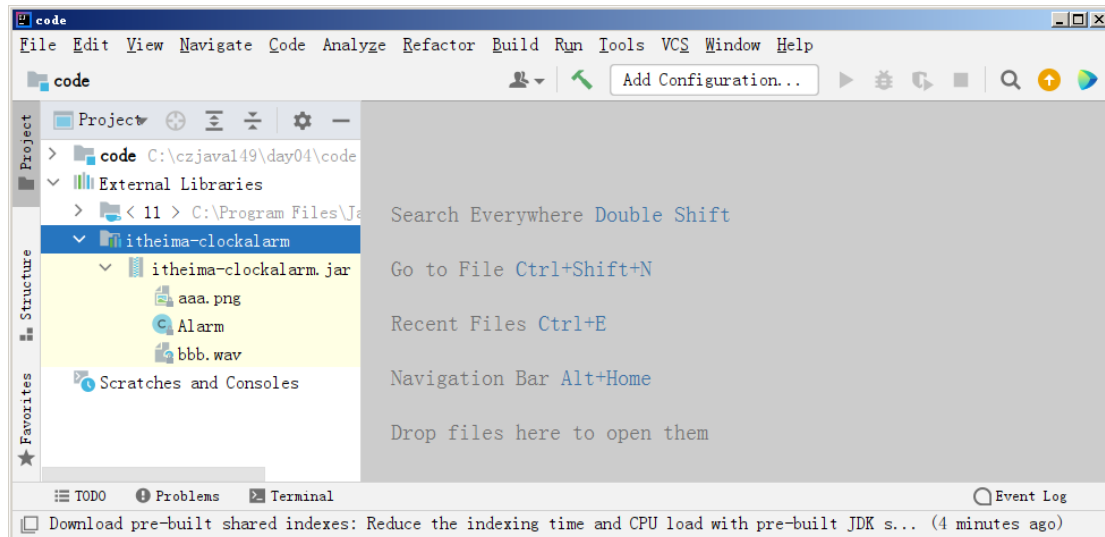


图 6-6 项目中的 Jar 包信息

2. 编写代码

创建一个名称为 TestAlarm 的类，在 main() 方法中调用 Alarm 类定义的方法，实现闹铃程序，具体代码如文件 6-2 所示。

文件 6-2 TestAlarm.java

```
public class TestAlarm {  
    public static void main(String[] args) {  
        Alarm.init();  
        while (Alarm.checkSetting()) { // 判断闹钟开关状态  
            Alarm.beep();  
        }  
    }  
}
```

3. 运行程序

单击  按钮运行程序，效果如图 6-7 所示。





图 6-7 文件 6-2 的运行结果

在图 6-7 中，闹铃的状态是可以修改的。默认情况下，闹铃的状态是开启，闹铃会一直响，如果我们手动将闹铃状态改为关闭，此时闹铃就会停止响。

6.1.3 应用案例：计算 100 以内数据和

本案例我们要计算 1~100 之间的数据和，我们可以按照下列思路实现：

- (1) 定义求和变量 sum，初始化值为 0；
- (2) 使用 while 循环获取 1~100 之间的数据；
- (3) 将每个数据进行累计，并赋值给 sum；
- (4) 循环结束，输出 sum 的值。

接下来，我们按照上述思路编写代码，具体代码如文件 6-3 所示。

文件 6-3 Test.java

```
public class Test {  
    // 计算 100 以内的数据和  
    public static void main(String[] args) {  
        int i = 1;  
        int sum = 0;  
        while (i <= 100) {  
            sum += i;  
            i++;  
        }  
        System.out.println(sum);  
    }  
}
```

程序运行结果如图 6-8 所示。



图 6-8 文件 6-3 的运行结果

6.2 do-while 循环

do-while 循环是另外一种 while 循环的写法，其语法格式具体如下：

```
do {  
    循环体语句;  
} while(判断条件语句);
```



上述格式中，程序首先会执行 do 中的循环体语句，然后进行 while 中的条件判断，如果判断条件为 true，那么会继续执行 do 中的循环体语句，如果判断条件为 false，那么就不再执行 do 中的循环体语句，循环结束。图 6-8 描述了 do-while 循环的执行流程。

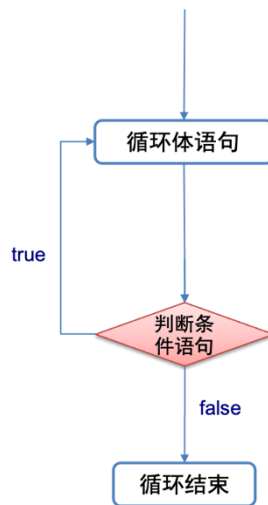


图 6-8 do-while 循环执行流程

注意：

1. 在 do-while 循环中，while 小括号后面的分号是不可省略的。
2. do...while 循环的循环体语句至少执行 1 遍。

下面看一个 do-while 循环的简单示例，如文件 6-4 所示。

文件 6-4 TestDoWhile.java

```
public class TestDoWhile {  
    public static void main(String[] args) {  
        int count = 1;  
        do {  
            System.out.println("haha");  
            count++;  
        } while (count <= 3);  
    }  
}
```

程序运行结果如图 6-9 所示。

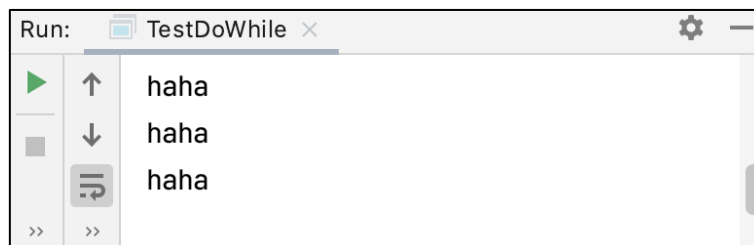


图 6-9 文件 6-4 的运行结果

从图 6-9 中可以看出，程序输出了 3 次“haha”，说明执行了 3 次 do 语句，执行过程如下：



第1次，程序执行 do 语句中的代码段，输出“haha”，并执行 count++操作，此时，count 的值为 2。

第2次，程序执行 do 语句中的代码，输出“haha”，并执行 count++操作，此时 count 的值为 3。

第3次，程序执行 do 语句中的代码，输出“haha”，并执行 count++操作，此时 count 的值为 4。

6.3 for 循环

6.3.1 while 循环入门

从功能和结构层面来看，for 循环和前面所学的 while 循环是非常类似的。for 循环的语法格式具体如下：

```
for (初始化语句；判断条件语句；控制条件语句) {  
    // 循环体  
}
```

上面的语法格式中，for 关键字后面()中包括了三部分内容，分别是初始化语句、判断条件语句以及控制条件语句，它们之间用分号(;)分隔，{}中的执行语句为循环体。

下面分别用①表示初始化语句，②表示判断条件语句，③表示控制条件语句，④表示循环体，通过序号分析 for 循环的执行流程。具体如下：

```
for (① ; ② ; ③) {  
    ④  
}
```

第一步，执行①

第二步，执行②，如果判断结果为 true，执行第三步，如果判断结果为 false，执行第五步

第三步，执行④

第四步，执行③，然后重复执行第二步

第五步，退出循环

接下来，通过一张图描述 for 循环的执行流程，如图 6-10 所示。

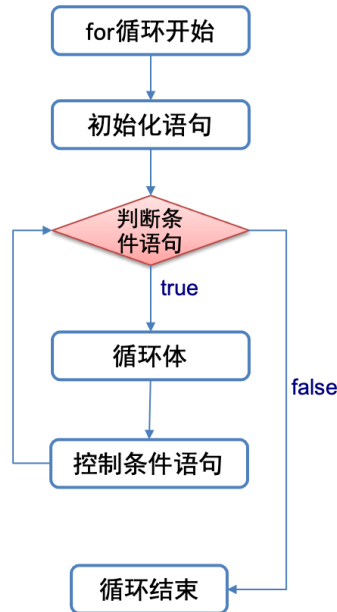


图 6-10 for 循环的执行流程

下面我们通过 for 循环输出 5 次 HelloWorld，如文件 6-5 所示。

文件 6-5TestFor. java

```
public class TestFor {  
    public static void main(String[] args) {  
        // for(初始化语句;判断条件语句;控制条件语句){ // 循环体内容}  
        for (int i = 0; i < 5; i++) {  
            System.out.println("HelloWorld");  
        }  
    }  
}
```

运行结果如图 6-11 所示。

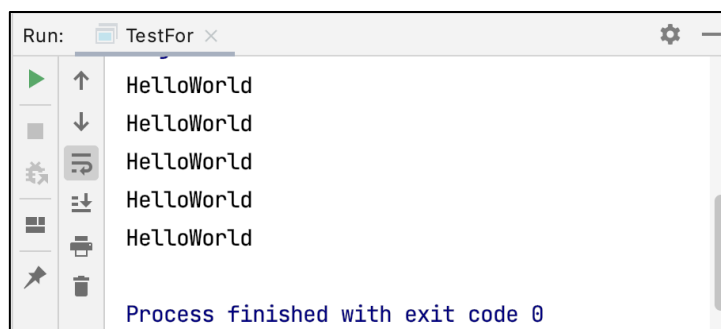


图 6-11 文件 6-5 的运行结果

小提示：

在 IDEA 中编写 for 循环的时候，我们可以输入“for i”，此时，编辑器会自动补全 for 循环格式，效果如图 6-12 所示。

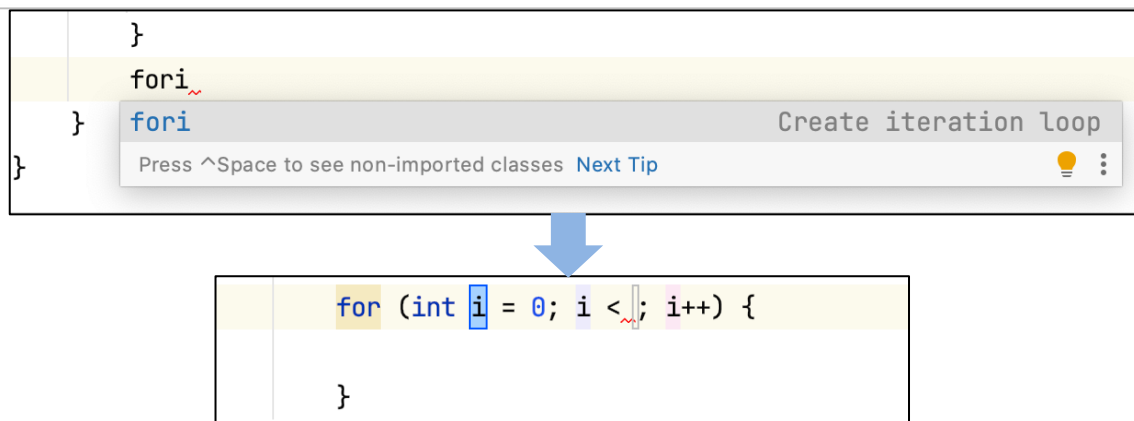


图 6-12 在 IDEA 中自动补全 for 循环结构

对于初学者来说，如果不记得 for 循环格式也不用担心，因为这种方式可以帮助我们编写好 for 循环格式，我们只需将 for 循环中的初始值、判断条件语句以及控制语句进行修改即可。

6.3.2 应用案例：水仙花

水仙花数，是指一个三位数，其各位数的立方和等于该数本身。例如，153 就是一个水仙花数： $1*1*1+5*5*5+3*3*3=153$ 。请编写代码，在控制台输出所有的“水仙花数”。

根据上述需求，我们可以按照下面思路实现：

- (1) 获取所有的三位数，即 100~1000 之间的数；
- (2) 获取每一个三位数的个位、十位、百位，示例如下：
 - 个位： $153 \% 10 = 3$
 - 十位： $153 \% 10 \% 10 = 5$
 - 个位： $153 / 10 / 10 \% 10 = 1$

(3) 将个位、十位、百位的立方与该三位数本身进行比较，如果相等，则在控制台打印这个三位数。

接下来，我们按照上述思路编写代码，具体如文件 6-6 所示。

文件 6-6 TestForLoop3.java

```
public class TestForLoop3 {
    public static void main(String[] args) {
        for (int i = 100; i < 1000; i++) {
            int ge = i % 10;
            int shi = i / 10 % 10;
            int bai = i / 100 % 10;
            if ((ge * ge * ge + shi * shi * shi + bai * bai * bai) == i) {
                System.out.println(i);
            }
        }
    }
}
```




上述代码中，先获取数字的个位、十位和百位，然后通过判断各位数的立方和是否等于数字本身，如果等于则输出这个数字。

由于判断水仙花数的代码是重复执行的，我们可以对上述代码进行优化，将判断水仙花数的实现代码抽取成一个方法，具体代码如文件 6-7 所示。

文件 6-7 TestForLoop3.java

```
public class TestForLoop3 {  
    public static void main(String[] args) {  
        for (int i = 100; i < 1000; i++) {  
            if (isNarcissisticNumber(i)) {  
                System.out.println(i);  
            }  
        }  
    }  
  
    private static boolean isNarcissisticNumber(int number) {  
        int ge = number % 10;  
        int shi = number / 10 % 10;  
        int bai = number / 100 % 10;  
        if ((ge * ge * ge + shi * shi * shi + bai * bai * bai) == number) {  
            return true;  
        } else {  
            return false;  
        }  
    }  
}
```

运行结果如图 6-13 所示。

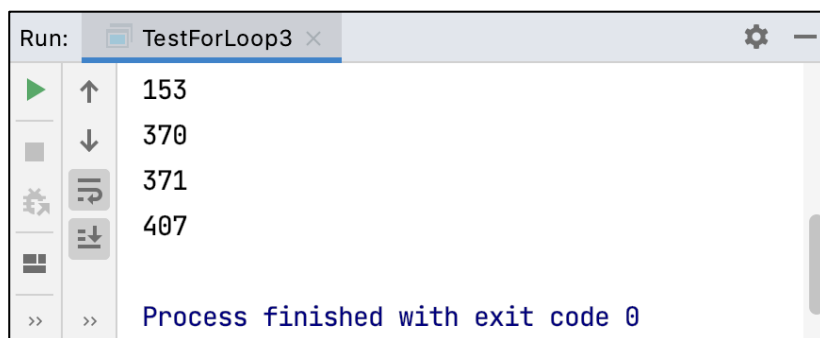


图 6-13 文件 6-7 的运行结果

接下来，我们对上述功能进行扩展，统计所有“水仙花数”的个数，可以对文件修改，再定义一个变量 count 用来计数，并设置初始化为 0，一旦判断三位数是水仙花数，那么就让 count 的值加 1。修改后的代码如文件 6-8 所示。

文件 6-8 TestForLoop3.java

```
public class TestForLoop3 {  
    public static void main(String[] args) {  
        int count = 0;  
        for (int i = 100; i < 1000; i++) {
```



```

        if (isNarcissisticNumber(i)) {
            count++;
            System.out.println(i);
        }
    }

    System.out.println("100-1000 之间水仙花的数量是: " + count);
}

private static boolean isNarcissisticNumber(int number) {
    int ge = number % 10;
    int shi = number / 10 % 10;
    int bai = number / 100 % 10;
    if ((ge * ge * ge + shi * shi * shi + bai * bai * bai) == number) {
        return true;
    } else {
        return false;
    }
}
}

```

运行结果如图 6-14 所示。

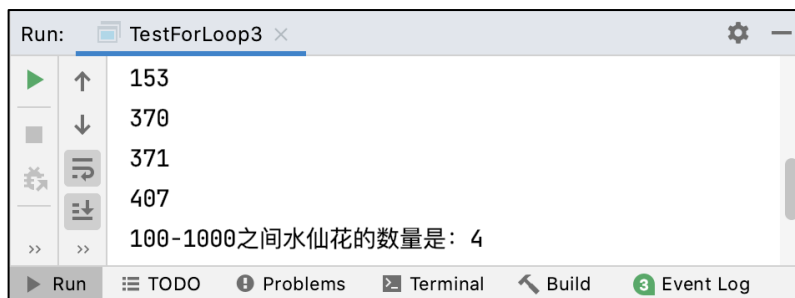


图 6-14 文件 6-8 的运行结果

6.3.3 应用案例：九九乘法表

本节我们使用 for 循环打印一个如图 6-15 所示的九九乘法表。

1×1=1									
1×2=2	2×2=4								
1×3=3	2×3=6	3×3=9							
1×4=4	2×4=8	3×4=12	4×4=16						
1×5=5	2×5=10	3×5=15	4×5=20	5×5=25					
1×6=6	2×6=12	3×6=18	4×6=24	5×6=30	6×6=36				
1×7=7	2×7=14	3×7=21	4×7=28	5×7=35	6×7=42	7×7=49			
1×8=8	2×8=16	3×8=24	4×8=32	5×8=40	6×8=48	7×8=56	8×8=64		
1×9=9	2×9=18	3×9=27	4×9=36	5×9=45	6×9=54	7×9=63	8×9=72	9×9=81	



图 6-15 九九乘法表

通过观察九九乘法表，可以得出一些规律：乘法表中共有 9 行，其中第 n 行，共有 n 个表达式，表达式是从 $1*n$ 开始，一直到 $n*n$ 结束。（ $9 \geq n \geq 1$ ）

根据上述规律，我们可以使用下面方式实现：

(1) 使用 for 循环控制行数 n ，行数是从 $1 \sim 9$ 。

(2) 使用 for 循环控制每行输出的表达式顺序及个数，表达式的顺序是从 $1*n$ 开始，一直到 $n*n$ 结束。

接下来，我们按照上述思路编写代码，具体如文件 6-9 所示。

文件 6-9 Test99.java

```
public class Test99 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 9; i++) {  
            printN(i);  
        }  
    }  
    // 打印九九乘法表中的一行  
    public static void printN(int n) {  
        for (int i = 1; i <= n; i++) {  
            System.out.print(i + "*" + n + "=" + (i * n) + " ");  
        }  
        System.out.println("");  
    }  
}
```

运行结果如图 6-16 所示。

```
Run: Test99 x  
1*1=1  
1*2=2 2*2=4  
1*3=3 2*3=6 3*3=9  
1*4=4 2*4=8 3*4=12 4*4=16  
1*5=5 2*5=10 3*5=15 4*5=20 5*5=25  
1*6=6 2*6=12 3*6=18 4*6=24 5*6=30 6*6=36  
1*7=7 2*7=14 3*7=21 4*7=28 5*7=35 6*7=42 7*7=49  
1*8=8 2*8=16 3*8=24 4*8=32 5*8=40 6*8=48 7*8=56 8*8=64  
1*9=9 2*9=18 3*9=27 4*9=36 5*9=45 6*9=54 7*9=63 8*9=72 9*9=81  
Process finished with exit code 0
```

图 6-16 文件 6-9 的运行结果

6.4 跳转语句

前面我们使用 while 和 for 循环控制代码循环执行。但是在某些特殊情况下，程序员需要根据情况改变循环的行为，break 和 continue 提供了这样的能力。



6.4.1 break 语句

break 表示中断，用于 switch 语句和循环语句。在 switch 语句中，表示结束 switch 代码块。在循环语句中，表示结束循环。

下面看一个案例演示 break 的作用，要求循环到一半的时候退出，如文件 6-10 所示。

文件 6-10 TestForLoop4.java

```
public class TestForLoop4 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            System.out.println(i);  
            if (i == 5) {  
                break;  
            }  
        }  
    }  
}
```

运行结果如图 6-17 所示。



图 6-17 6-10 的运行结果

6.4.2 continue 语句

continue 表示继续，用于循环语句，表示结束本次循环，继续下次循环。接下来，通过一个示例代码演示 continue 的作用，具体如文件 6-11 所示。

文件 6-11 Test.java

```
6 public class Test {  
7     public static void main(String[] args) {  
8         for (int i = 0; i < 5; i++) {  
9             if (i == 3) {  
10                // 跳出一次当前循环，不执行下面的语句，但是循环的逻辑还会继续  
11                continue;  
12            }  
13            System.out.println(i);  
14        }  
}
```



```
15     }  
16 }
```

运行结果如图 6-18 所示。



图 6-18 文件 6-11 的运行结果

从图 6-11 中可以看出，1~5 之间的整数，只有数字 3 没有输出。这是因为在 6-11 中，当变量 *i* 的值为 3 时，`continue` 语句结束了一次 `for` 循环，没有执行第 8 行代码输出 *i* 的值，从而继续跳转到第 3 行代码执行下一次 `for` 循环。

6.4.3 应用案例：逢七必过

为了帮助大家加深对 `continue` 语句的理解，接下来，开发一个“逢七必过”游戏。

逢七必过的游戏规则是：多人围坐在一起，依次快速说出从 1-100 的数字，所有含 7 或者 7 的倍数不能说，否则就失败接受惩罚。

根据上述需求，我们可以按照下面思路实现：

1. 使用 `for` 循环遍历 1-100 的数。
2. 在循环体中，判断数中是否含 7 或是否为 7 的倍数。
 - A. 是否含有 7：个位含 7（模以 10 等于 7），十位含 7（70-79）
 - B. 是否是 7 的倍数：对 7 取模，余数为 0，则是 7 的倍数。
3. 使用 `continue` 跳过所有含 7 和 7 的倍数的数，打印其他数。

接下来，我们按照上述思路编写代码，具体如文件 6-12 所示。

文件 6-12 TestForLoop5.java

```
public class TestForLoop5 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 100; i++) {  
            if (isContents7(i)) {  
                continue;  
            }  
            System.out.println(i);  
        }  
    }  
    // 检查是否包含 7 或者是 7 的倍数  
    private static boolean isContents7(int i) {  
        // 个位数包含 7  
        if (i % 10 == 7) {  
            return true;  
        }  
    }  
}
```



```
// 十位数包含 7
if (70 <= i && i <= 79) {
    return true;
}

// 7 的倍数
if (i % 7 == 0) {
    return true;
}

return false;
}
```

运行结果如图 6-19 所示。

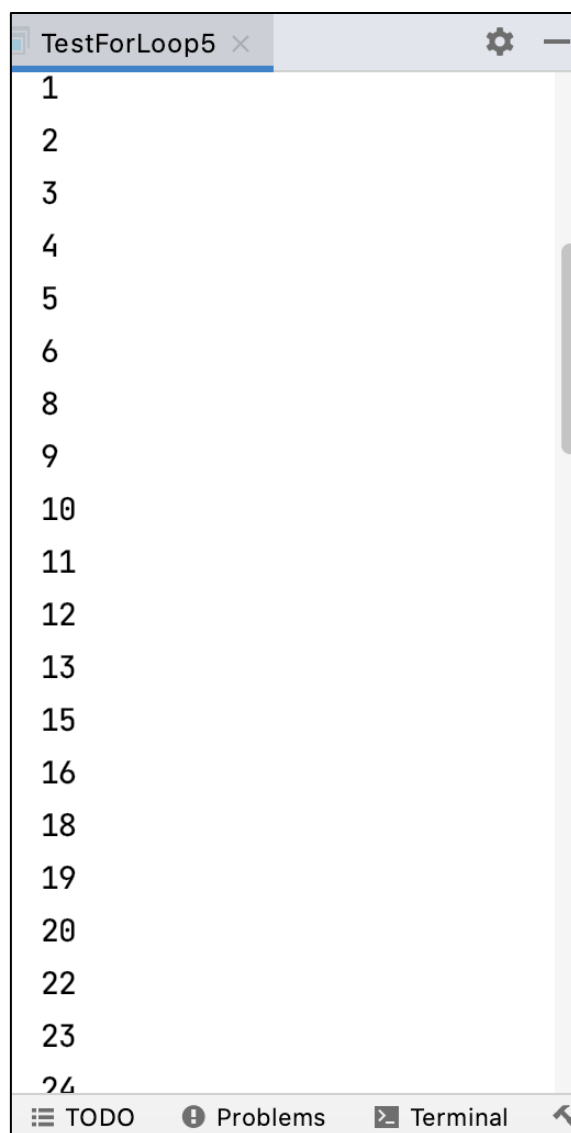


图 6-19 运行结果



第 7 章 数组

学习目标

- ◆ 熟悉什么是数组，能够准确定义数组并计算数组长度
- ◆ 掌握数组在函数和增强 for 循环中的使用，能够灵活使用数组

完成特定需求

- ◆ 熟悉二维数组的定义方式，能够准确说出二维数组元素的排列

方式

- ◆ 了解 Java 程序在内存中的加载方式，能够画出 Java 程序运行

过程中的内存情况

- ◆ 掌握数组动态扩容的方式，能够独立实现数组动态扩容

现在需要统计某公司员工的工资情况，例如，计算员工平均工资、最高工资等。假设该公司有 50 名员工，用前面所学的知识，程序首先需要声明 50 个变量分别存储每位员工的工资，这样做会很麻烦。在 Java 中，可以使用一个数组存储这 50 名员工的工资。数组是指一组类型相同的数据的集合，数组中的每个数据被称作元素。数组可以存放任意类型的元素，但同一个数组里存放的元素类型必须一致。数组可分为一维数组和多维数组，本章将围绕数组相关知识进行详细地讲解。

7.1 初识数组

7.1.1 数组的概念

在介绍数组之前，我们先了解一下小组的概念。

小组是为了工作、学习的方便而组成的小集体，比如，党小组、学习小组、讨论小组等。

而数组就是由数据组成的一个小组，里面可以按顺序存放某种类型的多条数据，方便计算机去存储和读取。

下面我们看一个数组的情景——随机点名，如图 7-1 所示。我们要开发一个随机点名的程序，班级里面的学生的花名册是：范剑，范统，夏建仁，朱逸群，秦寿生，庞光，杜琦燕，史珍香，沈京兵，毕云涛。



图 7-1 随机点名场景

如果按照我们之前所学的知识,如果要保存花名册的信息,需要定义 10 个字符串变量,如下所示:

```
String name1 = "范剑";  
String name2 = "范统";  
String name3 = "夏建仁";  
String name4 = "朱逸群";  
String name5 = "秦寿生";  
String name6 = "庞光";  
String name7 = "杜琦燕";  
String name8 = "史珍香";  
String name9 = "沈京兵";  
String name10 = "毕云涛";
```

显然,上述这种做法非常麻烦,而且书写起来也非常不方便。

这时,我们可以使用数组保持花名册信息。数组声明的格式比较简单,示例代码如下:

```
String[] names = {"范剑", "范统", "夏建仁", "朱逸群", "秦寿生",  
                  "庞光", "杜琦燕", "史珍香", "沈京兵", "毕云涛"};
```

上述代码中, **String[]** 表示是一个字符串数组, **names** 是字符串数组的名字, {} 中的内容是字符串数据存储的具体数据。

需要注意的是,数组中最小的索引(或下标)是 0,最大的索引(或下标)是“数组的长度-1”。为了方便获得数组的长度,Java 提供了一个 **length** 属性,在程序中可以通过“数组名.length”的方式获得数组的长度,即元素的个数。

下面通过一张示意图展示字符串和字符串存储数据的区别,如图 7-2 所示。

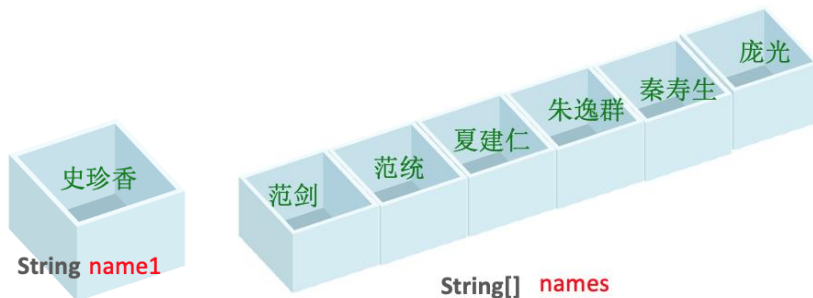




图 7-2 字符串和字符串数组存储数据的示意图

下面通过代码演示数组的使用，如文件 7-1 所示。

文件 7-1 ArrayDemo01.java

```
public class ArrayDemo01 {  
    public static void main(String[] args) {  
        String[] names = {"范剑", "范统", "夏建仁", "朱逸群", "秦寿生",  
                           "庞光", "杜琦燕", "史珍香", "沈京兵", "毕云涛"};  
        System.out.println(names[5]);  
    }  
}
```

上述代码中，首先定义了一个字符串数组 names，然后访问了 names 的第 6 个元素。程序的运行结果如图 7-3 所示。

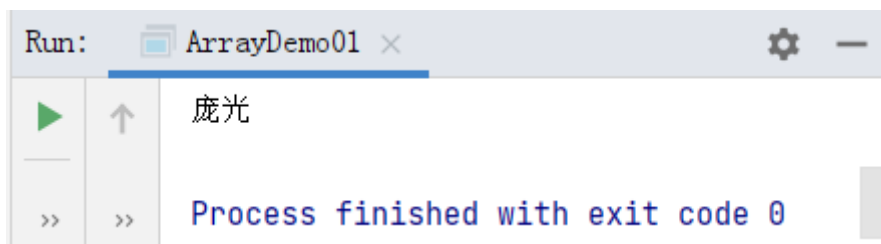


图 7-3 运行结果

当然，数组不仅可以是字符串类型，还可以是整型数组、浮点型数组或者双精度浮点数组，这些不同类型数组的初始化方式都比较类似，示例代码如下：

```
整数数组: int[] a = {1,2,3,4,5,7};  
浮点型数组: float[] b = {1.2f, 2.4f, 3.6f, 7.8f};  
双精度浮点数组 double[] c = {3.14,6.25, 9.65,3.88};
```

7.1.2 数组的长度

在 7.1.1 节，我们创建了一个数组 names[]，并调用 names[5] 访问了数组元素。但是，如果我们访问 names[12]，发现程序会报错，提示“Index 12 out of bounds for length 10”。这是因为数组 names 的长度是 10，如果访问索引是 12 的元素，超出了数组长度，所以我们访问数组元素时，要保证数组的下标不能越界。程序报错信息如图 7-4 所示。

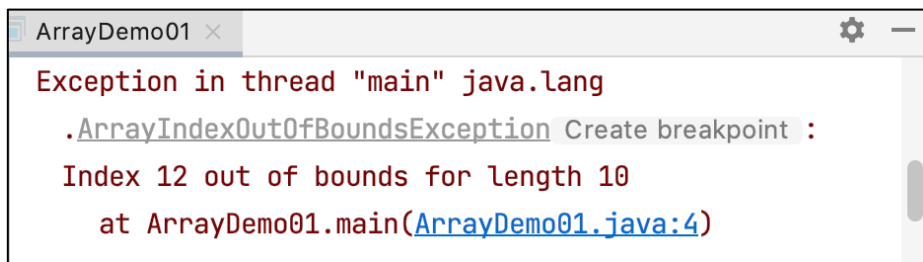


图 7-4 程序报错信息



回归我们场景的需求，现在我们要实现随机点名的功能，我们可以先生成 0-9 之间的随机数，然后将随机数作为数组下标访问数据，具体代码如下：

```
public class ArrayDemo01 {  
    public static void main(String[] args) {  
        String[] names = {"范剑", "范统", "夏建仁", "朱逸群", "秦寿生",  
                           "庞光", "杜琦燕", "史珍香", "沈京兵", "毕云涛"};  
        int index = (int) (Math.random() * 10);  
        System.out.println(names[index]);  
    }  
}
```

运行上述程序，发现程序每次运行的结果不一样，访问到的人名都是随机产生的。至此，一个随机点名程序就完成了。

这里再补充介绍一点的是，数组是一个复杂的数据类型，因为数组一旦创建，其大小就不会发生变化，所以数组有一个特殊的属性，叫做 `length`。

这时候，你可能会说，之前字符串有个方法是 `length()`，为什么是带小括号的，而数组的 `length` 不带小括号呢？其中的一个原因是，Java 程序在设计时，出于对性能方面的考虑，如果使用 `length` 属性，获取数据的速度和效率比调用方法都要快。再者，数组是一个很常用的复杂数据类型，所以设计了 `length` 属性，没有使用 `length()` 方法。

例如，输出数组 `names` 大小的示例代码具体如下：

```
System.out.println(names.length);
```

需要注意的是，数组大小一旦确定，就不能再修改。

7.2 数组在函数中的应用

数组作为一种数据类型，它既可以作为函数的参数，批量传入数据，也可以作为函数的返回值，批量输出结果。下面我们看几个应用场景，体验一下数组输入和输出，是如何和函数结合使用的。

7.2.1 应用案例：计算平均成绩

假设现在我们要计算某个班级的平均成绩，那么我们需要定义一个求平均值的函数，该函数需要接收一个数组，该数组中存储的数据是几门课的成绩。由于要计算数组元素的和，所有可以使用 `for` 循环遍历数组，并且在遍历的过程中累加求和，计算出成绩的总和，最后再计算平均成绩。具体代码如文件 7-2 所示。

文件 7-2 ArrayDemo02.java

```
public class ArrayDemo02 {  
    public static void main(String[] args) {  
        double[] a = {98, 99, 100, 96};  
        System.out.println(getAvg(a));  
    }  
}
```

```
// 求平均值的函数
public static double getAvg(double[] inputs) {
    double sum = 0;
    for (int i = 0; i < inputs.length; i++) {
        sum += inputs[i];
    }
    return sum / inputs.length;
}
}
```

程序运行结果如图 7-5 所示。

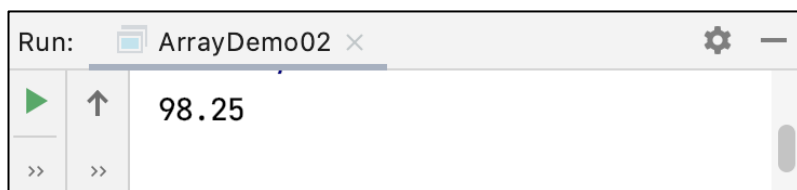


图 7-5 文件 7-2 的运行结果

7.2.2 应用案例：查找班级姓“范”的所有同学

根据需求可知，我们需要定义一个打印姓范的所有同学的函数，该函数传入一个数组，该数组存储的是班级里面所有学生的名字。因为要查询班级里面姓范的同学，所有我们可以使用 for 循环遍历数组，在遍历的过程中判断学生的名字是否以范字开头，如果是的话，则输出这个同学的名字。具体代码如文件 7-3 所示。

文件 7-3 ArrayDemo03.java

```
public class ArrayDemo03 {
    public static void main(String[] args) {
        String[] names = {"范剑", "范统", "夏建仁", "朱逸群", "秦寿生",
            "庞光", "杜琦燕", "史珍香", "沈京兵", "毕云涛"};
        printNames(names);
    }
    // 打印姓范的所有同学
    public static void printNames(String[] names) {
        for (int i = 0; i < names.length; i++) {
            if (names[i].startsWith("范")) {
                System.out.println(names[i]);
            }
        }
    }
}
```

程序运行结果如图 7-6 所示。

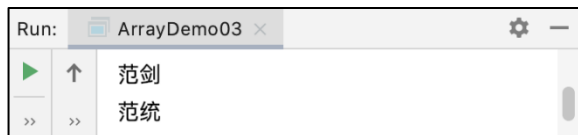


图 7-6 文件 7-3 的运行结果

当然，我们还可以对 `printNames()` 方法进行扩展，我们在定义 `printNames()` 方法时，要求不仅传入班级所有学生的数组，还得传入表示姓的字符串。如果在遍历数组的过程中，发现某个学生的姓与判定的姓一致，那么就打印出学生的姓名，接下来，我们对文件 7-3 进行修改，修改后的代码如文件 7-4 所示。

文件 7-4 ArrayDemo03.java

```
public class ArrayDemo03 {
    public static void main(String[] args) {
        String[] names = {"范剑", "范统", "夏建仁", "朱逸群", "秦寿生",
            "庞光", "杜琦燕", "史珍香", "沈京兵", "毕云涛"};
        printNames(names, "范");
    }
    // 查找某个姓的所有同学
    public static void printNames(String[] names, String perfix) {
        for (int i = 0; i < names.length; i++) {
            if (names[i].startsWith(perfix)) {
                System.out.println(names[i]);
            }
        }
    }
}
```

在文件 7-4 中，定义的 `printNames()` 方法需要两个参数，其中第 1 个参数表示遍历的数组，第 2 个参数表示要查找的姓。与文件 7-3 相比，文件 7-4 的代码扩展性更好，因为在文件 7-4 中，如果我们想查找班级中的某个姓的同学，只需要调用 `printNames()` 时传入指定的姓即可，而无需手动修改 `printNames()` 方法中的代码。

7.3 数组在增强 for 循环中的应用

增强 for 循环是个语法糖，你可以编写更简洁的代码，不需要使用计数器就可以遍历数组的每一个元素。

增强 for 循环的语法格式具体如下：

```
for (数据类型 参数 x: 数组名) {
    参数 x
}
```

下面通过一个示例代码带大家体会增强 for 循环的便捷，具体如文件 7-5 所示。

文件 7-5 ArrayDemo04.java

```
17 public class ArrayDemo04 {
```



```
18 public static void main(String[] args) {
19     int[] arr = {1, 2, 3, 4, 5, 6, 7};
20     for (int x : arr) {
21         System.out.println(x);
22     }
23 }
24 }
```

上述代码中，第4~6行代码与下面的代码是等同的。

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(i);
}
```

运行结果如图7-7所示。

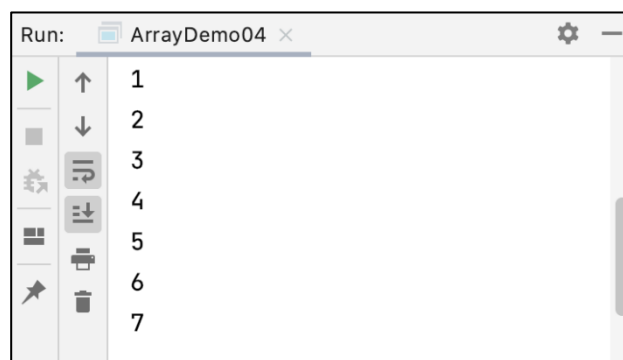


图 7-7 文件 7-5 的运行结果

7.4 二维数组

二维数组本质上是以数组作为数组元素的数组，即“数组的数组”。直接看概念的话，理解起来比较抽象，接下来，我们看一段代码，如文件7-6所示。

文件 7-6 ArrayDemo05.java

```
1 public class ArrayDemo05 {
2     public static void main(String[] args) {
3         int a = 3;
4         int[] b = {3, 4, 5, 6};
5         int[][] c = {{1, 2, 3}, {4, 5, 6, 9}, {7}};
6         System.out.println("数组 c 的第 1 行的长度为: " + c[0].length);
7         System.out.println("数组 c 的第 2 行的长度为: " + c[1].length);
8         System.out.println("数组 c 的第 3 行的长度为: " + c[2].length);
9     }
10 }
```

运行结果如图7-8所示。

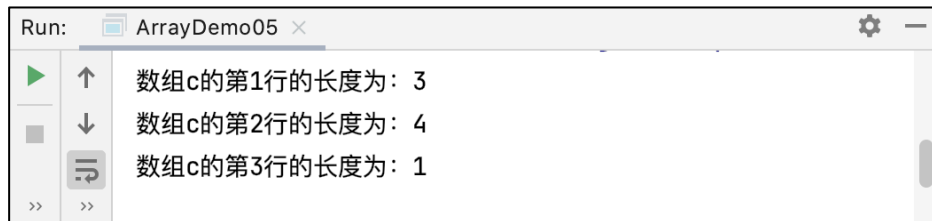


图 7-8 文件 7-6 的运行结果

上述代码中，第 5 行代码定义了一个名称为 c 的 二维数组，该二维数组存放了 3 个一维数组，分别是 {1, 2, 3}、{4, 5, 6, 9}、{7}，这些数据的存放形式如图 7-9 所示。

	Column 1	Column 2	Column 3	Column 4
Row1	1 c[0][0]	2 c[0][1]	3 c[0][2]	
Row2	4 c[1][0]	5 c[1][1]	6 c[1][2]	9 c[1][3]
Row3	7 c[2][0]			

图 7-9 二维数组 c 的数据存放形式

如果我们将二维数组 c 中的所有数据打印出来，可以通过双层 for 循环实现，其中外层 for 循环用来遍历二维数组的元素，也就是三个一维数组，而内层 for 循环用来遍历三个一维数组中的数据，具体代码如文件 7-7 所示。

文件 7-7 ArrayDemo05.java

```
public class ArrayDemo05 {  
    public static void main(String[] args) {  
        int a = 3;  
        int[] b = {3, 4, 5, 6};  
        int[][] c = {{1, 2, 3}, {4, 5, 6, 9}, {7}};  
        System.out.println("数组 c 的第 1 行的长度为: " + c[0].length);  
        System.out.println("数组 c 的第 2 行的长度为: " + c[1].length);  
        System.out.println("数组 c 的第 3 行的长度为: " + c[2].length);  
        for (int i = 0; i < c.length; i++) {  
            for (int j = 0; j < c[i].length; j++) {  
                System.out.println(c[i][j]);  
            }  
        }  
    }  
}
```



```
}
```

运行结果如图 7-10 所示。

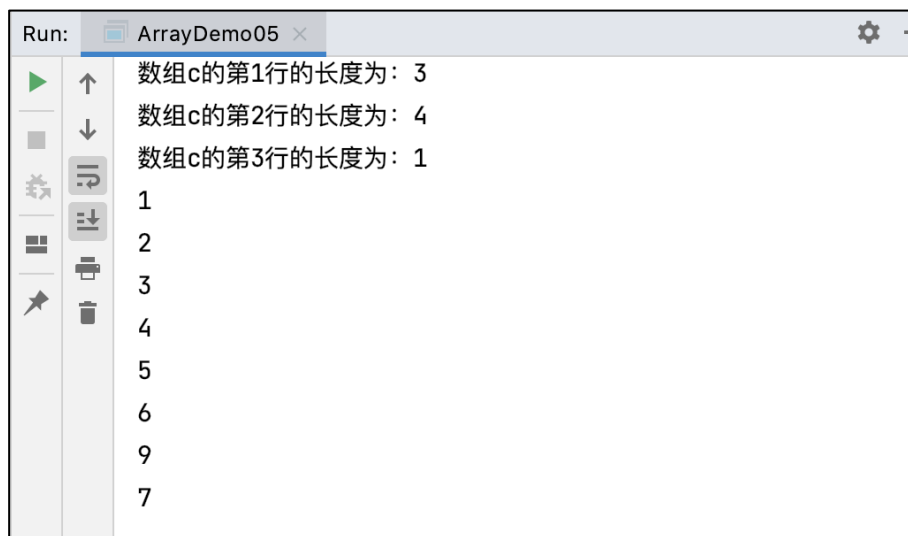


图 7-10 文件 7-7 的运行结果

当然了，我们也可以使用增强 for 循环打印二维数组中每个元素中的具体数据，示例代码如下：

```
public class ArrayDemo05 {
    public static void main(String[] args) {
        int a = 3;
        int[] b = {3, 4, 5, 6};
        int[][] c = {{1, 2, 3}, {4, 5, 6, 9}, {7}};
        System.out.println("数组 c 的第 1 行的长度为: " + c[0].length);
        System.out.println("数组 c 的第 2 行的长度为: " + c[1].length);
        System.out.println("数组 c 的第 3 行的长度为: " + c[2].length);
        for (int[] x : c) {
            for (int item : x) {
                System.out.println(item);
            }
            System.out.println(x);
        }
    }
}
```

值得一提的是，多维数组最简单形式是二维数组，存放二维数组数据的数组是三维数组，大家可以举一反三，这里不做过多演示了。

7.4 使用 new 的方式创建数组并赋值

使用 new 关键字的方式完成数组初始化。数组初始化方式有两种：

第 1 种方式：声明时赋值，示例代码如下：

```
int[] a={1,2,3,4,5};// 声明式赋值
```



第 2 种方式：先声明，后赋值，示例代码如下：

```
int[] a;  
a = new int[]{1, 2, 3, 4, 5};
```

需要注意的是，上述两种初始化数组的方式是互斥的，我们只能按照其中一种方式对数组进行初始化。

使用先声明后赋值的方式初始化时，可以不指定每个元素的值，但是必须指定数组大小。例如，声明一个 `int` 类型的数组，如果指定了数组的大小是 3，此时数组元素默认值都是 0，具体代码如文件 7-8 所示。

文件 7-8 ArrayDemo06.java

```
public class ArrayDemo06 {  
    public static void main(String[] args) {  
        int[] b;  
        b = new int[3];  
        for (int x : b) {  
            System.out.println(x);  
        }  
    }  
}
```

运行结果如图 7-11 所示。

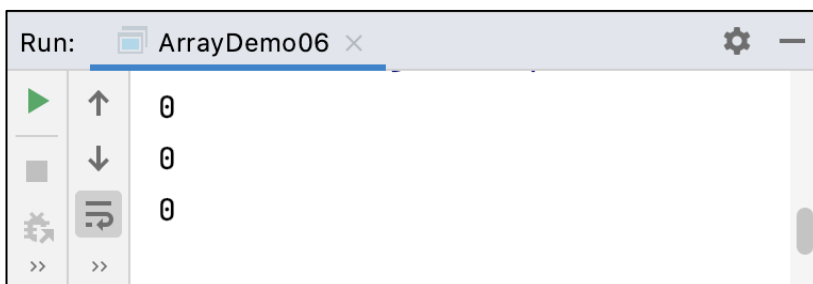


图 7-11 文件 7-8 的运行结果

7.5 Java 程序内存分析

7.5.1 程序加载的内存区域

我们知道，程序加载到内存中后，数据都是以二进制形式存储的。对于计算机来说，是无法区分这些二进制数据到底是函数，还是某个数值。所以无论是哪种计算机语言编写的程序加载到内存中，都被划分为 4 个部分，分别是 `code`、`data`、`stack` 和 `heap`，具体介绍如下：

1. `code`：表示代码段区，用于存储方法的二进制数据。
2. `data`：表示数据区，通常存放一些静态、不可变的数据。
3. `stack`：表示栈，用于存放方法中定义的临时变量或者参数。
4. `heap`：表示堆，Java 中所有 `new` 关键字创建的数据都存在堆内存。



7.5.2 函数调用过程内存分析

为了大家更好理解内存区域的作用，接下来，我们通过一段代码演示函数调用过程中的内存分配情况。程序代码如文件 7-9 所示。

文件 7-9 Test.java

```
1 public class Test{
2     public static void main(String[] args) {
3         int[] a = {1, 2, 3, 4, 5};
4         int[] b = {1, 2, 3, 4, 5, 6, 7};
5         print(a);
6     }
7     public static void print(int[] x) {
8         for (int i = 0; i < x.length; i++) {
9             System.out.println(x[i]);
10        }
11    }
12 }
```

上述程序运行后，内存区域会被分为 4 部分，由于程序中没有定义静态数据，并且没有使用 new 关键字创建对象，所以 data 区域和 heap 区域是没有数据的。而 code 区和 stack 区的内存情况如下：

(1) code 区：一旦程序编译运行，程序中的 main() 函数和 print() 函数的二进制数据地址就会存储在 code 区，如图 7-12 所示。

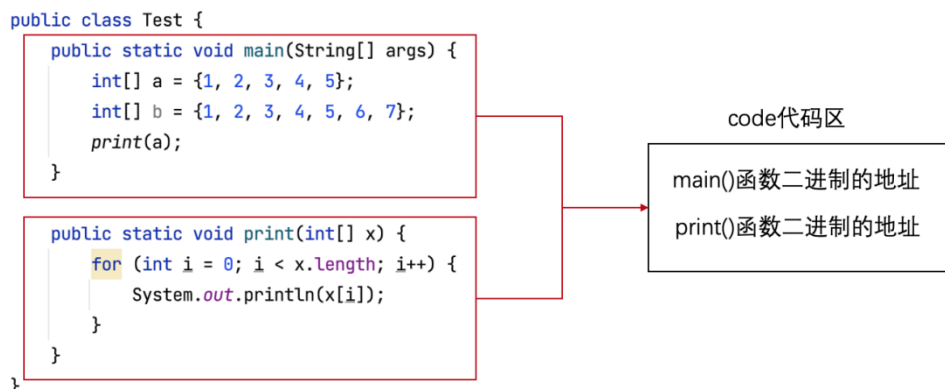


图 7-12 程序在 code 区的内存

(2) stack 区：由于程序是从上至下执行，所以当程序执行 main() 方法时，栈内存的情况如图 7-13 所示。

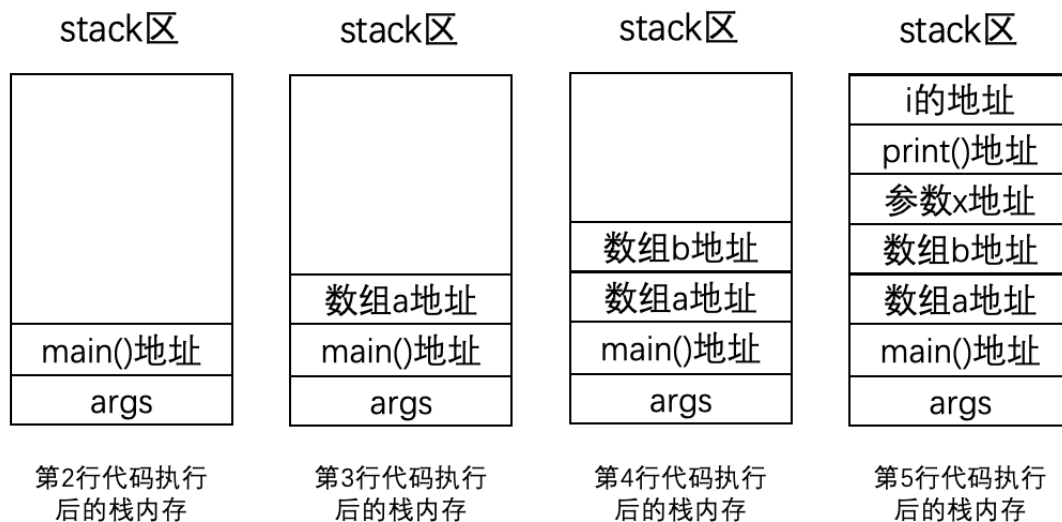


图 7-13 程序在栈内存的情况

在图 7-13 中，由于程序调用 print() 函数传入的是数组 a，所以参数 x 的地址和数组 a 的地址是相同的。

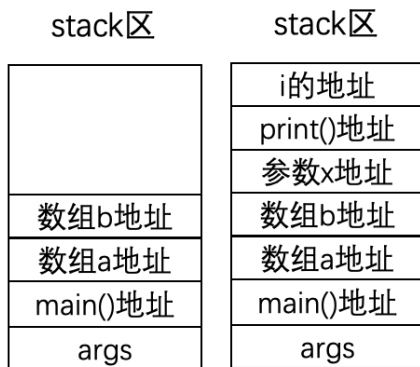
假设，现在我们对 main() 函数进行修改，不仅输出数组 a 的值，而且输出数组 b 的值，具体代码如下：

```

1 public class Test {
2     public static void main(String[] args) {
3         int[] a = {1, 2, 3, 4, 5};
4         int[] b = {1, 2, 3, 4, 5, 6, 7};
5         print(a);
6         print(b);
7     }
8     public static void print(int[] x) {
9         for (int i = 0; i < x.length; i++) {
10             System.out.println(x[i]);
11         }
12     }
13 }

```

当程序执行第 6 行代码时，栈内存中的数据首先会进行出栈操作，将第 5 行代码在栈内存中的数据弹出，然后才会将调用 print(b) 的数据入栈，程序执行第 6 行代码时，栈内存的变化过程如图 7-14 所示。



执行第6行代码栈内存的变化

图 7-14 执行第 6 行代码栈内存的变化过程

在图 7-14 中，由于第 6 行代码调用 print() 函数传入的是数组 b，所以参数 x 的地址和数组 b 的地址是相同的。

7.5.3 两个数字交换的内存分析

我们先来看一个数字交换的案例，具体代码如文件 7-10 所示。

文件 7-10 Test.java

```
1 public class Test {  
2     public static void main(String[] args) {  
3         int a = 3; // 看做是瓶子 A,装的是酱油  
4         int b = 5; // 看做是瓶子 B,装的是醋  
5         int temp; // 看做是瓶子 C,目前是空瓶子  
6         temp = a; // 将瓶子 A 的酱油装入瓶子 C  
7         a = b;     // 将瓶子 B 的醋装入瓶子 A  
8         b = temp;  // 将瓶子 C 的酱油装入瓶子 B  
9         System.out.println(a); // 此时，瓶子 A 装的是醋  
10        System.out.println(b); // 此时，瓶子 B 装的是酱油  
11    }  
12 }
```

程序运行结果如图 7-15 所示。



图 7-15 文件 7-10 的运行结果

从图 7-15 可以看出，程序输出的 a、b 的值分别是 5 和 3，说明成功实现了数据交换的功能。

为了帮助大家更好地理解文件 7-10 中数据在栈内存的情况，接下来，我们通过一张图描述，具体如图 7-16 所示。

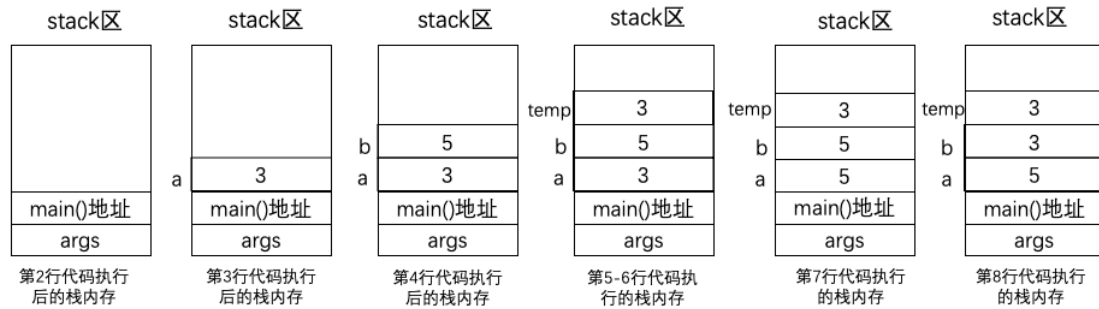


图 7-16 文件 7-10 的栈内存

如果我们对文件 7-10 进行修改，修改后的代码如文件 7-11 所示。

文件 7-11 Test.java

```
1 public class Test {  
2     public static void main(String[] args) {  
3         int a = 3;  
4         int b = 5;  
5         swap(a, b);  
6         System.out.println(a);  
7         System.out.println(b);  
8     }  
9     public static void swap(int a, int b) {  
10        int temp;  
11        temp = a;  
12        a = b;  
13        b = temp;  
14    }  
15 }
```

程序运行结果如图 7-17 所示。



图 7-17 文件 7-11 的运行结果

从图 7-17 中可以看出，程序输出的 a、b 的结果是 3、5，并没有实现数据的交换。这是为什么呢？接下来，我们对文件 7-11 的程序执行过程进行分析，具体如图 7-18 所示。

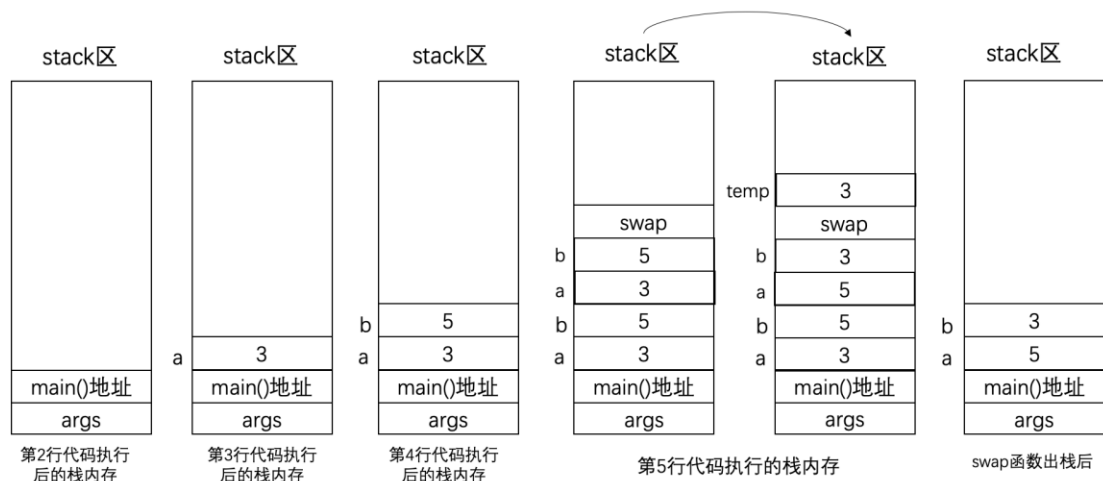


图 7-18 文件 7-11 的栈内存

从图 7-18 中可以看出，当文件 7-11 执行第 5 行代码后，交换的数据其实是 `swap()` 函数参数 `a` 和 `b` 的值，而不是 `main()` 方法中变量 `a`、`b` 的值，所以文件 7-11 并没有实现数据的交换功能。

知道了文件 7-11 的栈内存情况后，接下来，我们对文件 7-11 进行修改，使程序调用 `swap()` 函数后可以实现数据交换，修改后的代码如文件 7-12 所示。

文件 7-12 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        int a = 3;  
        int b = 5;  
        swap(a, b);  
    }  
    public static void swap(int a, int b) {  
        int temp;  
        temp = a;  
        a = b;  
        b = temp;  
        System.out.println(a);  
        System.out.println(b);  
    }  
}
```

程序的运行结果如图 7-19 所示。



图 7-19 文件 7-12 的运行结果



7.5.4 使用数组交换数据的内存分析

上个小节我们演示了两个变量的数据交换，接下来，我们使用数组实现数据交换，具体代码如文件 7-13 所示。

文件 7-13 Test.java

```
1 public class Test {  
2     public static void main(String[] args) {  
3         int[] arr = {3, 5};  
4         swap(arr);  
5         System.out.println(arr[0]);  
6         System.out.println(arr[1]);  
7     }  
8     public static void swap(int[] arr) {  
9         int temp;  
10        temp = arr[0];  
11        arr[0] = arr[1];  
12        arr[1] = temp;  
13    }  
14 }
```

程序运行结果如图 7-20 所示。



图 7-20 文件 7-13 的运行结果

从图 7-20 可以看出，文件 7-13 输出的数组元素的值是 5 和 3，说明程序成功实现了数组元素的交换。为了大家更清楚数组交换数据的内存情况，接下来通过一张图描述程序调用 swap() 函数的栈内存情况，如图 7-21 所示。

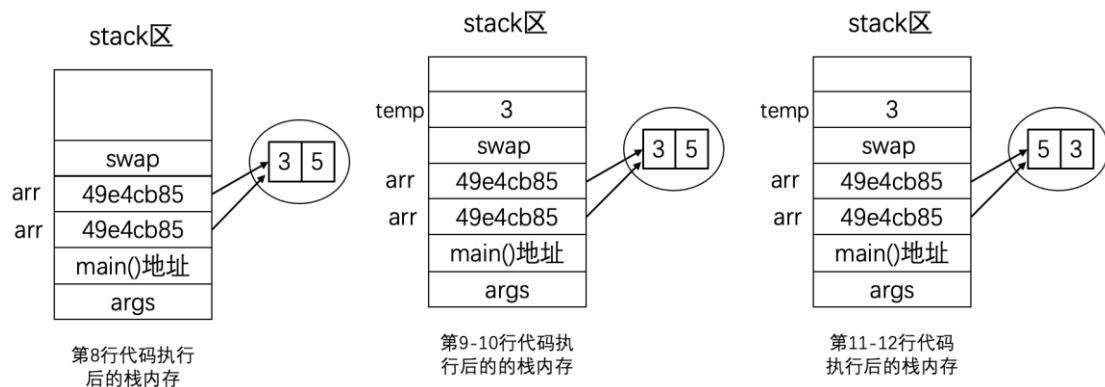


图 7-21 程序调用 swap() 函数的栈内存情况

图 7-21 描述了文件第 4 行代码调用 swap() 函数的栈内存情况。当第 4 行代码执行完毕后，swap() 函数相关的数据会在内存中执行出栈操作，出栈后的效果如图 7-22 所示。

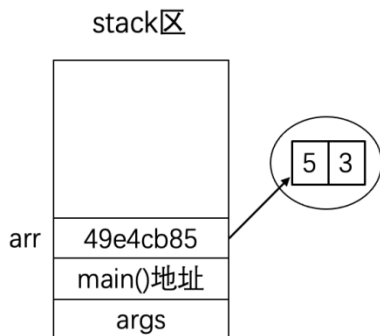


图 7-22 出栈后的效果

从图 7-22 可知，此时数组 arr 的第 1 个元素是 5，第 2 个元素是 3，正好与图 7-20 所示的输出结果是一样的。

7.6 数组的动态扩容

根据前面所学的内容，我们知道数组一旦创建，其长度是不可变的。那如果想对数组进行扩容，我们可以自定义一个方法实现数组的扩容，这个方法所做的事情，就是传入一个原始数组以及要添加的一个元素，然后将元素添加到原始数组中，生成并返回一个新的数组。

用于实现数组扩容的 `addAnElement()` 方法，具体代码如下：

```
/**
 * 在原始的数据末尾添加一个新的元素，并返回
 *
 * @param srcArray 原始数组
 * @param element 添加的元素
 * @return 添加完元素后新的数组
 */
public static int[] addAnElement(int[] srcArray, int element) {
    // 新的数组，多一个元素，长度是原来数组长度+1
    int[] destArray = new int[srcArray.length + 1];
    for (int i = 0; i < srcArray.length; i++) {
        destArray[i] = srcArray[i];
    }
    // 把新添加的元素放在 destArray 的末尾
    destArray[srcArray.length] = element;
    return destArray;
}
```

在 `addAnElement()` 方法中，先定义了一个 `destArray` 数组作为目标数组，由于目标数组比原始数组多一个元素，所以其长度为原始数组的长度加 1。然后通过遍历原始数组的方式，将原始数组中的元素依次添加到目标数组中，最后将新添加的元素也放入目标数组，至此，



实现了数组的扩容。

接下来，在测试类 ArrayDemo07 中调用 addAnElement()方法，验证一下数组扩容的功能，具体代码文件 7-14 所示。

文件 7-14 ArrayDemo07.java

```
public class ArrayDemo07 {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3, 4, 5};  
        int[] newArray = addAnElement(a, 6);  
        System.out.println("新的数组长度: " + newArray.length);  
        System.out.println("新的数组的每个元素: ");  
        for (int item : newArray) {  
            System.out.println(item);  
        }  
    }  
    /**  
     * 在原始的数据末尾添加一个新的元素，并范围  
     *  
     * @param srcArray 原始数组  
     * @param element 添加的元素  
     * @return 添加完元素后新的数组  
     */  
    public static int[] addAnElement(int[] srcArray, int element) {  
        // 新的数组，多一个元素，长度是原来数组长度+1  
        int[] destArray = new int[srcArray.length + 1];  
        for (int i = 0; i < srcArray.length; i++) {  
            destArray[i] = srcArray[i];  
        }  
        // 把新添加的元素放在 destArray 的末尾  
        destArray[srcArray.length] = element;  
        return destArray;  
    }  
}
```

运行结果如图 7-23 所示。



图 7-23 文件 7-14 的运行结果

第 8 章面向对象入门

学习目标

- ◆ 熟悉面向对象的概念，能够编写代码描述现实生活中的事物
- ◆ 掌握类和对象，能够区分类和对象的关系，并对类进行实例化
- ◆ 掌握构造方法的作用，能够使用构造方法创建对象
- ◆ 掌握 this 关键字，能够使用 this 关键字解决重名问题
- ◆ 了解 toString 方法的使用
- ◆ 掌握访问控制符的类型及作用范围
- ◆ 了解包的作用

前面学习的知识都属于 Java 的基本程序设计范畴，属于结构化的程序开发，若使用结构化方法开发软件，其稳定性、可修改性和可重用性都比较差。在软件开发过程中，用户的需求随时都有可能发生变化，为了更好地适应用户需求的变化，Java 语言采用了面向对象的程序设计思想。在接下来的两个章节中，将为读者详细讲解 Java 语言面向对象的特性。

8.1 面向对象概述

8.1.1 认识面向对象

对象的英文名称叫 Object，其实我个人认为这个翻译不太准确，Object 更准确的翻译应该叫“东西”。面向对象编程，准确讲就是面向“东西”编程。

日常生活中的东西，通常包含两种特性，静态属性和动态行为。其中：

- 静态属性：field、state，通常是静止的，可以用数据描述；
- 动态行为：method、behavior，通常是一个动作或者过程。

为了大家更好理解静态属性和动态行为，下面我们举几个生活中的例子。

1. 狗

- 静态属性：名字、颜色，是不是饿了……
- 动态行为：打招呼、拿耗子、哇哇叫……

2. 自行车

- 静态属性：颜色，大小，当前车速……
- 动态行为：刹车，摇铃……

3. 台灯

- 静态属性：亮不亮……
- 动态行为：关灯，开灯……

4. 笔记本电脑

- 静态属性：电池电量，电源是否接通，屏幕是否打开，声音音量……
- 动态行为：打开，关闭，充电，删除文件，下载歌曲，播放小电影，发送邮件……

综上所述，现实生活中的任意一个东西，我们都可以分析出它的属性和行为，从而了解这个东西的用处。Java 编程其实采用的就是这种面向东西的编程思想组织管理代码，只要看到代码的名称和组织结构，基本可以猜测出代码的作用是什么，例如，各个变量用来描述什么属性，各个方法用来描述什么行为等。

8.1.2 使用面向对象描述事物

对象就是一个东西，世间万物都是对象，人是对象，老师也是个对象，假设现在我们要使用面向对象的思想描述老师，老师具有的静态属性和动态行为如下：

- 静态属性：姓名、职称、学科
- 动态行为：出试卷、改试卷

如果使用 UML 描述老师这个类，具体如图 8-1 所示。



老师

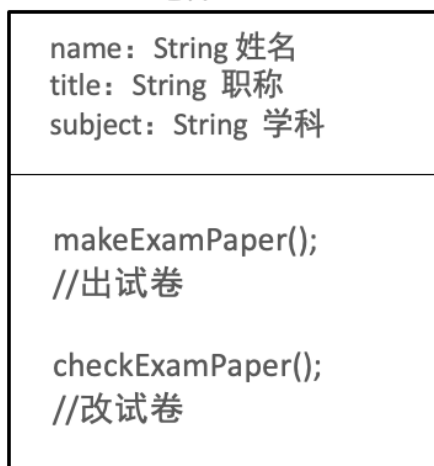


图 8-1 老师对象的 UML 图

接下来，我们按照图 8-1 所示的 UML 图编写一个表示老师的 Teacher 类，具体代码如下文件 8-1 所示。

文件 8-1 Teacher.java

```
15 public class Teacher {  
16     String name; // 姓名  
17     String title; // 职称  
18     String subject; // 学科  
19     public void makeExamPaper() {  
20         System.out.println("一套试卷出好了");  
21     }  
22     public void checkExamPaper() {  
23         System.out.println("正在批改试卷");  
24     }  
25 }
```

文件 8-1 定义的 Teacher 类表示老师类，其中，第 2~4 行代码定义的变量用来描述老师的姓名、职称和学科这些静态属性，第 5~7 行代码定义的 makeExamPaper() 方法用来描述老师出试卷的行为，第 8~10 行代码定义的 checkExamPaper() 方法用来描述老师批改试卷的行为。

8.2 类和对象

8.2.1 类和对象的关系

现实生活中，其实是不存在类的，类是人类为了方便沟通定义的一个概念，指的是类别，它是一个抽象的概念，而对象则是真实存在的，例如，狗是一个抽象的概念，它指的是某个类型，但具体的一条叫旺财的狗则是一个对象。



类关注的是事物共同的属性和行为，比如，狗的颜色、体重等属性，啃骨头、拿耗子等行为。可以说，类是通用的，而对象是具体的个体。

为了更直观体现类和对象的关系，我们可以把类看做汽车图纸，把对象看做是真实汽车，具体如图 8-2 所示。

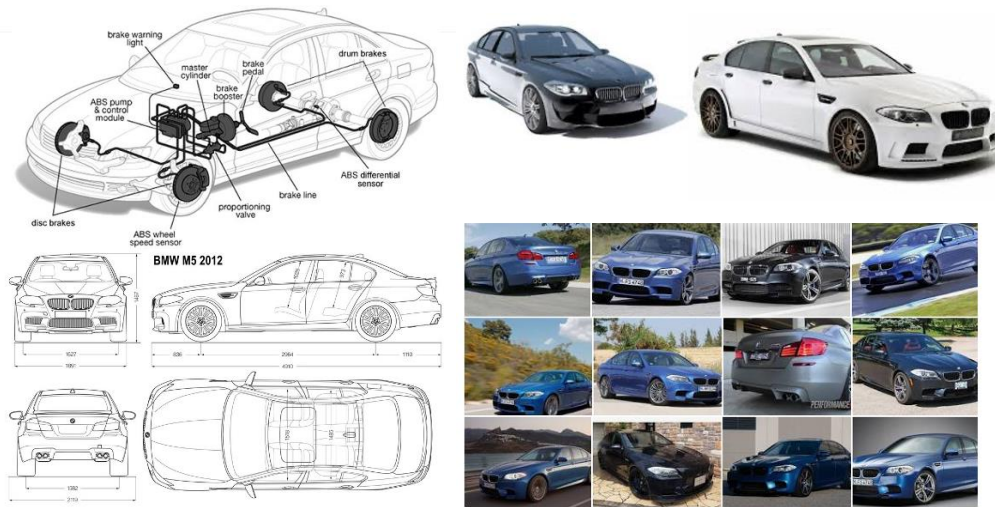


图 8-2 汽车图纸与汽车

在图 8-2 中，汽车图纸可以看做是类（class），而真实的汽车是基于图纸制造出来的，可以看做是类对应的对象（Object）。如果使用 UML 表示汽车类与汽车对象的关系，如图 8-3 所示。

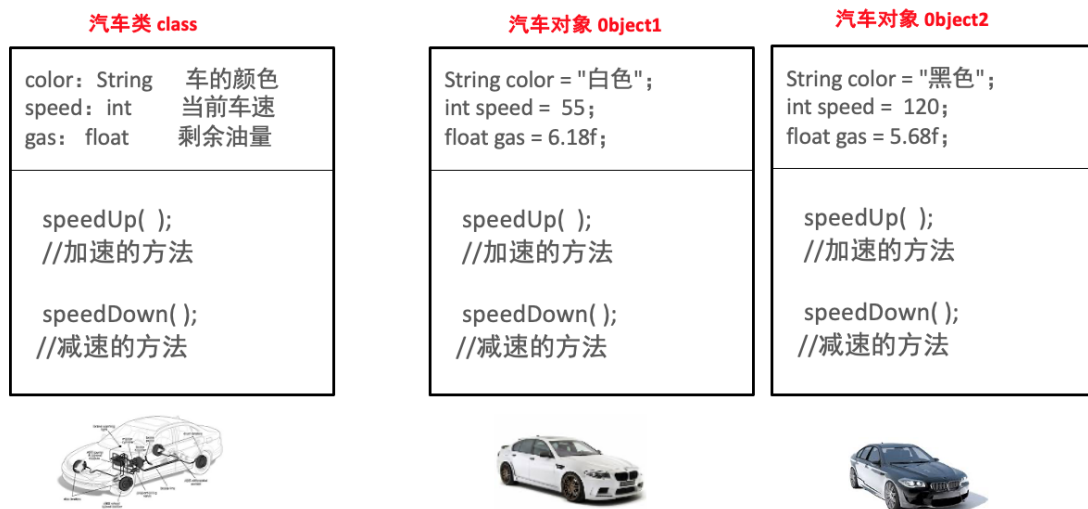


图 8-3 汽车类与汽车对象的 UML 图

在图 8-3 中，汽车类包含了一些属性和方法，这些方法都没有值。我们可以通过对类进行实例化，从而创建真实的汽车对象。真实的小汽车对象有不同的状态，例如，汽车颜色可以是白色、黑色。但是，真实小汽车的行为是一样的，例如，都可以加速、减速等。

总之，类可以实例化出对象，或者叫通过类创建对象，不同的类会有不同的状态，但是他们的行为都是相似的。



8.2.2 类的实例化

通过 8.2.1 节的介绍我们知道，类是抽象的，对象才是具体的。类可以使 `class` 定义，那么对象怎么创建呢？在 Java 程序中可以使用 `new` 关键字创建对象，具体格式如下。

```
类名 对象名称 = null;  
对象名称 = new 类名();
```

上述格式中，创建对象分为声明对象和实例化对象两步，也可以直接通过下面的方式创建对象，具体格式如下。

```
类名 对象名称 = new 类名();
```

假设现在有一个定义好的 `Dog` 类，具体代码如文件 8-2 所示，

文件 8-2 Dog.java

```
public class Dog {  
    String name;  
    String color;  
    boolean isHungary;  
    public void sayHello() {  
        System.out.println("挥动我的小尾巴，打个招呼！");  
    }  
    public void catchMouse() {  
        System.out.println("跑跑跑，抓耗子");  
    }  
    public void wangwangwang() {  
        System.out.println("汪汪汪");  
    }  
}
```

接下来，我们在测试类 `Test` 中创建 `Dog` 类的对象，并调用 `Dog` 类对象的方法，具体代码如文件 8-3 所示。

文件 8-3 TestDog.java

```
26 public class TestDog {  
27     public static void main(String[] args) {  
28         // 创建 Dog 对象  
29         Dog dog1 = new Dog();  
30         // dog1 是对象，是一个具体的东西  
31         dog1.name = "小白";  
32         dog1.color = "白";  
33         dog1.isHungary = true;  
34         dog1.sayHello();  
35         dog1.catchMouse();  
36         dog1.wangwangwang();  
37     }  
38 }
```



程序的运行结果如图 8-4 所示。

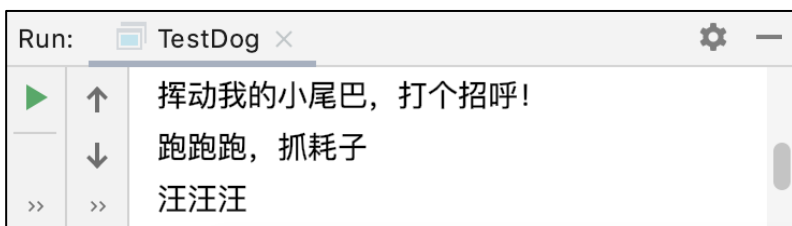


图 8-4 文件 8-3 的运行结果

8.3 构造方法

8.3.1 定义构造方法

构造方法是类在实例化对象时自动执行的方法，我们可以在构造方法中做一些变量初始化的操作。与之前学的普通方法相比，构造方法是一个特殊的方法，它具有以下特点：

- (1) 构造方法的名称和类名相同。
- (2) 构造方法没有返回值。注意，没有返回值和返回值是 `void` 是两个概念。

需要注意的是，如果我们不写构造方法，Java 编译器会自动为每个类创建一个空的构造方法。

接下来，通过一个案例来学习构造方法的使用，具体步骤如下：

- (1) 定义一个 `Person` 类，具体代码如文件 8-4 所示。

文件 8-4 Person.java

```
public class Person {  
    int age;  
    String name;  
    public void eat() {  
        System.out.println(name + "在吃饭");  
    }  
    /**  
     * 构造方法  
     */  
    Person() {  
        System.out.println("我是 Person 的构造方法，我被调用了");  
    }  
}
```

- (2) 定义一个测试类 `Test`，在 `main()` 方法中创建 3 个 `Person` 对象，具体代码如文件 8-5 所示。

文件 8-5 Test.java

```
public class Test {  
    public static void main(String[] args) {
```



```
Person p = new Person();  
Person p1 = new Person();  
Person p2 = new Person();  
p.age = 18;  
p.name = "张三";  
p.eat();  
}  
}
```

运行结果如图 8-5 所示。

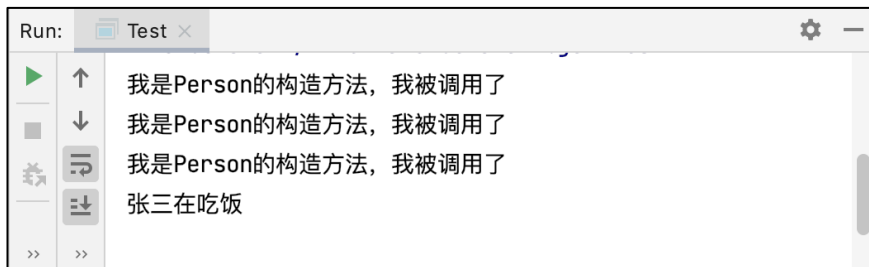


图 8-5 文件 8-5 的运行结果

从图 8-5 中可以看出，控制台输出了三次“我是 Person 的构造方法，我被调用了”。这是因为我们创建了三个 Person 对象，每次创建 Person 对象时，都会自动调用 Person 类的构造方法，所以会输出三次“我是 Person 的构造方法，我被调用了”。

前面我们说过，构造方法可以做一些变量的初始化工作。接下来，我们对 Person 类的构造方法进行修改，通过向构造方法传入参数的方式，对变量进行初始化操作。修改后的 Person 类的代码如文件 8-6 所示。

文件 8-6 Person.java

```
public class Person {  
    int age;  
    String name;  
    public void eat() {  
        System.out.println(name + "在吃饭");  
    }  
    /**  
     * 构造方法  
     */  
    Person(int a, String n) {  
        System.out.println("我是 Person 的构造方法，我被调用了");  
        age = a;  
        name = n;  
    }  
}
```

对 Person 类的代码修改完毕后，此时在 Test 测试类中创建 Person 对象的代码也需要修改，我们需要在使用 new 关键字创建对象的时候，传入初始化变量的值，具体代码如下：

```
public class Test {  
    public static void main(String[] args) {
```



```
        Person p = new Person(18, "张三");  
        Person p1 = new Person(19, "李四");  
        Person p2 = new Person(20, "赵六");  
    }  
}
```

在构造方法中进行初始化的好处是，可以防止程序员在创建对象的时候，忘记给对象的静态变量进行赋值。为此，我们可以在构造方法中完成类静态属性的初始化工作。

8.3.2 构造方法的重载

与普通方法一样，构造方法也可以重载，在一个类中可以定义多个构造方法，只要每个构造方法的参数或参数个数不同即可。在创建对象时，可以通过调用不同的构造方法为不同的属性赋值。

接下来定义一个表示机器人的类 Robot，该类定义了两个构造方法，具体代码如文件 8-7 所示。

文件 8-7 Robot.java

```
public class Robot {  
    String name;  
    String model;  
    // 无参构造方法  
    public Robot() {  
        name = "未知";  
        model = "待定";  
    }  
    // 有两个参数的构造方法  
    public Robot(String name, String model) {  
        this.name = name;  
        this.model = model;  
    }  
    @Override  
    public String toString() {  
        return "Robot{" +  
            "名字='" + name + '\'' +  
            ", 型号='" + model + '\'' +  
            '}';  
    }  
}
```

接下来，在测试类 Test 中使用不同的构造方法创建 Robot 对象，具体代码如文件 8-8 所示。

文件 8-8 Test.java

```
public class Test {  
    public static void main(String[] args) {
```




```
Robot robot = new Robot(); // 使用空参的构造方法创建对象
System.out.println(robot);

// 使用两个参数的构造方法创建对象
Robot robot2 = new Robot("终结者", "T1000");
System.out.println(robot2);

}

}
```

程序的运行结果如图 8-6 所示。

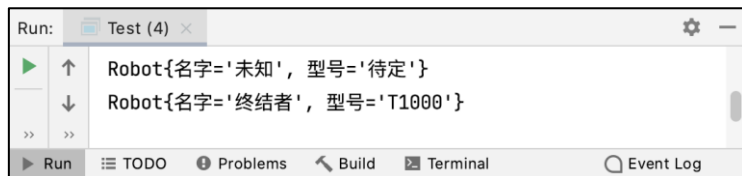


图 8-6 文件 8-8 的运行结果

8.4 this 关键字

在 8.3 小节中，这里有个细节需要注意，我们在 Person 类中定义构造方法时，构造方法传入的参数是 a, n, b, ack 这样的简写。如果我们将参数的名字修改为 age、name、blood 和 attack，此时，构造方法的参数名称与 Person 类的属性名是一样的，示例代码如下：

```
Person(int age, String name, int blood, int attack) {
    System.out.println("我是 Person 的构造方法，我被调用了");
    age = age;
    name = name;
    blood = blood;
    attack = attack;
}
```

此时，如果在 Test 类中创建对象，并调用对象的 eat() 方法时，运行结果如图 8-7 所示。

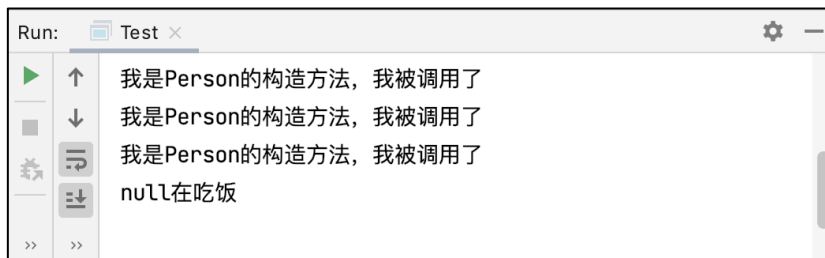


图 8-7 运行结果

从图 8-7 可以看出，Person 类的构造方法被调用了，但是使用 Person 对象调用 eat() 方法后，输出的结果是“null 在吃饭”，说明 Person 对象的 name 没有初始化成功。

那这是为什么呢？因为在 Person 类的构造方法中使用“age = age;”这种方式赋值时，参数 age 的作用域范围是其所在的大括号范围，是无法为 Person 成员变量赋值的。图 8-8 描述了 Person 构造方法参数的作用域。



```
1 public class Person {
2     int age;
3     String name;
4     int blood; // 生命力
5     int attack; // 攻击力
6     public void eat() {
7         System.out.println(name + "在吃饭");
8     }
9     /**
10     * 构造方法
11     */
12     Person(int age, String name, int blood, int attack) {
13         System.out.println("我是Person的构造方法，我被调用了");
14         age = age;
15         name = name;
16         blood = blood;
17         attack = attack;
18     }
```

Person的成员变量

构造方法中参数的作用域

图 8-8 Person 构造方法中参数的作用域

为了解决上述问题，我们可以通过 this 关键字实现，使用 this 关键字可以方便的引用当前对象的成员变量或方法。下面我们对 Person 类的构造方法进行修改，修改后的代码具体如下：

```
Person(int age, String name, int blood, int attack) {
    System.out.println("我是 Person 的构造方法，我被调用了");
    this.age = age;
    this.name = name;
    this.blood = blood;
    this.attack = attack;
}
```

此时，再次运行 Test 测试类，运行结果如图 8-9 所示。

```
Run: Test x
我是Person的构造方法，我被调用了
我是Person的构造方法，我被调用了
我是Person的构造方法，我被调用了
赵六在吃饭
Process finished with exit code 0
```

图 8-9 运行结果

从运行结果可以看出，程序输出了“赵六在吃饭”，说明我们使用 this 成功的将参数的值传递给了 Person 对象。



综上所述，使用 this 关键字可以方便的引用当前对象的成员变量或者方法，它最主要的一个用处就是可以更清楚的指明这个变量

8.5 应用案例：格斗游戏

8.5.1 格斗游戏入门

假设我们现在要开发一款格斗游戏，该格斗游戏的每个游戏角色的名称以及生命值各不相同，我们在选定游戏角色的时候（new 对象的时候），每个游戏角色的信息就应该被确定下来。下面我们分步骤实现这个案例，具体如下：

(1) 定义一个表示游戏角色的 Person 类，具体代码如文件 8-9 所示。

文件 8-9 Person.java

```
/**
 * 游戏的角色类
 */
public class Person {
    String name;    // 游戏角色的名字
    int blood = 100; // 生命值

    public Person(String name) {
        this.name = name;
    }

    /**
     * 攻击人的方法
     *
     * @param p 被打对象
     */
    public void attack(Person p) {
        System.out.println(name + "提起了拳头用力打了" + p.name + "一下");
        p.blood -= 10;
        System.out.println(p.name + "脸上肿起来一个大包");
        System.out.println(p.name + "还剩下" + p.blood + "点血");
    }
}
```

上述代码中，Person 类的构造方法对 age、name、blood 和 attack 四个变量进行了初始化。

(2) 定义测试类 Test，在该类中创建两个游戏角色，具体代码如文件 8-10 所示。

文件 8-10 Game.java

```
public class Game {
    public static void main(String[] args) {
```



```
Person p1 = new Person("特朗普");  
Person p2 = new Person("希拉里");  
p1.attack(p2);  
p1.attack(p2);  
p1.attack(p2);  
}  
}
```

程序运行结果如图 8-10 所示。

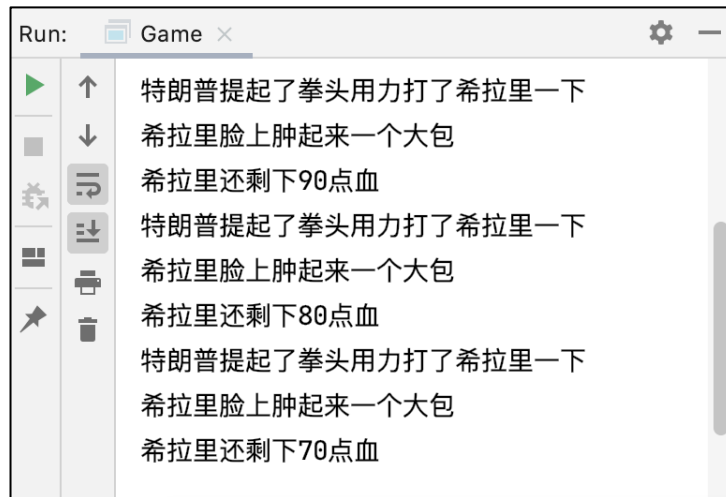


图 8-10 文件 8-10 的运行结果

8.5.2 打斗过程的内存分析

8.5.1 节开发了一个简单的格斗游戏，接下来，我们对执行文件 8-10 的内存进行分析，具体如下：

(1) 代码 `Person p1 = new Person("特朗普");` 执行完毕后，内存情况如图 8-11 所示。

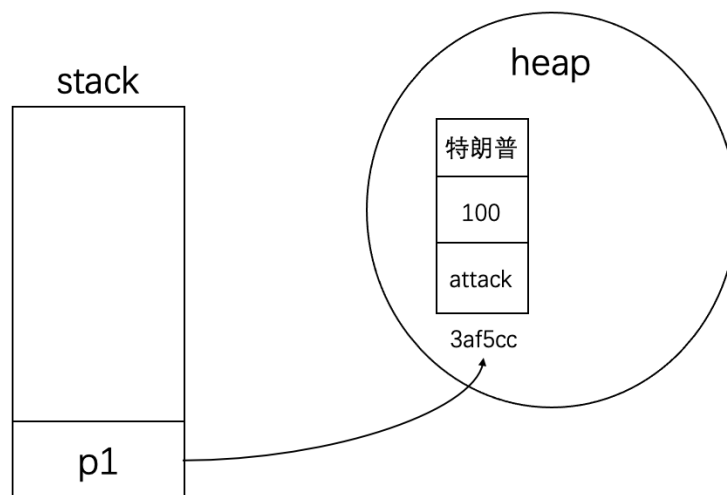


图 8-11 内存情况 (1)



(2) 代码 `Person p2 = new Person("希拉里");` 执行完毕后，内存情况如图 8-12 所示。

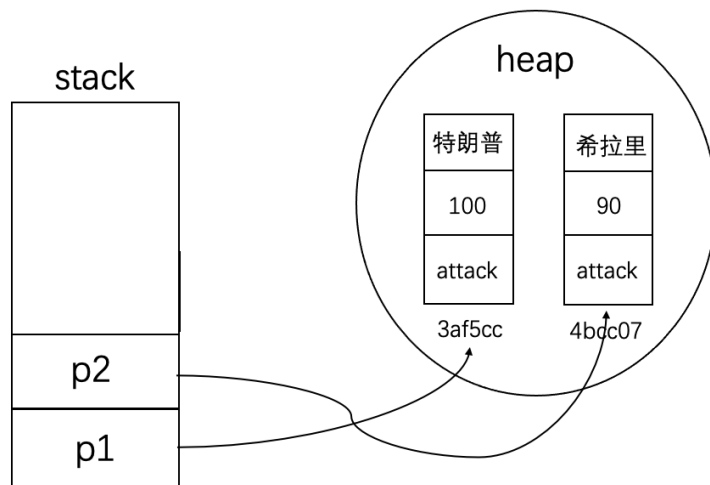


图 8-12 内存情况 (2)

(3) 代码 `p1.attack(p2);` 执行完毕后，内存情况如图 8-13 所示。

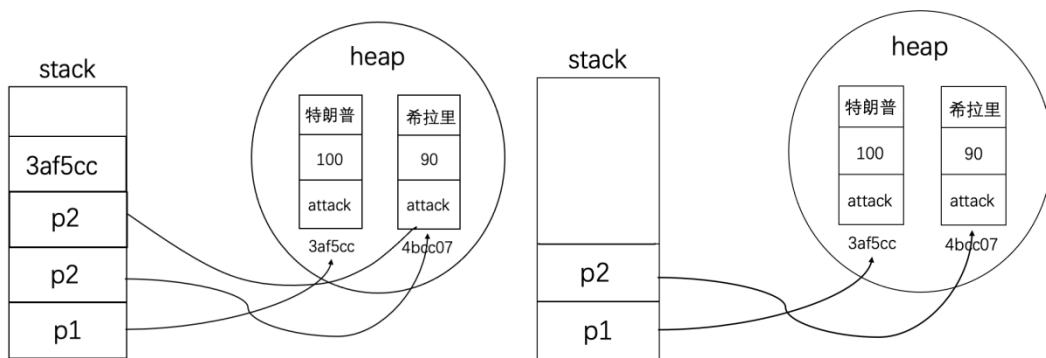


图 8-13 内存情况 (3)

8.5.3 游戏角色的长相设计

接下来，我们对格斗游戏进行升级，对游戏角色的长相进行设计。我们希望创建游戏角色时，游戏可以根据性别，随机分配游戏角色的长相。

已知游戏角色的长相库信息如下：

```
String[] boyfaces= {"风流俊雅","气宇轩昂","相貌英俊","五官端正",
                    "相貌平平","一塌糊涂","面目狰狞"}
String[] girlfaces ={"美奂绝伦","沉鱼落雁","婷婷玉立","身材娇好",
                     "相貌平平","相貌简陋","惨不忍睹"};
```

接下来，对表示游戏角色的 `Person` 类进行修改，修改后的代码如文件 8-11 所示。

文件 8-11 `Person.java`

```
39 /**
40  * 游戏的角色类
```



```
41  */
42 public class Person {
43     String[] boyfaces = {"风流倜傥", "器宇轩昂", "相貌英俊", "五官端正",
44                          "相貌平平", "一塌糊涂", "面目狰狞"};
45     String[] girlfaces = {"美奂绝伦", "沉鱼落雁", "亭亭玉立", "身材姣好",
46                          "相貌平平", "相貌简陋", "惨不忍睹"};
47     String name;    // 游戏角色的名字
48     int blood = 100; // 生命值
49     char gender; // 性别
50     String faceDescription; // 长相
51     public Person(String name, char gender) {
52         this.name = name;
53         if (gender == '男') {
54             faceDescription =
55                 boyfaces[(int) (Math.random() * boyfaces.length)];
56         } else if (gender == '女') {
57             faceDescription =
58                 girlfaces[(int) (Math.random() * girlfaces.length)];
59         } else {
60             faceDescription = "面目狰狞";
61         }
62     }
63     /**
64      * 攻击人的方法
65      *
66      * @param p 被打对象
67      */
68     public void attack(Person p) {
69         System.out.println(name + "提起了拳头用力打了" + p.name + "一下");
70         p.blood -= 10;
71         System.out.println(p.name + "脸上肿起来一个大包");
72         System.out.println(p.name + "还剩下" + p.blood + "点血");
73     }
74     public void showDescription() {
75         System.out.println("我是" + name);
76         System.out.println("相貌: " + faceDescription);
77         System.out.println("生命值" + blood);
78     }
79 }
```

在文件 8-11 中，第 13~24 行代码是 Person 类的构造方法，用于对 Person 类型的对象的姓名和长相进行初始化。第 36~40 行代码定义了 showDescription() 用于输出当前对象的姓名、长相以及生命值。



接下来，在测试类 Game 中进行测试，具体代码如文件 8-12 所示。

文件 8-12 Game.java

```
public class Game {  
    public static void main(String[] args) {  
        Person p1 = new Person("特朗普", '男');  
        Person p2 = new Person("希拉里", '女');  
        p1.showDescription();  
        p2.showDescription();  
    }  
}
```

程序的运行结果如图 8-14 所示。

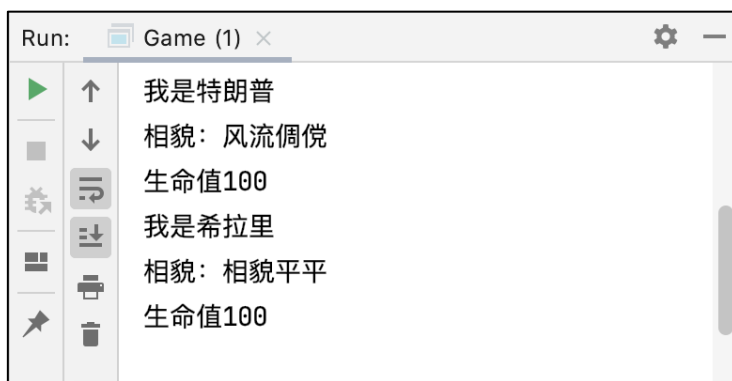


图 8-14 文件 8-12 的运行结果

8.5.4 完善文字版的格斗游戏

完成游戏角色长相的设计后，我们对格斗游戏继续完善，增加游戏角色打斗环节，实现一个文字版的格斗游戏。文字版的格斗游戏规则如下：

- (1) 游戏角色介绍，介绍内容包括姓名、长相和生命值。
- (2) 游戏角色轮流发起攻击，受到攻击的游戏角色，其生命值会减少，受伤程度也不同。
- (3) 只要其中一个游戏角色的生命值为 0，表示格斗游戏结束，公布格斗结果。

接下来，我们分步骤实现上述规则的文字游戏，具体如下：

- (1) 对 8.5.3 小节的 Person 类进行修改，在 attack() 方法中随机发起攻击，并根据攻击程度减少生命值，以及输出攻击后的受伤程度。除此之外，为了看到格斗的过程，我们在 attack() 方法中对程序进行延迟操作，修改后的 Person 类如文件 8-13 所示。

文件 8-13 Person.java

```
80 /**  
81  * 游戏的角色类  
82  */  
83 public class Person {  
84     String[] boyfaces = {"风流倜傥", "器宇轩昂", "相貌英俊", "五官端正",  
85                          "相貌平平", "一塌糊涂", "面目狰狞"};
```



```
86     String[] girlfaces = {"美奂绝伦", "沉鱼落雁", "亭亭玉立", "身材姣好",
87                           "相貌平平", "相貌简陋", "惨不忍睹"};
88     // 攻击的相关描述
89     String[] attacks_desc = {
90         "一招【背心钉】，转到对方的身后，一掌猛向%s 背心灵台穴拍去。",
91         "一招【游空探爪】，飞起身形自半空中变掌为抓锁向%s。",
92         "大喝一声，身形下伏，一招【劈雷坠地】，捶向%s 双腿。",
93         "运气于掌，一瞬间掌心变得血红，一式【掌心雷】，推向%s。",
94         "阴手翻起阳手跟进，一招【没遮拦】，结结实实的捶向%s。",
95         "上步抢身，招中套招，一招【劈挂连环】，连环攻向%s。"
96     };
97     // 受伤的相关描述
98     String[] injureds_desc = {
99         "结果%s 退了半步，毫发无损",           // >80
100        "结果给%s 造成一处瘀伤",           // 70-80
101        "结果一击命中， %s 痛得弯下腰",       // 60-70
102        "结果%s 痛苦地闷哼了一声，显然受了点内伤", // 50-60
103        "结果%s 摇摇晃晃，一跤摔倒在地",       // 40-50
104        "结果%s 脸色一下变得惨白，连退了好几步", // 30-40
105        "结果『轰』的一声， %s 口中鲜血狂喷而出", // 8-30
106        "结果%s 一声惨叫，像滩软泥般塌了下去" // 0-8
107    };
108    String name;    // 游戏角色的名字
109    int blood = 100; // 生命值
110    char gender; // 性别
111    String faceDescription; // 长相
112    public Person(String name, char gender) {
113        this.name = name;
114        if (gender == '男') {
115            faceDescription =
116                boyfaces[(int) (Math.random() * boyfaces.length)];
117        } else if (gender == '女') {
118            faceDescription =
119                girlfaces[(int) (Math.random() * girlfaces.length)];
120        } else {
121            faceDescription = "面目狰狞";
122        }
123    }
124    /**
125     * 攻击的方法
126     *
127     * @param p 被打对象
128     */
```




```
129     public void attack(Person p) {
130         int randomNum = (int) (Math.random() * attacks_desc.length);
131         // 运气于掌，一瞬间掌心变的通红，一式【掌心雷】，推向%s
132         System.out.printf(name + attacks_desc[randomNum], p.name);
133         System.out.println("");
134         p.blood -= 7 * randomNum;
135         if (p.blood < 0) {
136             p.blood = 0;
137         }
138         if (p.blood >= 80) {
139             System.out.printf(injureds_desc[0], p.name);
140             System.out.println("");
141         } else if (p.blood >= 70) {
142             System.out.printf(injureds_desc[1], p.name);
143             System.out.println("");
144         } else if (p.blood >= 60) {
145             System.out.printf(injureds_desc[2], p.name);
146             System.out.println("");
147         } else if (p.blood >= 50) {
148             System.out.printf(injureds_desc[3], p.name);
149             System.out.println("");
150         } else if (p.blood >= 40) {
151             System.out.printf(injureds_desc[4], p.name);
152             System.out.println("");
153         } else if (p.blood >= 30) {
154             System.out.printf(injureds_desc[5], p.name);
155             System.out.println("");
156         } else if (p.blood >= 8) {
157             System.out.printf(injureds_desc[6], p.name);
158             System.out.println("");
159         } else {
160             System.out.printf(injureds_desc[7], p.name);
161             System.out.println("");
162         }
163         // 为了看到格斗过程，对程序进行延迟操作
164         try {
165             Thread.sleep(1500);
166         } catch (InterruptedException e) {
167             e.printStackTrace();
168         }
169     }
170     public void showDescription() {
```



```
171         System.out.println("我是" + name);
172         System.out.println("相貌: " + faceDescription);
173         System.out.println("生命值" + blood);
174     }
    }
```

(2) 在测试类 Game 中进行测试，具体代码如文件 8-14 所示。

文件 8-14 Game.java

```
175     public class Game {
176         public static void main(String[] args) {
177             Person p1 = new Person("特朗普", '男');
178             Person p2 = new Person("希拉里", '女');
179             p1.showDescription();
180             p2.showDescription();
181             while (true) {
182                 p1.attack(p2);
183                 if (p2.blood <= 0) {
184                     System.out.println("游戏战斗结束" + p1.name + "K.O." +
p2.name);
185                     break;
186                 }
187                 p2.attack(p1);
188                 if (p1.blood <= 0) {
189                     System.out.println("游戏战斗结束" + p2.name + "K.O." +
p1.name);
190                     break;
191                 }
192             }
193         }
194     }
```

在文件 8-14 中，第 7~18 行代码通过 for 循环模拟对象 p1 和 p2 的格斗过程，先是 p1 攻击 p2，然后 p2 攻击 p1，只要任何一方的生命值小于等于 0，那么格斗游戏结束，输出格斗结果。

程序的运行结果如图 8-15 所示。



图 8-15 文件 8-14 的运行结果

8.6 toString 方法

toString()方法是 Java 编程中很常用的一个方法。通过 System.out.print()打印某个类的对象，默认打印的是对象在内存中的地址。但是，如果我们编写类的 toString()方法，可以让打印输出的内容更容易阅读，也更方便调试和排错。

现在有一个 Person 类，代码如文件 8-15 所示。

文件 8-15 Person.java

```
public class Person {  
    int age;  
    String name;  
    public Person(int age, String name) {  
        this.age = age;  
        this.name = name;  
    }  
}
```

我们在测试类 Test 类中创建一个 Person 对象，并使用 System.out.print()将创建的 Person 打印出来，具体代码如文件 8-16 所示。

文件 8-16 Test.java

```
public class Test {  
    public static void main(String[] args) {
```



```
Person person = new Person(18, "小芳");  
System.out.println(person);  
  
}  
  
}
```

程序的运行结果如图 8-16 所示。

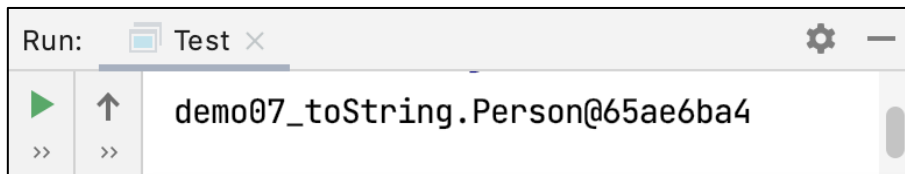


图 8-16 文件 8-10 的运行结果

在图 8-16 中，使用 System.out.print()打印的 Person 对象是“Person@XX”，其中，XX 是 Person 对象哈希值的十六进制表示，也就是对象在虚拟机中的地址。很显然，这样是看不出 Person 对象中的具体信息的。

如果大家想要输出对象的具体信息，可以在 IDEA 的 Person 类中，按【Ctrl+O】组合键，在弹出的窗口中选择 toString()方法，如图 8-17 所示。

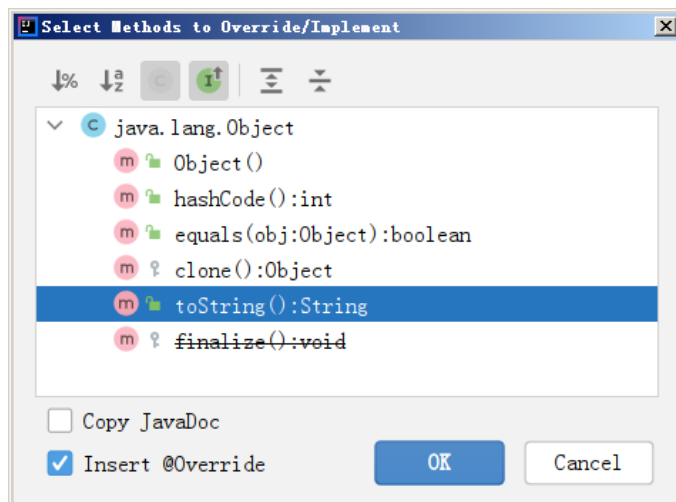


图 8-17 选择 toString()方法

选中 toString()后，会在 Person 类中自动生成一个 toString()方法，具体代码如下：

```
@Override  
public String toString(){  
    return "Person{" +  
        "age=" + age +  
        ", name='" + name + '\'' +  
        '}';  
}
```

此时，再次运行测试类 Test，程序的运行结果如图 8-18 所示。

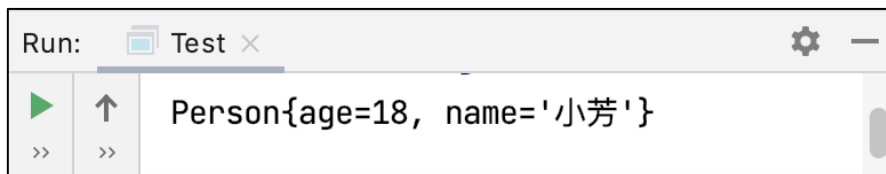


图 8-18 再次运行的结果



从图 8-16 中可以看出，此时打印出的 Person 对象的信息可以很明显看懂。当然，如果我们希望定制控制台打印信息的输出形式，可以对 toString() 方法修改，

8.7 静态方法

如果想要使用类中的成员方法，就需要先将这个类实例化，而在实际开发时，开发人员有时希望在不创建对象的情况下，通过类名就可以直接调用某个方法，要实现这样的效果，只需要在成员方法前加上 static 关键字，使用 static 关键字修饰的方法通常称为静态方法。

静态方法可以通过类名和对象访问，具体如下所示。

类名.方法

接下来通过一个案例来学习静态方法的使用。我们先定义一个 Human 类，该类内部定义了一个实例方法和一个静态方法，具体代码如文件 8-17 所示。

文件 8-17 Human.java

```
public class Human {  
    String name;  
    int age;  
    public static void printStatic() {  
        System.out.println("我是 static 方法");  
    }  
    public void printInstance() {  
        System.out.println("我是实例方法");  
    }  
}
```

此时，我们定义一个测试类 Test，在该类内部调用 Human 中的方法，具体代码如文件 8-18 所示。

文件 8-18 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        Human human=new Human();  
        // 调用 Human 的实例方法 printInstance  
        human.printInstance();  
        // 调用 Human 的静态方法 printStatic  
        Human.printStatic();  
    }  
}
```

程序的运行结果如图 8-19 所示。

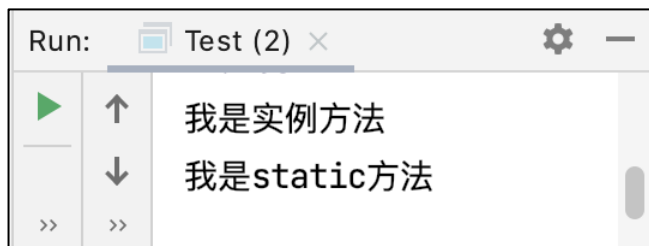


图 8-19 文件 8-18 的运行结果

注意：

静态方法只能访问静态成员，因为非静态成员需要先创建对象才能访问，即随着对象的创建，非静态成员才会分配内存。而静态方法在被调用时可以不创建任何对象。

8.8 访问修饰符

前面的学习中，其实我们已经使用过了一些访问修饰符，例如，`public`。除此之外，Java 还提供了其他的访问修饰符，具体如下：

- `public`：公开的。
- `private`：私有的。
- `protected`：受保护的。
- `default`：包内可见。

不同访问修饰符的作用范围是不同的，接下来通过一张表描述访问修饰符的作用范围，如表 8-1 所示。

表 8-1 访问修饰符的作用范围

访问修饰符	Class	Package	Subclass	Global
<code>public</code>	Yes	Yes	Yes	Yes
<code>protected</code>	Yes	Yes	Yes	No
<code>default</code>	Yes	Yes	No	No
<code>private</code>	Yes	No	No	No

那么，访问修饰符究竟有什么应用场景呢？我们通过示例代码进行演示。

假设我们开了一个图书馆，图书馆藏有大量的图书，所以我们先定义一个 `Book` 类用来描述图书信息，具体代码如文件 8-19 所示。

文件 8-19 `Book.java`

```
public class Book {  
    String title; // 书名  
    String author; // 作者  
    public Book(String title, String author) {  
        this.title = title;  
        this.author = author;  
    }  
    @Override  
    public String toString() {  
        return "书名:" + title + ", 作者:" + author;  
    }  
}
```



此时，如果在测试类中打印图书的信息，具体代码如文件 8-20 所示。

文件 8-20 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        Book book1 = new Book("毛泽东思想", "毛泽东");  
        Book book2 = new Book("邓小平理论", "邓小平");  
        System.out.println(book1);  
        System.out.println(book2);  
    }  
}
```

运行结果如图 8-20 所示。

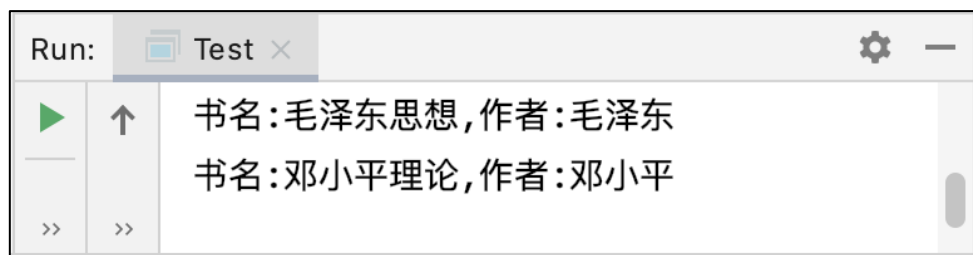


图 8-20 文件 8-20 的运行结果

但是上述代码存在一个问题，假设我们在输出 Book 对象之前，对 Book 对象的信息进行了修改，将 book1 的作者改为了三毛，代码如文件 8-21 所示。

文件 8-21 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        Book book1 = new Book("毛泽东思想", "毛泽东");  
        Book book2 = new Book("邓小平理论", "邓小平");  
        book1.author = "三毛";  
        System.out.println(book1);  
        System.out.println(book2);  
    }  
}
```

此时，运行程序的结果如图 8-21 所示。

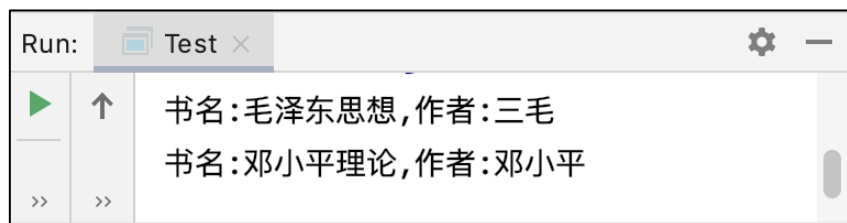


图 8-21 运行结果

从图 8-21 中可以看出，书名是毛泽东思想的作者被变更为了三毛。很显然这样的操作是不合理也不应该的。

为了保证图书的信息不能被随意修改，我们可以对 Book 类进行，将 title 和 author 属性使用 private 修饰，修改后的 Book 类，具体代码如文件 8-22 所示。



文件 8-22 Book.java

```
public class Book {  
    private String title; // 书名  
    private String author; // 作者  
    public Book(String title, String author) {  
        this.title = title;  
        this.author = author;  
    }  
    @Override  
    public String toString() {  
        return "书名:" + title + ",作者:" + author;  
    }  
}
```

上述代码中，使用 private 修饰的 title 和 author 只能在 Book 类中使用。此时如果在测试类 Test 中使用类似 “book1.author” 方式进行赋值是不允许的，程序会报错，提示 author 是私有的，不能被访问，如图 8-22 所示。



图 8-22 程序报错

还有一种情况，希望图书的信息能被修改。例如，图书有一个属性 isBorrowed，用于记录图书是否被借出，如果图书被借，那么 isBorrowed 的值是 true，如果图书被归还，那么 isBorrowed 的值是 false。如果将 isBorrowed 使用 private 修饰的话，我们显然是无法更改 isBorrowed 的值。

为了解决这个问题，我们可以在 Book 类中定义几个方法用于更新图书的借还信息。具体代码如文件 8-23 所示。

文件 8-23 Book.java

```
public class Book {  
    private String title; // 书名  
    private String author; // 作者  
    private boolean isBorrowed; // 是否被借出  
    public void borrowBook() {  
        isBorrowed = true;  
    }  
}
```




```
public void returnBook() {
    isBorrowed = false;
}

public boolean isBorrowed() {
    return isBorrowed;
}

public Book(String title, String author) {
    this.title = title;
    this.author = author;
}

@Override
public String toString() {
    return "书名:" + title + ",作者:" + author+",是否被借出:"+isBorrowed;
}
}
```

这样一来，如果想借阅某本书，只需要调用 borrowBook() 方法即可，具体代码如文件 8-24 所示。

文件 8-24 Test.java

```
public class Test {
    public static void main(String[] args) {
        Book book1 = new Book("毛泽东思想", "毛泽东");
        Book book2 = new Book("邓小平理论", "邓小平");
        book1.borrowBook();
        System.out.println(book1);
        System.out.println(book2);
    }
}
```

运行结果如图 8-23 所示。

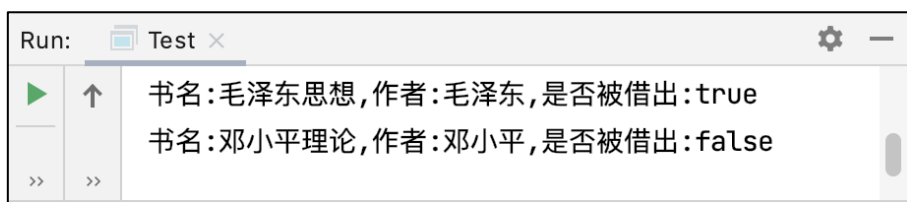


图 8-23 文件 8-24 的运行结果

总结：

- (1) 静态属性一般先尝试使用 **private** 关键字。除非我们希望属性直接暴露出来，才会考虑使用 **public** 关键字。
- (2) 私有的静态属性，我们会通过构造方法作为输入，从而实例化静态属性。
- (3) 我们可以创建 **public** 修饰的方法，使用这个方法修改私有的静态属性，这个方法的名字叫 **setters**。
- (4) 我们可以创建 **public** 修饰的方法，使用这个方法读取类的私有静态属性，这样可

以避免程序员不小心修改属性值，这个方法的学名叫 `getters`。

8.9 包的使用

在实际 Java 应用开发时，如果我们的项目比较大，可能会包含上百个甚至上千个 Java 源代码，如图 8-24 所示。如果我们把所有的 Java 源代码放在一个文件夹下，这样势必会导致项目结构不清晰，不仅在查找 Java 源代码时耗时，而且很可能出现命名冲突的问题。



图 8-24 项目中多个 Java 源代码

为了解决上述问题，Java 引入了包（Package）的概念，包其实就是通过文件夹实现的，示例如图 8-25 所示。

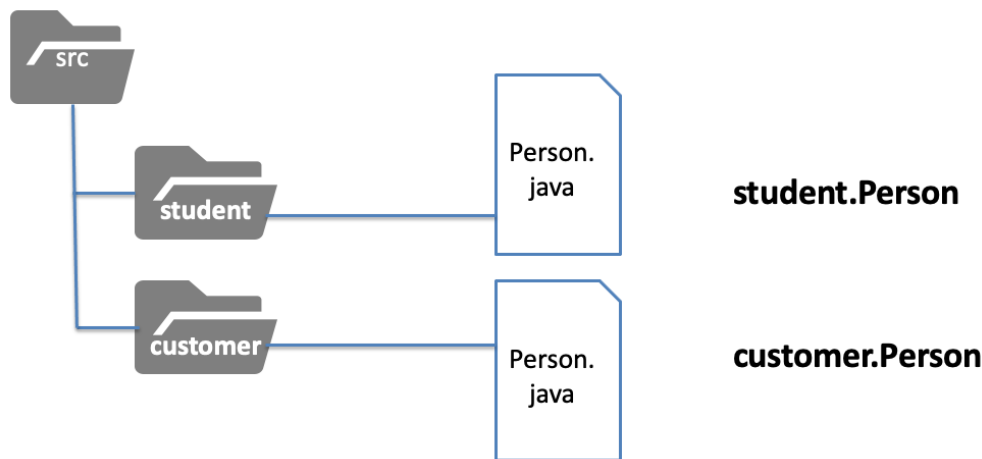


图 8-25 包的形式

在图 8-25 中，假设要定义两个 `Person` 类，其中一个用来表示学生 `student`，一个用来表示客户 `customer`，由于这两个 `Person` 类分别存放在不同的文件夹中，所以这两个类之间不会出现冲突。

下面我们通过代码的方式演示包的操作，具体步骤如下：

(1) 在 IDEA 中右击 `src`，选择【New】→【Package】，创建一个名称为 `student` 的包和一个名称为 `customer` 的包。



(2) 分别在 `student` 包和 `customer` 包中新建 `Person` 类，具体代码如下：

`student` 包中的 `Person` 类：

```
package student;

public class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    @Override
    public String toString() {
        return "学生类: name=" + name + ",age=" + age;
    }
}
```

`customer` 包中的 `Person` 类：

```
package customer;

public class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    @Override
    public String toString() {
        return "顾客类: name=" + name + ",age=" + age;
    }
}
```

(3) 在测试类 `Test` 中创建 `Person` 对象并输出对象的值，具体代码如下：

```
public class Test {
    public static void main(String[] args) {
        student.Person p1 = new student.Person("张三", 18);
        customer.Person p2 = new customer.Person("李四", 28);
        System.out.println(p1);
        System.out.println(p2);
    }
}
```

(4) 运行程序，结果如图 8-26 所示。

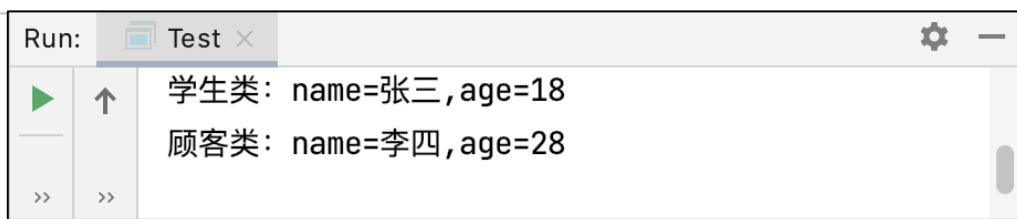


图 8-26 运行结果

需要注意的是，包在命名时，最好注意下列两点：

- 包的名称最好是带上公司名，因为通常情况下，公司的域名是唯一的，加上公司域名可以很好地避免包的重名，如图 8-27 所示。

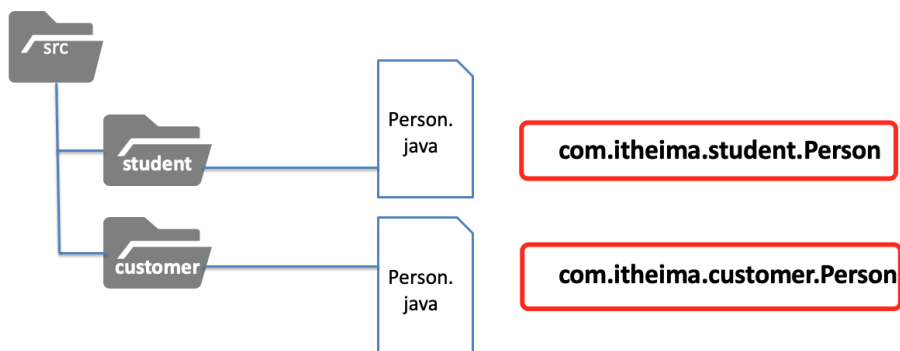


图 8-27 使用公司域名为包命名

- 包的名称最好可以顾名思义，通过阅读包名就可以大致明白各个包的作用，具体示例如下：

com.itheima.db 包， 数据库相关的代码

com.itheima.ui 包， 界面相关的代码

com.itheima.controller 包， 数据控制相关的代码

第 9 章 面向对象进阶

学习目标

- ◆ 了解面向对象编程的意义，能够说出面向对象编程的好处
- ◆ 掌握封装和继承，能够编写代码实现封装和继承
- ◆ 熟悉 Object 类的作用及常用方法
- ◆ 熟悉 Java 中的抽象，能够根据需求编写抽象类和抽象方法
- ◆ 掌握接口，能够独立实现接口，并使用接口实现多继承

◆ 掌握 Java 中的多态，能够使用多态实现业务需求

在上一章中，介绍了面向对象的的基本用法，本章中将继续讲解面向对象的一些高级特性，如封装、继承、多态。

9.1 面向对象编程思考

9.1.1 面向对象编程的意义

面向对象是一种编程思想，下面我们通过生活中的一些案例来理解面向对象编程的意义。

比如，平时我们吃火龙果（如图 9-1）时，并不需要知道火龙果长在树上，还是长在地上，大家只需要知道吃火龙果时，不要吃皮，不用挑籽就可以了。



图 9-1 吃火龙果

再比如，穿 T 恤（如图 9-2）的时候，我们不需要关心 T 恤是如何做出来的，也不用考虑 T 恤的颜色是如何染的，只需要知道不同身材的人穿不同号码的 T 恤即可，不同季节穿 T 恤时，可能会凉快也可能会冷。



图 9-2 穿 T 恤

再比如，使用洗衣机（如图 9-3）洗衣服时，我们不需要知道洗衣机内部电路原理，也不需要知道洗衣机的电机是怎么制作的，只需关心洗衣机的容量是多少、噪音大小、洗的衣服是否干净。



图 9-3 使用洗衣服

上述举例的事物，例如，火龙果、T 恤、洗衣机都可以看做是一个东西，我们使用这些东西的时候，只需要关心与我们相关的内容即可。

社会大分工的出现，标志着人类进入文明时代，社会越发达，分工就越细。我们程序员写代码，已经告别了单打独斗，一个人写几万行的年代了。程序员写代码，也是有明确的分工的，我们应该站在巨人的肩膀上进行开发，后面大家在学习 Java EE 高级课程的时候，会学习各种各样的框架，这些其实都是站在巨人的肩膀上进行开发。我们做编程，不应该重复的创造轮子，而要善于利用和改造轮子，大家在编程初学阶段，学会使用【Ctrl+C】和【Ctrl+V】，这其实也是一种代码的复用。

9.1.2 应用案例:使用面向对象思想协同开发洗衣机

程序员开发是有明确分工的，有的程序员是做硬件开发的，有的是做嵌入式开发的，还有做软件开发的等等，而软件开发相关程序员，又分为不同的方向，有移动端开发、有 Web 前端开发、后台开发等。接下来，通过一个面向对象的实战——洗衣机，看一下程序员是如何分工，一起协同开发工作的。

首先思考两个问题：（1）如何洗衣服。（2）洗衣服需要使用什么工具。

最初人们洗衣服使用的是搓衣板，搓衣板不仅能洗衣服，而且还可以跪搓衣板。后来随着社会的发展，搓衣板逐渐被淘汰，人们使用洗衣机洗衣服。

洗衣机出现后，我们考虑一下洗衣机的社会分工：

洗衣机的生产商（程序员 A）：

- 电机工作原理
- 排水系统设计
- 电压转化方式
- 保险丝和用电安全设计
-

使用者（程序员 B）：

- 买了个洗衣机



- 设置模式，启动洗衣机开始洗衣服
-

下面使用 Java 代码演示洗衣机生产商和顾客分别是如何生产洗衣机和使用洗衣机的，不同程序员又是如何协同分工的。我们把洗衣机生产厂商的事情交给程序员 A 完成，顾客使用洗衣机的事情交给程序员 B 完成，代码实现过程如下：

(1) 程序员 A 在 IDEA 中创建一个包 `com.itheima.demo01`，在这个包中创建一个表示洗衣机的类 `WashMachine`，具体代码如文件 9-1 所示。

文件 9-1 WashMachine.java

```
package com.itheima.demo01;

public class WashMachine {

    private String logo;
    private int size;
    public WashMachine(String logo, int size) {
        this.logo = logo;
        this.size = size;
        System.out.println(this.toString());
    }

    @Override
    public String toString() {
        return "我是" + logo + "洗衣机,我的容量是:" + size;
    }
}
```

(2) 顾客首先购买了一台洗衣机，他想要洗衣服的话，需要做三件事，分别是打开门、关上门、洗衣服。所以，程序员 B 在包 `com.itheima.demo01` 中创建了一个类 `Customer`，具体代码如文件 9-2 所示。

文件 9-2 Customer.java

```
package com.itheima.demo01;

/**
 * 顾客类，代表的是程序员 B
 */
public class Customer {

    public static void main(String[] args) {
        WashMachine machine = new WashMachine("小天鹅", 24);
        machine.openDoor(); // 打开门
        machine.closeDoor(); // 关闭门
        machine.wash(); // 洗衣服
    }
}
```

(3) 当顾客购买了洗衣机，在使用洗衣机的过程中，如果发现洗衣机厂商并没有提供对应的功能，那这个洗衣机厂商肯定要倒闭。所以，作为程序员 A，必须在 `WashMachine` 类中定义 `openDoor()`、`closeDoor()`、`wash()` 这三个方法，分别实现开门、关门与洗衣服功能。修改后的 `WashMachine` 类的代码如文件 9-3 所示。



文件 9-3 WashMachine.java

```
package com.itheima.demo01;

public class WashMachine {

    private String logo;
    private int size;

    public WashMachine(String logo, int size) {

        this.logo = logo;
        this.size = size;
        System.out.println(this.toString());
    }

    @Override
    public String toString() {

        return "我是" + logo + "洗衣机,我的容量是:" + size;
    }

    public void openDoor() {

        System.out.println("洗衣机的门开开了, 请放衣服");
    }

    public void closeDoor() {

        System.out.println("洗衣机的门, 关上了");
    }

    public void wash() {

        System.out.println("自动加水...");
        System.out.println("水加满了!");
        System.out.println("洗衣机开始转圈, 洗衣服");
    }

}
```

(4) 此时，运行 Customer 类，结果如图 9-4 所示。

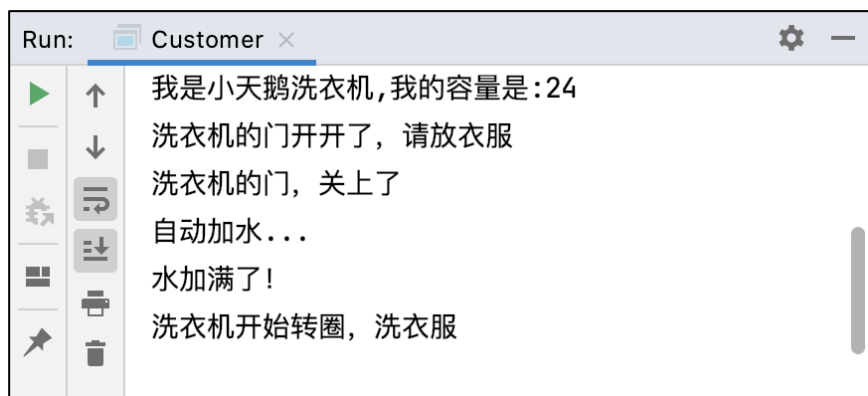


图 9-4 运行结果

至此，我们就演示了面向对象在实际开发中的应用场景。作为程序员 B，他不需要关心洗衣机内部是如何工作的，他只需要关心洗衣机的门能否打开关闭，能否洗衣服即可。那程序员 A 作为洗衣机厂商，他所做的事情就是构造出一个洗衣机，并实现洗衣机应该具有的功能。这两个程序员只需要协同分工，就可以保证程序的正常运行。

9.2 封装

9.2.1 认识封装

面向对象这种编程思想的目标是不仅让我们的代码更容易使用、被理解，而且可以让我们的代码更容易被维护、增加或修改代码的功能。理解面向对象编程，必须要深入理解面向对象的三大特征，分别是封装、继承和多态，接下来，我们先来学习封装。

为了大家更好理解封装，我们看几个现实生活中封装的例子，如图 9-5 和图 9-6 所示。



图 9-5 电箱



图 9-6 电子标签

图 9-5 是一个电箱，电箱内部有密密麻麻的电线。对于非专业人员来说，我们无需知道电箱内部的电路是如何工作的，只需要会操作外部的按钮就可以接通电源。

图 9-6 是一个电子标签，该便签内部是什么样的，我们无需关心，我们只需知道标签一旦被打开，就意味着产品被开封，可能面临的的就是失去保修资格。

由此，我们可以这么定义封装的概念：

- (1) 封装，就是隐藏对象的属性和实现细节，仅对外公开接口，控制在程序中属性的读和修改的访问级别。
- (2) 封装就是把细节部分包装、隐藏起来的方法。
- (3) 封装可以被认为是一个保护屏障、防止该类的代码和数据被外部类定义的代码随机访问。



9.2.2 封装的实现

理解了什么是封装之后，下面我们通过代码演示封装是如何实现的，以及封装的好处。

这里，同样表示洗衣机的 WashMachine 为例，该类拥有打开门、关上门、洗衣服的方法。假设顾客操作洗衣机的流程设计为“打开门→洗衣服”，具体代码如文件 9-4 所示。

文件 9-4 Customer.java

```
package com.itheima.demo01;

/**
 * 顾客类，代表的是程序员 B
 */
public class Customer {
    public static void main(String[] args) {
        WashMachine machine = new WashMachine("小天鹅", 24);
        machine.openDoor();
        machine.wash();
    }
}
```

程序的运行结果如图 9-7 所示。

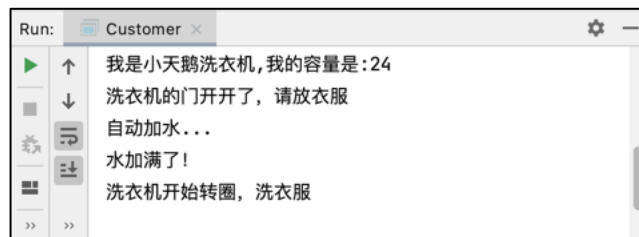


图 9-7 文件 9-4 的运行结果

从图 9-7 展示的洗衣服流程来看，洗衣机的门即使没有关上，也是可以开始洗衣服的，显然在现实生活中这种程序带来的后果是可想而知的。

为了避免出现上述的情况，我们可以在洗衣服之前，检测一下洗衣机门的状态。下面我们对 WashMachine 类进行修改，在该类中添加一个布尔类型的变量 isDoorOpen，在调用洗衣服的方法 wash() 时，通过检测 isDoorOpen 的值，判断是否能够开始执行洗衣服的动作。为了防止 isDoorOpen 的值被强制修改为 false，我们将 isDoorOpen 的访问修饰符设置为 private。

修改后的 WashMachine 类的代码如文件 9-5 所示。

文件 9-5 WashMachine.java

```
package com.itheima.demo01;

public class WashMachine {
    private String logo;
    private int size;
    private boolean isDoorOpen;
```



```
public WashMachine(String logo, int size) {
    this.logo = logo;
    this.size = size;
    System.out.println(this.toString());
}
@Override
public String toString() {
    return "我是" + logo + "洗衣机,我的容量是:" + size;
}
public void openDoor() {
    System.out.println("洗衣机的门开开了, 请放衣服");
    isDoorOpen = true;
}
public void closeDoor() {
    System.out.println("洗衣机的门, 关上了");
    isDoorOpen = false;
}
public void wash() {
    if (isDoorOpen) {
        System.out.println("洗衣机的门没关好, 哗哗哗哗哗哗");
    } else {
        System.out.println("自动加水...");
        System.out.println("水加满了!");
        System.out.println("洗衣机开始转圈, 洗衣服");
    }
}
}
```

此时，如果我们按照“打开门→洗衣服”的流程洗衣服的话，程序会提示没有关好门，而且不会自动开始洗衣服的。再次运行程序，结果如图 9-8 所示。

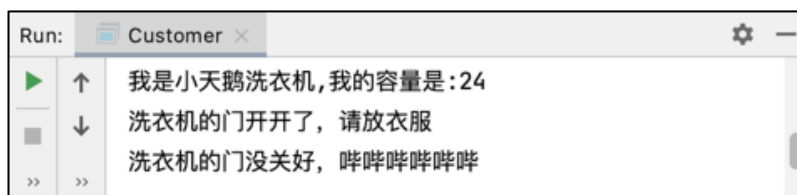


图 9-8 运行结果

如果我们对洗衣机的功能再次升级，为洗衣机设置洗衣模式，这里我们设置两种洗衣模式，1 表示轻柔模式，2 表示狂揉模式，其中轻柔模式的电机转速是 1000 转/分，狂揉模式的电机转速是 5000 转/分。修改后的 WashMachine 类，具体代码如文件 9-6 所示。

文件 9-6 WashMachine.java

```
195 package com.itheima.demo01;
196 public class WashMachine {
197     private String logo;
198     private int size;
```



```
199     private boolean isDoorOpen;
200     public WashMachine(String logo, int size) {
201         this.logo = logo;
202         this.size = size;
203         System.out.println(this.toString());
204     }
205
206     @Override
207     public String toString() {
208         return "我是" + logo + "洗衣机,我的容量是:" + size;
209     }
210     public void openDoor() {
211         System.out.println("洗衣机的门开开了，请放衣服");
212         isDoorOpen = true;
213     }
214     public void closeDoor() {
215         System.out.println("洗衣机的门，关上了");
216         isDoorOpen = false;
217     }
218     public void wash(int mode) {
219         if (isDoorOpen) {
220             System.out.println("洗衣机的门没关好，哗哗哗哗哗哗");
221         } else {
222             System.out.println("自动加水...");
223             System.out.println("水加满了！");
224             if (mode == 1) {
225                 System.out.println("洗衣模式:轻柔");
226                 setMotorSpeed(1000);
227             } else if (mode == 2) {
228                 System.out.println("洗衣模式:狂揉");
229             } else {
230                 System.out.println("洗衣模式设置不正确");
231                 return;
232             }
233             System.out.println("洗衣机开始转圈，洗衣服");
234         }
235     }
236     private void setMotorSpeed(int speed) {
237         System.out.println("洗衣机电机转速:" + speed + "转/每分钟");
238     }
239 }
```

需要注意的是，第 42~44 行代码是用于设置电机转速的方法，这个我们是不希望暴露给



终端用户的，所以需要将这个方法使用 `private` 修饰。

此时，在 `Customer` 类中再次调用 `WashMachine` 类的相关方法实现洗衣服的功能，具体代码如文件 9-7 所示。

文件 9-7 Customer.java

```
package com.itheima.demo01;

/**
 * 顾客类，代表的是程序员 B
 */
public class Customer {
    public static void main(String[] args) {
        WashMachine machine = new WashMachine("小天鹅", 24);
        machine.openDoor();
        machine.closeDoor();
        System.out.println("把你的袜子放进去");
        machine.wash(1);
    }
}
```

程序的运行结果如图 9-9 所示。

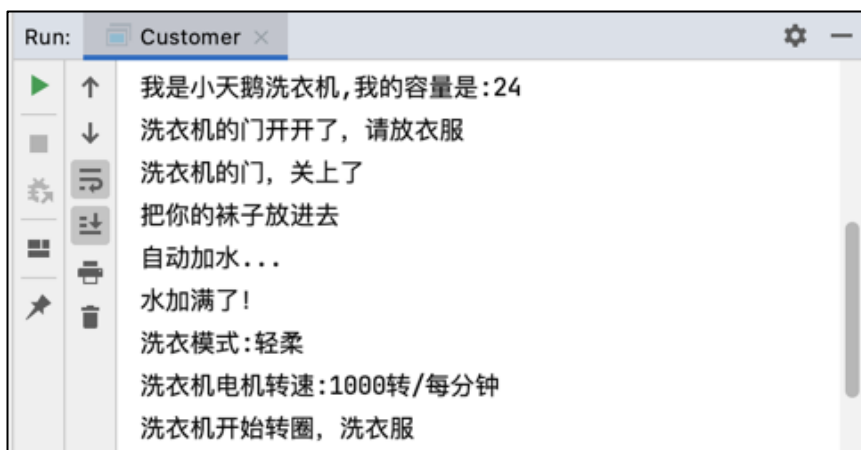


图 9-9 文件 9-7 的运行结果

9.2.3 getter 和 setter 方法

对数据封装之后，想要访问或者设置私有数据可以创建 `getter` 和 `setter` 方法。`getter` 和 `setter` 的命名遵循的是 `JavaBean` 命名约定，例如，现在有一个私有变量 `name`：

```
private String name;
```

按照 `JavaBean` 命名约束，变量 `name` 对应的 `setter` 和 `getter` 方法，具体如下：

```
public void setName(String name){}
public String getName(){} 
```

其中，`setName()` 方法是变量 `name` 的 `setter` 方法，用于设置 `name` 的值；`getName()` 方法是变量 `name` 的 `getter` 方法，用于获取 `name` 的值。



为了大家更好地理解 getter 和 setter 方法的作用,接下来,我们定义一个 Person 类,具体代码如文件 9-8 所示。

文件 9-8 Person. java

```
public class Person {  
    private String name;  
    private int age;  
    public String getName() {  
        return name;  
    }  
    public int getAge() {  
        return age;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public void setAge(int age) {  
        if (age >= 0 && age <= 100) {  
            this.age = age;  
        }  
    }  
}
```

在测试类Test 内部创建Person类型的对象并调用 setter 方法设置属性值,调用 getter 方法获取属性值,具体代码如文件 9-9 所示。

文件 9-9 Test. java

```
public class Test {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.setAge(80);  
        p.setName("张三");  
        System.out.println(p.getAge());  
        System.out.println(p.getName());  
    }  
}
```

程序的运行结果如图 9-10 所示。

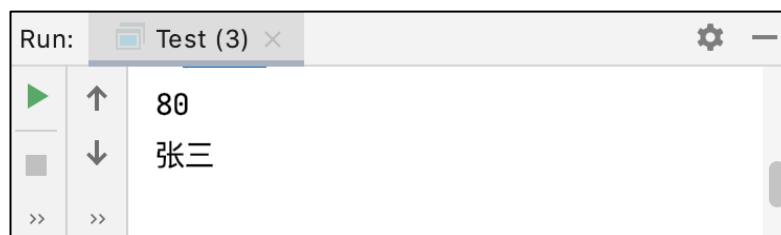


图 9-10 文件 9-9 运行结果



9.3 继承

9.3.1 认识继承

假设我们要开发一个交通工具管理系统，该系统里面有汽车、飞机、轮船这三种交通工具，这些交通工具都有自己的属性，比如，重量、速度等。不同的交通工具，他们的属性有些是相同的，有些是不同的。

例如，汽车、飞机、轮船这些交通工具都有重量、速度这些共同属性，除此之外，飞机有飞行高度的属性、汽车有轴距属性、轮船有吃水深度的属性，如图 9-11 所示。

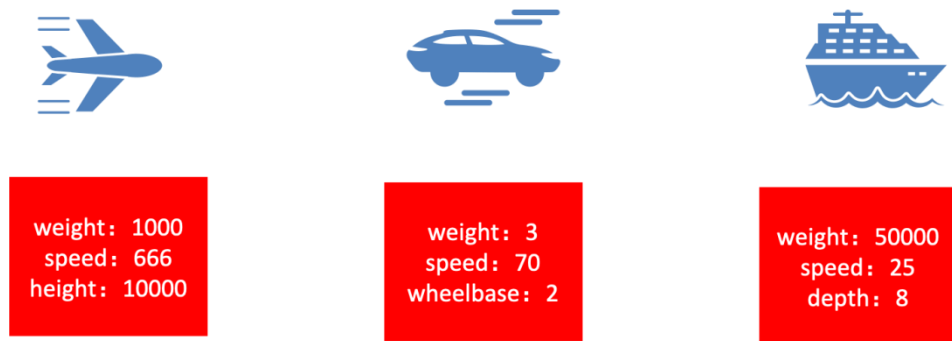


图 9-11 三种交通工具的部分属性

图 9-11 中的三种交通工具的属性都有对应的数据，那这些数据是如何使用类定义呢？如果定义为一个表示交通工具的 TrafficTools 类，代码如图 9-12 所示。

```
class TrafficTools {  
    int type ; //类型，1表示飞机，2表示汽车，3表示轮船  
    float weight ; //重量  
    float speed ; //速度  
    float height ; //飞机的飞行高度  
    float wheelbase ; //汽车的轮胎轴距  
    float depth ; //轮船的吃水深度  
}
```

图 9-12 TrafficTools 类

但是上述定义的 TrafficTools 类有个问题，就是有些属性是某个交通工具特有的，不是每个交通工具都有的。那可能有的同学会说，我们定义三个不同的类，如图 9-13 所示。

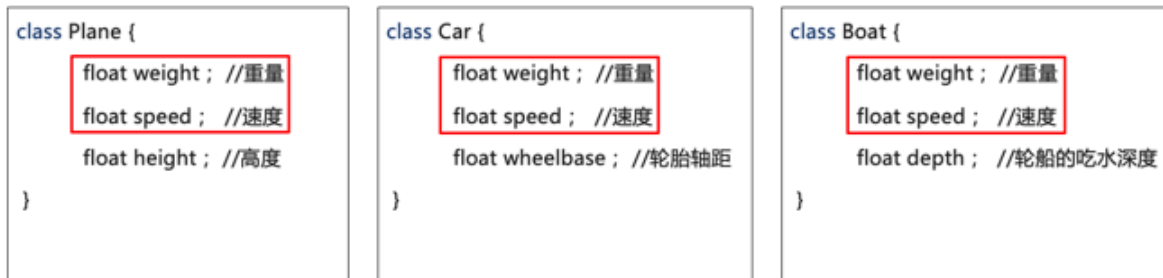


图 9-13 三个不同的类

上述定义每个类包含的都是它自身应该有的属性。但是这样的做法，会导致相同的属性被声明了三次。如果后续需求有变化，要求在每种交通工具中添加一个表示编号的属性，我们需要在三个类中每个都添加一次，如图 9-14 所示。



图 9-14 添加属性后的类

上面在三个类中添加一个属性，可能大家觉得不会过于麻烦。但是在实际生产中，修改这样需求的工作量是非常大的，如果按时上述方式编写代码，一个小小的改动，对程序员来说简直就是一个噩梦。

那有什么办法能对上面的情况进行改进呢？Java 提供了继承这样的特性，它允许我们先定义一个基本的类，然后在基本类上进行扩展。于是，我们可以把前面的类的共同属性抽取出来，形成下列类的定义形式，如图 9-15 所示。

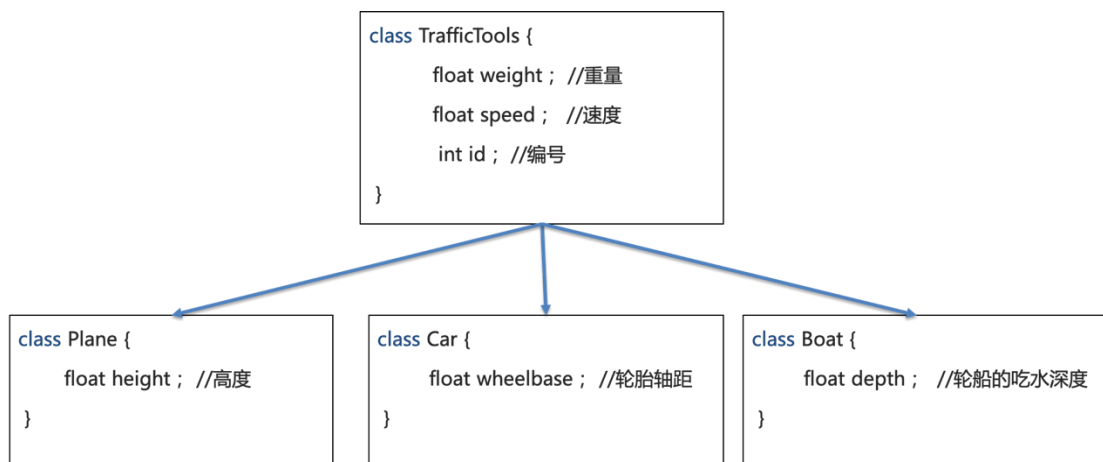


图 9-15 提取共同属性后的类

在图 9-15 中，TrafficTools 类包含的是共同的属性，Plane、Car 和 Boat 这三个类是从



TrafficTools 类扩展的，每个扩展类中都有自己特有的属性。

如何让 Plane、Car 和 Boat 这三个类具有 TrafficTools 共同的属性呢，在 Java 中可以通过 extend 关键字实现，具体如图 9-16 所示。

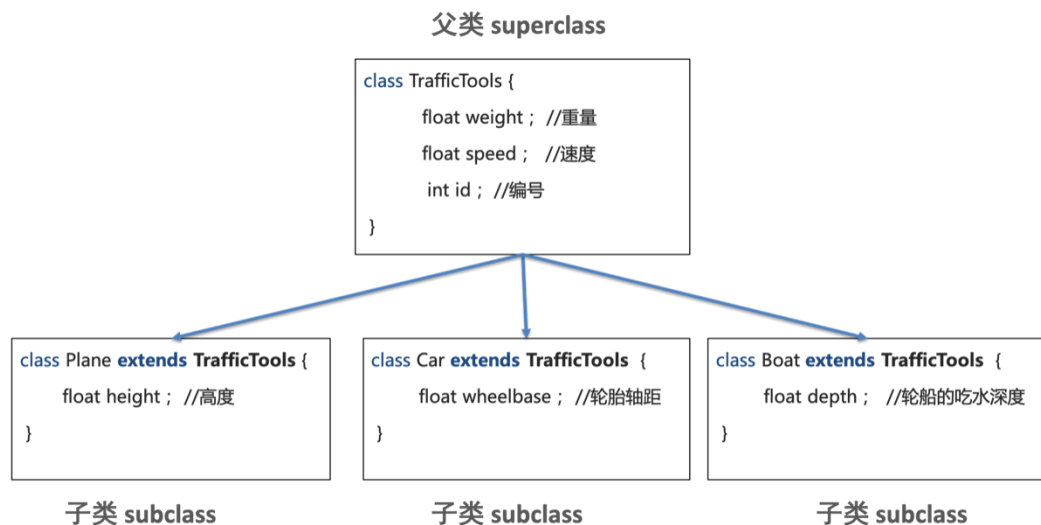


图 9-16 使用 extend 继承 TrafficTools

在图 9-16 中，TrafficTools 是父类，Plane、Car 和 Boat 是继承了 TrafficTools 的子类，这三个子类继承父类后，不仅拥有自己独特的属性，同时拥有了 TrafficTools 类的属性。由此可以看出，使用继承还可以最大程度的减少代码的重复，还保留了子类之间的差异，可以保证代码的灵活性和独立性。

9.3.2 继承的实现

下面我们通过编写代码的方式，演示 9.3.1 节继承的实现，具体步骤如下：

(1) 在包 com.itheima.demo03 中，创建父类 TrafficTools，具体代码如文件 9-10 所示。

文件 9-10 TrafficTools.java

```
package com.itheima.demo03;  
  
public class TrafficTools {  
    float weight; //重量  
    float speed; //速度  
    int id; // 编号  
    public void setId(int id) {  
        this.id = id;  
    }  
    public int getId(){  
        return id;  
    }  
}
```

(2) 创建继承自 TrafficTools 的子类 Plane，具体代码如文件 9-11 所示。



文件 9-11 Plane.java

```
package com.itheima.demo03;

public class Plane extends TrafficTools {

    float height;// 高度

}
```

(3) 创建继承自 TrafficTools 的子类 Car，具体代码如文件 9-12 所示。

文件 9-12 Car.java

```
package com.itheima.demo03;

public class Car extends TrafficTools{

    float wheelbase; //表示轴距

}
```

(4) 创建继承自 TrafficTools 的子类 Boat，具体代码如文件 9-13 所示。

文件 9-13 Boat.java

```
package com.itheima.demo03;

public class Boat extends TrafficTools{

    float depth; // 吃水深度

}
```

(5) 创建一个测试类 Test，在 Test 中创建一个子类对象，此时，子类对象不仅能够继承父类的属性，而且还可以继承父类的方法，如图 9-17 所示。

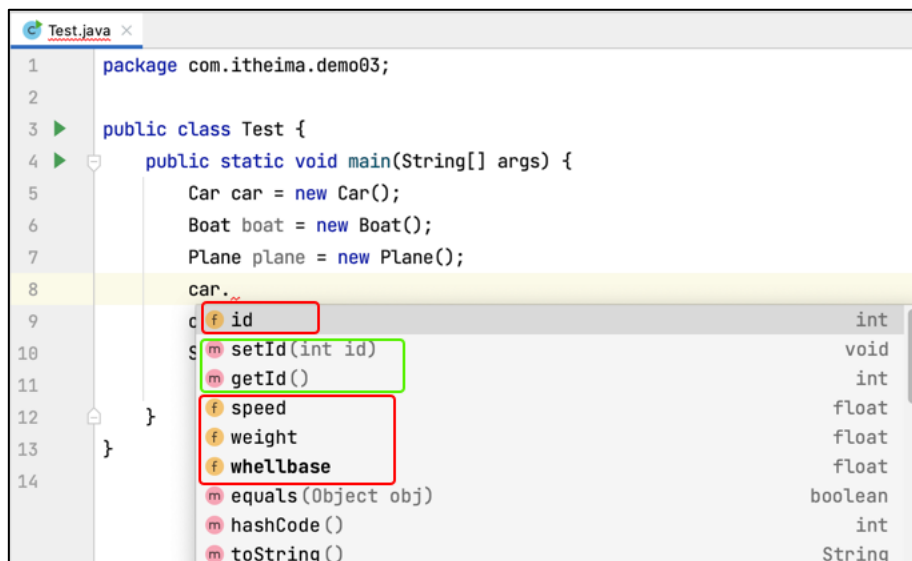


图 9-17 调用父类属性或者方法

需要注意的是，如果父类的某个属性使用 private 修饰，那么继承的子类是拥有这个属性的，但是无法看到这个属性的。这就好比，你的父亲有一个保险箱，你继承了父亲的财产，拥有了保险箱，但是由于保险箱是私有的，没有拿到保险箱的钥匙，就无法看到保险箱中存放的东西。

下面，我们将 TrafficTools 类的 id 使用 private 修饰，此时，我们在 Test 类中通过 Car 对象是无法直接调用 id 属性的，如图 9-18 所示。

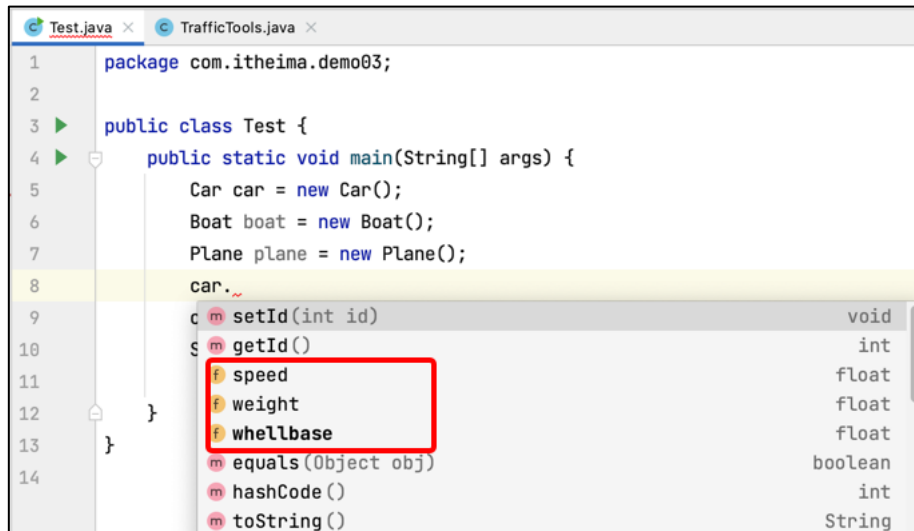


图 9-18 无法调用父类的 id 属性

接下来，我们在 Test 类中通过通过 setId() 和 getId() 操作 id 属性，具体代码如文件 9-14 所示。

文件 9-14 Test.java

```
package com.itheima.demo03;

public class Test {

    public static void main(String[] args) {

        Car car = new Car();

        Boat boat = new Boat();

        Plane plane = new Plane();

        car.setId(10);

        System.out.println(car.getId());

    }

}
```

运行结果如图 9-19 所示。

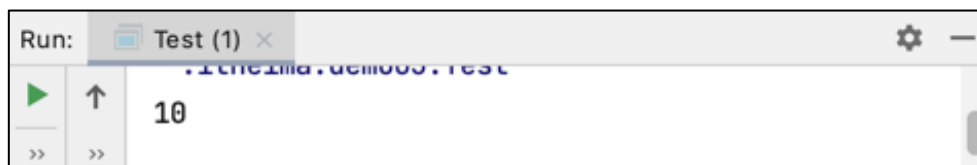


图 9-19 运行结果

9.3.3 方法的重写

在继承关系中，子类会自动继承父类中定义的方法，但有时在子类中需要对继承的方法进行一些修改，即对父类的方法进行重写。在子类中重写的方法需要和父类被重写的方法具有相同的方法名、参数列表以及返回值类型，且在子类重写的方法不能拥有比父类方法更加严格的访问权限。

接下来，通过一个案例来讲解如何进行方法重写，具体步骤如下：

- (1) 定义一个表示哺乳动物的类 Mammal，具体代码如文件 9-15 所示。



文件 9-15 Mammal.java

```
public class Mammal {  
    public void sayHello() {  
        System.out.println("哺乳动物的问候方式：叽里呱啦");  
    }  
}
```

在文件 9-15 中，定义了一个 sayHello() 方法，该方法用于表示哺乳动物 Mammal 的问候方式。

(2) 定义一个表示人类的类 Human，该类继承了 Mammal，具体代码如文件 9-16 所示。

文件 9-16 Human.java

```
public class Human extends Mammal {  
    @Override  
    public void sayHello() {  
        System.out.println("人类的问候方式：您好");  
    }  
}
```

文件 9-16 中，@Override 是方法重写的标识，用于标识 sayHello() 方法是重写的方法，重写后的 sayHello() 方法用于表示人类的问候方式。

(3) 定义一个表示猫类 Cat，该类继承了 Mammal，具体代码如文件 9-17 所示。

文件 9-17 Cat.java

```
public class Cat extends Mammal {  
    @Override  
    public void sayHello() {  
        System.out.println("猫的问候方法：喵喵喵");  
    }  
}
```

在文件 9-17 中，重写的 sayHello() 方法用于表示猫的问候方式。

(4) 定义测试类 Test，在该类中分别创建 Mammal、Human、Cat 类对象，并调用 sayHello() 方法，具体代码如文件 9-18 所示。

文件 9-18 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        Cat cat = new Cat();  
        Human human = new Human();  
        Mammal mammal = new Mammal();  
        cat.sayHello();  
        human.sayHello();  
        mammal.sayHello();  
    }  
}
```

程序的运行结果如图 9-20 所示。

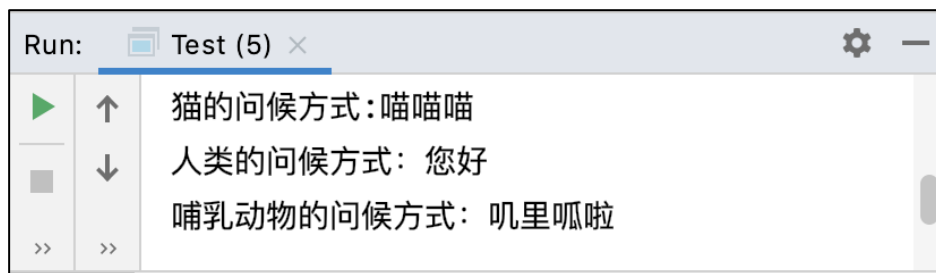


图 9-20 文件 9-18 的运行结果

从图 9-20 可以看出，当子类重写父类的 sayHello() 方法后，Human、Cat 子类调用的都是各自重写的 sayHello() 方法，而不是父类 Mammal 的 sayHello() 方法。

9.4 super 关键字

当子类重写父类的方法后，子类对象将无法访问父类被重写的方法，为了解决这个问题，Java 提供了 super 关键字，super 关键字可以在子类中调用父类的普通属性、方法以及构造方法。

接下来通过一个案例来理解 super 关键字的作用，具体如下：

(1) 定义一个 Person 类，该类内部定义定义了若干个属性及对应的 setter 和 getter 方法，具体代码如文件 9-19 所示。

文件 9-19 Person.java

```
public class Person {  
    protected String name;  
    protected String dayofBirth;  
    protected String address;  
    // 带有三个参数的构造方法  
    public Person(String name, String dayofBirth, String address) {  
        this.name = name;  
        this.dayofBirth = dayofBirth;  
        this.address = address;  
    }  
    public String getName() {  
        return name;  
    }  
    public String getDayofBirth() {  
        return dayofBirth;  
    }  
    public String getAddress() {  
        return address;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```



```
    }  
    public void setDayofBirth(String dayofBirth) {  
        this.dayofBirth = dayofBirth;  
    }  
    public void setAddress(String address) {  
        this.address = address;  
    }  
}
```

(2) 定义一个继承了 **Person** 的类 **Employee**，具体代码如文件 9-20 所示。

文件 9-20 Employee.java

```
240 public class Employee extends Person {  
241     protected String date; // 入职时间  
242     protected Long salary; // 薪资情况  
243     public Employee(String name, String dayofBirth, String address,  
244                     String date, Long salary) {  
245         super(name, dayofBirth, address);  
246         this.date = date;  
247         this.salary = salary;  
248     }  
249     @Override  
250     public String toString() {  
251         return "Employee{" +  
252             "date='" + date + '\'' +  
253             ", salary=" + salary +  
254             ", name='" + name + '\'' +  
255             ", dayofBirth='" + dayofBirth + '\'' +  
256             ", address='" + address + '\'' +  
257             '}';  
258     }  
259 }
```

在文件 9-20 中，第 6 行代码使用 `super()` 调用了父类中有三个参数的构造方法，第 11~19 行代码重写了 `toString()` 方法。

(3) 定义测试类 **Test**，在该类中创建 **Employee** 类型的对象，并输出对象的信息，具体代码如文件 9-21 所示。

文件 9-21 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        // 创建 Employee 类的对象  
        Employee employee = new Employee("张三", "19900101",  
                                           "西二旗", "2021", 10000L);  
        // 输出 Employee 对象信息  
        System.out.println(employee);  
    }  
}
```



```
}
```

程序的运行结果如图 9-21 所示。

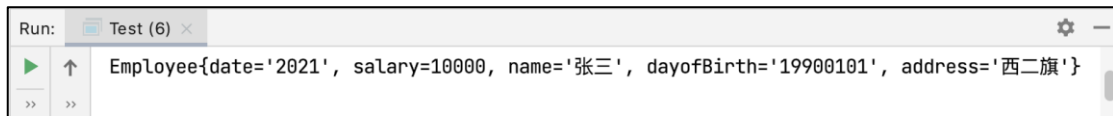


图 9-21 文件 9-21 的运行结果

注意:

通过 `super()`调用父类构造方法的代码必须位于子类构造方法的第一行，并且只能出现一次。

9.5 Object 类

Java 中的万物皆对象，所有的类都继承自 `Object`，该类内置了很多属性和方法。下面我们在 `TrafficTools` 类中，按【Ctrl+0】快捷键，可以查看 `Object` 类的方法，具体如图 9-22 所示。

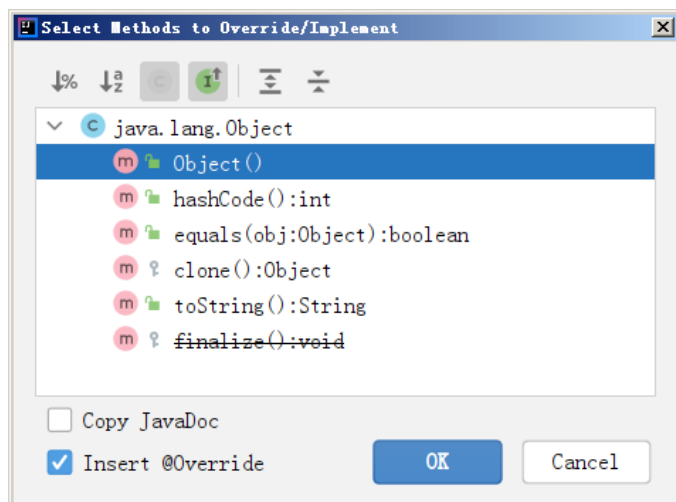


图 9-22 查看 `Object` 类的方法

关于图 9-22 中 `Object` 类相关方法的具体介绍如下：

- (1) `clone()`：用于克隆对象的方法。
- (2) `equals()`：用于比较对象是否相等。
- (3) `hashCode()`：计算对象的哈希值。
- (4) `toString()`：用于输出对象的字符串表示。

9.6 final 关键字

`final` 的英文意思是“最终”。在 Java 中，可以使用 `final` 关键字声明类、属性、方法，在声明时需要注意以下几点：

- (1) 使用 `final` 修饰的类不能有子类。



- (2) 使用 **final** 修饰的方法不能被子类重写。
- (3) 使用 **final** 修饰的变量（成员变量和局部变量）是常量，常量不可修改。
- 接下来，我们以 **final** 修饰方法为例，带大家加深对 **final** 的理解，具体步骤如下：
- (1) 定义一个父类 **Room**，具体代码如文件 9-22 所示。

文件 9-22 Room.java

```
public class Room {  
    int roomWidth;  
    int roomLong;  
    public Room(int roomWidth, int roomLong) {  
        this.roomWidth = roomWidth;  
        this.roomLong = roomLong;  
    }  
    public final int getRoomArea() {  
        return roomWidth * roomLong;  
    }  
}
```

- (2) 定义一个继承了 **Room** 的子类 **LivingRoom**，并在该类中重写 **getRoomArea()** 方法，具体代码如文件 9-23 所示。

文件 9-23 LivingRoom.java

```
public class LivingRoom extends Room {  
    public LivingRoom(int roomWidth, int roomLong) {  
        super(roomWidth, roomLong);  
    }  
    @Override  
    public final int getRoomArea() {  
        return roomWidth * roomLong;  
    }  
}
```

- (3) 定义一个测试类 **Test**，在该类中创建 **LivingRoom** 对象并调用 **getRoomArea()** 方法，具体代码如文件 9-24 所示。

文件 9-24 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        LivingRoom lr = new LivingRoom(3, 5);  
        System.out.println("面积是" + lr.getRoomArea());  
    }  
}
```

编译文件 4-7，编译器报错，如图 9-23 所示。

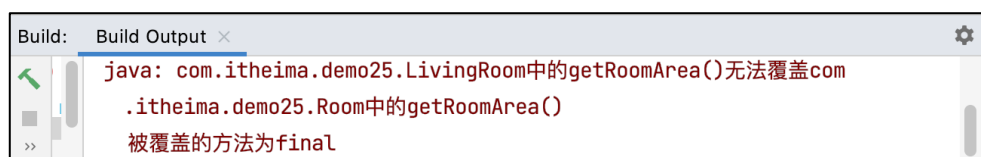


图 9-23 文件 9-24 编译报错

图 9-23 提示编译报错。这是因为 Room 类的 `getRoomArea()` 方法被 `final` 修饰，而被 `final` 关键字修饰的方法为最终方法，子类不能对该方法进行重写。因此，当在父类中定义某个方法时，如果不希望被子类重写，就可以使用 `final` 关键字修饰该方法。

9.7 Java 中的抽象

9.5.1 规则是更高层次的抽象

不知道大家有没有听过这样的句子。

一流的企业卖规则，二流的企业卖技术，三流的企业卖产品，四流的企业卖力气。

下面我们看一张企业的生态系统示意图，如图 9-24 所示。



图 9-24 企业的生态系统示意图

在图 9-24 中，ARM 公司是一家制作知识产权供应商，它主要出售芯片设计的架构规范，以及芯片设计软件设计的授权。可以说，基本上大家使用的电子产品的芯片都是基于 ARM 架构的，例如，华为的麒麟 9000 处理器、苹果的 M1 处理器、高通的处理器等都是基于 ARM 架构设计的。像中芯国际、TSMC、三星这样的企业，他们具备的是基于向华为、苹果、高通这样企业设计的 CPU 的图纸进行 CPU 生产的能力，将图纸变成真正的产品。像 VIVO、荣耀、小米、魅族这样的企业，做的就是通过采购物件进行产品组装的事情。

总而言之，从设计到生产，只有打通了全链路，才能不被“卡脖子”，而做生态设计的，永远位于生态系统的顶端。

程序员的逻辑与上面是类似的，大牛程序员就是定规则，做架构，做技术方案的，而刚入门的小白程序员只能根据规则写代码，卖苦力。

Java 中的抽象类是一个方便我们定规则、做架构的工具，在接下来的几个小节，我们将介绍抽象类的以下知识：

- 学习并掌握抽象类的语法格式。
- 通过一个案例体会小白程序员如何使用大牛程序员写好的抽象类模板开发代码。



- 模仿大牛程序员使用抽象类设计一个简易游戏框架给小白程序员使用。

9.5.2 抽象类和抽象方法的语法

抽象类的定义非常简单，只需要在普通类定义的访问修饰符后面加上 `abstract` 关键字，格式如下：

```
public abstract class Test {  
}
```

需要和大家说明的是，单纯定义一个抽象类是没什么意义的，一般情况下，抽象类都要与抽象方法放在一起定义才有用处，这里说的抽象方法，指的是没有实现方法体的方法。

那定义抽象类有什么用途呢？

大牛程序员写框架：大牛程序员在抽象类里面可以定义抽象方法，这些抽象方法就等同于在定义标准。

小白程序员搬砖做苦力：小白程序员根据大牛程序员定义的标准实现这些抽象方法。

在抽象类中，抽象方法必须使用 `abstract` 关键字修饰，抽象方法不包含方法体，抽象方法的格式具体如下：

```
访问修饰符 abstract 返回值类型 抽象方法名 (参数);
```

如果想将普通方法改造成抽象方法，只需要做两步：

第1步：删除方法体，将 `{}` 换成分号。

第2步：在访问修饰符后添加 `abstract` 关键字。

在抽象类中定义抽象方法的示例代码具体如下：

```
public abstract class A {  
    public abstract void hello();  
}
```

注意：

- (1) 抽象类里面的抽象方法只需要声明，不需要去实现。
- (2) 一个抽象类可以含有一个或者多个抽象方法。
- (3) 抽象类可以包含普通的方法。
- (4) 抽象类不能实例化，也就是说不能用 `new` 的方式产生对象。
- (5) 我们只能 `new` 抽象类的子类，这个抽象类的子类需要实现抽象方法。

9.5.3 应用案例：计算器

为了大家更好地理解抽象类和抽象方法，下面我们做一个计算器的案例。

再回顾一下前面我们讲过大牛程序员和小白程序员做的事情，大牛程序员写框架，他们可以定义抽象类，把一些懒得写业务逻辑定义成抽象方法。小白程序员搬砖做苦力，专门实现抽象类子类，实现那些大牛程序员懒得写的业务逻辑。



作为大牛程序和小白程序员，他们分别要做下列事情：

大牛程序员，首先要思考如何定义计算器的抽象类架构，包括计算器有什么功能，每个功能的方法名，方法参数以及返回值等，例如，下面的代码：

```
public abstract int add(int x, int y);
```

小白程序员，根据大牛程序员的设计，定义子类实现抽象方法的业务逻辑，例如下面的代码：

```
public abstract class A {  
    public int add(int x, int y) {  
        return x + y;  
    }  
}
```

现在，我们在 IDEA 中编写代码实现计算器，具体步骤如下：

(1) 定义 `AbstractCalc` 类，该类是大牛程序员写的抽象类，类中定义了两个数加减乘除运算的抽象方法，具体代码如文件 9-25 所示。

文件 9-25AbstractCalc.java

```
package com.itheima.demo04;  
  
/**  
 * 大牛程序员写的抽象类  
 */  
  
public abstract class AbstractCalc {  
    public abstract int add(int x, int y);  
    public abstract int sub(int x, int y);  
    public abstract int multi(int x, int y);  
    public abstract int devide(int x, int y);  
}
```

(2) 定义 `Calc` 类，该类用于实现了抽象类的业务逻辑，具体代码如文件 9-26 所示。

文件 9-26 Calc.java

```
package com.itheima.demo04;  
  
public class Calc extends AbstractCalc {  
    @Override  
    public int add(int x, int y) {  
        return x + y;  
    }  
    @Override  
    public int sub(int x, int y) {  
        return x - y;  
    }  
    @Override  
    public int multi(int x, int y) {  
        return x * y;  
    }  
    @Override  
    public int devide(int x, int y) {
```



```
        return x / y;  
    }  
}
```

(3) 定义 Test 类，对计算器的功能进行测试，具体代码如文件 9-27 所示。

文件 9-27 Test.java

```
package com.itheima.demo04;  
  
public class Test {  
    public static void main(String[] args) {  
        int a = 3;  
        int b = 5;  
        Calc c = new Calc();  
        System.out.println(c.add(a, b));  
        System.out.println(c.sub(a, b));  
        System.out.println(c.multi(a, b));  
        System.out.println(c.devide(a, b));  
    }  
}
```

程序的运行结果如图 9-25 所示。

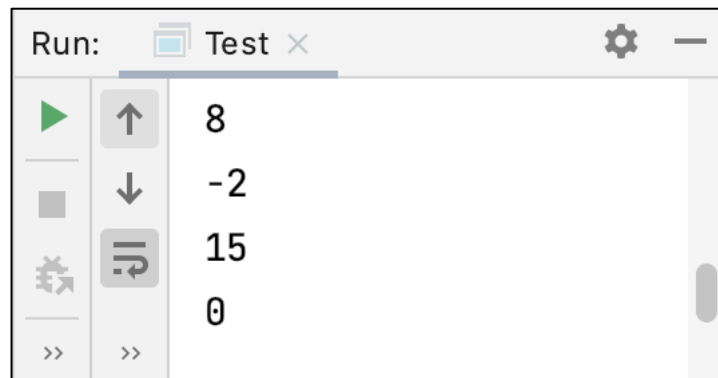


图 9-25 运行结果

9.8 案例：使用抽象类模板开发乒乓球游戏

9.6.1 乒乓球游戏需求分析

这节课我们通过一个游戏案例体会一下如何使用大牛程序员开发的抽象类模板快速开发一款小游戏。游戏效果如图 9-26 所示。

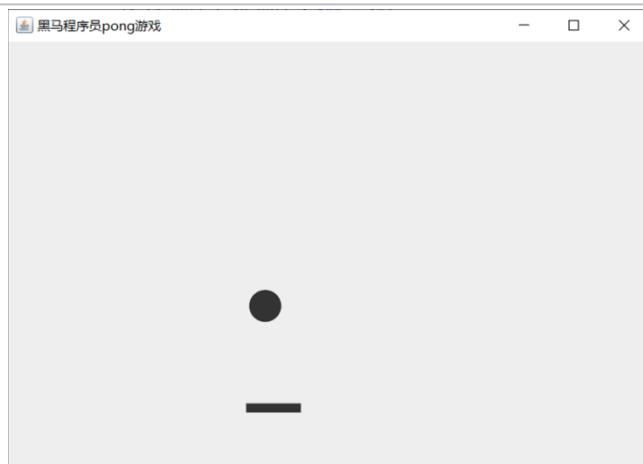


图 9-26 游戏效果

这里给大家分享个小故事，我们这节课开发的这款游戏看似简单，但它的玩法和世界上第一款交互式游戏—Pong 是基本类似的。1972 年，Pong 游戏最初出现在酒吧，大家争先恐后的玩这个游戏，酒吧人满为患。第 2 年夏天，Pong 游戏推出了家用版，一台游戏机的售价 45 美金，截止圣诞节前夕，Pong 游戏机的销量多达 15 万台。大家想想，如果你听完了这节课，穿越回 1972 年卖这个游戏的话，是不是就会变成百万级的大富翁呀？

通过分析游戏玩法可知，开发这个游戏需要实现的功能及技术点如下：

- (1) 如何在创建一个游戏窗体。(GUI 图形化编程)
- (2) 如何在窗体上画一个球 (GUI 绘制圆形)
- (3) 如何在窗体上画一个球拍。(GUI 绘制矩形)
- (4) 如何通过键盘控制球拍左右移动。(处理键盘事件)
- (5) 已知球的位置和球拍的位置，如何判断球有没有落在球拍上。(业务逻辑)

上述罗列的功能中，第 (1)~(4) 个都是我们目前实现不了的，需要大牛程序员帮忙。

其中，第 (5) 个功能，我们可以通过 if...else...实现。

那如何向大牛程序员寻求帮助呢？我们可以采用下列方式向大牛程序员寻求帮助：

大哥， 我现在精通 Java 的 helloworld，熟悉条件判断，循环和数组，

但是我 GUI 编程，窗体绘制，键盘事件处理一窍不通，我想开发个乒乓球游戏，

您能给我提供些一个开发游戏的抽象类模板吗？GUI 编程，窗体绘制，键盘事件处理您能给我提供一些示例代码吗？

相信大牛程序员听到你的诉求后，一定会给你提供一套 jar 包和开发文档，作为小白程序员，我们要做的事情就是熟悉 jar 包的内容。

9.6.2 游戏框架介绍

大牛程序员提供的游戏框架 jar 包，主要包括两个类，分别是 Game 和 GameEngine，其中，GameEngine 类是游戏引擎的抽象模板类，Game 类是游戏的公共类，具体如图 9-27 和图 9-28 所示。



程序包 com.itheima.game

类 GameEngine

java.lang.Object
com.itheima.game.GameEngine

```
public abstract class GameEngine  
extends java.lang.Object
```

由大牛程序员编写的游戏引擎的抽象模板类

需要调用者实现两个抽象方法，更新逻辑方法updateLogic

更新ui方法 renderUI

图 9-27 GameEngine 类

程序包 com.itheima.game

类 Game

java.lang.Object
java.awt.Component
java.awt.Container
javax.swing.JComponent
javax.swing.JPanel
com.itheima.game.Game

```
public class Game  
extends javax.swing.JPanel
```

大牛程序员写的游戏的公共类

使用Game.init()方法初始化游戏

使用Game.gameOver()方法退出游戏

图 9-28 Game 类

这里，我们建议同学们打开准备好的 API 文档进行查阅，并借助代码实践加深对 API 的理解。

9.6.3 通过实验理解抽象类中方法的作用

为了帮助大家更好地理解大牛程序员编写的抽象类及抽象方法，下面我们通过代码方式介绍一下抽象类中不同方法的作用。

为了方便后续代码的编写，我们先做一些准备工作，具体如下：

(1) 我们先新建一个项目 OOP2，并在项目中导入准备好的 game-engine.jar 包。

(2) 大牛程序员共为我们提供了两个类，一个是 Game 类，一个是 GameEngine 类。

由于 GameEngine 类是一个抽象类，所以我们在 com.itheima.gamedemo 包中定义



RealGameEngine 类实现抽象类 GameEngine 中的方法，RealGameEngine 的具体代码如下文件 9-28 所示。

文件 9-28 RealGameEngine.java

```
package com.itheima.gamedemo;
import com.itheima.game.GameEngine;
public class RealGameEngine extends GameEngine {
    @Override
    public void updateLogic() {
        System.out.println("updatellogic");
    }
    @Override
    public void renderUI(Graphics2D g2d) {
        System.out.println("render ui");
    }
}
```

完成准备工作后，我们首先创建一个游戏对象。大牛程序员为我们提供了一个 Game 类，该类是游戏的公共类，它提供了初始化游戏的方法 Game.init()方法，如图 9-29 所示。

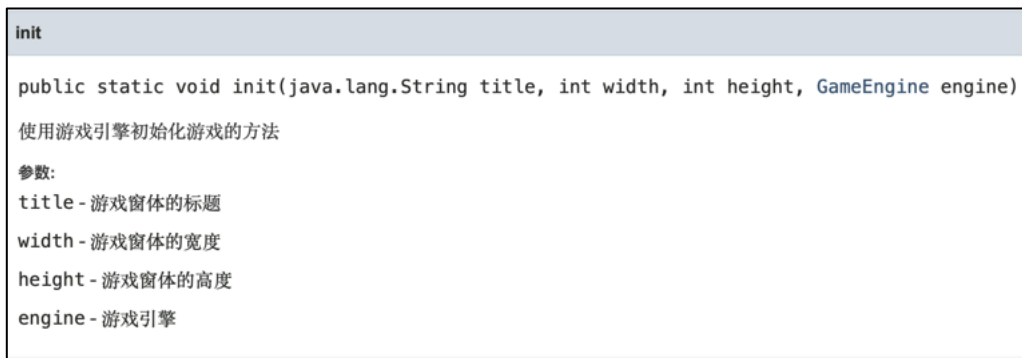


图 9-29 Game.init()方法介绍

下面我们在测试类 Test 中创建一个 Game 对象，并完成 Game 对象的初始化，具体代码如下文件 9-29 所示。

文件 9-29 Test.java

```
package com.itheima.gamedemo;
import com.itheima.game.Game;
public class Test {
    public static void main(String[] args) {
        RealGameEngine engine = new RealGameEngine();
        Game.init("黑马乒乓球", 300, 400, engine);
    }
}
```

上述代码使用游戏引擎对游戏进行了初始化，其中参数“黑马乒乓球”用于设置游戏窗体的标题，300 用于设置游戏窗体的宽度，400 用于设置游戏窗体的高度，engine 表示的初始化游戏的引擎对象。



程序运行后，发现会弹出一个游戏窗口，如图 9-30 所示。

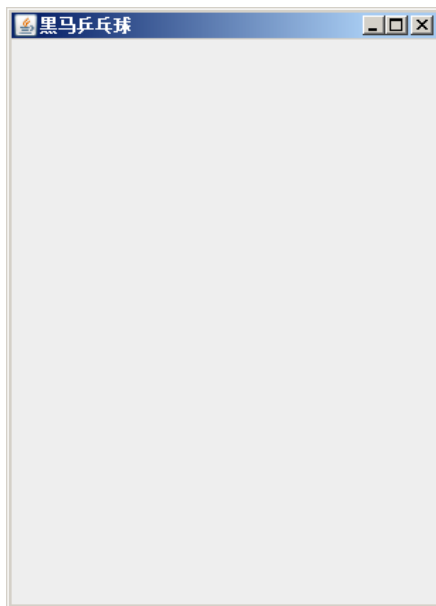


图 9-30 游戏窗口

与此同时，控制台在不停的打印“updateLogic”和“render ui”，说明游戏引擎在不停的更新逻辑和游戏界面，其实这点是符合实际游戏场景的，因为无论什么游戏，游戏引擎会一直监控用户的操作，并及时更新游戏画面。控制台输出效果如图 9-31 所示。

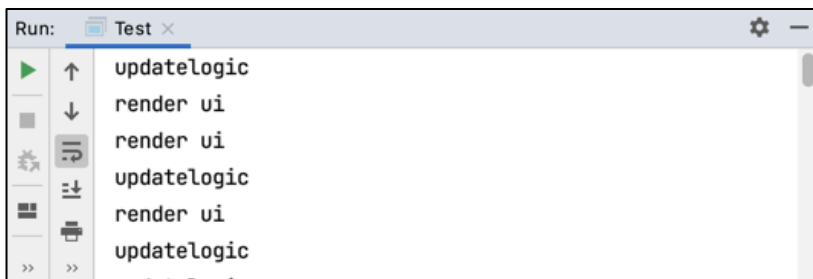


图 9-31 控制台输出效果

除此之外，大牛程序员也考虑到了小白程序员不知道如何处理键盘，所以提前在 GameEngine 类中提供了一个 getCurrentPressedKeyCode()方法，该方法用于获取当前用户的按钮代码，如图 9-32 所示。



图 9-32getCurrentPressedKeyCode()方法

接下来，我们在 RealGameEngine 中使用 getCurrentPressedKeyCode()方法获取用户的按键代码，并对按键代码进行判断，示例代码如下：



```
public void updateLogic() {  
    int keycode = getCurrentPressedKeyCode();  
    if (keycode == KeyEvent.VK_LEFT) {  
        System.out.println("左");  
    } else if (keycode == KeyEvent.VK_RIGHT) {  
        System.out.println("右");  
    }  
}  
  
public void renderUI(Graphics2D g2d) {  
}
```

再次运行测试类 Test 进行按键测试。我们按一下键盘←，发现控制台会打印出“左”，按→，控制台会打印出“右”，说明我们使用代码的方式成功获取到了用户的按键。

9.6.4 快速开发乒乓球游戏

在开发游戏之前，我们先补充几个技术关键点。

1. 坐标系统

以游戏窗口界面为例的坐标系如图 9-33 所示。

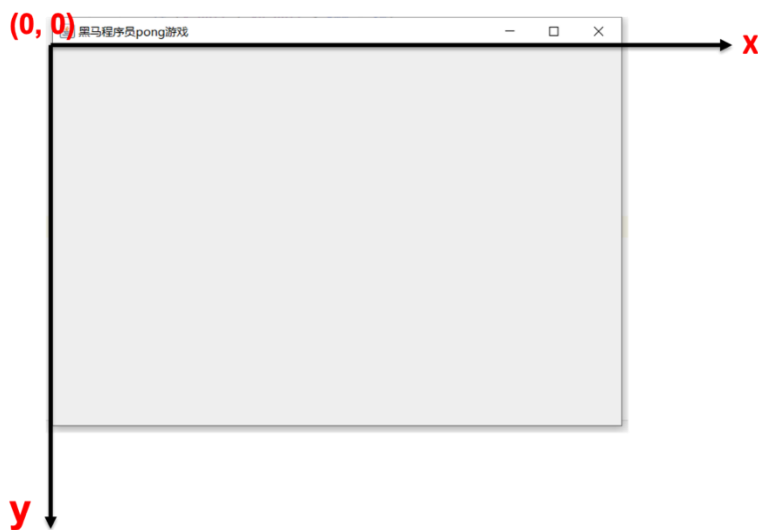


图 9-33 游戏界面的坐标系

在图 9-33 中，左上角的 (0, 0) 是坐标原点，原点的水平向右是 x 轴方向，垂直向下是 y 轴方向。

2. 如何画矩形和椭圆

在大牛程序员提供的 API 中，有一个 renderUI() 方法，如图 9-34 所示。



renderUI

```
public abstract void renderUI(java.awt.Graphics2D g2d)
```

根据用户需求更新绘制图形化界面的内容

示例代码：绘制矩形： `g2d.fillRect(左上角x的坐标, 左上角y的坐标, 矩形的宽度, 矩形的高度);`

示例代码：绘制椭圆形： `g2d.fillOval(椭圆外包裹矩形左上角x的坐标, 椭圆外包裹矩形y的坐标, 要填充椭圆形的宽度, 要填充椭圆形的高度);`

图 9-34 renderUI()方法

在 `renderUI()` 方法的 API 文档中，提供了绘制图形的示例代码，具体如下：

(1) 绘制矩形：

`g2d.fillRect(左上角 x 的坐标, 左上角 y 的坐标, 矩形的宽度, 矩形的高度);`

(2) 绘制椭圆形：

`g2d.fillOval(椭圆外包裹矩形左上角 x 的坐标, 椭圆外包裹矩形 y 的坐标, 要填充椭圆形的宽度, 要填充椭圆形的高度);`

图 9-35 和图 9-36 分别描述了使用 `g2d.fillRect(30,40,30,30)` 绘制矩形以及使用 `g2d.fillOval(30,30,10,20)` 绘制椭圆的效果。

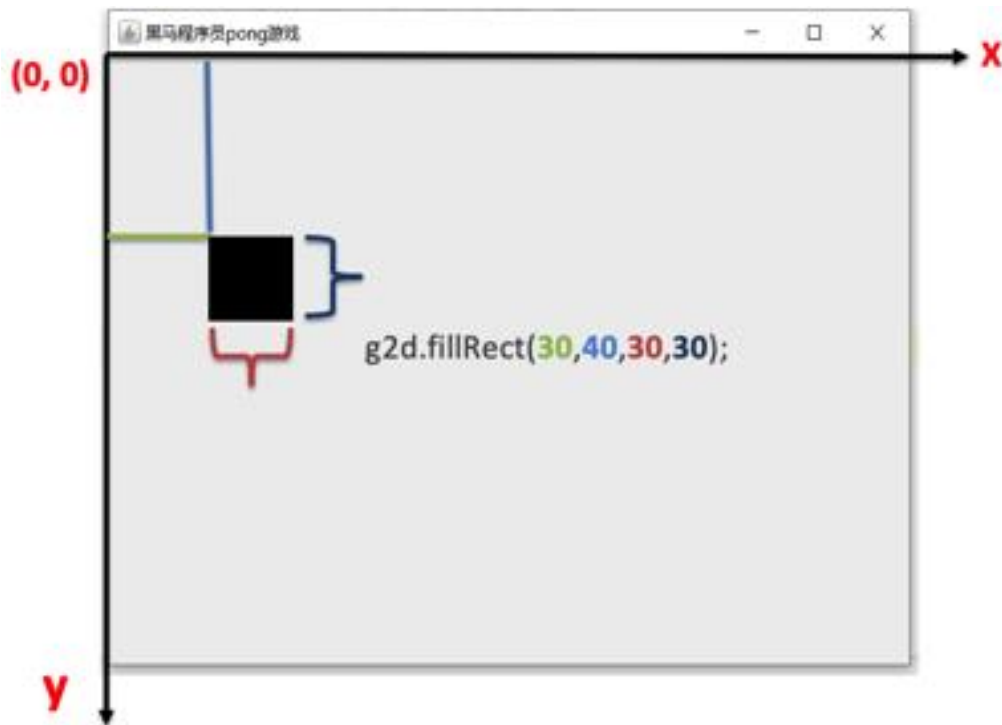


图 9-35 `g2d.fillRect(30,40,30,30)` 绘制矩形

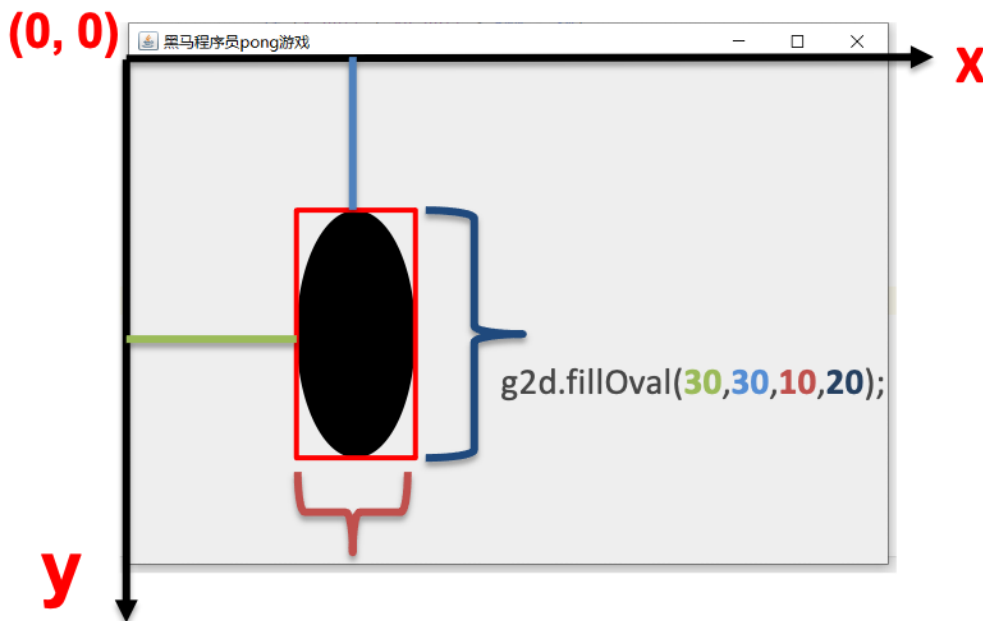


图 9-36 g2d.fillOval(30,30,10,20)绘制椭圆

3. 如何更新球的位置

根据游戏效果可知，球的大小是不变的，变化的是球在 x 和 y 轴的距离，所以如果我们希望球呈现出移动的效果，只需要不停的更改 x、y 的坐标位置即可。

接下来，我们在 RealGameEngine 类中编写代码，实现球移动的效果，具体代码如文件 9-30 所示。

文件 9-30 RealGameEngine.java

```
260 import com.itheima.game.GameEngine;
261 import java.awt.*;
262 import java.awt.event.KeyEvent;
263 public class RealGameEngine extends GameEngine {
264     int x = 30; //
265     int y = 30;
266     int ax = 1; // x 方向移动的距离
267     int ay = 1; // y 方向移动的距离
268     @Override
269     public void updateLogic() {
270         if (x < 0) { // 如果小球移出最左边屏幕，控制小球向右移动
271             ax = 1;
272         } else if (y < 0) { // 如果小球移出最上边屏幕，控制小球向下移动
273             ay = 1;
274         } else if (x > 300 - 30) { // 如果小球到达屏幕最右边，控制小球向左移动
275             ax = -1;
276         } else if (y > 400 - 30) { // 如果小球到达屏幕最下边，控制小球向上移动
```



```
277         ay = -1;
278     }
279     x = x + ax;
280     y = y + ay;
281     int keycode = getCurrentPressedKeyCode();
282     if (keycode == KeyEvent.VK_LEFT) {
283         System.out.println("左");
284     } else if (keycode == KeyEvent.VK_RIGHT) {
285         System.out.println("右");
286     }
287 }
288 // 渲染 UI 界面的方法
289 @Override
290 public void renderUI(Graphics2D g2d) {
291     g2d.fillOval(x, y, 30, 30); // 绘制圆形的球
292 }
293 }
```

至此，就完成了小球移动的效果，当小球接触到边缘后，小球会自动弹回。

4. 实现球拍的绘制与控制

绘制球拍比较简单，只需要在 `renderUI()` 方法中调用 `fillRect()` 方法绘制一个矩形即可。

如果想使用键盘控制球拍左右移动，可以通过监听键盘按键设置球拍 `x` 的坐标。接下来，继续完善 `RealGameEngine` 类，具体代码如文件 9-31 所示。

文件 9-31 RealGameEngine.java

```
import com.itheima.game.GameEngine;
import java.awt.*;
import java.awt.event.KeyEvent;

public class RealGameEngine extends GameEngine {
    int x = 30;
    int y = 30;
    int ax = 1;
    int ay = 1;
    int ballpannel = 30;
    @Override
    public void updateLogic() {
        if (x < 0) {
            ax = 1;
        } else if (y < 0) {
            ay = 1;
        } else if (x > 300 - 30) {
            ax = -1;
        } else if (y > 400 - 30) {
            ay = -1;
        }
    }
}
```



```

    }

    x = x + ax;
    y = y + ay;
    // 通过键盘控制球拍的移动方向

    int keycode = getCurrentPressedKeyCode();
    if (keycode == KeyEvent.VK_LEFT) {
        System.out.println("左");
        ballpanel--; // 球拍向左移动
    } else if (keycode == KeyEvent.VK_RIGHT) {
        System.out.println("右");
        ballpanel++; // 球拍向右移动
    }
}

@Override
public void renderUI(Graphics2D g2d) {
    g2d.fillOval(x, y, 30, 30);

    g2d.fillRect(ballpanel, 390, 60, 10); // 绘制球拍
}
}

```

5. 检测球是否落在球拍上

最后，我们要解决的一个问题，就是检测球是否落在球拍上。当球落下来的时候，我们可以通过判断球与球拍的 x 坐标位置，进而判断球是否落在球拍上。如果落在球拍上，那么让球反弹，否则提示游戏结束。

继续对 RealGameEngine 进行修改，修改后的代码如文件 9-32 所示。

文件 9-32 RealGameEngine.java

```

import com.itheima.game.Game;
import com.itheima.game.GameEngine;
import java.awt.*;
import java.awt.event.KeyEvent;

public class RealGameEngine extends GameEngine {
    int x = 30;
    int y = 30;
    int ax = 1;
    int ay = 1;
    int ballpanel = 30;

    @Override
    public void updateLogic() {
        if (x < 0) {
            ax = 1;
        } else if (y < 0) {
            ay = 1;
        } else if (x > 300 - 30) {

```



```
        ax = -1;
    } else if (y > 400 - 30) {
        // 检查球拍是否在球的下方
        if (x > ballpanel && x < ballpanel + 60) {
            ay = -1;
        } else {
            Game.gameOver("游戏结束", "友情提示");
        }
    }
}

x = x + ax;
y = y + ay;

int keycode = getCurrentPressedKeyCode();
if (keycode == KeyEvent.VK_LEFT) {
    System.out.println("左");
    ballpanel--;
} else if (keycode == KeyEvent.VK_RIGHT) {
    System.out.println("右");
    ballpanel++;
}
}

@Override
public void renderUI(Graphics2D g2d) {
    g2d.fillOval(x, y, 30, 30);
    g2d.fillRect(ballpanel, 390, 60, 10);
}
}
```

程序的运行结果如图 9-37 所示。



图 9-39 运行结果

6. 显示游戏得分信息



显示游戏的得分信息，我们分两步完成，具体如下：

(1) 在 RealGameEngine 类中定义一个变量 count 用来统计分数，每当乒乓球在 Y 轴方向弹起一次，那么 count 的值加 1，表示得一分，示例代码如下：

```
public class RealGameEngine extends GameEngine {  
    // 省略部分代码  
    int count=0;  
    @Override  
    public void updateLogic() {  
        if (x < 0) {  
            ax = 1;  
        } else if (y < 0) {  
            ay = 1;  
        } else if (x > 300 - 30) {  
            ax = -1;  
        } else if (y > 400 - 30) {  
            // 检查球拍是否在球的下方  
            if (x > ballpannel && x < ballpannel + 60) {  
                count++;  
                ay = -1;  
            } else {  
                Game.gameOver("游戏结束", "友情提示");  
            }  
        }  
    }  
    // 省略代码  
}
```

(2) 在 RealGameEngine 类的 renderUI 中绘制游戏得分的文本信息，具体代码如下：

```
g2d.drawString("游戏得分:"+count,0,30);
```

此时，再次运行游戏，游戏效果如图 9-38 所示。

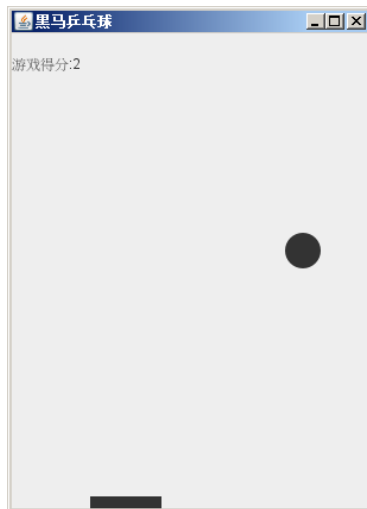


图 9-38 游戏得分效果



9.9 静态常量

在学习静态常量声明之前，我们先来认识一下魔法数字。

魔法数字指的是我们写代码的时候用到的一些数字，这些数字我们自己知道代表什么含义，但是其他程序员可能有所疑惑，所以在开发过程中要尽量避免魔法数字。

那怎么做呢？这里就引出了我们要讲的静态常量了。

静态常量指的是使用 `public static final` 修饰的变量，其定义格式如下：

```
public static final 数据类型 变量名 = 值;
```

上述格式中，变量名通常采用全部字母大写形式，多个单词使用下划线连接。

例如，在 `Config` 类中定义一个静态常量，示例代码如下：

```
public class Config{  
    public static final int WINDOW_WIDTH=300;  
}
```

如果我们想使用类的静态成员，不需要创建对象，直接使用类名就可以访问了，示例代码如下：

```
System.out.println(Config.WINDOW_WIDTH);
```

接下来，我们对 9.5 小节的案例进行修改，使用类的静态常量对之前编写的代码进行重构，具体步骤如下：

(1) 定义一个 `Config` 类，该类定义了 4 个静态常量，分别表示窗体宽度、窗体高度、球的直径、球拍宽度、球拍高度，具体代码如文件 9-33 所示。

文件 9-33 Config.java

```
public class Config {  
    public static final int WINDOW_WIDTH = 300; // 窗体宽度  
    public static final int WINDOW_HEIGHT = 400; // 窗体高度  
    public static final int BALL_SIZE = 30; // 球的直径  
    public static final int PANEL_WIDTH = 60; // 球拍的宽度  
    public static final int PANEL_HEIGHT = 60; // 球拍的高度  
}
```

(2) 修改 `Test` 类，使用 `Config` 类中定义的静态常量替换魔法数字，修改后的代码如文件 9-34 所示。

文件 9-34 Test.java

```
import com.itheima.game.Game;  
public class Test {  
    public static void main(String[] args) {  
        RealGameEngine engine = new RealGameEngine();  
        Game.init("黑马乒乓球",Config.WINDOW_WIDTH,  
                Config.WINDOW_HEIGHT, engine);  
    }  
}
```




(3) 修改 RealGameEngine 类，使用静态常量替换魔法数字，修改后的代码如文件 9-35 所示。

文件 9-35 RealGameEngine.java

```
import com.itheima.game.Game;
import com.itheima.game.GameEngine;
import java.awt.*;
import java.awt.event.KeyEvent;

public class RealGameEngine extends GameEngine {
    int x = 30;
    int y = 30;
    int ax = 1;
    int ay = 1;
    int ballpanel = 30;
    int count=0;
    @Override
    public void updateLogic() {
        if (x < 0) {
            ax = 1;
        } else if (y < 0) {
            ay = 1;
        } else if (x > Config.WINDOW_WIDTH - Config.BALL_SIZE) {
            ax = -1;
        } else if (y > Config.WINDOW_HEIGHT - Config.BALL_SIZE) {
            // 检查球拍是否在球的下方
            if (x > ballpanel && x < ballpanel + Config.PANEL_WIDTH) {
                count++;
                ay = -1;
            } else {
                Game.gameOver("游戏结束", "友情提示");
            }
        }
        x = x + ax;
        y = y + ay;

        int keycode = getCurrentPressedKeyCode(); // 获取键盘按键
        if (keycode == KeyEvent.VK_LEFT) {
            System.out.println("左");
            ballpanel--;
        } else if (keycode == KeyEvent.VK_RIGHT) {
            System.out.println("右");
            ballpanel++;
        }
    }
}
```



```
@Override
public void renderUI(Graphics2D g2d) {
    g2d.fillOval(x, y, Config.BALL_SIZE, Config.BALL_SIZE);
    g2d.fillRect(ballpanel, 390,
        Config.PANEL_WIDTH, Config.PANEL_HEIGHT);
    g2d.drawString("游戏得分:"+count,0,30);
}
}
```

至此，我们使用静态变量完成了对 9.8 小节乒乓球游戏的改写。

9.10 接口

9.8.1 认识接口

接口是一种规范。怎么理解呢？我们先看一下现实生活中的接口，如图 9-39 所示。



图 9-39 现实生活中的接口

在图 9-39 中，左边是联想笔记本的电源线接口，右边是西门子插座。大家有没有想过，为什么不同厂家生产的东西可以完美的配套使用？其实背后的原因就是他们遵守了相同的规范。我们国家的标准委员会已经定义了插头插座的标准，所有的厂商必须严格的按照这种标准去执行，如果你尝试修改这个规范定义的数据是不允许的。图 9-40 展示的就是插座的国家标准。

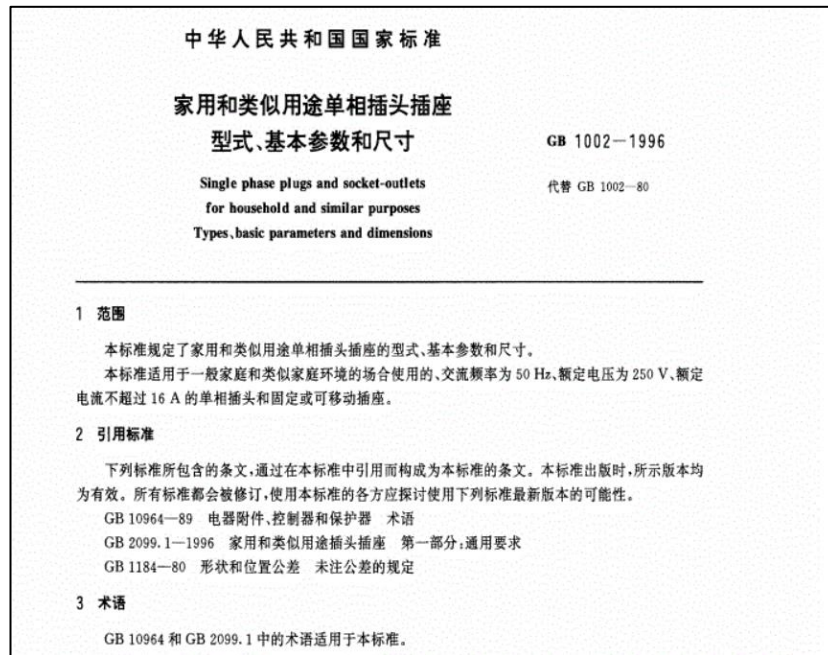


图 9-40 插座的国家标准

Java 中的接口，和现实中的接口含义是非常类似的。下面我们在 IDEA 中定义一个表示插头的接口。创建一个项目 code，在项目中新建包 com.itheima.demo01,右击包选择【New】→【Java Class】，填写接口名称 Socket，并选择 Interface，如图 9-41 所示。

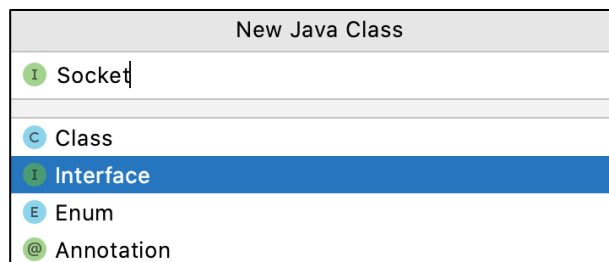


图 9-41 创建接口

创建好的接口文件，如图 9-42 所示。

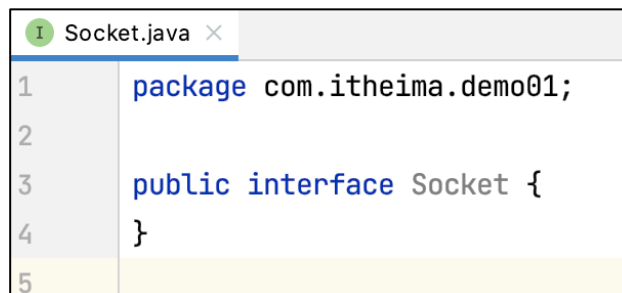


图 9-42 创建好的接口 Socket 文件

从图 9-42 中可以看出，接口的声明使用 interface 关键字修饰。

接下来，我们在接口中定义 distance 表示两向插头的两个金属片之间的距离，具体代码如下：

```
public interface Socket {
```



```
float distance = 12.7f; // 二向插头，两个金属片之间的距离  
}
```

为了体现接口的作用，我们编写一个测试类 `Test`，并在测试类中调用接口中的数据，具体代码如下：

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(Socket.distance);  
    }  
}
```

需要注意的是，默认情况下，接口中定义的变量默认都是被 `public static final` 修饰的，也就是说，接口中定义的数据都是静态常量，这些接口中的静态常量的数据是不允许被修改的。

9.8.2 接口定义行为标准

接口的声明使用 `interface` 关键字修饰，类的声明使用 `class` 修饰，如图 9-43 所示。



图 9-43 接口和类的声明方式

通过上节课学习可知，接口定义的成员变量都是常量，它是自动被 `public static final` 修饰的，接口中定义的变量是不可以被修改的，这正是接口是一种规范的具体体现。

其实，接口不仅可以定义数值标准，更重要的是还定义了行为标准。怎么理解呢，我们看一个现实生活中的例子。

我们知道刑法标准定义了很多犯罪的标准，例如强奸犯的立案标准如图 9-44 所示。

立案标准

根据刑法第236条的规定，有下列两种情形之一的，都应当以强奸罪立案追究：

- (1) 以暴力、胁迫或者其他手段强奸妇女的；
- (2) 奸淫不满14周岁幼女的。

强奸罪是行为犯，只要行为人实施了强奸妇女或者奸淫幼女的行为，不论是否达到奸淫的目的，都应当立案追究。

[详情](#)

图 9-44 强奸犯的立案标准

在图 9-44 的立案标准中，只是定义了强奸犯的行为，刑法本身并没有实现强奸犯的行为。如果有个法外狂徒张三想去按照这个标准实现立案标准，就需要实现图 9-44 中的（1）或者（2）的行为，那么张三就会变成一个刑法认证的强奸犯。



大家思考一下公安局的作用，它的作用之一就是检测有没有人实现了强奸犯的接口（更准确的讲就是实现了刑法标准里面定义的行为），实现了这个接口的人就会被抓起来。

上述现实生活的举例，希望大家能明白什么是定义标准，什么是实现标准。定义标准指的就是客观的将内容进行描述，而实现标准就是基于标准做的一些事情。

Java 中的行为，使用方法描述。那么在接口中定义行为标准，其实就是定义方法。在接口中声明方法的方式与抽象类方法的声明方式基本类似，都是没有方法体的。抽象类和接口中声明方法的具体示例如图 9-45 所示。



图 9-45 抽象类和接口声明方法

需要注意的是，默认情况下，声明接口时省略了 `abstract` 关键字，也就是说，默认情况下，接口是一个特殊的抽象类，它里面声明的方法都是抽象的。例如，下面的两段代码效果是等同的。

```
public interface ITest {
    public abstract void hi();
}
```

等价于：

```
public interface ITest {
    void hi();
}
```

由此，我们对接口中方法声明的语法进行总结：

- （1）接口中可以定义方法。
- （2）接口中的方法不需要实现。
- （3）接口中方法的签名 `abstract` 是可以省略的。

9.8.3 实现接口

讲完了接口的声明，接下来看一下如何实现接口。我们知道，抽象类使用 `extends` 关键字实现，而接口的实现需要使用 `implements` 关键字。例如，定义一个 `MyTest` 类实现 `ITest` 接口的示例代码具体如下：

```
294 public class MyTest implements ITest {
295     @Override
296     public void hi() {
297         System.out.println("哈哈");
298     }
299     public static void main(String[] args) {
```



```
300         MyTest mytest = new MyTest();  
301         mytest.hi();  
302     }  
303 }
```

上述代码中，第2行代码是一个注解，表示重写，该注解可写可不写。不过写上的话，编译器会为我们验证@Override下面的方法是否来自父类，如果没有，则会提示报错。例如，我们删除@Override，如果下面的方法名写错了，此时，编译器可以编译通过的，因为编译器会以为这个方法是子类增加的方法，而不是来自父类的方法。

9.8.4 检查对象是否实现接口规范

可能有的同学会对抽象类和接口产生疑问，既然接口是一种特殊的抽象类，那我们只需要掌握抽象类就可以了，为什么还要学习接口？其实接口是一个更高层级的规范，我们通过一个实际案例进行解释。

假设我们现在想定义一个接口IDrive，该接口里面声明了一个方法drive()表示如何开车。由于车的类型有很多种，例如公交车，货车、三轮车等，所以每种开车的规范是不一样的。对于定义开车规范的人来说，需要根据开车的类型发放对应的驾照。

下面我们在IDEA中新建一个接口IDrive，并在接口中声明drive()方法，示例代码如下文件9-36所示。

文件 9-36 IDrive.java

```
public interface IDrive {  
    void drive();  
}
```

PersonA 实现了接口IDrive，具体代码如下文件9-37所示。

文件 9-37 PersonA.java

```
public class PersonA implements IDrive {  
    private String name;  
    public PersonA(String name) {  
        this.name = name;  
    }  
    @Override  
    public void drive() {  
        System.out.println(name + "在大马路上开车...");  
    }  
}
```

PersonB 没有实现IDrive接口，但是却定义了方法drive，具体代码如下文件9-38所示。

文件 9-38 PersonB.java

```
public class PersonB {
```



```
private String name;
public PersonB(String name) {
    this.name = name;
}
public void drive() {
    System.out.println(name + "在大马路上开车...");
}
}
```

在测试类 Test 中分别创建 PersonA 和 PersonB 对象并调用 drive()方法，具体代码如文件 9-39 所示。

文件 9-39 Test.java

```
public class Test {
    public static void main(String[] args) {
        PersonA a = new PersonA("张三");
        a.drive();
        PersonB b = new PersonB("李四");
        b.drive();
    }
}
```

运行代码，结果如图 9-46 所示。

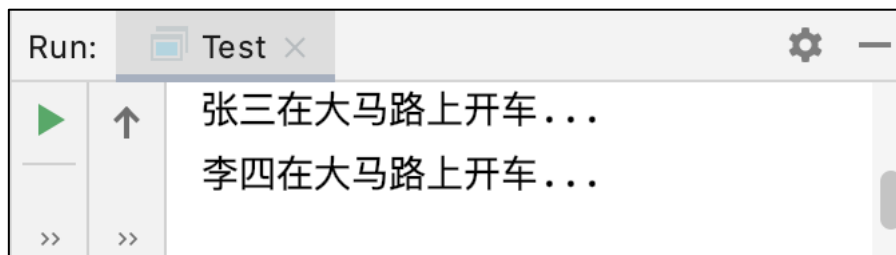


图 9-46 运行结果

从图 9-46 的运行结果可以看出，张三和李四都在大马路上开车，说明 PersonA 实现了接口 IDrive 和 PersonB 没有实现 IDrive 效果是一样的。也就是说，无论是否有驾照，都可以在马路上开车，这点在现实生活中显然是不可取的。按照交规来说，只有实现了 IDrive 接口的类，才会发放驾照，有资格在大马路上开车。

那如何判断某个类是否实现了接口呢？Java 提供了一个 instanceof 关键字，它可以检查某个类是否实现了接口，接下来，我们在测试类 Test 中添加对接口实现的检测代码，具体代码如文件 9-40 所示。

文件 9-40 Test.java

```
public class Test {
    public static void main(String[] args) {
        PersonA a = new PersonA("张三");
        a.drive();
        PersonB b = new PersonB("李四");
        b.drive();
        if (a instanceof IDrive) {
```



```
        System.out.println("a, 拥有驾照, 合法开车");  
    }  
    if (b instanceof IDrive) {  
    } else {  
        System.out.println("b, 没有驾照, 拘留半个月");  
    }  
}  
}
```

上述代码中，使用 instanceof 关键字分别检测了对象 a、b 是否实现了接口 IDrive，并根据检测结果做了不同的处理。此时运行程序，运行结果如图 9-47 所示。

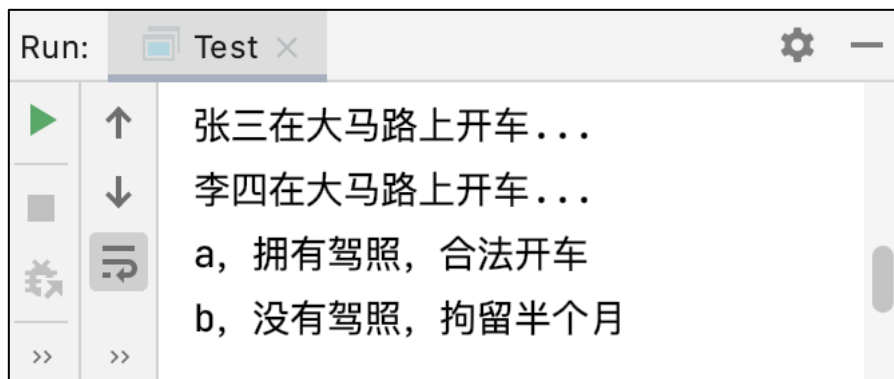


图 9-47 运行结果

9.8.5 接口可以做标记

上节课的案例，我们通过是否实现接口，进而判断驾驶行为是否合法，如图 9-48 所示。

<pre>xxx class implements IDrive { public void drive(){ System.out.println("drive"); } }</pre>	<pre>xxx class { public void drive(){ System.out.println("drive"); } }</pre>
---	---

图 9-48 合法开车与无证驾驶的鉴定

基于上述案例的规则，我们理解一下国际插座的接口规范。假设现在有一个表示国际插座的接口规范 ISocket，我们将实现了接口的类认为是国标插座，如果没有实现国际插座的接口规范，我们就认为是山寨插座，如图 9-49 所示。



xxx class implements ISocket {

xxx class {



}

}

图 9-49 国标插座与山寨插座

在图 9-49 中，我们可以把接口作为识别国标插座和山寨插座的标记。如果后续大家在阅读 JDK 文档时，发现某个接口中既没有声明成员变量，也没有声明方法，此时这个接口就可以看做是一个标记，例如，JDK 中的 `Serializable` 就是一个空的接口，它用来标记某个类是否可以被序列化，`Serializable` 接口的声明如图 9-50 所示。

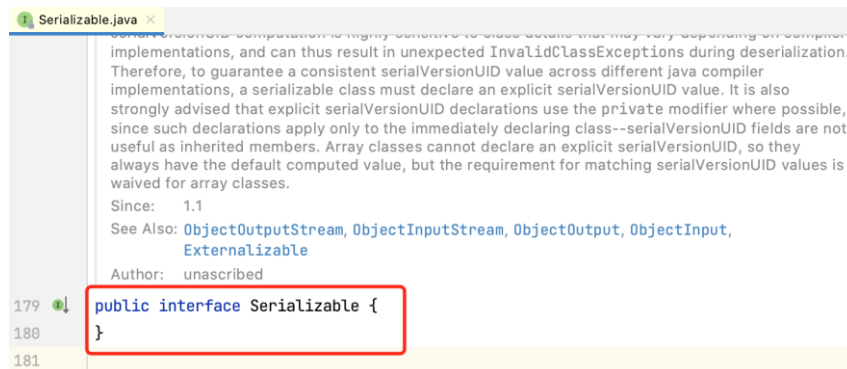


图 9-50 `Serializable` 接口的声明

值得一提的是，Java 中使用接口做标记是一种常见的做法，这就好比通过判断证件是否印有 VIP 进而判断人员是否可以享受 VIP 服务的一种标志，希望大家可以理解接口可以做标记的这个用途。

9.8.6 抽象类和接口的区别

抽象类和接口的区别：

- (1) 抽象类是一个物体的本质。
- (2) 接口是一个物体的能力。

为了大家更好理解抽象类和接口的区别，我们看一个现实生活中的例子，如图 9-51 所示。



图 9-51 充电宝

在图 9-51 中，不同类型手机的充电宝的充电接口是不一样的，不管是什么接口的充电宝，其本质都是充电宝，即使我们的充电宝接口坏了，其本质还是充电宝，只不过充电宝失去了充电的能力而已。

大家在面向对象编程时，先思考一下事物的本质，将本质定义为一个抽象类，将其拥有的能力定义为接口，一个事物可以同时拥有多个接口的。例如，上面充电宝的例子，可以按照下面思路进行设计：

（1）本质：充电宝，容量 cap，品牌 logo，返回当前电量方法 getRemainingPower，充电的方法 charge。

（2）能力：给 Android 手机充电的能力，给华为品牌手机充电的能力，给苹果品牌手机充电的能力，发光的能力。

9.8.7 应用案例：充电宝

上节课我们介绍了抽象类和接口，并且以充电宝的案例分析了面向对象编程，如何设计抽象类和接口，这节课，我们实现一下充电宝的案例，具备步骤如下：

（1）定义抽象类 PowerBank，用来描述充电宝的本质，具体代码如文件 9-41 所示。

文件 9-41 PowerBank.java

```
public abstract class PowerBank {  
    private int cap;  
    private String logo;  
    public abstract void getRemainingPower();  
    // 初始化方法  
    public void init(int cap, String logo) {  
        this.cap = cap;  
        this.logo = logo;  
    }  
}
```



```
@Override
public String toString() {
    return "PowerBank{" +
        "cap=" + cap +
        ", logo='" + logo + '\'' +
        '}';
}
}
```

(2) 定义接口 IAndroid，用来描述 Android 手机的充电接口，具体代码如文件 9-42 所示。

文件 9-42 IAndroid.java

```
public interface IAndroid {
    public void chargeAndroid();
}
```

(3) 定义接口 IHuawei，用来描述华为手机的充电接口，具体代码如文件 9-43 所示。

文件 9-43 IHuawei.java

```
public interface IHuawei {
    public void chargeHuawei();
}
```

(4) 定义接口 IApple，用来描述苹果手机的充电接口，具体代码如文件 9-44 所示。

文件 9-44 IApple.java

```
public interface IApple {
    public void chargeApple();
}
```

(5) 定义继承了 PowerBank 并且实现了 IApple 接口的类 JieDianPowerBank，用来描述苹果接口的充电宝，具体代码如文件 9-45 所示。

文件 9-45 JieDianPowerBank.java

```
public class JieDianPowerBank extends PowerBank implements IApple {
    public JieDianPowerBank() {
        init(10000, "街电");
    }
    @Override
    public void chargeApple() {
        System.out.println("给苹果手机充电");
    }
    @Override
    public void getRemainingPower() {
        System.out.println("用 led 灯显示剩余电量");
    }
}
```

(6) 定义继承了 PowerBank 且同时实现了 IApple、IAndroid、IHuawei 三种接口的类 GuaiShouPowerBank，用来描述可以同时提供三种接口的充电宝，具体代码如文件 9-46 所示。



文件 9-46 GuaiShouPowerBank.java

```
public class GuaiShouPowerBank extends PowerBank implements
    IApple, IAndroid, IHuawei {

    public GuaiShouPowerBank() {
        init(20000, "怪兽");
    }

    @Override
    public void chargeAndroid() {
        System.out.println("为安卓充电");
    }

    @Override
    public void chargeApple() {
        System.out.println("为苹果充电");
    }

    @Override
    public void chargeHuawei() {
        System.out.println("为华为充电");
    }

    @Override
    public void getRemainingPower() {
        System.out.println("用液晶屏显示剩余电量");
    }
}
```

(7) 定义测试类 Test，在该类中创建 GuaiShouPowerBank 和 JieDianPowerBank 对象，并调用其定义的方法，具体代码如文件 9-47 所示。

文件 9-47 Test.java

```
public class Test {

    public static void main(String[] args) {
        GuaiShouPowerBank gb = new GuaiShouPowerBank();
        gb.getRemainingPower();
        gb.chargeAndroid();
        System.out.println(gb);
        System.out.println("-----");
        JieDianPowerBank jb = new JieDianPowerBank();
        jb.getRemainingPower();
        jb.chargeApple();
        System.out.println(jb);
        if (jb instanceof IHuawei) {
            System.out.println("可以为华为充电");
        } else {
            System.out.println("不可以为给华为充电，接口不同");
        }
    }
}
```



}

程序的运行结果如图 9-52 所示。

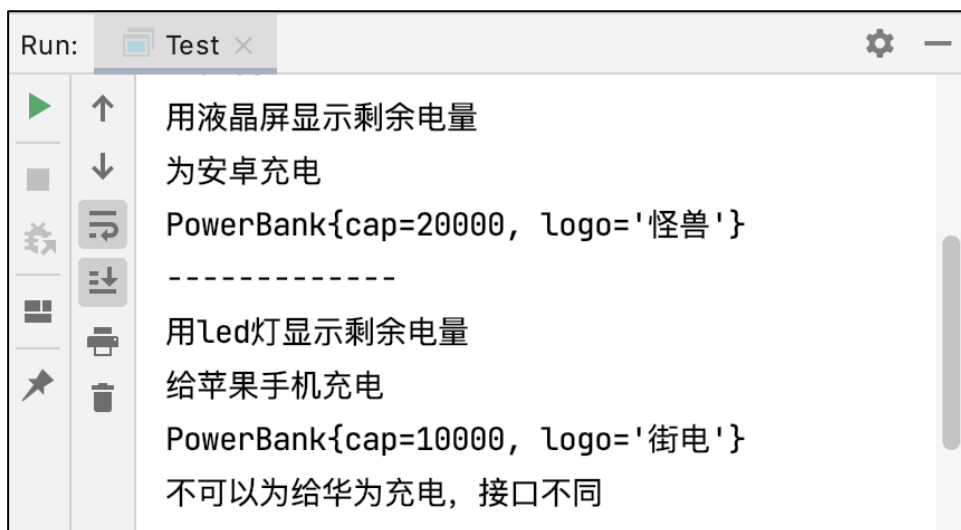


图 9-52 运行结果

9.11 通过接口实现多继承

通过前面的学习我们知道，一个类只能继承一个抽象类，但是却可以同时实现多个接口。Java 中，一个类只能扩展一个单独的类，换句话说，一个子类只能有一个父类，如果一个子类有多个父类，就会产生多重继承，多重继承会产生歧义。

有一些编程语言是支持多继承的，例如 C++，但是这样会带来很多问题，下面我们看一个例子，如图 9-53 所示。

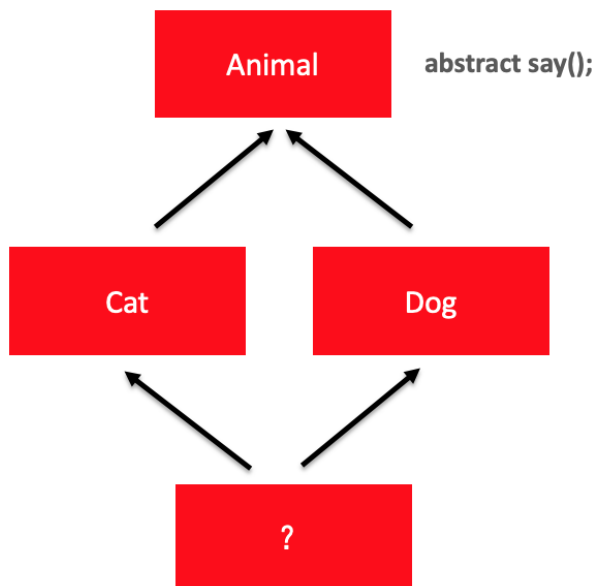


图 9-53 多重继承的问题

在图 9-53 中，Animal 类声明了一个表示动物叫声的方法 say()，Cat 类和 Dog 类都继承了 Animal 类，都拥有了 say() 方法，假设 Cat 类的 say() 方法输出的动物叫声是喵喵，Dog



类的 say() 方法输出的动物叫声是汪汪，那么如果有个类同时继承了 Cat 和 Dog 类，那么叫声既是喵喵，又是汪汪，这种情况出现了多重继承的冲突，显然是有问题的。

针对上述问题，Java 的解决方案是不允许多重继承。在 Java 中，可以通过接口解决多重继承的问题。接口是定义类的一种能力，定义接口的唯一目的就是能够被其他类实现，设计接口的人要站在更高层次考虑问题，在接口中只需要定义事物能力的规范是什么，不需要定义如何做。换句话说，接口只定义了类中的方法，不需要实现这些方法，Java 引入接口就是为了解决多重继承的问题。

一个类可以实现多个接口，这样我们的代码可以变的更加灵活，另外，由于接口定义的只是规范，并不会有的具体的实现过程，所以接口可以很好地消除多继承的问题。为了大家更好地理解，我们看一张图，如图 9-54 所示。

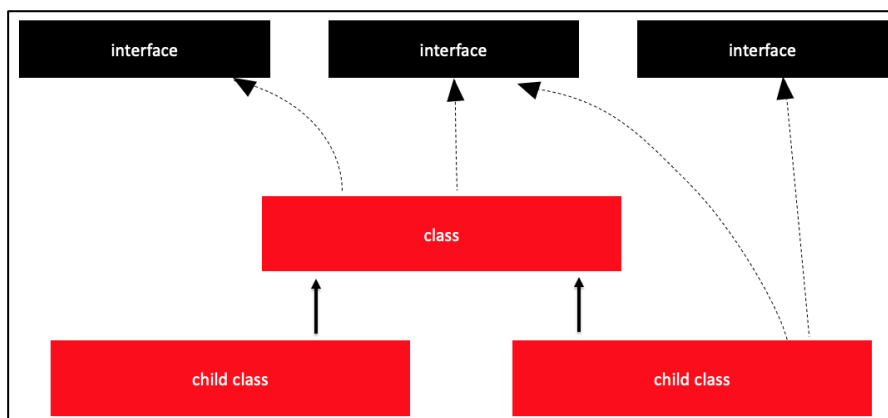


图 9-54 一个类实现多个接口

在图 9-54 中，任何一个类都可以实现接口，并且可以同时实现多个接口。例如，父类实现了某个接口，子类也是可以实现同样接口的。这样的话，子类实现接口后重写的方法就属于子类独有，与父类实现接口重写的方法是完全没有关系的。

接下来，我们通过一个房车的例子，来演示如何使用接口实现多继承。

我们知道，房车，既有房子的功能，也有汽车的功能。如果我们想要得到一个房车型，如果先定义一个房子类，再定义一个汽车类，是无法通过多继承同时拥有房子和汽车功能的。所以最好的做法是设计两个接口，其中一个接口是 Liveable，具有居住的功能，另外一个接口是 Moveable，具有移动的功能，这两个接口的相关介绍如下：

- Moveable 接口，声明了 void move(int distance) 方法表示房车具有移动的能力，其中参数 distance 表示移动的距离。
- Liveable 接口，声明了 boolean canLive(int person) 表示房车具有住的能力，其中参数 person 表示人数。

我们的房车只要同时实现了 Moveable 和 Liveable 接口，就可以具备房子和汽车的功能啦，下面我们分步骤进行实现，具体如下：

(1) 定义一个接口 IMoveable，具体代码如文件 9-48 所示。

文件 9-48 IMoveable.java

```
public interface IMoveable {  
    void move(int distance);  
}
```

(2) 定义一个接口 ILiveable，具体代码如文件 9-49 所示。

文件 9-49 ILiveable.java

```
public interface ILiveable {
```



```
boolean canLive(int person);  
}
```

(3) 定义一个房车类 **CarHouse**，同时实现了 **IMoveable** 和 **ILiveable** 接口，具体代码如文件 9-50 所示。

文件 9-50 CarHouse.java

```
public class CarHouse implements ILiveable, IMoveable {  
    @Override  
    public boolean canLive(int person) {  
        if (person <= 4) {  
            return true;  
        } else {  
            return false;  
        }  
    }  
    @Override  
    public void move(int distance) {  
        System.out.println("移动了" + distance + "km");  
    }  
}
```

(4) 定义测试类 **Test**，在该类中创建 **CarHouse** 对象并调用相关方法，具体代码如文件 9-51 所示。

文件 9-51 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        CarHouse ch = new CarHouse();  
        ch.move(300);  
        System.out.println(ch.canLive(2));  
    }  
}
```

程序的运行结果如图 9-55 所示。



图 9-55 运行结果



9.12 抽象类和接口的选取规则

9.10.1 接口和抽象类的设计

接下来，我们通过一个 PPAP 的案例来学习如何在程序中设计接口和抽象类，PPAP 示意图如图 9-56 所示。



图 9-56 PPAP 示意图

在图 9-56 中，笔和水果是可以进行任意组合的，组合示意图如图 9-57 所示。

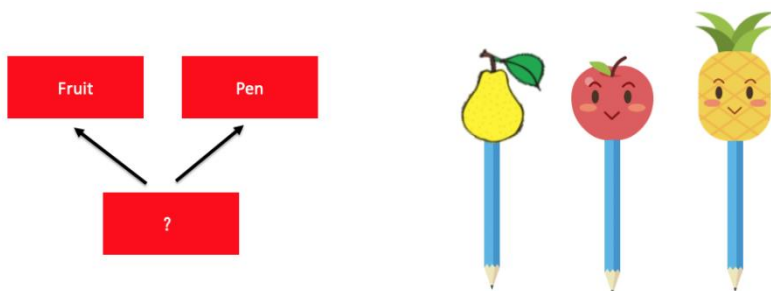


图 9-57 组合示意图

这个 PPAP 案例反映的就是现实生活中的物体组合在一起的思想，例如 Apple 和 Pen 组合在一起是 ApplePen, Pineapple 和 Pen 组合在一起就是 PineapplePen。

我们在代码开发时，如果遇到物体组合的情况，就需要考虑到底应该使用抽象类还是使用接口呢？

假设目前的业务场景是使用相同的笔插很多不同的水果，那么无论任何水果插在笔上，其本质还是水果，所以我们可以定义一个抽象类 Fruit，通过继承 Fruit 得到不同的水果。除此之外，水果与笔组合后，还具备一个写字的能力，所以我们可以定义一个接口 Writeable，其设计思路如图 9-58 所示。

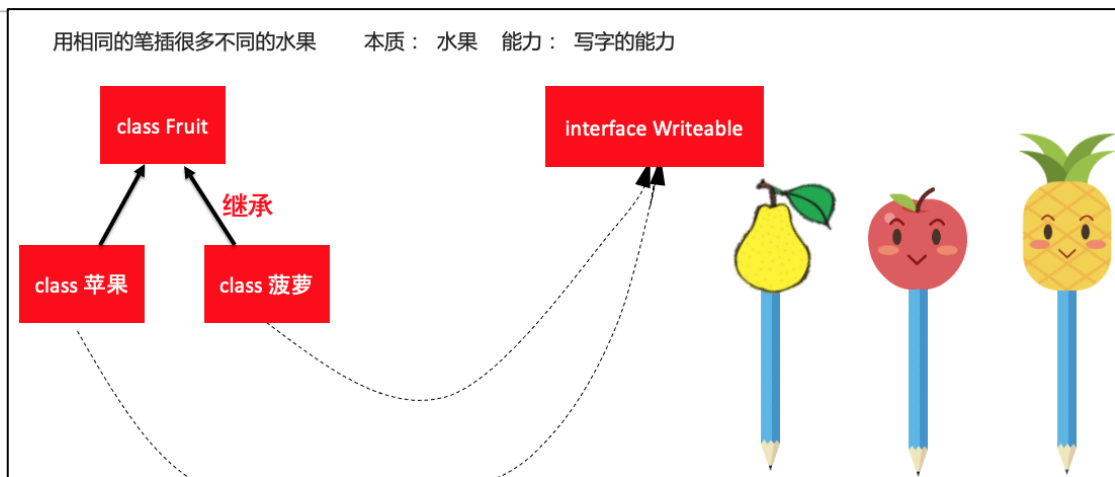


图 9-58 设计思路（1）

假设上述需求发生了变化，要用很多种不同的笔插水果。由于笔的种类比较多，我们可以设计一个抽象类 Pen，通过继承 Pen 得到不同种类的笔。而水果可以被吃，我们可以定义一个接口 eatable 接口，其设计思路如图 9-59 所示。

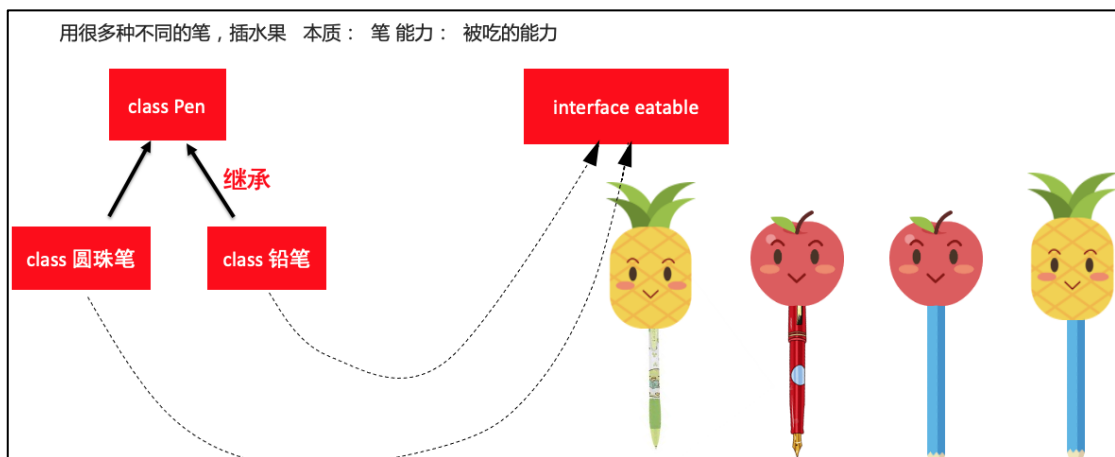


图 9-59 设计思路（2）

Java 语言用来描述现实生活中的事物时，大家要学会根据业务场景设计不同的代码。面向对象编程，不是一门技术，而是一种编程思想，使用这种编程思想进行开发时，要充分考虑代码的阅读性、扩展性以及后期的维护。

9.10.2 PPAP 代码的实现

为了帮助大家理解前面讲解的内容，我们这里使用代码的方式实现图 9-60 的业务场景。

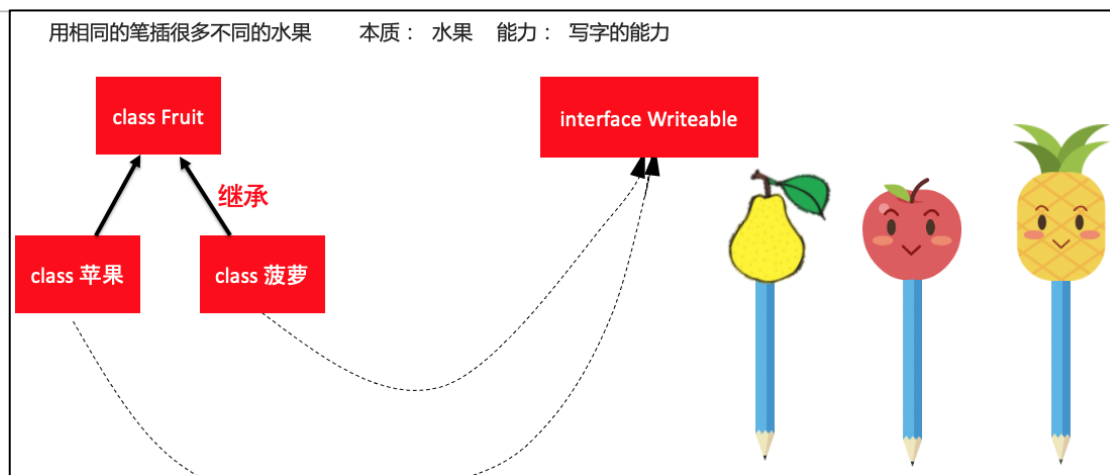


图 9-60 业务场景

接下来，我们分步骤进行实现，具体如下：

(1) 定义一个接口 `IWriteable`，具体代码如文件 9-52 所示。

文件 9-52 `IWriteable.java`

```
public interface IWriteable {
    void write();
}
```

(2) 定义一个抽象类 `Fruit`，具体代码如文件 9-53 所示。

文件 9-53 `Fruit.java`

```
public abstract class Fruit {
    public abstract void qiwei();
    public abstract void weidao();
}
```

(3) 定义一个类 `Apple`，该类继承了 `Fruit` 并实现了 `IWriteable` 接口，具体代码如文件 9-54 所示。

文件 9-54 `Apple.java`

```
public class Apple extends Fruit implements IWriteable {
    @Override
    public void qiwei() {
        System.out.println("气味:香甜");
    }
    @Override
    public void weidao() {
        System.out.println("吃起来很脆");
    }
    @Override
    public void write() {
        System.out.println("可以写字");
    }
}
```



(4) 定义一个类 PineApple，该类继承 Fruit 并实现 IWriteable 接口，具体代码如下文件 9-55 所示。

文件 9-55 PineApple.java

```
public class PineApple extends Fruit implements IWriteable {  
    @Override  
    public void qiwei() {  
        System.out.println("气味:酸甜");  
    }  
    @Override  
    public void weidao() {  
        System.out.println("吃起来 zhe 嘴");  
    }  
    @Override  
    public void write() {  
        System.out.println("可以写字");  
    }  
}
```

(5) 定义测试类 Test，该类中创建 Apple 和 PineApple 对象并调用对应的方法，具体代码如下文件 9-56 所示。

文件 9-56 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        Apple a = new Apple();  
        PineApple p = new PineApple();  
        a.qiwei();  
        a.weidao();  
        a.write();  
  
        p.qiwei();  
        p.weidao();  
        p.write();  
    }  
}
```

程序的运行结果如图 9-61 所示。

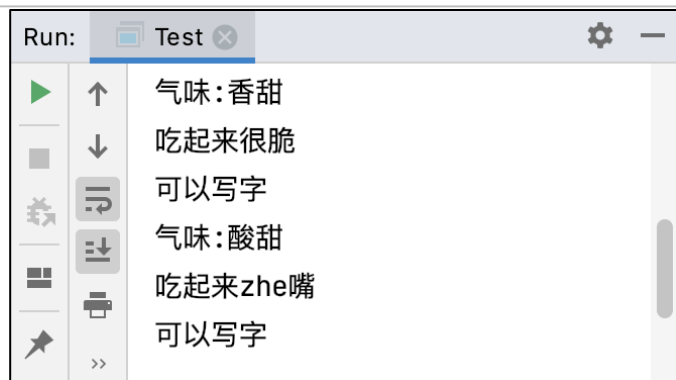


图 9-61 运行结果

9.13 多态

多态（英文：polymorphism）它在生物学中是指一个物种的同一种群存在两种或者多种明显不同的表现。这就好比，我们在不同环境下的身份是不同的，例如，在家里是父亲的角色，在公司是老师的角色，在超市购物是顾客的角色。同理，人类这个群体，存在男人和女人这两种形态。

Java 中的多态，指的是同一个方法在不同子类中表现的不同形态，简单来说就是同种功能，不同的表现形态。例如，同样去厕所撒尿，男人和女人的表现形态是不一样的。

接下来，我们通过一个案例来演示多态，案例相关类的设计如图 9-62 所示。

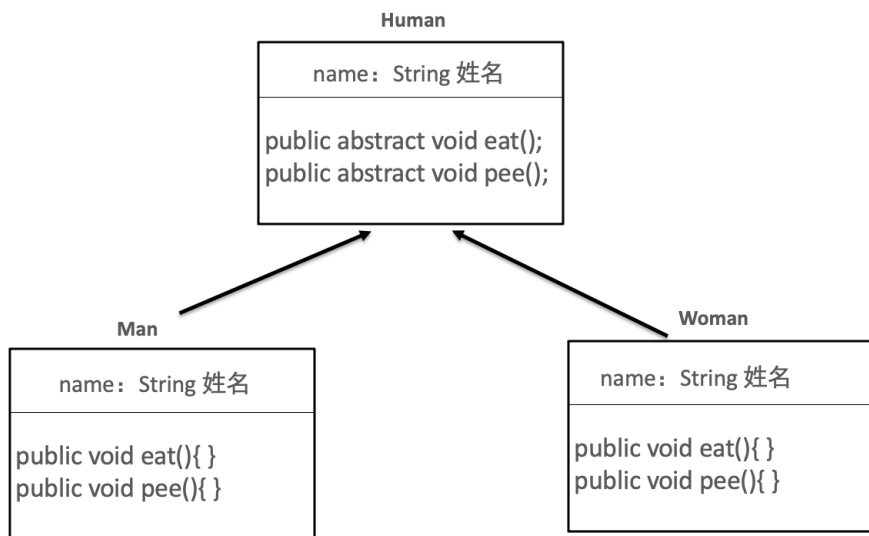


图 9-62 类的设计

案例实现步骤具体如下：

（1）定义抽象类 Human，该类声明了成员变量 name、抽象方法 eat() 和 pee()，具体代码如文件 9-57 所示。

文件 9-57 Human.java

```
public abstract class Human {
    String name;

    public abstract void eat();
    public abstract void pee();
}
```



```
}
```

(2) 定义一个类 Man，该类继承了 Human 类，并重写了 eat() 和 pee() 方法，具体代码如文件 9-58 所示。

文件 9-58 Man.java

```
public class Man extends Human {  
    @Override  
    public void eat() {  
        System.out.println(name + "狼吞虎咽大口吃");  
    }  
  
    @Override  
    public void pee() {  
        System.out.println(name + "站着尿");  
    }  
}
```

(3) 定义一个类 Woman，该类继承了 Human 类，并重写了 eat() 和 pee() 方法，具体代码如文件 9-59 所示。

文件 9-59 Woman.java

```
public class Woman extends Human {  
    @Override  
    public void eat() {  
        System.out.println(name + "小口细嚼慢咽");  
    }  
  
    @Override  
    public void pee() {  
        System.out.println(name + "蹲着尿");  
    }  
}
```

(4) 定义测试类 Test，在该类中创建名称为希拉里、武则天的 Woman 对象以及名称金正日和特朗普的两个 Man 对象，然后调用这些对象的方法，具体代码如文件 9-60 所示。

文件 9-60 Test.java

```
304 public class Test {  
305     public static void main(String[] args) {  
306         Woman w1 = new Woman();  
307         w1.name = "希拉里";  
308         Woman w2 = new Woman();  
309         w2.name = "武则天";  
310         Man m1 = new Man();  
311         m1.name = "金正日";  
312         Man m2 = new Man();  
313         m2.name = "特朗普";  
314         Human[] houses = {w1, w2, m1, m2};  
315         for (Human h : houses) {
```



```
316         h.eat();  
317         h.pee();  
318     }  
319 }  
320 }
```

上述代码中，由于 w1、w2、m1、m2 都属于 Human 类，所以在第 11 行代码定义了一个 Human 类型的数组用于存放创建好的 4 个对象。第 12~15 行代码通过遍历数组的方式分别输出了 4 个对象调用 eat() 和 pee() 方法的结果。

程序的运行结果如图 9-63 所示。

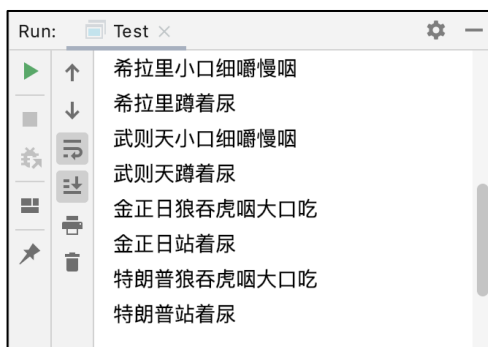


图 9-63 运行结果

从运行结果可以发现，虽然我们没有显式调用 Woman 或者 Man 类中的 eat() 和 pee() 方法，但是程序自动调用子类的 eat() 和 pee() 方法，从而显示出不同的输出结果。这就是 Java 多态的体现，也就是同一个方法，在不同子类里面表现出不同的形态。

第 10 章集合和泛型

学习目标

- ◆ 了解集合的概念，能够说出集合的特点
- ◆ 掌握 List 常见 API 的使用，能够使用 ArrayList 和 LinkedList

对元素进行增删改查

- ◆ 掌握对集合的遍历，能够使用 for 循环和增强 for 循环对集合进行遍历

- ◆ 掌握学生信息管理系统开发，能够使用 List 集合开发学生信息管理系统



- ◆ 了解栈和队列，能够说出栈和队列的特点, 以及常用 API
- ◆ 掌握泛型的使用，能够在集合中正确使用泛型
- ◆ 掌握 Map 的使用，能够使用 HashMap 对元素进行添加和获取

数组可以存储多个对象，但是数组只能存储相同类型的对象，如果要存储一批不同类型的对象，数组便无法满足需求了。为此，Java 提供了集合，集合可以存储不同类型的多个对象。本章将针对 Java 中的集合类进行详细地讲解。

10.1 集合概述

为了存储不同类型的多个对象，Java 提供了一系列特殊的类，这些类可以存储任意类型的对象，并且存储的长度可变，被统称为集合。集合可以简单理解为一个长度可变，可以存储不同数据类型的动态数组。集合都位于 `java.util` 包中，使用集合时必须导入 `java.util` 包。

集合按照数据的存储方式，可以分为四类，分别是列表、栈、键值对、队列，具体如图 10-1 所示。

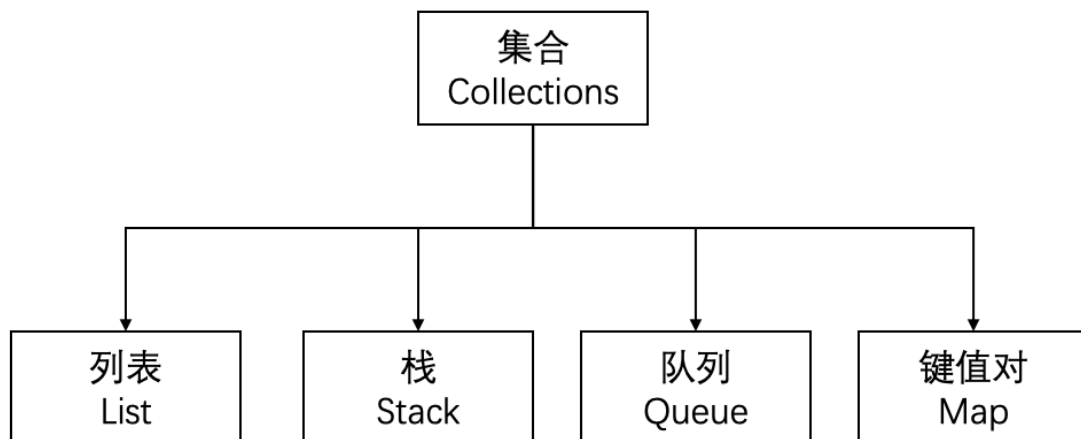


图10-1 集合的分类

与数组相比，集合不仅可以很方便的存储数据，而且增删数据非常便捷，这点是数组所不具备的。

10.2 List 接口

List 是一个接口，List 有一个特点就是元素有序，即元素的存入顺序和取出顺序一致，并且可以通过索引来访问集合中的指定元素。



10.2.1 ArrayList 集合

ArrayList 是 List 接口的一个实现类，它是程序中最常见的一种集合。ArrayList 集合内部封装了一个长度可变的数组对象，当存入的元素超过数组长度时，ArrayList 会在内存中分配一个更大的数组来存储这些元素，因此可以将 ArrayList 集合看作一个长度可变的数组。ArrayList 插入元素的过程如图 10-2 所示。

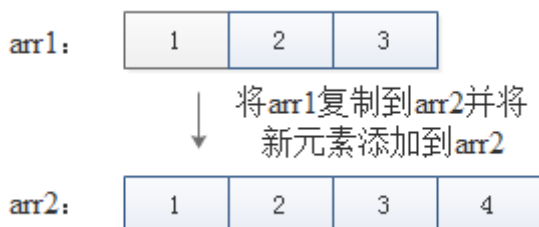


图10-2 ArrayList 插入元素的过程

我们先看一段代码，具体如下：

```
ArrayList names = new ArrayList();  
names.add("张三");  
names.add("李四");  
names.add("王五");  
names.remove("李四");
```

上述代码中，第1行代码创建了一个 ArrayList 集合 list，第2~4行代码通过调用 add() 方法向 list 集合添加元素，第5行代码通过调用 remove() 方法移除指定的数据。

为了大家更好地理解 ArrayList 的使用，接下来，我们通过一个案例来演示，具体代码如文件 10-1 所示。

文件 10-1 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        ArrayList list=new ArrayList();  
        list.add("haha");  
        list.add("gaga");  
        list.add("guagua");  
        list.add("xixi");  
        list.add("gugu");  
        list.add("miaomiao");  
        System.out.println(list.get(2));  
        System.out.println("移除前的列表数据是: "+list);  
        list.remove(1);  
        System.out.println("移除后的列表数据是: "+list);  
    }  
}
```

程序的运行结果如图 10-3 所示。

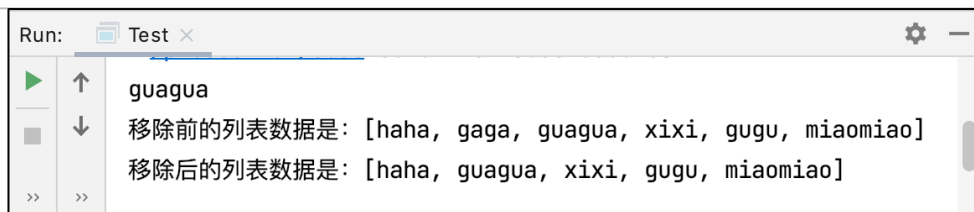


图10-3 运行结果

从图 10-3 可以得出, list 集合中角标为 2 的元素是 guagua, list 通过 remove() 方法将角标为 1 的元素在 list 集合中移除, 原角标为 1 之后的所有元素自动向前补位。

10.2.2 LinkedList 集合

除了 ArrayList, 实现了 List 接口的类还有很多, 比如 LinkedList。LinkedList 内部维护了一个双向循环链表, 链表中的每一个元素都使用引用的方式记录它的前一个元素和后一个元素, 从而可以将所有的元素彼此连接起来。当插入一个新元素时, 只需要修改元素之间的引用关系即可, LinkedList 添加元素的过程如图 10-4 所示。

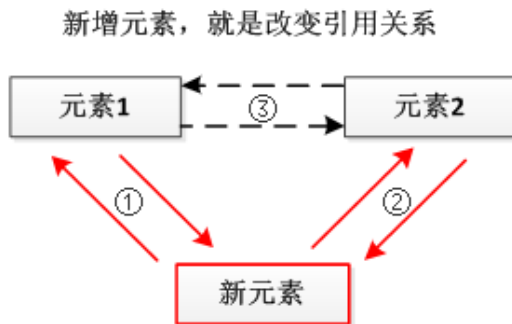


图10-4 LinkedList 添加元素的过程

ArrayList 与 LinkedList 相比, ArrayList 海量数据的效率更快, 但是添加和删除元素的效率却不及 LinkedList。

10.3 List 常见 API+循环遍历集合元素

通过上节课的学习我们知道, List 是一个接口, 基于 List 接口有很多不同的实现类, 常见的实现类是 ArrayList、LinkedList。

下面我们看一下 List 接口中常见方法, 具体如下。

- add(E element): 添加一个元素
- add(int index, E element): 在指定位置添加一个元素
- get(int index): 返回某个位置的元素
- contains(Object o): 判断 list 里面是否包含某个元素
- remove(int index): 删除某个位置的元素
- size(): 查看 list 集合里面元素的个数
- clear(): 清空集合的元素

如果我们希望遍历集合中的每个元素, 既可以使用 for 循环实现, 也可以使用增强 for 循环实现, 示例代码如下:



for 循环遍历集合示例：

```
for (int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

增强 for 循环集合示例：

```
for (Object s:list){  
    System.out.println(s);  
}
```

10.4 学生信息管理系统开发

下面我们在 IDEA 中模拟开发一个学生信息管理系统，具体步骤如下：

(1) 定义一个类 `SystemEngine`，该类是模板系统类，定义了创建各种不同系统的通用方法，包含获取用户的键盘输入、更新逻辑、更新显示，具体代码如文件 10-2 所示。

文件 10-2 `SystemEngine.java`

```
public abstract class SystemEngine {  
    public String getCurrentInput(){  
        Scanner sc = new Scanner(System.in);  
        String str = sc.next();  
        return str;  
    }  
  
    public abstract void updateLogic();  
    public abstract void renderUI();  
}
```

(2) 定义一个类 `StudentSystem`，该类用于定义学生管理系统，类中包含系统的初始化和系统退出，具体代码如文件 10-3 所示。

文件 10-3 `StudentSystem.java`

```
public class StudentSystem {  
    private static boolean isRuning = true;  
    public static void init(SystemEngine engine){  
        isRuning = true;  
        while(isRuning){  
            engine.updateLogic();  
            engine.renderUI();  
        }  
    }  
  
    public static void gameOver(){  
        java.lang.System.out.println("系统退出了");  
        isRuning = false;  
    }  
}
```



(3) 定义一个类 MySystemEngine 继承自 SystemEngine，并实现具体的业务逻辑，具体代码如文件 10-4 所示。

文件 10-4 MySystemEngine.java

```
import java.util.ArrayList;

public class MySystemEngine extends SystemEngine{
    ArrayList list = new ArrayList();
    @Override
    public void updateLogic() {
        System.out.println("=====欢迎使用学生信息管理系统=====");
        System.out.println("查询所有学生，请输入 1");
        System.out.println("删除所有学生，请输入 2");
        System.out.println("添加一个学生，请输入 3");
        String input = getCurrentInput();
        switch (input){
            case "1":
                queryStudent();
                break;
            case "2":
                deleteAllStudent();
                break;
            case "3":
                addStudent();
                break;
            default:
                System.out.println("非法输入，请重新输入");
        }
    }
    private void queryStudent() {
        if(list.size()<=0){
            System.out.println("学生信息为空，没有任何学生信息在系统中");
        }
        else {
            System.out.println("全部的学生为：");
            System.out.println(list);
        }
    }
    private void deleteAllStudent() {
        list.clear();
    }
    private void addStudent() {
        System.out.println("请输入学生姓名");
        String name = getCurrentInput();
        list.add(name);
    }
}
```



```
    }  
  
    @Override  
    public void renderUI() {  
  
    }  
  
}
```

(4) 在测试类 Test 中调用学生系统初始化方法，具体代码如文件 10-5 所示。

文件 10-5 Test.java

```
public class Test {  
    public static void main(String[] args) {  
        StudentSystem.init(new MySystemEngine());  
    }  
  
}
```

运行程序，按照提示输入数字执行不同的操作，程序的部分运行结果如图 10-5 所示。

```
Run: Test x
"C:\Program Files\Java\jdk-11.0.11\bin\java.exe" "
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
3
请输入学生姓名
张三
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
1
全部的学生为：
[张三]
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
3
请输入学生姓名
李四
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
1
全部的学生为：
[张三， 李四]
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
2
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
1
学生信息为空，没有任何学生信息在系统中
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
```

图10-5 运行结果（1）



上述实现的学生信息管理系统是比较精简的，其中学生的信息只有学号。如果我们需要存入更完善的学生信息到学生信息管理系统中，需要对该系统进行升级，升级的步骤如下。

(1) 创建学生类。创建一个名称为 Student 的类用于存放学生的信息，具体代码如文件 10-6 所示。

文件 10-6 Student.java

```
public class Student {  
    private String name;  
    private String id;  
    private String address;  
    public Student(String name, String id, String address) {  
        this.name = name;  
        this.id = id;  
        this.address = address;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getId() {  
        return id;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
    public String getAddress() {  
        return address;  
    }  
    public void setAddress(String address) {  
        this.address = address;  
    }  
    @Override  
    public String toString() {  
        return "姓名=" + name + "\t" + "学号=" + id + "\t" + "籍贯=" + address ;  
    }  
}
```

(2) 优化添加学生信息的方法。之前添加学生信息时，只需获取学生姓名并添加即可，优化后添加到信息需要包含学生姓名、学号、籍贯，优化后的 addStudent() 方法代码如下所示。

```
private void addStudent() {  
    System.out.println("请输入学生姓名");  
    String name = getCurrentInput();
```



```
System.out.println("请输入学生学号");  
String id = getCurrentInput();  
System.out.println("请输入学生籍贯");  
String address = getCurrentInput();  
list.add(new Student(name,id,address));  
}
```

(3) 运行程序，按照提示输入数字执行不同的操作，程序的部分运行结果如图 10-6 所示。



```
Run: Test x
"C:\Program Files\Java\jdk-11.0.11\bin\java.exe" "-javaagent:D
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
3
请输入学生姓名
张三
请输入学生学号
111
请输入学生籍贯
北京
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
1
全部的学生为：
[姓名=张三 学号=111 籍贯=北京]
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
3
请输入学生姓名
李四
请输入学生学号
222
请输入学生籍贯
深圳
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
1
全部的学生为：
[姓名=张三 学号=111 籍贯=北京, 姓名=李四 学号=222 籍贯=深圳]
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
2
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
1
学生信息为空，没有任何学生信息在系统中
=====欢迎使用学生信息管理系统=====
查询所有学生，请输入1
删除所有学生，请输入2
添加一个学生，请输入3
```




图10-6 运行结果（2）

10.5 栈和队列

栈和队列是特殊的数据结构，其中，**栈的特点是后进先出**，**队列的特点是先进先出**，接下来，通过一张图来描述栈和队列的特点，如图 10-7 所示。

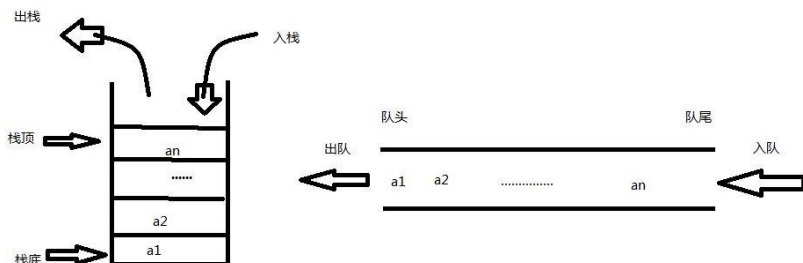


图10-7 栈和队列的特点

关于栈和队列的相关 API 介绍具体如下：

1. 栈—Stack

- `push(E item)`: 在栈顶添加一个元素，入栈。
- `pop()`: 移除栈顶一个元素，出栈。
- `peek()`: 读取栈顶的元素。
- `empty()`: 检查栈是否为空。
- `search(Object o)`: 检查栈中是否包含某个元素，返回元素的位置。

2. 队列—Queue

- `add(E element)`: 在队列中添加一个元素。
- `poll()`: 获取队首的元素，并把他从队列中移除。

接下来，我们通过代码方式演示栈和队列的相关 API，具体代码如下：

1. 栈——后进先出

```
import java.util.Stack;

public class TestStack {

    public static void main(String[] args) {

        Stack stack = new Stack();

        stack.push("zhangsan");

        stack.push("李四");

        stack.push("王五");

        String name = (String) stack.pop(); // 后进先出

        System.out.println(name);

        System.out.println(stack.size());

    }

}
```

上述代码中，首先创建了一个 `stack` 对象，然后调用 `push()` 方法添加了 3 个元素，接着调用 `pop()` 方法移除一个元素并输出移除的元素，最后输出了 `stack` 的大小，也就是元素的个数。

运行结果如图 10-8 所示。

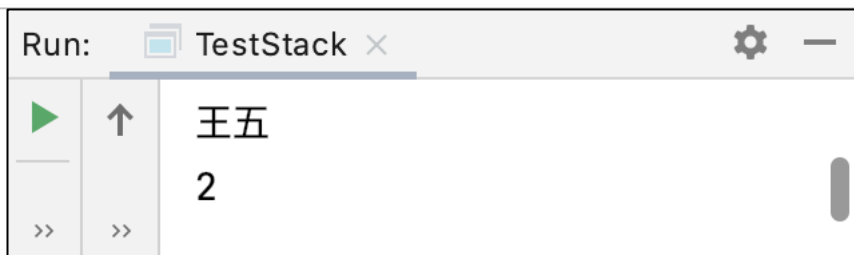


图10-8 运行结果

从图 10-8 可以看出，第 1 个被移除的元素是王五，说明在栈中后进入栈的元素先被移除，这正是栈后进先出特点的体现。

3. 队列——先进先出

Java 中 Queue 是个接口规范，LinkedList 间接实现了 Queue，在此可以有通过 LinkedList 演示队列的 API，以及队列的特点。

```
import java.util.LinkedList;

public class TestQueue {

    public static void main(String[] args) {

        LinkedList queue = new LinkedList();

        queue.add("张三");
        queue.add("李四");
        queue.add("王五");
        queue.add("赵六");
        queue.add("钱七");

        System.out.println(queue.poll());
        System.out.println(queue.poll());
        System.out.println(queue.poll());

    }

}
```

由于上述代码中，首先创建了一个 LinkedList 对象，然后调用 add() 方法添加了 4 个元素到队列中，最后输出了前 3 个被移除的元素。

程序的运行结果如图 10-9 所示。

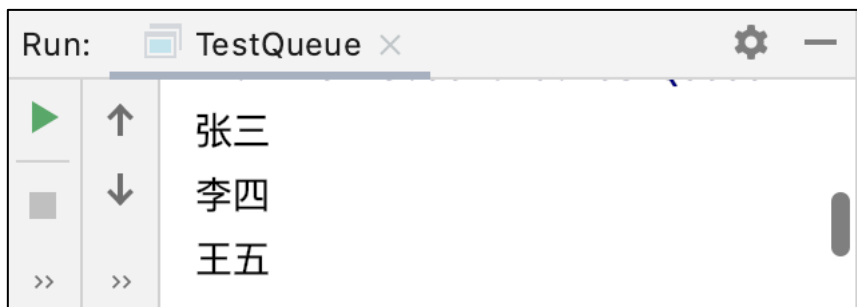


图10-9 运行结果

从图 10-9 可以看出，先被移除的三个元素正是最先添加到队列中的元素，这正好队列先进先出特点的体现。



10.6 泛型

泛型可以看做是 Java 编译器提供的一种能力，让一种容器中只存放一种数据类型，这种能力能让开发人员编写的代码更加健壮。下面，我们通过一段代码来理解，具体代码如文件 10-7 所示。

文件 10-7 FanXingDemo.java

```
321 import java.util.ArrayList;
322 public class FanXingDemo {
323     public static void main(String[] args) {
324         ArrayList list = new ArrayList();
325         list.add("abad");
326         list.add("hellowo");
327         list.add(new Student("zhangsan", 111));
328         for (Object obj : list) {
329             String str = (String) obj;
330             System.out.println(str);
331         }
332     }
333 }
```

上述代码创建了一个 ArrayList 集合，并向该集合添加了三个元素，但是这三个元素的数据类型不完全相同，此时如果遍历集合时，对遍历出的元素进行某种类型的强制转换，程序会报错，具体如图 10-10 所示。

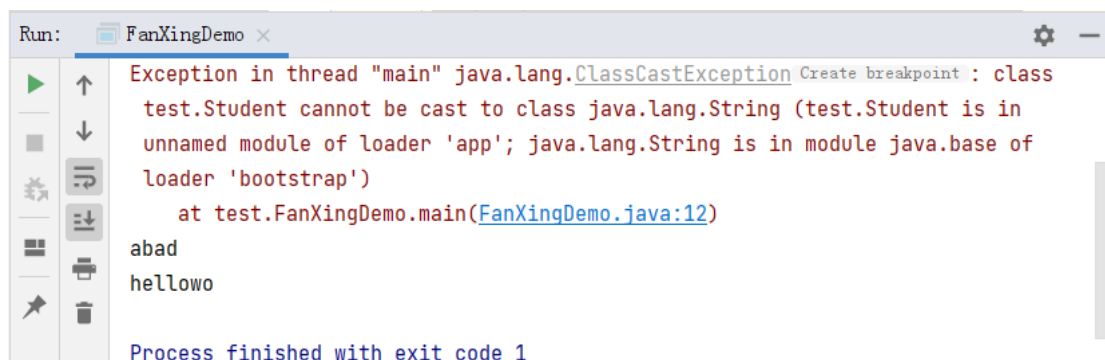


图10-10 运行结果报错

虽然集合很强大，可以存储任意类型的数据。但是使用 ArrayList 存储数据时，应该与数组类似，存储相同类型的数据。上述代码编译时程序并没有提示错误，在运行时才出错，提示无法将对象类型转为字符串。

我们在定义 ArrayList 对象时，可以使用泛型的方式指定元素的类型，例如下面的代码。

```
ArrayList<Student> list = new ArrayList();
```

上述代码中，使用泛型指定存入 ArrayList 集合的元素的类型是 Student，如果添加到集合中的元素的类型不是 Student，那么代码会在编译时报错。例如，如果使上述代码替换文件 10-2 中的第 4 行代码，则代码会出现编译时异常。

接下来，我们对文件 10-7 进行修改，修改后的代码如下所示：

```
import java.util.ArrayList;
```



```
public class FanXingDemo {  
    public static void main(String[] args) {  
        ArrayList<Student> list = new ArrayList();  
        list.add(new Student("李四", 222));  
        list.add(new Student("赵六", 444));  
        list.add(new Student("zhangsan", 111));  
        for (Object obj : list) {  
            Student stu = (Student) obj;  
            System.out.println(stu);  
        }  
    }  
}
```

程序运行结果如图 10-11 所示。

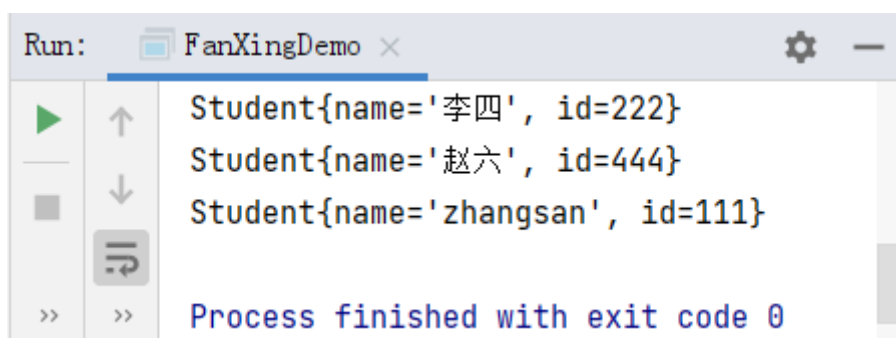


图10-11 运行结果

泛型可以指定数据类型，让代码变得更加严谨，例如下面的情况也是比较常见的。

```
ArrayList<String> list = new ArrayList();  
list.add("abc");  
list.add("hahah");  
list.add("111");
```

10.7Map

Map 是一种键值对数据类型，它可以快速方便查找集合元素，因为采用键值对方式存储的数据，底层数据查询的效率是非常高的。日常使用较多的 Map 是 HashMap，HashMap 内部使用哈希算法，可以根据要查找的键，自动去内存中获取对应的值。这就好比一本词典，可以根据词典的目录快速找到对应的内容。

接下来，通过一段代码学习 HashMap 的使用，具体如下：

```
import java.util.HashMap;  
public class DemoHashMap {  
    public static void main(String[] args) {  
        HashMap<Integer, String> map = new HashMap<>();  
        map.put(1, "一");  
        map.put(2, "二");  
        map.put(3, "三");  
        map.put(4, "四");  
    }  
}
```



```
map.put(5, "五");  
map.put(6, "六");  
System.out.println(map);  
System.out.println(map.get(1));  
System.out.println(map.get(2));  
System.out.println(map.get(3));  
System.out.println(map.get(4));  
System.out.println(map.get(5));  
System.out.println(map.get(6));  
}  
}
```

上述代码中，首先创建了一个 HashMap 对象 map，该对象存储的数据，键是 Integer 类型，值是 String 类型，然后使用 put() 方法为 map 中添加了 6 个元素，最后将 map 中的元素打印输出到控制台。

程序的运行结果如图 10-12 所示。

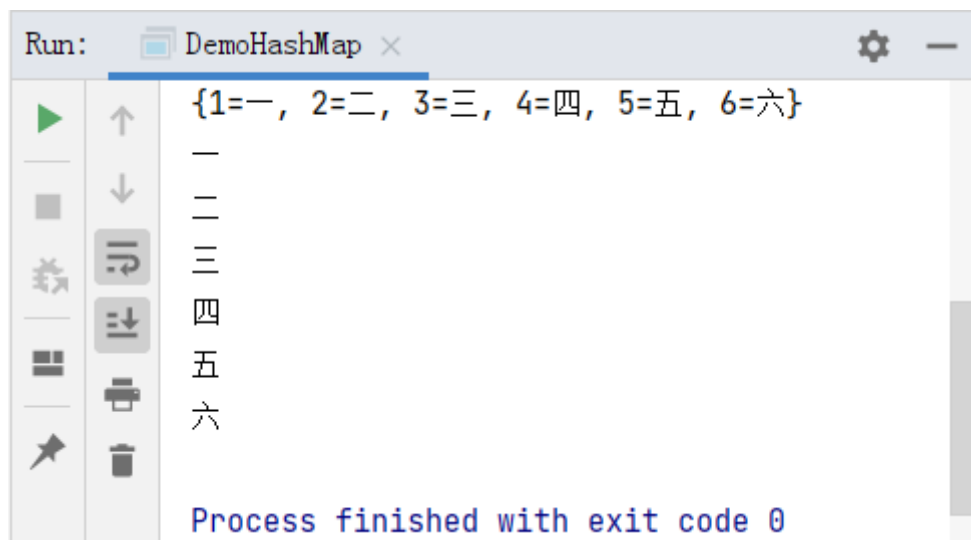


图10-12 运行结果

从图 10-12 可以得出，HashMap 中根据键可以快速获取到键对应的值。

第 11 章 GUI 编程

学习目标

- ◆ 熟悉常见图形的绘制
- ◆ 熟悉使用面向对象思想设计直线
- ◆ 熟悉动画的绘制



◆ 掌握键盘事件的应用

GUI 全称是 Graphical User Interface，即图形用户界面。顾名思义，GUI 就是可以让用户直接操作的图形化界面，包括窗口、菜单、按钮、工具栏和其他各种图形界面元素。目前，图形用户界面已经成为一种趋势，几乎所有的程序设计语言都提供了 GUI 设计功能。接下来，本章将针对 GUI 编程的相关知识进行讲解。

11.1 绘制常见图形

11.1.1 绘制对话框

对话框是人机对话提供的交互工具。在应用程序中，经常使用对话框向用户提供信息，或者从用户获得信息。接下来，我们分别演示如何绘制弹出对话框和文本输入对话框。

(1) 弹出对话框，示例代码如下：

```
import javax.swing.*;

public class Test {
    public static void main(String[] args) {
        JOptionPane.showMessageDialog(null, "哈哈");
    }
}
```

上述代码中，通过调用 `JOptionPane` 类的 `showMessageDialog()` 方法弹出了一个消息对话框，运行弹出对话框的示例代码效果如图 11-1 所示。

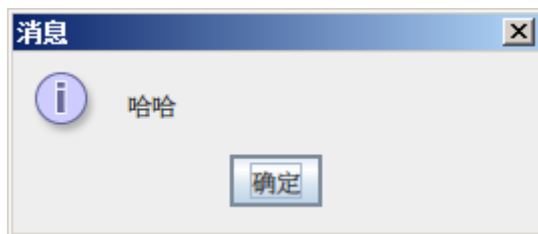


图11-1 对话框效果

(2) 文本输入对话框，示例代码如下：

```
public class Test02 {
    public static void main(String[] args) {
        String name=JOptionPane.showInputDialog(null, "请输入您的姓名");
        System.out.println(name);
        JOptionPane.showMessageDialog(null, "您的姓名为: "+name);
    }
}
```

上述代码中，通过调用 `JOptionPane` 类的 `showInputDialog()` 方法弹出了一个输入对话框，运行文本输入对话框的示例代码效果如图 11-2 所示。

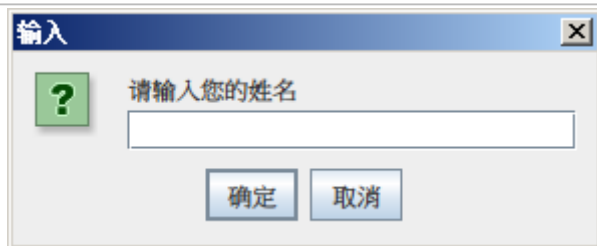


图11-2 输入对话框效果

在图 1-2 中，当用户输入姓名“张三”后，单击确定按钮，会弹出一个消息对话框提示用户的姓名，具体效果如图 11-3 所示。



图11-3 消息对话框

11.1.2 绘制划线

JPanel 是一种面板组件，它提供了绘图方法 `paint()` 方法，我们可以通过重写 `paint()` 方法绘制各种各样的图案，接下来，我们通过一个案例来演示如何绘制如图 11-4 所示的划线。

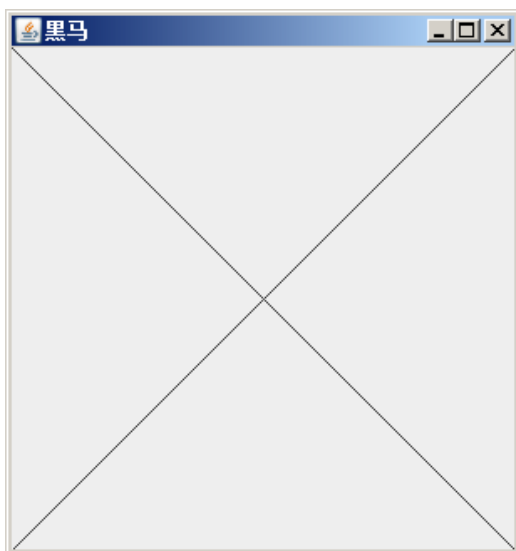


图11-4 效果图

图 11-4 中显示了两条划线，接下来，我们分步骤进行实现，具体如下。

(1) 定义一个用于绘制图形的面板 `DrawPanel` 类，该类继承自 `JPanel`，在 `DrawPanel` 类中重写用于绘图的 `paint()` 方法，在 `paint()` 方法中获取面板的宽高，并调用 `drawLine()` 方法绘制 2 条线，具体代码如下：

```
334 import javax.swing.*;
```



```
335     import java.awt.*;
336     public class DrawPanel extends JPanel {
337         // 子类重写了父类的画画方法
338         @Override
339         public void paint(Graphics g) {
340             // 获取当前面板的宽高
341             int width = getWidth();
342             int height = getHeight();
343             g.drawLine(0, 0, width, height);
344             g.drawLine(width, 0, 0,height);
345         }
346     }
```

上述代码中，第10~11行代码调用drawLine()方法用于绘制两个线条，其中drawLine()方法的4个参数依次表示第1个点的x坐标、第1个点的y坐标，第2个点的x坐标，第2个点的y坐标。

(2) 在测试类Test中绘制图形，具体代码如下：

```
347     import javax.swing.*;
348     import java.awt.*;
349     public class Test{
350         public static void main(String[] args) {
351             JFrame app=new JFrame("黑马");
352             app.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
353             DrawPanel dp=new DrawPanel();
354             dp.setPreferredSize(new Dimension(300,300));
355             app.add(dp);
356             app.pack();
357             app.setVisible(true);
358         }
359     }
```

关于上述代码的介绍如下：

第5行代码创建了一个标题为“黑马”的窗体对象JFrame。

第6行代码用于设置窗体右上角的按钮是一个是关闭按钮。

第7行代码创建了一个面板对象DrawPanel。

第8行代码用于设置面板的默认大小，这里设置为像素是300*300。

第9行代码用于将面板加入窗体。

第10行代码用于通知窗体计算自身显示的大小。需要注意的是，这行代码非常关键，因为如果没有这行代码，窗体是不会基于我们设置的内容进行显示的。

第11行代码用于显示窗体内容。

程序的运行结果如图11-5所示。

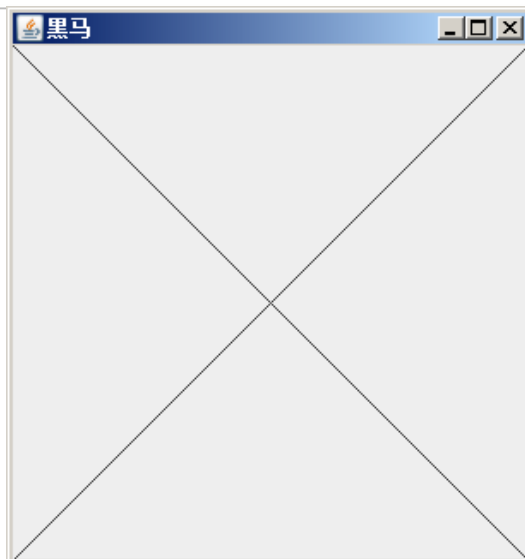


图11-5 绘制划线的效果图

11.1.3 对抗锯齿

前面小节我们绘制的线条，放大后发现线条是锯齿状的。为了解决线条不平滑的问题，我们可以在绘制线条的时候，通过下面代码对抗锯齿，消除锯齿状，让线条更加平滑，示例代码如下：

```
// 对抗锯齿
Graphics2D g2d = (Graphics2D) g;
g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
    RenderingHints.VALUE_ANTIALIAS_ON);
```

为了大家更直观看到对抗锯齿的效果，接下来，我们对 11.1.2 节的代码进行修改，将 11.1.2 节绘制的线条消除锯齿状，修改后的代码如文件 11-1 所示。

文件11-1 DrawPanel.java

```
import javax.swing.*;
import java.awt.*;

public class DrawPanel extends JPanel {
    // 子类重写了父类的画画方法
    @Override
    public void paint(Graphics g) {
        // 对抗锯齿
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        // 获取当前面板的宽高
        int width = getWidth();
        int height = getHeight();

        g.drawLine(0, 0, width, height); // 参数 0, 0 表示坐标位置
        g.drawLine(width, 0, 0, height);
    }
}
```



```
}  
}
```

此时，再次运行代码查看图形效果，如图 11-6 所示。

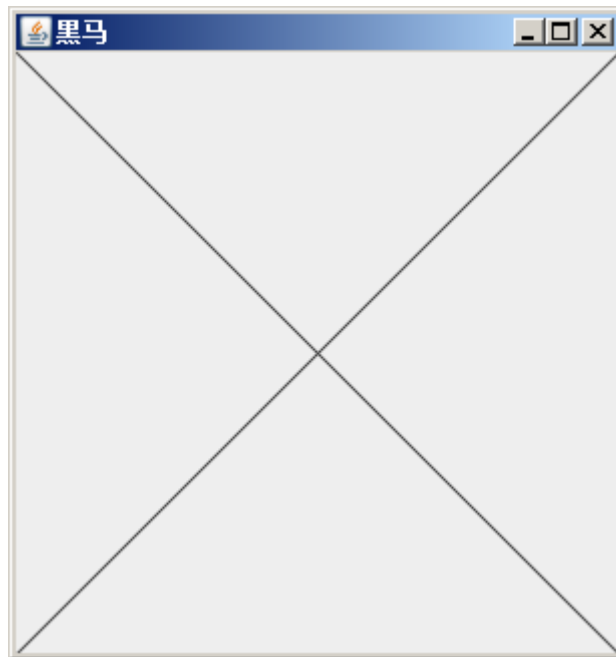


图11-6 对抗锯齿后的效果

11.1.4 绘制好看的线条

这节课我们使用之前学习的内容，绘制一些比较复杂的图形，如图 11-7 所示。

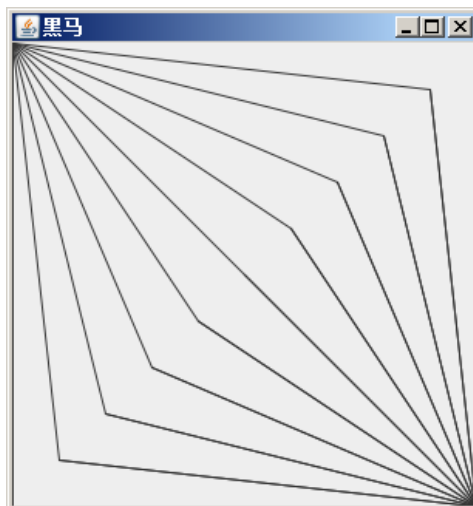


图11-7 绘制图形效果图

接下来，我们对之前的案例进行改写，在 IDEA 中定义一个用于绘制图形的面板 DrawPanel02，通过 for 循环调用 drawLine() 方法绘制线条，具体代码如下：

```
360    import javax.swing.*;  
361    import java.awt.*;
```



```
362     public class DrawPanel02 extends JPanel {
363         // 子类重写了父类的画画方法
364         @Override
365         public void paint(Graphics g) {
366             Graphics2D g2d = (Graphics2D) g;
367             g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
368                                 RenderingHints.VALUE_ANTIALIAS_ON);
369             // 获取当前面板的宽高
370             int width = getWidth();
371             int height = getHeight();
372             for (int i = 0; i < 10; i++) {
373                 g.drawLine(0, 0, 0 + 30 * i, height - 30 * i);
374                 g.drawLine(width, height, 0 + 30 * i, height - 30 * i);
375             }
376             for(int i = 0;i<10;i++) {
377                 g.drawLine(0+i*30, height-i*30, width, height);
378             }
379         }
380     }
```

从效果图分析可知，共需要绘制 18 个线条，我们可以看做是 9 组线条。由于画板的像素大小是 300*300，我们可以将 x、y 坐标等分为 10 份，每份以 30 像素的倍数递增/递减。

新建测试类 Test02，创建 DrawPanel02 对象，具体代码如下：

```
public class Test02 {
    public static void main(String[] args) {
        JFrame app=new JFrame("黑马");
        app.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        DrawPanel02 dp=new DrawPanel02();
        dp.setPreferredSize(new Dimension(300,300));
        app.add(dp);
        app.pack();
        app.setVisible(true);
    }
}
```

程序运行结果如图 11-8 所示。

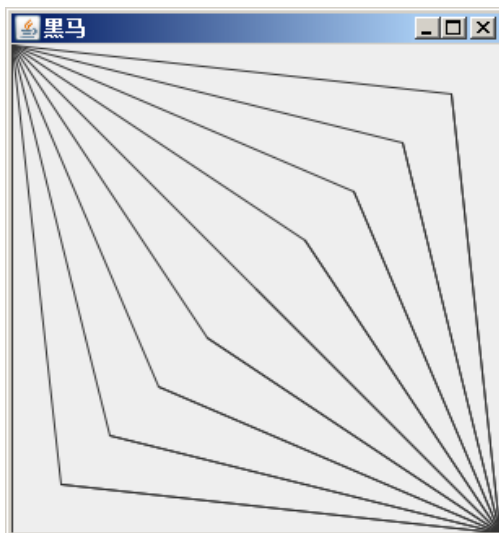


图11-8 运行结果

11.1.5 绘制矩形

下面我们以图 11-9 所示的效果为例，介绍如何使用 GUI 绘制矩形。



图11-9 矩形效果

案例开发步骤具体如下：

(1) 创建一个表示面板的类 RectPanel, 在该类中通过调用 drawRect() 方法绘制矩形，具体代码如下：

```
import javax.swing.*;
import java.awt.*;

public class RectPanel extends JPanel {
    @Override
    public void paint(Graphics g) {
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                              RenderingHints.VALUE_ANTIALIAS_ON);
    }
}
```



```
        for (int i = 0; i < 10; i++) {  
            g.drawRect(10 + 5 * i, 10 + i * 5, 50 + i * 10, 50 + i * 10);  
        }  
    }  
}
```

(2) 创建测试类 TestRect，具体代码如下：

```
import javax.swing.*;  
import java.awt.*;  
public class TestRect {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("黑马测试窗体");  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        RectPanel panel = new RectPanel();  
        panel.setPreferredSize(new Dimension(200, 200));  
        frame.add(panel);  
        frame.pack();  
        frame.setVisible(true);  
    }  
}
```

程序运行结果如图 11-10 所示。



图11-10 运行结果

11.1.6 绘制圆形

绘制圆形的方式与前面绘制其他图形的做法基本类似，只不过绘制圆形对应的 API 不同。绘制圆形使用的方法是 drawOval() 方法。

下面我们通过代码简单演示一下绘制圆形，具体步骤如下：

(1) 定义一个绘制圆形的面板类 CyclePanel，用于绘制圆形，具体代码如下：

```
import javax.swing.*;  
import java.awt.*;
```



```
public class CyclePanel extends JPanel {  
    @Override  
    public void paint(Graphics g) {  
        Graphics2D g2d = (Graphics2D) g;  
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,  
                               RenderingHints.VALUE_ANTIALIAS_ON);  
        for (int i = 0; i < 10; i++) {  
            g.drawOval(10 + 5 * i, 10 + i * 5, 50 + i * 20, 50 + i * 20);  
        }  
    }  
}
```

(2) 在测试类 TestCycle 绘制图形，具体代码如下：

```
import javax.swing.*.*;  
import java.awt.*.*;  
public class TestCycle {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("黑马测试窗体");  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        CyclePanel panel = new CyclePanel();  
        panel.setPreferredSize(new Dimension(500, 500));  
        frame.add(panel);  
        frame.pack();  
        frame.setVisible(true);  
    }  
}
```

程序的运行结果如图 11-11 所示。

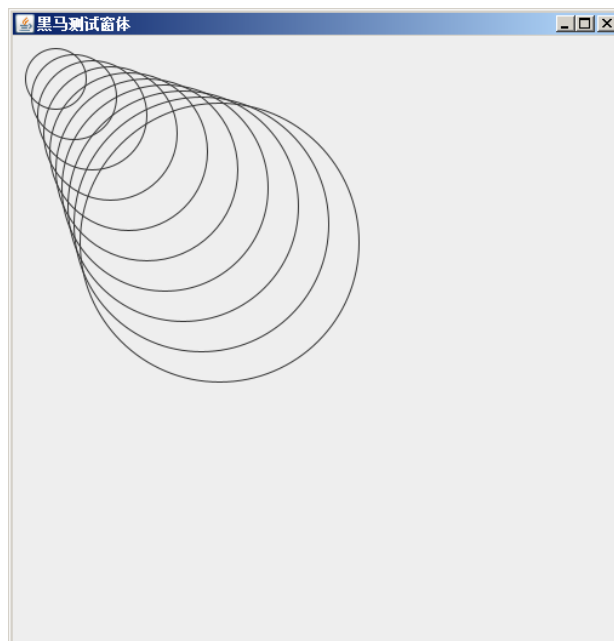


图11-11 运行结果

从图 11-11 可以看出，此时绘制出的圆形是空心的圆形，如果想绘制出的圆形为实心的圆形，也就是将图 11-11 中的图形进行填充，只需将绘制圆形的 `drawOval()` 方法替换为 `fillOval()` 方法即可。

修改绘制圆形的面板类 `CyclePanel` 的代码，修改后的具体代码如下：

```
import javax.swing.*;
import java.awt.*;

public class CyclePanel extends JPanel {
    @Override
    public void paint(Graphics g) {
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                               RenderingHints.VALUE_ANTIALIAS_ON);
        for (int i = 0; i < 10; i++) {
            g.fillOval(10 + 5 * i, 10 + i * 5, 50 + i * 20, 50 + i * 20);
        }
    }
}
```

运行程序，填充圆形的效果如图 11-11 所示。

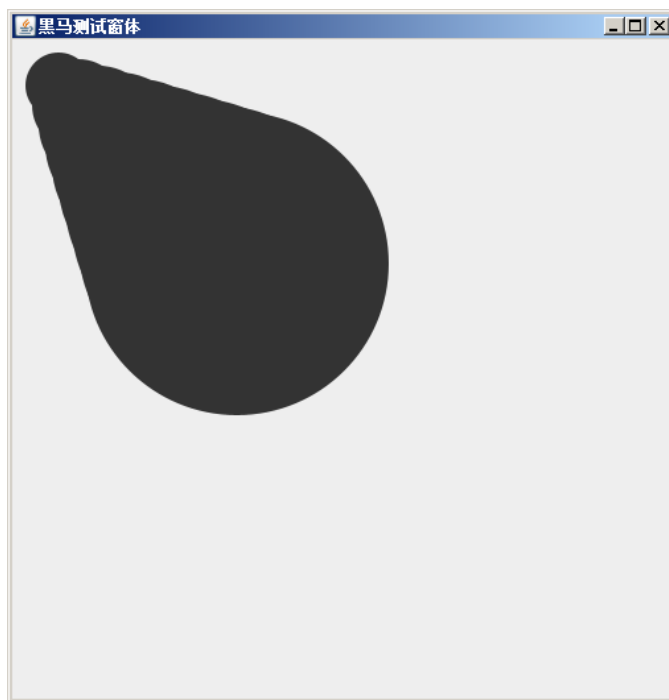


图11-12 填充圆形的效果

11.1.7 绘制文字

这节课，我们接着使用 GUI 绘制文字，效果图如图 11-13 所示。

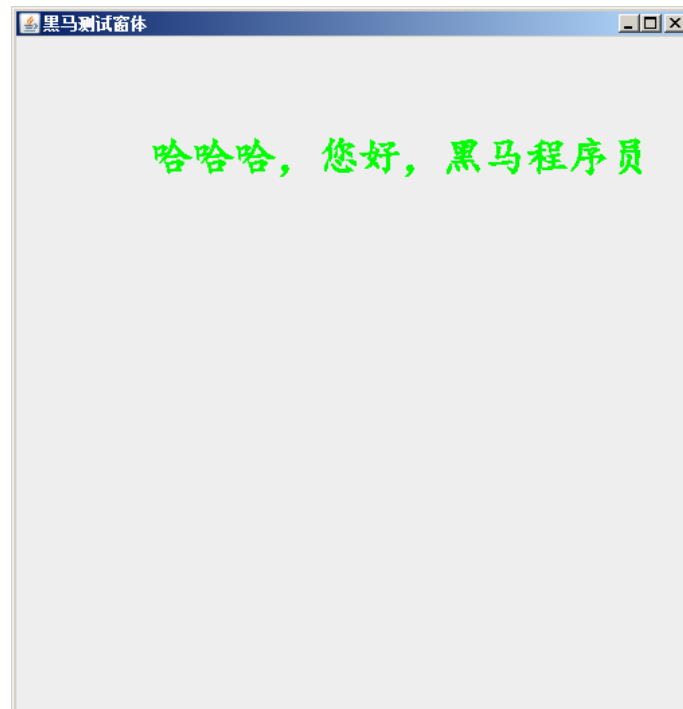


图11-13 绘制文字效果图

实现代码具体如下：

(1) 新建 StringPanel 类继承自 JPanel，在该类的 paint() 方法中设置字体的颜色、字体样式以及文字内容，具体代码如下：

```
import javax.swing.*;
import java.awt.*;

public class StringPanel extends JPanel {
    @Override
    public void paint(Graphics g) {
        g.setColor(Color.green);
        g.setFont(new Font("楷体", Font.BOLD, 30));
        g.drawString("哈哈，您好，黑马程序员", 100, 100);
    }
}
```

(2) 在测试类 TestString 中创建 StringPanel 对象绘制文字，具体代码如下：

```
import javax.swing.*;
import java.awt.*;

public class TestString {
    public static void main(String[] args) {
        JFrame frame = new JFrame("黑马测试窗体");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        StringPanel panel = new StringPanel();
        panel.setPreferredSize(new Dimension(500, 500));
        frame.add(panel);
        frame.pack();
    }
}
```



```
frame.setVisible(true);  
  
}  
  
}
```

程序的运行结果如图 11-14 所示。



图11-14 运行结果

需要注意的是，当调用 `setFont()` 方法设置的字体，一定是我们当前系统所支持的字体库。如果大家不知道当前系统支持哪些字体库，最简单的方式，就是通过查看 PPT 中的字体样式，从而得知我们当前的系统安装了哪些字体。

11.1.8 绘制图片

本节我们来学习如何绘制图片，效果如图 11-15 所示。



图11-15 图片效果

其实绘制图片的代码是一个模板代码，由于绘制图片的操作涉及到后面的学习内容，所以这里我们只需要知道模板代码的含义以及我们要显示图片的话，修改代码的哪个参数即可。

接下来，通过写代码的方式实现上述的图片效果，具体步骤如下：

(1)在 IDEA 中新建一个文件夹 resources 用于存放资源文件，右击 resources 文件夹，选择【Mark Directory as】→【Resources Root】，如图 11-16 所示。

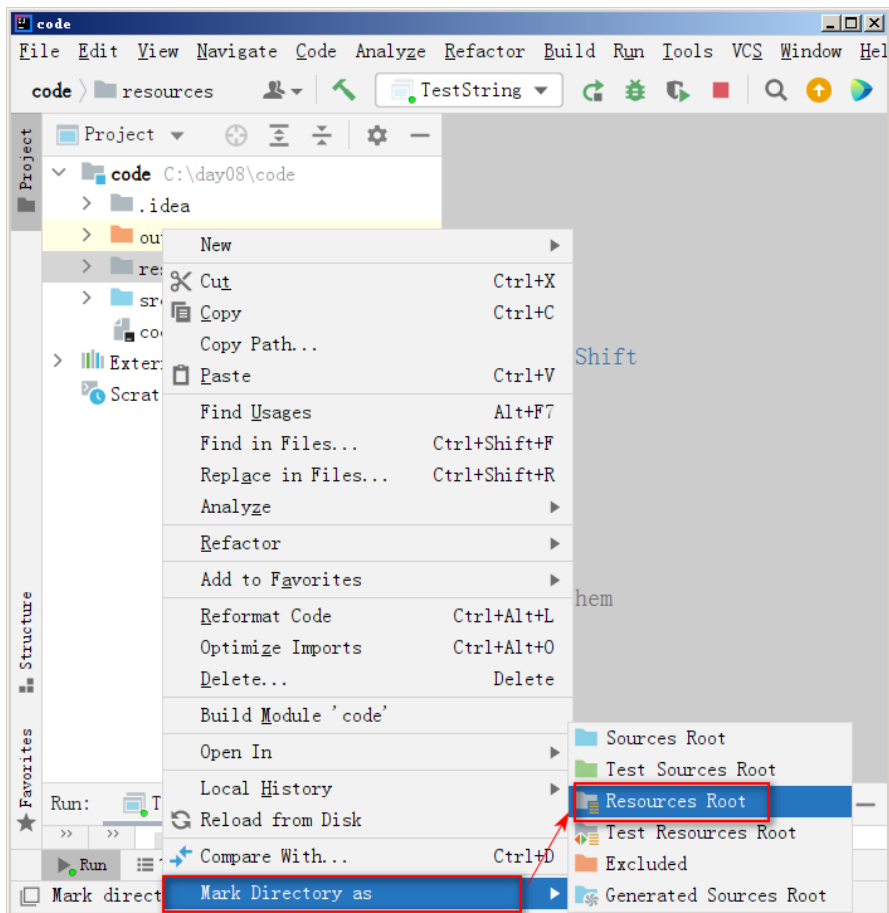


图11-16 新建资源文件并打包



这么做的目的是将 resources 文件夹作为项目的资源目录，程序编译时会自动将资源打包到项目中。打包完成后，我们将 logo.jpg 图片拷贝到 resources 文件夹。

(2) 创建一个继承自 JPanel 类的 ImagePanel 类，在该类中读取图片并设置图片的显示方式，具体代码如下：

```
381     import javax.imageio.ImageIO;
382     import javax.swing.*;
383     import java.awt.*;
384     import java.io.IOException;
385     public class ImagePanel extends JPanel {
386         @Override
387         public void paint(Graphics g) {
388             try {
389                 g.drawImage(ImageIO.read(getClass().getClassLoader().
390                     getResourceAsStream("logo.png")), 10, 10, null);
391             } catch (IOException e) {
392                 e.printStackTrace();
393             }
394         }
395     }
```

上述代码中，第 9-10 行代码是绘制图片的核心代码，其中 logo.png 是位于 resources 文件夹下的图片的名称，如果我们想绘制其他图片，只需修改图片名称即可。

(3) 在测试类 TestImage 中绘制图片，具体代码如下：

```
import javax.swing.*;
import java.awt.*;

public class TestImage {
    public static void main(String[] args) {
        JFrame frame = new JFrame("heima");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ImagePanel panel = new ImagePanel();
        panel.setPreferredSize(new Dimension(500, 500));
        frame.add(panel);
        frame.pack();
        frame.setVisible(true);
    }
}
```

程序运行效果如图 11-17 所示。



图11-17 运行结果

11.1.9 绘制笑脸

学习了图片的绘制之后，这节课我们使用前面所学的知识绘制一个如图 11-18 所示的笑脸效果图。

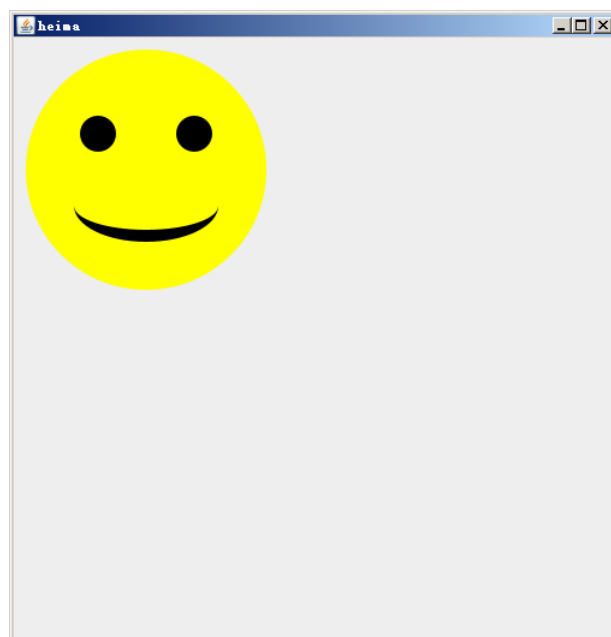


图11-18 笑脸效果图

下面通过编写代码的方式实现，具体步骤如下：

(1) 创建继承自 Panel 的类 SmilePanel，用于绘制笑脸图形，具体代码如下：

```
396 import java.awt.*;  
397 public class SmilePanel extends Panel {  
398     @Override  
399     public void paint(Graphics g) {
```



```
400         // 消除锯齿
401         Graphics2D g2d = (Graphics2D) g;
402         g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
403             RenderingHints.VALUE_ANTIALIAS_ON);
404         // 画脑袋
405         g.setColor(Color.yellow);
406         g.fillOval(10, 10, 200, 200);
407         // 画眼睛
408         g.setColor(Color.black);
409         g.fillOval(55, 65, 30, 30);
410         g.fillOval(135, 65, 30, 30);
411         // 画嘴巴
412         g.fillOval(50, 110, 120, 60);
413         g.setColor(Color.yellow);
414         g.fillRect(50, 110, 120, 30);
415         g.fillOval(50, 120, 120, 40);
416     }
417 }
```

(2) 在测试类 Test 中创建 SmilePanel 对象并，具体代码如下：

```
418     import javax.swing.*;
419     import java.awt.*;
420     public class Test {
421         public static void main(String[] args) {
422             JFrame frame = new JFrame("heima");
423             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
424             SmilePanel panel = new SmilePanel();
425             panel.setPreferredSize(new Dimension(500, 500));
426             frame.add(panel);
427             frame.pack();
428             frame.setVisible(true);
429         }
430     }
```

程序运行结果如图 11-19 所示。

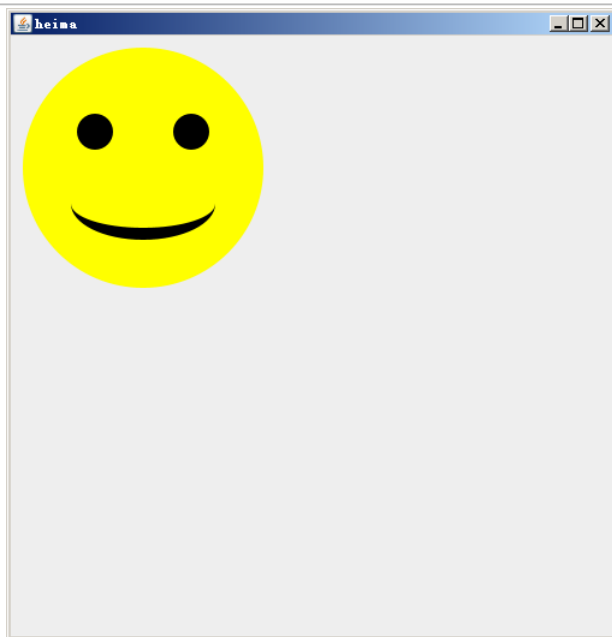


图11-19 运行结果

11.2 使用面向对象思想设计直线

这节课我们来学习如何使用面向对象的编程思想设计一个直线类，具体步骤如下：

(1) 创建一个类 Line 表示直线，该类定义了直线起点和终点的坐标、线条颜色以及线条的绘制方法，具体代码如下：

```
import java.awt.*;

public class Line {

    private int x1, y1; // 线的开始坐标
    private int x2, y2; // 线的结束坐标
    private Color color; // 线条的颜色

    public Line(int x1, int y1, int x2, int y2, Color color) {
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
        this.color = color;
    }

    // 绘制直线
    public void draw(Graphics2D g2d) {
        g2d.setColor(color);
        g2d.drawLine(x1, y1, x2, y2);
    }
}
```

(2) 创建一个画板类 MyPanel, 在该类中创建一个 Line 类型的对象，具体代码如下：



```
import javax.swing.*;
import java.awt.*;

public class MyPanel extends JPanel {

    @Override
    public void paint(Graphics g) {
        // 消除锯齿
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
        Line line = new Line(10, 10, 60, 60, Color.red);
        line.draw(g2d);
    }
}
```

(3) 创建测试类 Test，具体代码如下：

```
public class Test {

    public static void main(String[] args) {
        JFrame frame = new JFrame("linedemo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        MyPanel panel = new MyPanel();
        panel.setPreferredSize(new Dimension(500, 500));
        frame.add(panel);
        frame.pack();
        frame.setVisible(true);
    }
}
```

程序的运行结果如图 11-20 所示。

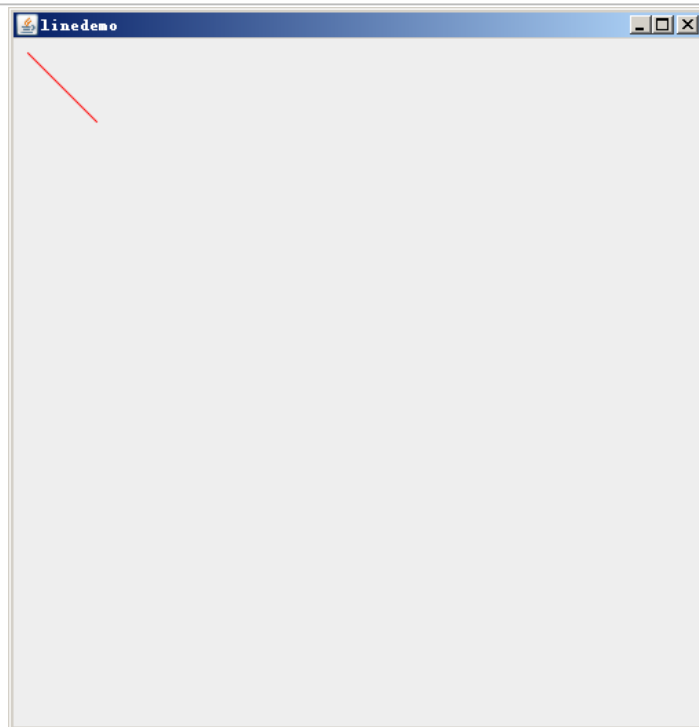


图11-20 运行结果（1）

从图 11-20 中可以看出，画板上成功绘制一条红色的线条。

假设，现在要实现随机绘制多条直线的功能，我们对 MyPanel 类进行修改，将 Line 对象的坐标、颜色都设置为随机数，具体代码如下：

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class MyPanel extends JPanel {
    Random random = new Random();

    @Override
    public void paint(Graphics g) {
        // 消除锯齿
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        for (int i = 0; i < 40; i++) {
            Line line = new Line(random.nextInt(500), random.nextInt(500),
                random.nextInt(500), random.nextInt(500),
                new Color(random.nextInt(256),
                    random.nextInt(256), random.nextInt(256)));
            line.draw(g2d);
        }
    }
}
```

再次运行测试类 Test，程序的运行结果如图 11-21 所示。

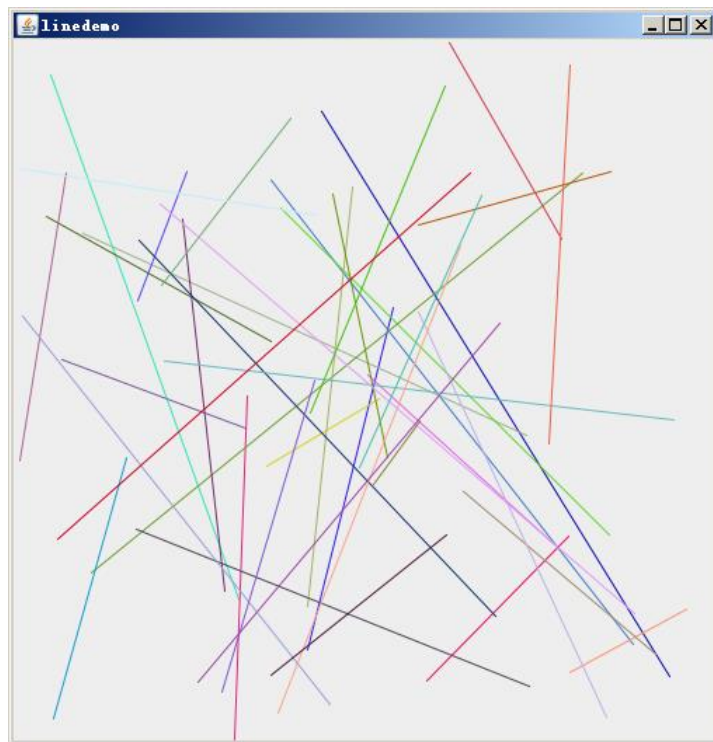


图11-21 运行结果（2）

11.3 绘制动画

绘制动画，其实就是按照一定的顺序绘制多个图片，这些图片连续播放从而形成动画效果。接下来，我们通过一个案例来演示如何使用 GUI 绘制动画，具体步骤如下：

（1）将绘制动画效果的图片导入 resources 资源包。

（2）创建一个名称为 Bang 的动画类，在该类中定义一个数组 images 用于存放多张图片，定义一个 draw() 方法用来实现爆炸效果。Bang 类的具体代码如下：

```
431 import javax.imageio.ImageIO;
432 import java.awt.*;
433 import java.io.IOException;
434 public class Bang {
435     private int index = 0;
436     private String[] images = {"blast_1.gif", "blast_2.gif",
"blast_3.gif",
437                               "blast_4.gif", "blast_5.gif",
"blast_6.gif",
438                               "blast_7.gif", "blast_8.gif"};
439     public void draw(Graphics2D g2d) {
440         try {
441             g2d.drawImage(ImageIO.read(getClass().getClassLoader()
442                               .getResourceAsStream(images[index % images.length])), 10,
10, null);
```



```
443         } catch (IOException e) {  
444             e.printStackTrace();  
445         }  
446         index++;  
447         if (index < 0) {  
448             index = 0;  
449         }  
450     }  
451 }
```

上述代码中，第 6~8 行代码定义了一个数组用来存放图片名称。第 9~20 行代码定义了 draw() 方法绘制图片，为了防止图片索引超出范围，第 11~12 代码通过计算 “index % images.length” 的值来控制图片索引。

(3) 创建一个画板类 BangPanel，具体代码如下：

```
import javax.swing.*;  
import java.awt.*;  
public class BangPanel extends JPanel {  
    Bang bang=new Bang();  
    @Override  
    public void paint(Graphics g) {  
        super.paint(g);  
        // 消除锯齿  
        Graphics2D g2d = (Graphics2D) g;  
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,  
            RenderingHints.VALUE_ANTIALIAS_ON);  
        bang.draw(g2d);  
    }  
}
```

(4) 创建测试类 Test，具体代码如下：

```
import javax.swing.*;  
import java.awt.*;  
public class Test {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("Bang");  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        BangPanel panel = new BangPanel();  
        panel.setPreferredSize(new Dimension(500, 500));  
        frame.add(panel);  
        frame.pack();  
        frame.setVisible(true);  
        while (true){  
            panel.repaint(); // panel 的 draw 方法会被重新执行  
            try {  
                Thread.sleep(20);  
            }  
        }  
    }  
}
```



```
        }catch (InterruptedException e){  
            e.printStackTrace();  
        }  
    }  
}
```

上述代码中，为了体现动画效果，我们使用 while 循环不断调用 repaint() 方法重复执行 draw() 方法绘制图片，同时，为了明显看到图片的切换效果，调用 “Thread.sleep(20)” 让每次绘制图片的时候休眠一会。

程序的运行后会依次绘制数组 images 中的图片，显示效果的顺序如图 11-1 所示。

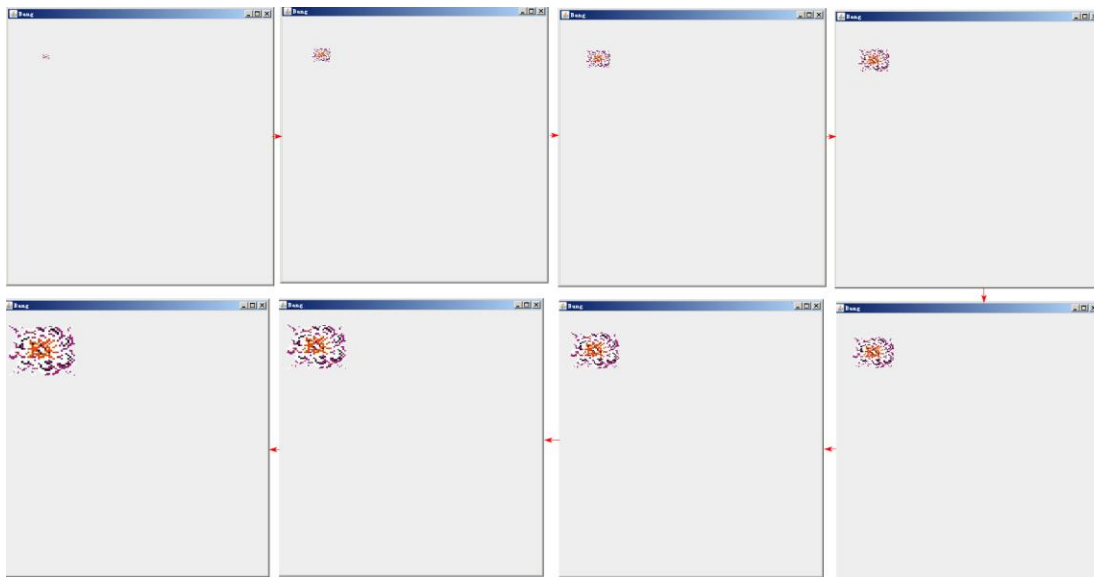


图11-22 显示效果的顺序

11.3 键盘事件

当我们使用键盘按下、释放或键入键时会生成键盘事件，我们可以在指定的组件上监听键盘事件以获取当前按下或释放的键是哪一个。接下来，我们通过一个案例来演示键盘事件的使用，具体步骤如下：

(1) 创建一个面板类 MyPanel，用于显示生成的键盘事件，具体代码如下所示。

```
1 import javax.swing.*;  
2 import java.awt.*;  
3  
4 public class MyPanel extends JPanel {  
5     private String info = "";  
6     public void setInfo(String info){  
7         this.info = info;  
8     }  
9     @Override  
10    public void paint(Graphics g) {
```



```
11     super.paint(g);
12     g.drawString(info ,100,100);
13 }
14 }
```

(2) 创建一个测试类，在测试类中创建窗体对象和面板对象，为窗体对象添加键盘事件的监听，并将监听到的事件在面板中显示，具体代码如下所示。

```
1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.KeyEvent;
4  import java.awt.event.KeyListener;
5
6  public class Test {
7      public static void main(String[] args) {
8          JFrame frame = new JFrame("keyevent");
9          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
10         MyPanel panel = new MyPanel();
11         panel.setPreferredSize(new Dimension(500,500));
12         frame.add(panel);
13         frame.addKeyListener(new KeyListener() {
14             @Override
15             public void keyTyped(KeyEvent e) {
16
17             }
18             @Override
19             public void keyPressed(KeyEvent e) {
20                 panel.setInfo(e.getKeyChar()+"被按下了");
21                 System.out.println(e.getKeyChar()+"被按下了");
22             }
23             @Override
24             public void keyReleased(KeyEvent e) {
25                 panel.setInfo(e.getKeyChar()+"被弹起了");
26                 System.out.println(e.getKeyChar()+"被弹起了");
27             }
28         });
29         frame.pack();
30         frame.setVisible(true);
31         while (true) {
32             panel.repaint();
33             try {
34                 Thread.sleep(10);
35             } catch (InterruptedException e) {
36                 e.printStackTrace();
37             }
38         }
39     }
40 }
```



```
38     }  
39 }  
40 }
```

上述代码中，第 13~28 行代码为 JFrame 对象添加了键盘事件的监听，其中第 15~17 行的 keyTyped() 方法会在在键入键时调用，第 19~22 行的 keyPressed() 方法会在按下某个键时调用，第 15~17 行的 keyReleased() 方法会在释放键时调用。默认情况 panel 的内容不会重复进行渲染，只有当窗体的大小发生改变或者程序员手动设置需要重新渲染时，panel 的内容才会再次渲染。对此在第 31~38 行中，指定 panel 每隔 10 毫秒重新进行渲染一次。

运行程序后，在窗体上按下键盘下的“a”键，效果如图 11-23 所示。



图11-23 键盘按下的效果

此时，如果松开按键“a”，面板显示的内容如图 11-24 所示。



图11-24 键盘弹起的效果

第 12 章综合项目-贪吃蛇

学习目标

- ◆ 了解贪吃蛇游戏框架,能够说出 GameEngine 和 Game 的内部实现原理
- ◆ 掌握贪吃蛇游戏环境的搭建
- ◆ 掌握贪吃蛇的网格设计,能够使用绘制多条线条的方式在面板中绘制网格
- ◆ 掌握贪吃蛇的身子绘制,能够根据面向对象的编程思想在面板中绘制贪吃蛇的身子
- ◆ 掌握贪吃蛇的上下左右移动,能够使用键盘事件实现贪吃蛇的移动
- ◆ 掌握苹果的绘制,能够在随机位置绘制红色的方块
- ◆ 掌握吃苹果的操作,能够在贪吃蛇吃到苹果后,再随机生成新的苹果
- ◆ 掌握贪吃蛇的速度,能够让贪吃蛇自动移动
- ◆ 掌握贪吃蛇的优化,能够解决贪吃蛇咬自己的问题,以及消除面板中的网格

通过前面章节的学习,相信读者已经掌握了 Java 基础的相关知识。学习编程语言的目的是将其应用到项目开发中解决实际问题,在不断的应用中增强开发技能,锻炼编程思维,加深对程序设计语言的认识和理解。接下来,本章将利用前面所学的知识开发一个贪吃蛇的游戏,加深读者对 Java 基础知识的理解。



12.1 游戏框架介绍

前面我们使用过一个封装了游戏引擎的 jar 包 game.engine.jar, 它内部有两个核心类, 分别是 GameEngine 和 Game, 接下来, 我们分析一下这两个类的内部实现原理。

我们先分析 GameEngine 类, 该类是由大牛程序员编写的游戏引擎抽象模板类, 具体代码如下:

```
import java.awt.*;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
/**
 * 由大牛程序员编写的游戏引擎的抽象模板类
 * 需要调用者实现两个抽象方法, 更新逻辑方法 updateLogic
 * 更新 ui 方法 renderUI
 */
public abstract class GameEngine {
    KeyListener listener;
    private int currentPressedKeyCode;
    /**
     * 构造方法, 创建出游戏引擎
     */
    public GameEngine(){
        this.listener = new KeyListener() {
            @Override
            public void keyTyped(KeyEvent e) {
            }
            @Override
            public void keyPressed(KeyEvent e) {
                currentPressedKeyCode = e.getKeyCode();
            }
            @Override
            public void keyReleased(KeyEvent e) {
                currentPressedKeyCode = -1;
            }
        };
    }
    /**
     * 获取当前用户的按键代码 按键代码的定义在 KeyEvent 对象中定义
     * @return 返回 KeyEvent 的按键代码
     */
    public int getCurrentPressedKeyCode(){
        return currentPressedKeyCode;
    }
}
```



```
/**
 * 根据用户行为或者游戏的时间更新游戏的逻辑
 */
public abstract void updateLogic();
/**
 * 根据用户需求更新绘制图形化界面的内容
 * 示例代码，绘制矩形：
    g2d.fillRect(左上角 x 的坐标，左上角 y 的坐标，矩形的宽度，矩形的高度)
 * 示例代码：绘制椭圆形：
    g2d.fillOval(椭圆外包裹矩形左上角 x 的坐标，椭圆外包裹矩形 y 的坐标，要填充椭圆形的宽度，要填充椭圆形的高度);
 * @param g2d
 */
public abstract void renderUI(Graphics2D g2d);
}
```

上述代码中，GameEngine 内部定义了若干方法，其中 updateLogic 和 renderUI 是抽象方法，分别用于更新逻辑以及更新 UI。GameEngine 类还有一个构造方法，该构造方法接收一个键盘监听器，用户按下或者弹起键盘的信息都被存储到 currentPressedKeyCode 变量中。如果想获取键盘信息，则可以通过调用 getCurrentPressedKeyCode() 实现。

接下来，再看 Game 类，具体代码如下：

```
import javax.swing.*;
import java.awt.*;
/**
 * 大牛程序员写的游戏的公共类 <p></p>
 * 使用 Game.init() 方法初始化游戏<p></p>
 * 使用 Game.gameOver() 方法退出游戏<p></p>
 */
public class Game extends JPanel {
    private GameEngine engine;
    private Graphics2D g2d;
    private static JFrame frame;
    @Override
    public void paint(Graphics g) {
        super.paint(g);
        g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
        engine.renderUI(g2d);
    }
}
/**
 * 使用游戏引擎初始化游戏的方法
 * @param title 游戏窗体的标题
 */
```




```
* @param width 游戏窗体的宽度
* @param height 游戏窗体的高度
* @param engine 游戏引擎
*/
public static void init(String title, int width, int height, GameEngine engine){
    frame = new JFrame(title);
    Game game = new Game();
    game.engine = engine;
    game.addKeyListener(engine.listener);
    game.setFocusable(true);
    game.setPreferredSize(new Dimension(width, height));
    frame.add(game);
    frame.pack();
    frame.setVisible(true);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    while (true) {
        engine.updateLogic();
        game.repaint();
        try {
            Thread.sleep(10);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
/**
 * 显示退出游戏对话框的方法
 * @param message 对话框提示的消息
 * @param title 对话框提示的标题
 */
public static void gameOver(String message,String title) {
    JOptionPane.showMessageDialog(frame, message, title,
                                   JOptionPane.YES_NO_OPTION);

    System.exit(ABORT);
}
}
```

上述代码中，Game 继承自 JPanel，所以 Game 类可以看做是我们自定义的一个图形化界面。在 Game 类的 init() 方法中，先实现了图像化界面的绘制，然后通过 while 循环不断更新业务逻辑以及 UI 界面。除此之外，Game 类还定义了一个退出游戏的方法，通过显示对话框的方式提示用户游戏结束。

综上所述，GameEngine 和 Game 类虽然代码不是很多，但是却实现了一个经典游戏引擎具备的基本功能，希望大家能认真体会这些代码的设计思想。



12.2 游戏环境搭建

从这节开始，我们将带领大家开发一个贪吃蛇的游戏。

在开发游戏之前，我们需要先搭建贪吃蛇游戏的环境，具体如图 12-1 所示。



图 12-1 吃蛇游戏环境

在图 12-1 中，每个类都有特定功能，具体介绍如下：

- (1) Game.java: 大牛程序员编写的游戏公共类。
- (2) GameEngine.java: 大牛程序员编写的游戏引擎的抽象模板类。
- (3) Config.java: 魔法数字的配置文件。
- (4) SnakeGameEngine.java: 小牛程序员编写的，GameEngine 的实现类。
- (5) SnakeGame.java: 游戏的入口类，该类提供了 main() 方法。

接下来，我们在 IDEA 中按照图 12-1 搭建环境，具体步骤如下：

- (1) 将 gameEngin.jar 中的 GameEngine 和 Game 导入 demo09_snake 包。
- (2) 定义 Config 类，用于存放魔法数字，具体代码如下：

```
public class Config {  
    /**  
     * 窗体宽带  
     */  
    public static final int SCREEN_WIDTH=500;  
    /**  
     * 窗体高带  
     */  
    public static final int SCREEN_HEIGHT=500;  
}
```

(3) 定义 SnakeGameEngine 类，继承自 GameEngine，该类是小牛程序员编写的业务逻辑，具体代码如下：

```
import java.awt.*;  
  
public class SnakeGameEngine extends GameEngine{  
    @Override  
    public void updateLogic() {  
    }  
    @Override  
    public void renderUI(Graphics2D g2d) {  
    }  
}
```



```
}
```

(4) 定义 SnakeGame 类，该类是贪吃蛇程序的入口，具体代码如下：

```
public class SnakeGame {  
    public static void main(String[] args) {  
        Game.init("贪吃蛇",  
            Config.SCREEN_WIDTH, Config.SCREEN_HEIGHT, new SnakeGameEngine());  
    }  
}
```

至此，我们就把贪吃蛇游戏需要的 Java 文件都创建好了，后续我们只需要根据需求在对应的 Java 文件中编写具体代码即可。

12.3 贪吃蛇的网格设计

在贪吃蛇游戏中，贪吃蛇的移动是在网格中进行的，如图 12-2 所示。

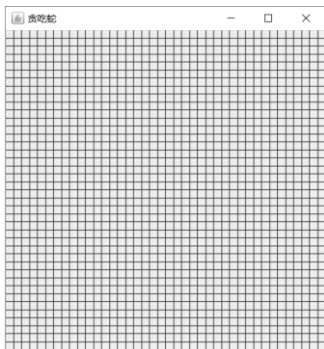


图 12-2 游戏网格

如果要实现图 12-2 所示的网格效果，我们可以在面板中绘制多条线条的方式实现。

根据面向对象的编程思想，我们可以定义一个表示网格的类 Grid，该类提供绘制小格子方法，具体代码如下：

```
public class Grid {  
    public void draw(Graphics2D g2d) {  
        // 绘制线条的代码  
    }  
}
```

由于我们要绘制的线条很多，我们可以将绘制线条的一些魔法数字在 Config.java 文件中设置，包括格子的大小，水平方向格子的数量以及垂直方向格子的数量。

Config.java 文件的具体代码如下：

```
public class Config {  
    /**  
     * 窗体宽带  
     */  
    public static final int SCREEN_WIDTH = 500;  
    /**  
     * 窗体高带  
     */  
}
```



```
    */  
    public static final int SCREEN_HEIGHT = 500;  
    /**  
     * 格子大小  
     */  
    public static final int GRID_SIZE = 10;  
    /**  
     * 垂直方向小格子数量  
     */  
    public static final int HEIGHT_NUMBER = SCREEN_HEIGHT / GRID_SIZE;  
    /**  
     * 水平方向小格子数量  
     */  
    public static final int WIDTH_NUMBER = SCREEN_WIDTH / GRID_SIZE;  
}
```

此时，在 Grid 类的 draw 中实现绘制线条，具体代码如下：

```
import java.awt.*;  
  
public class Grid {  
    public void draw(Graphics2D g2d) {  
        for (int i = 0; i < Config.HEIGHT_NUMBER; i++) {  
            g2d.drawLine(0, i * Config.GRID_SIZE,  
                          Config.SCREEN_WIDTH, i * Config.GRID_SIZE);  
        }  
        for (int i = 0; i < Config.HEIGHT_NUMBER; i++) {  
            g2d.drawLine(i * Config.GRID_SIZE,  
                          0, i * Config.GRID_SIZE, Config.SCREEN_WIDTH);  
        }  
    }  
}
```

为了验证上述代码的绘制效果，我们在 SnakeGameEngine 类中创建 Grid 对象，并在 renderUI() 方法中通过 Grid 对象调用 draw()，具体代码如下：

```
import java.awt.*;  
  
public class SnakeGameEngine extends GameEngine{  
    Grid grid=new Grid();  
    @Override  
    public void updateLogic() {  
    }  
    @Override  
    public void renderUI(Graphics2D g2d) {  
        grid.draw(g2d);  
    }  
}
```



在 SnakeGame 类中进行测试，具体代码如下：

```
public class SnakeGame {  
    public static void main(String[] args) {  
        Game.init("贪吃蛇", Config.SCREEN_WIDTH,  
                  Config.SCREEN_HEIGHT, new SnakeGameEngine());  
    }  
}
```

程序的运行结果如图 12-3 所示。

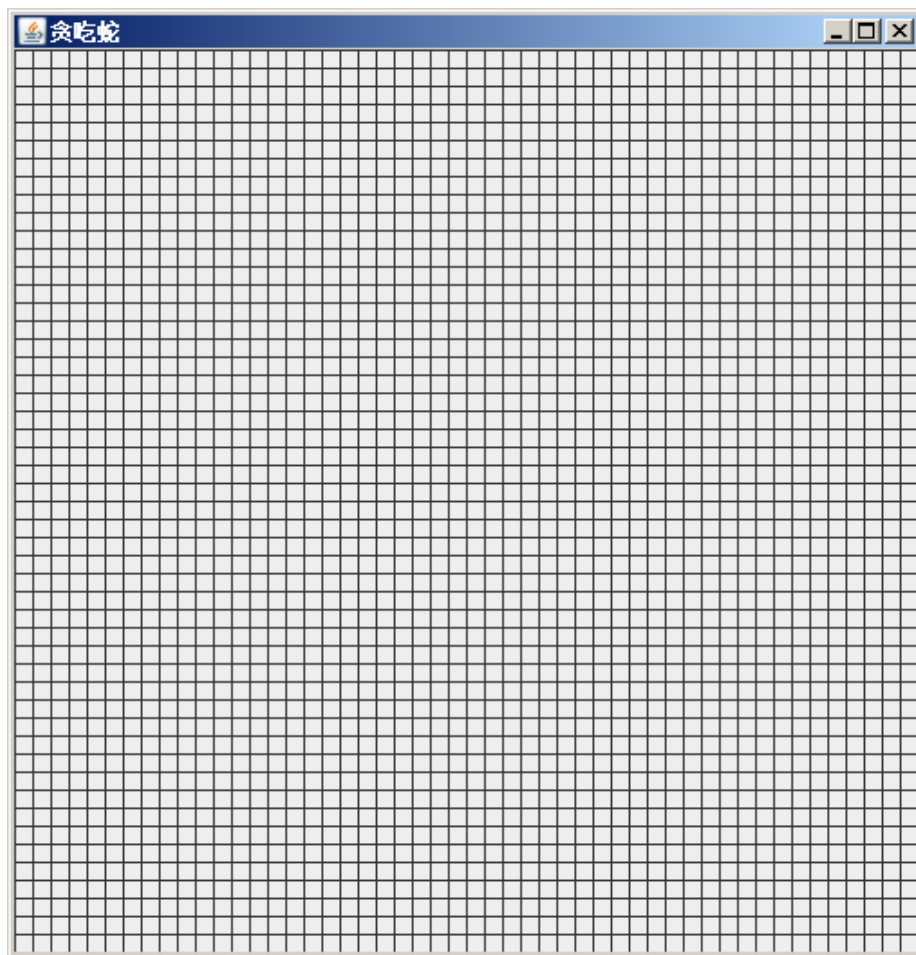


图 12-3 运行结果

其实，最终贪吃蛇游戏的效果，网格是要隐藏的，这么做的目的，一是游戏界面更干净，二是增加用户玩游戏的难度。所以，其实我们还可以在 Config.java 文件中定义一个布尔类型的常量 DEBUG，用于标识网格是否显示。修改后的 Config.java 文件具体如下：

```
public class Config {  
    /**  
     * 窗体宽带  
     */  
    public static final int SCREEN_WIDTH = 500;  
    /**  
     * 窗体高带  
     */
```



```
public static final int SCREEN_HEIGHT = 500;
/**
 * 格子大小
 */
public static final int GRID_SIZE = 10;
/**
 * 垂直方向小格子数量
 */
public static final int HEIGHT_NUMBER = SCREEN_HEIGHT / GRID_SIZE;
/**
 * 水平方向小格子数量
 */
public static final int WIDTH_NUMBER = SCREEN_WIDTH / GRID_SIZE;
/**
 * 开发调试的 flag，用于标识网格是否显示
 */
public static final boolean DEBUG = true;
}
```

与此同时，可以在 Grid 类的 draw() 方法中添加判断条件，如果 Config.DEBUG 的值为 true，才绘制网格，具体代码如下：

```
import java.awt.*;
public class Grid {
    public void draw(Graphics2D g2d) {
        if (Config.DEBUG) {
            for (int i = 0; i < Config.HEIGHT_NUMBER; i++) {
                g2d.drawLine(0, i * Config.GRID_SIZE,
                    Config.SCREEN_WIDTH, i * Config.GRID_SIZE);
            }
            for (int i = 0; i < Config.HEIGHT_NUMBER; i++) {
                g2d.drawLine(i * Config.GRID_SIZE, 0,
                    i * Config.GRID_SIZE, Config.SCREEN_WIDTH);
            }
        }
    }
}
```

后续开发人员如果想调试程序，可以通过修改 Config.java 中的 DEBUG 的值来完成。

12.4 贪吃蛇的身子绘制

贪吃蛇的身体，其实可以看做网格中的若干格子组成。根据面向对象的编程思想，我们可以先定义一个类 Node，该类代表的是网格中的一个小格子，具体代码如下：

```
import java.awt.*;
```



```
public class Node {  
    /**  
     * 小格在屏幕上的位置 x  
     */  
    private int x;  
    /**  
     * 小格在屏幕上的位置 y  
     */  
    private int y;  
    public Node(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public int getX() {  
        return x;  
    }  
    public void setX(int x) {  
        this.x = x;  
    }  
    public int getY() {  
        return y;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
    public void draw(Graphics2D g2d) {  
        g2d.fillRect(x * Config.GRID_SIZE, y * Config.GRID_SIZE,  
                     Config.GRID_SIZE, Config.GRID_SIZE);  
    }  
}
```

接下来，定义一个表示贪吃蛇的类 Snake，具体代码如下：

```
import java.awt.*;  
import java.util.LinkedList;  
public class Snake {  
    /**  
     * 定义一个集合（复杂的数组）存放蛇的身子（node）  
     */  
    private LinkedList<Node> snake = new LinkedList<>();  
    public Snake() { // 初始状态由三个小格子组成  
        snake.add(new Node(6, 4)); // 蛇头  
        snake.add(new Node(5, 4)); // 身子
```



```
        snake.add(new Node(4, 4)); // 尾巴
    }
    /**
     * 画蛇的方法
     *
     * @param g2d
     */
    public void draw(Graphics2D g2d) {
        for (Node node : snake) { // 循环画蛇的每一个 node
            node.draw(g2d);
        }
    }
}
```

在 Snake 类中，首先定义了一个 LinkedList 集合，然后在 Snake 类的构造方法中，向 LinkedList 集合添加三个元素，分别表示贪吃蛇的蛇头、身子和尾巴，最后通过调用 Node 类的 draw() 方法完成蛇身子的绘制。

在 SnakeGameEngine 类中创建 Snake 对象，并在 renderUI() 方法中调用 Snake 类的 draw() 方法，具体代码如下：

```
import java.awt.*;
/**
 * 小牛程序员写的业务逻辑
 */
public class SnakeGameEngine extends GameEngine {
    Grid grid = new Grid();
    Snake snake = new Snake();
    @Override
    public void updateLogic() {
    }
    @Override
    public void renderUI(Graphics2D g2d) {
        grid.draw(g2d);
        snake.draw(g2d);
    }
}
```

程序的运行结果如图 12-4 所示。

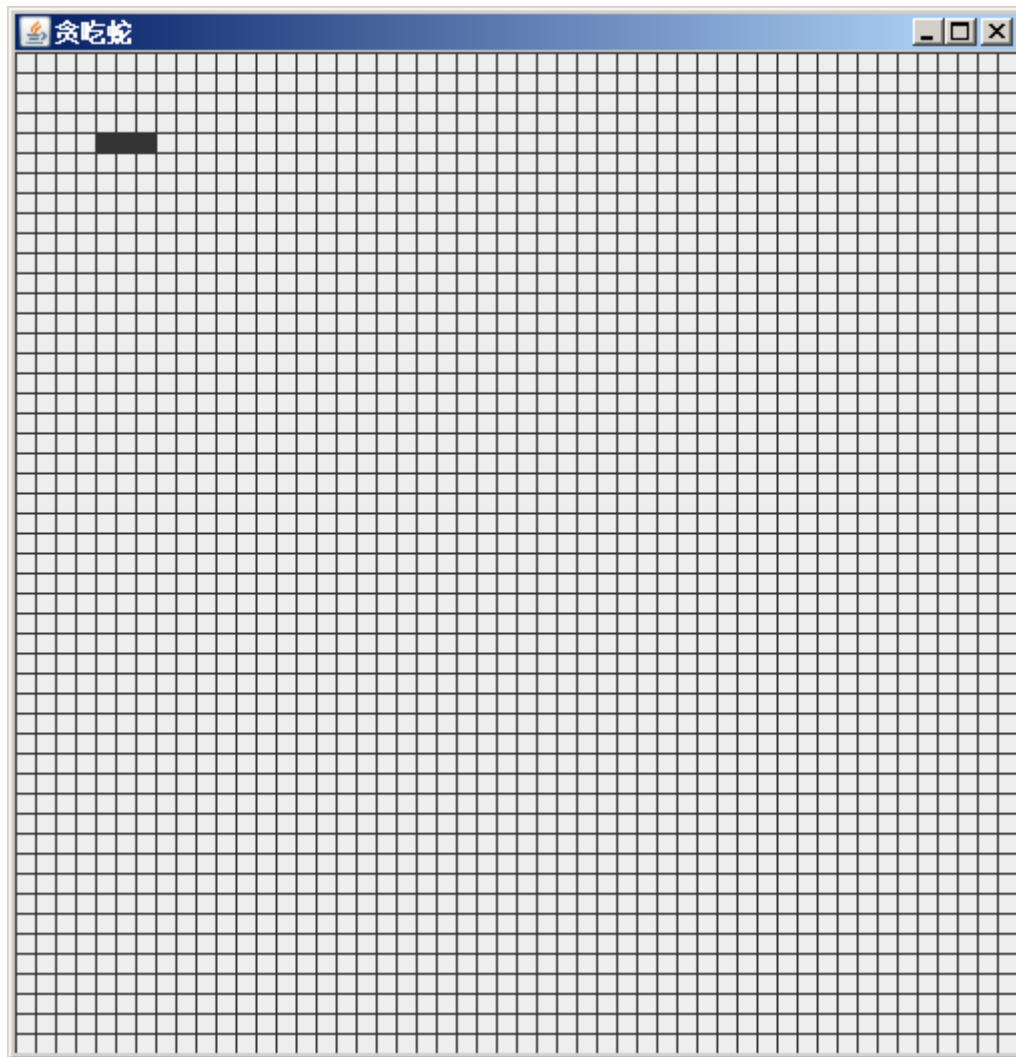


图 12-4 运行结果

12.5 贪吃蛇的上下左右移动

贪吃蛇的上下左右移动需要借助键盘事件实现。

贪吃蛇在移动时，只能沿着蛇头方向前进、向左以及向右移动，所以我们首先必须要明确的是当前蛇头的方向。这里我们将表示蛇头方向的常量值定义在 Config.java 中，具体代码如下：

```
public class Config {  
    /**  
     * 窗体宽带  
     */  
    public static final int SCREEN_WIDTH = 500;  
    /**  
     * 窗体高带  
     */  
    public static final int SCREEN_HEIGHT = 500;
```



```
public static final int GRID_SIZE = 10;
/**
 * 垂直方向小格子数量
 */
public static final int HEIGHT_NUMBER = SCREEN_HEIGHT / GRID_SIZE;
/**
 * 水平方向小格子数量
 */
public static final int WIDTH_NUMBER = SCREEN_WIDTH / GRID_SIZE;
/**
 * 开发调试的 flag
 */
public static final boolean DEBUG = true;
/**
 * 蛇头方向
 */
public static final int UP = 1;
public static final int DOWN = 2;
public static final int LEFT = 3;
public static final int RIGHT = 4;
}
```

接下来，我们在 Snake 中定义一个变量 direction，用于定义蛇头方向，默认情况下，蛇头是向右的，除此之外，我们还定义了贪吃蛇上下左右移动的方法，具体代码如下：

```
import java.awt.*;
import java.util.LinkedList;
public class Snake {
    /**
     * 贪吃蛇创建出来后，默认头是向右的。
     */
    private int direction = Config.RIGHT;
    /**
     * 定义一个集合（复杂的数组）存放蛇的身子（node）
     */
    private LinkedList<Node> snake = new LinkedList<>();
    public Snake() { // 初始状态由三个小格子组成
        snake.add(new Node(6, 4)); // 蛇头
        snake.add(new Node(5, 4)); // 身子
        snake.add(new Node(4, 4)); // 尾巴
    }
    /**
     * 画蛇的方法
     * @param g2d
     */
}
```



```
public void draw(Graphics2D g2d) {
    for (Node node : snake) { // 循环画蛇的每一个 node
        node.draw(g2d);
    }
}

/**
 * 蛇向右移动的方法
 */
public void moveRight() {
    if (direction == Config.LEFT) {
        // 蛇头是向右的，不可以处理向左键
        return;
    }

    System.out.println("向右移动");
    direction = Config.RIGHT; // 更新蛇头方向向右
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();

    Node node = new Node(x + 1, y);
    snake.addFirst(node);
    // 删除蛇的尾巴
    snake.removeLast();
}

/**
 * 蛇向左移动的方法
 */
public void moveLeft() {
    if (direction == Config.RIGHT) {
        // 蛇头是向左的，不可以处理向右键
        return;
    }

    System.out.println("向左移动");
    direction = Config.LEFT; // 更新蛇头方向向左
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();

    Node node = new Node(x - 1, y);
    snake.addFirst(node);
    // 删除蛇的尾巴
    snake.removeLast();
}
```



```
/**
 * 蛇向上移动的方法
 */
public void moveUp() {
    if (direction == Config.DOWN) {
        // 蛇头是向上的，不可以处理向下键
        return;
    }
    System.out.println("向上移动");
    direction = Config.UP; // 更新蛇头方向向上
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();

    Node node = new Node(x, y - 1);
    snake.addFirst(node);
    // 删除蛇的尾巴
    snake.removeLast();
}
/**
 * 蛇向下移动的方法
 */
public void moveDown() {
    if (direction == Config.UP) {
        // 蛇头是向下的，不可以处理向上键
        return;
    }
    System.out.println("向下移动");
    direction = Config.DOWN; // 更新蛇头方向向下
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();

    Node node = new Node(x, y + 1);
    snake.addFirst(node);
    // 删除蛇的尾巴
    snake.removeLast();
}
}
```

定义好贪吃蛇上下左右移动的方法之后，需要更新贪吃蛇业务，在更新逻辑方法中，根据监听到按下的按键，调用贪吃蛇对应上下左右移动的方法，具体代码如下所示。

```
/**
```



```
* 小牛程序员写的业务逻辑，贪吃蛇的业务
*/

public class SnakeGameEngine extends GameEngine {

    Grid grid=new Grid();

    Snake snake = new Snake();

    @Override

    public void updateLogic() {

        int keyCode = getCurrentPressedKeyCode();

        //根据按键行为 控制贪吃蛇的上下左右移动。 必须要知道蛇头当前的方向

        if(keyCode == KeyEvent.VK_RIGHT){

            //向右移动

            snake.moveRight();

        }

        if(keyCode == KeyEvent.VK_LEFT){

            //向左移动

            snake.moveLeft();

        }

        if(keyCode == KeyEvent.VK_DOWN){

            //向下移动

            snake.moveDown();

        }

        if(keyCode == KeyEvent.VK_UP){

            //向上移动

            snake.moveUp();

        }

    }

    @Override

    public void renderUI(Graphics2D g2d) {

        grid.draw(g2d);

        snake.draw(g2d);

    }

}
```

因为贪吃蛇创建出来后，默认头是向右的。程序运行后，贪吃蛇可以向右、向上、向下移动，本次以向右移动为例，效果如图 12-5 所示。

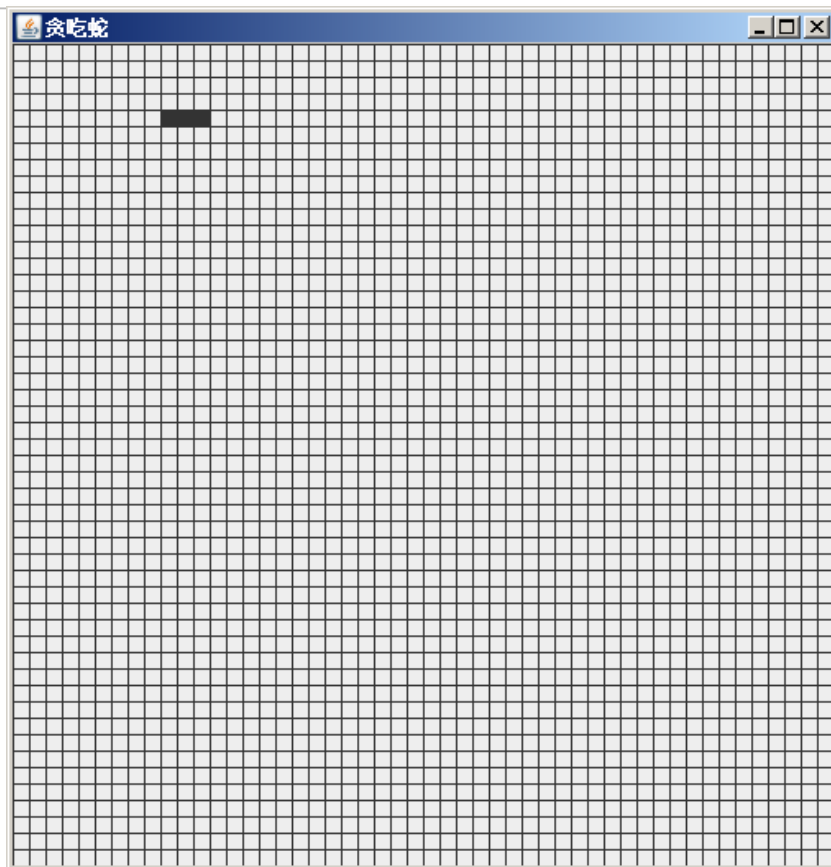


图 12-5 运行效果

从图 12-5 可以看出，蛇的位置相对初始位置向右移动了。

12.6 绘制苹果

贪吃蛇游戏是需要让蛇去吃苹果，贪吃蛇的身子和移动实现好了之后，缺少的就是苹果。贪吃蛇游戏中苹果本质就是一个小方块，我们可以在随机位置绘制一个红色的小方块作为苹果。

接下来，定义一个 Apple 类代表游戏中的苹果，具体代码如下所示。

```
/**
 * 贪吃蛇吃的果实
 */
public class Apple {
    private Node node;
    private Random random = new Random();
    public int getAppleX(){
        return node.getX();
    }
    public int getAppleY(){
        return node.getY();
    }
    public Apple(){
```



```
        node = new Node(random.nextInt(Config.WIDTH_NUMBER),
                        random.nextInt(Config.HEIGHT_NUMBER));
    }

    public void draw(Graphics2D g2d) {
        g2d.setColor(Color.RED);
        node.draw(g2d);
    }
}
```

上述代码中，Apple 的构造方法中创建了一个随机坐标的 Node，作为苹果，并在 draw() 方法中将这个 Node 设置为红色，并绘制出来。

Apple 类定义好之后，需要在贪吃蛇的业务类中将苹果绘制出来，SnakeGameEngine 修改后的代码如下所示。

```
/**
 * 小牛程序员写的业务逻辑，贪吃蛇的业务
 */
public class SnakeGameEngine extends GameEngine {
    Grid grid=new Grid();
    Snake snake = new Snake();
    Apple apple=new Apple();
    @Override
    public void updateLogic() {
        int keyCode = getCurrentPressedKeyCode();
        //根据按键行为 控制贪吃蛇的上下左右移动。 必须要知道蛇头当前的方向
        if(keyCode == KeyEvent.VK_RIGHT){
            //向右移动
            snake.moveRight();
        }
        if(keyCode == KeyEvent.VK_LEFT){
            //向左移动
            snake.moveLeft();
        }
        if(keyCode == KeyEvent.VK_DOWN){
            //向下移动
            snake.moveDown();
        }
        if(keyCode == KeyEvent.VK_UP){
            //向上移动
            snake.moveUp();
        }
    }
    @Override
    public void renderUI(Graphics2D g2d) {
        grid.draw(g2d);
    }
}
```



```
snake.draw(g2d);  
apple.draw(g2d);  
}  
}
```

程序的运行结果如图 12-6 所示。

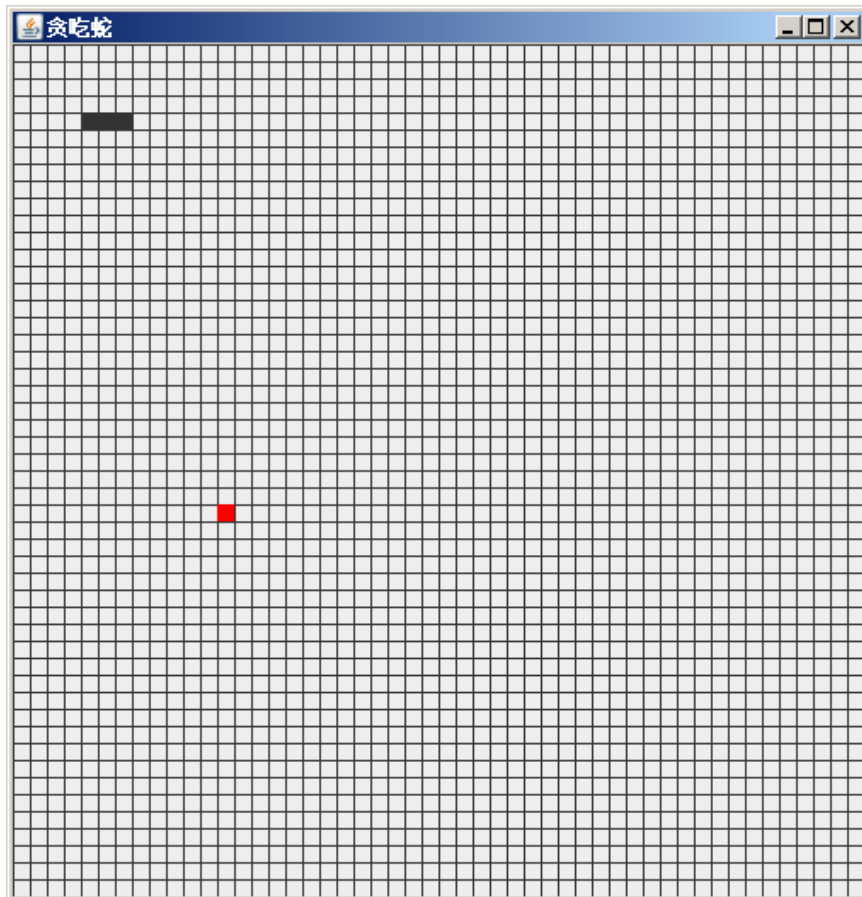


图 2-6 运行结果

12.7 吃苹果操作

绘制完苹果之后，接下来我们继续实现蛇吃苹果的功能。蛇吃苹果的本质，其实就是当蛇移动时，判断蛇的头部有没有苹果（蛇头的坐标和苹果的坐标是否一样），如果有苹果，则吃掉苹果。吃掉苹果后，需要再次随机生成一个苹果，并且蛇的长度应该多一格，此时只需将原来应该删除的蛇的尾部不进行移除即可。

下面更新 Snake 类，在类中新增吃苹果的方法，并且更新上下左右移动的方法，在移动的方法中，判断是否吃到苹果来确定是否需要移除蛇的尾巴。具体代码如下所示。

```
public class Snake {  
    /**  
     * 贪吃蛇创建出来后，默认头是向右的。  
     */  
    private int direction = Config.RIGHT;  
    /**
```




```
* 定义一个集合（复杂的数组）存放蛇的身子（node）
*/
private LinkedList<Node> snake = new LinkedList<>();
public Snake() { // 初始状态由三个小格子组成
    snake.add(new Node(6, 4)); // 蛇头
    snake.add(new Node(5, 4)); // 身子
    snake.add(new Node(4, 4)); // 尾巴
}
/**
 * 画蛇的方法
 * @param g2d
 */
public void draw(Graphics2D g2d) {
    for (Node node : snake) { // 循环画蛇的每一个 node
        node.draw(g2d);
    }
}
/**
 * 蛇向右移动的方法
 */
public void moveRight(boolean haveApple) {
    if (direction == Config.LEFT) {
        // 蛇头是向右的，不可以处理向左键
        return;
    }
    System.out.println("向右移动");
    direction = Config.RIGHT; // 更新蛇头方向向右
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();
    Node node = new Node(x + 1, y);
    snake.addFirst(node);

    if (!haveApple)
        // 删除蛇的尾巴
        snake.removeLast();
}
/**
 * 蛇向左移动的方法
 */
public void moveLeft(boolean haveApple) {
    if (direction == Config.RIGHT) {
        // 蛇头是向左的，不可以处理向右键
        return;
    }
}
```



```
    }

    System.out.println("向左移动");
    direction = Config.LEFT; // 更新蛇头方向向左
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();
    Node node = new Node(x - 1, y);
    snake.addFirst(node);
    if(!haveApple)
    // 删除蛇的尾巴
    snake.removeLast();
}

/**
 * 蛇向上移动的方法
 */
public void moveUp(boolean haveApple) {
    if (direction == Config.DOWN) {
        // 蛇头是向上的，不可以处理向下键
        return;
    }

    System.out.println("向上移动");
    direction = Config.UP; // 更新蛇头方向向上
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();
    Node node = new Node(x, y - 1);
    snake.addFirst(node);
    if(!haveApple)
    // 删除蛇的尾巴
    snake.removeLast();
}

/**
 * 蛇向下移动的方法
 */
public void moveDown(boolean haveApple) {
    if (direction == Config.UP) {
        // 蛇头是向下的，不可以处理向上键
        return;
    }

    System.out.println("向下移动");
    direction = Config.DOWN; // 更新蛇头方向向下
    // 拿到蛇头
    int x = snake.getFirst().getX();
```



```
int y = snake.getFirst().getY();
Node node = new Node(x, y + 1);
snake.addFirst(node);

if(!haveApple)
// 删除蛇的尾巴
snake.removeLast();
}
/**
 * 头部是不是有苹果
 * @return true 有苹果 false 没苹果
 */
public boolean headHaveApple(Apple apple){
    if(apple.getAppleX()==snake.getFirst().getX() &&
        apple.getAppleY()==snake.getFirst().getY()){
        //再随机生成个苹果
        apple.generateNewApple();
        return true;
    }else{
        return false;
    }
}
```

接下来，更新贪吃蛇业务类 SnakeGameEngine 中的 updateLogic() 方法，将当前是否吃到苹果的结果传递给移动方法，updateLogic() 方法修改后的代码如下所示。

```
public void updateLogic() {
    int keyCode = getCurrentPressedKeyCode();
    //根据按键行为 控制贪吃蛇的上下左右移动。 必须要知道蛇头当前的方向
    if(keyCode == KeyEvent.VK_RIGHT){
        boolean haveapple = snake.headHaveApple(apple);
        //向右移动
        snake.moveRight(haveapple);
    }
    if(keyCode == KeyEvent.VK_LEFT){
        boolean haveapple = snake.headHaveApple(apple);
        //向左移动
        snake.moveLeft(haveapple);
    }
    if(keyCode == KeyEvent.VK_DOWN){
        boolean haveapple = snake.headHaveApple(apple);
        //向下移动
        snake.moveDown(haveapple);
    }
    if(keyCode == KeyEvent.VK_UP){
```



```
        boolean haveapple = snake.headHaveApple(apple);  
        //向上移动  
        snake.moveUp(haveapple);  
    }  
}
```

12.8 贪吃蛇的速度

至此可以操作贪吃蛇进行吃苹果，但是默认情况下只有按下上下左右键时，贪吃蛇才会移动。如果想要让贪吃蛇自己跟随蛇头的方向默认移动，可以在贪吃蛇的业务类的 `updateLogic()` 方法中进行设置。

设置蛇的默认移动行为之前，需要在 `Snake` 类中新增一个获取蛇头方向的方法，具体代码如下所示。

```
public int getDirection(){  
    return direction;  
}
```

能获取到蛇头的方向之后，在 `SnakeGameEngine` 类中的 `updateLogic()` 方法中新增蛇默认移动的代码，修改后 `updateLogic()` 方法的具体代码如下所示。

```
public void updateLogic() {  
    int keyCode = getCurrentPressedKeyCode();  
    //根据按键行为 控制贪吃蛇的上下左右移动。 必须要知道蛇头当前的方向  
    if(keyCode == KeyEvent.VK_RIGHT){  
        boolean haveapple = snake.headHaveApple(apple);  
        //向右移动  
        snake.moveRight(haveapple);  
    }  
    if(keyCode == KeyEvent.VK_LEFT){  
        boolean haveapple = snake.headHaveApple(apple);  
        //向左移动  
        snake.moveLeft(haveapple);  
    }  
    if(keyCode == KeyEvent.VK_DOWN){  
        boolean haveapple = snake.headHaveApple(apple);  
        //向下移动  
        snake.moveDown(haveapple);  
    }  
    if(keyCode == KeyEvent.VK_UP){  
        boolean haveapple = snake.headHaveApple(apple);  
        //向上移动  
        snake.moveUp(haveapple);  
    }  
    boolean haveapple = snake.headHaveApple(apple);  
}
```



```
//蛇的默认行为
switch (snake.getDirection()) {
    case Config.RIGHT:
        //向右
        snake.moveRight(haveapple);
        break;
    case Config.LEFT:
        //向左
        snake.moveLeft(haveapple);
        break;
    case Config.UP:
        //向上
        snake.moveUp(haveapple);
        break;
    case Config.DOWN:
        //向下
        snake.moveDown(haveapple);
        break;
}
}
```

此时运行程序，蛇默认会向蛇头的方向移动。由于 Game 类初始化游戏时设置的是每 10 毫秒执行一次业务更新和界面渲染，所以此时蛇默认移动的速度太快，想要设置蛇的移动速度，可以修改 Game 类 init() 方法中的休眠时间。在此设置休眠的时间为 100 毫秒，代码如下。

```
public static void init(String title, int width, int height,
    GameEngine engine){
    frame = new JFrame(title);
    Game game = new Game();
    game.engine = engine;
    game.addKeyListener(engine.listener);
    game.setFocusable(true);
    game.setPreferredSize(new Dimension(width, height));
    frame.add(game);
    frame.pack();
    frame.setVisible(true);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    while (true) {
        engine.updateLogic();
        game.repaint();
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```



```
    }  
    }  
}
```

12.5 贪吃蛇的优化

贪吃蛇游戏的整体功能基本上已经完成，贪吃蛇的可以根据设置的默认速度进行移动和吃苹果。随着贪吃蛇吃的苹果越来越多，贪吃蛇的身体也会越来越长，此时的程序是允许贪吃蛇吃到自己，如果想要设置当贪吃蛇吃到自己时，就结束游戏，可以对程序进一步优化。可以在贪吃蛇移动时，判断蛇头要移动的下一个 Node 和蛇的身体中某一个 Node 是否一致，如果一致说明贪吃蛇咬到自己了，弹出结束游戏的对话框。

为了确保每一个网格所创建的 Node 对象是同一个，需要在 Node 类中重写 equals() 方法和 hashCode() 方法，重写后 Node 类的代码如下所示。

```
public class Node {  
    /**  
     * 小格在屏幕上的位置 x  
     */  
    private int x;  
    /**  
     * 小格在屏幕上的位置 y  
     */  
    private int y;  
    public Node(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public int getX() {  
        return x;  
    }  
    public void setX(int x) {  
        this.x = x;  
    }  
    public int getY() {  
        return y;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
    public void draw(Graphics2D g2d) {  
        g2d.fillRect(x * Config.GRID_SIZE, y * Config.GRID_SIZE,  
            Config.GRID_SIZE, Config.GRID_SIZE);  
    }  
}
```



```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Node node = (Node) o;
    return x == node.x && y == node.y;
}

@Override
public int hashCode() {
    return Objects.hash(x, y);
}
}
```

接下来，在 Snake 类的上下左右的移动方法中加入贪吃蛇是否咬自己的判断，并对判断结果进行对应的处理，如果具体代码如下所示。

```
/**
 * 蛇向右移动的方法
 */
public void moveRight(boolean haveApple) {
    if (direction == Config.LEFT) {
        // 蛇头是向右的，不可以处理向左键
        return;
    }
    System.out.println("向右移动");
    direction = Config.RIGHT; // 更新蛇头方向向右
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();
    Node node = new Node(x + 1, y);
    if (snake.contains(node)) {
        Game.gameOver("自己咬着自己了", "游戏失败");
    }
    snake.addFirst(node);
    if (!haveApple)
        // 删除蛇的尾巴
        snake.removeLast();
}

/**
 * 蛇向左移动的方法
 */
public void moveLeft(boolean haveApple) {
    if (direction == Config.RIGHT) {
        // 蛇头是向左的，不可以处理向右键
        return;
    }
}
```



```
    }

    System.out.println("向左移动");
    direction = Config.LEFT; // 更新蛇头方向向左
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();
    Node node = new Node(x - 1, y);
    if (snake.contains(node)) {
        Game.gameOver("自己咬了自己", "游戏失败");
    }
    snake.addFirst(node);
    if (!haveApple)
        // 删除蛇的尾巴
        snake.removeLast();
}

/**
 * 蛇向上移动的方法
 */
public void moveUp(boolean haveApple) {
    if (direction == Config.DOWN) {
        // 蛇头是向上的，不可以处理向下键
        return;
    }
    System.out.println("向上移动");
    direction = Config.UP; // 更新蛇头方向向上
    // 拿到蛇头
    int x = snake.getFirst().getX();
    int y = snake.getFirst().getY();
    Node node = new Node(x, y - 1);
    if (snake.contains(node)) {
        Game.gameOver("自己咬着自己了", "游戏失败");
    }
    snake.addFirst(node);
    if (!haveApple)
        // 删除蛇的尾巴
        snake.removeLast();
}

/**
 * 蛇向下移动的方法
 */
public void moveDown(boolean haveApple) {
    if (direction == Config.UP) {
```




```
// 蛇头是向下的，不可以处理向上键
return;

}

System.out.println("向下移动");
direction = Config.DOWN; // 更新蛇头方向向下
// 拿到蛇头
int x = snake.getFirst().getX();
int y = snake.getFirst().getY();
Node node = new Node(x, y + 1);
if (snake.contains(node)) {
    Game.gameOver("自己咬着自己了", "游戏失败");
}
snake.addFirst(node);
if (!haveApple)
// 删除蛇的尾巴
snake.removeLast();
}
```

解决完贪吃蛇可以咬自己的问题后，为了界面更美观，此时可以设置不显示界面中的网格，将 Config 类中 DEBUG 属性的值设置为 false 即可，代码如下。

```
/**
 * 开发调试的 flag，用于标识网格是否显示
 */
public static final boolean DEBUG = false;
```

12.5 案例总结

贪吃蛇是应用面向对象非常经典的一款游戏，通过实现贪吃蛇案例可以很好的锻炼面向对象思维，对于初学者来说，建议大家根据实现步骤逐步实现。现在对贪吃蛇的实现步骤进行回顾，具体如下。

1. 游戏环境搭建

游戏环境的搭建主要是在项目中引入游戏的公共类和模板类、创建游戏的配置文件、创建贪吃蛇的游戏的业务逻辑类、创建游戏入口。这些类的作用具体如下。

- (1) Game.java: 大牛程序员写的游戏的公共类
- (2) GameEngine.java: 大牛程序员编写的游戏引擎的抽象模板类
- (3) Config.java: 魔法数字的配置文件
- (4) SnakeGameEngine.java: 小牛程序员编写的，GameEngine 的实现类
- (5) SnakeGame.java: 游戏的入口类，提供一个 main 方法

2. 贪吃蛇的网格设计

通过网格的形式更好的理解贪吃蛇的构成，以及业务逻辑的实现。

3. 贪吃蛇的身子绘制

根据网格的设计对贪吃蛇的身子进行绘制。

4. 贪吃蛇的上下左右移动

给贪吃蛇创建移动的方法，通过按键可以对贪吃蛇的方向进行控制，蛇头只能安装蛇前进的方向移动，或者左右移动，不可以倒退。

蛇移动的原理，蛇头添加一个，蛇尾删除。

5. 吃苹果操作

蛇吃苹果，判断蛇的头部有没有苹果，吃掉苹果的话，蛇的尾部无需移除

6. 绘制苹果

在随机位置绘制一个红色 Node 作为苹果。

7. 贪吃蛇的速度

设置贪吃蛇的自动移动行为，蛇的自动移动就是沿着蛇头的方向，继续移动一下

8. 贪吃蛇的优化

优化贪吃蛇自己咬自己的问题，并且去掉网格。

本案例实现的贪吃蛇功能没有太多，但是大家通过案例的学习加深了面向对象思想的理解，可以自行进行拓展，例如，蛇不能超出屏幕范围、地图添加障碍物、添加游戏关卡，蛇移动的速度越来越快等等